Morten Hjorth-Jensen

Quantum Machine Learning

October 11, 2026

Springer

Contents

1 Introduction and Motivation 1
1.1 Two fields and a question 1
1.2 What machine learning is 2
1.3 What quantum machine learning is 3
1.4 Why quantum? 4
1.5 Challenges 5
1.6 Variational circuits reconsidered 6
1.7 Relation to the companion volume 7
1.8 Loading classical data into qubits 8
1.9 The plan of the book 10
1.10 Conventions, notation and software 11
1.11 Exercises 11
Programs for this chapter 12

2 Quantum Support Vector Machines 13
2.1 Introduction 13
2.2 Hyperplanes and the margin 14
2.3 Lagrange multipliers and the dual problem 15
2.4 The soft margin 17
2.5 Kernels 18
2.6 Solving the dual problem 19
2.7 Quantum feature maps and quantum kernels 21
2.8 Estimating the kernel on a quantum computer 23
2.9 The quantum support-vector machine 25
2.9.1 Kernels from a finite number of shots 29
2.10 Training the feature map 31
2.11 Perspectives 32
2.12 Summary 33
2.13 Exercises 33
Programs for this chapter 34

3 The Quantum Approximate Optimization Algorithm 37
3.1 Introduction 37
3.2 From a cost function to a Hamiltonian 38
3.3 The algorithm 40
3.4 The algorithm at work 43
3.4.1 One edge, by hand 43
3.4.2 The ring and a six-vertex graph 43

3.4.3 Performance, landscape and cost . . . 44
3.5 Gradients . . . 45
3.6 A physical register: two electrons in a double quantum well . . . 47
3.7 An application: selecting a portfolio . . . 48
3.8 QAOA as a machine-learning model . . . 52
3.9 Summary . . . 53
3.10 Exercises . . . 54
Programs for this chapter . . . 55

4 Quantum Neural Networks . . . 57
4.1 Introduction . . . 57
4.2 The model . . . 58
4.3 The output as a Fourier series . . . 60
4.4 Training . . . 61
4.5 Geometry: the quantum Fisher information . . . 63
4.6 Barren plateaus . . . 66
4.7 Quantum neural networks at work . . . 69
4.7.1 Regression . . . 69
4.7.2 Classification . . . 70
4.7.3 A QNN as a scoring function: portfolio selection . . . 71
4.8 Physics-informed quantum neural networks . . . 72
4.8.1 From data to equations . . . 72
4.8.2 Derivatives with respect to the input . . . 73
4.8.3 The Poisson equation . . . 74
4.8.4 An initial-value problem and a wave equation . . . 75
4.8.5 What the method can and cannot do . . . 77
4.9 Summary and perspectives . . . 78
4.10 Exercises . . . 78
Programs for this chapter . . . 79

5 Quantum Boltzmann Machines . . . 81
5.1 Introduction . . . 81
5.2 Energy-based models and the Boltzmann machine . . . 82
5.2.1 Learning as the minimization of a divergence . . . 82
5.2.2 Energy-based models . . . 83
5.2.3 The Boltzmann machine and its restricted form . . . 83
5.2.4 Gibbs sampling and contrastive divergence . . . 84
5.3 The quantum Boltzmann machine . . . 86
5.3.1 Definition . . . 86
5.3.2 The gradient: Duhamel's formula . . . 87
5.3.3 The clamped-state bound . . . 89
5.4 Boltzmann machines on classical data . . . 90
5.5 Learning quantum states . . . 92
5.6 Preparing Gibbs states on a quantum computer . . . 94
5.6.1 The free-energy principle . . . 94
5.6.2 The thermofield double . . . 95
5.6.3 The variational quantum thermalizer . . . 96
5.7 Connections . . . 98
5.7.1 Quantum neural networks and quantum Boltzmann machines . . . 98
5.7.2 The transverse-field Ising model as a learning problem . . . 99
5.7.3 Expressivity, hardware and honest expectations . . . 99
5.8 Summary and perspectives . . . 100

5.9 Exercises . . . 100
Programs for this chapter . . . 101

6 Reinforcement Learning and Quantum Reinforcement Learning . . . 103
6.1 Scope of this chapter . . . 103
6.2 Exercises . . . 104

7 Summary and Perspectives . . . 105
7.1 What the book has shown . . . 105
7.2 The reassessment, with the evidence in hand . . . 106
7.3 A priority list for the next years . . . 107
7.4 For many-body physics and quantum sensing . . . 108
7.5 Perspectives . . . 108
References . . . 111

Index . . . 115

Acronyms

The following abbreviations are used throughout the book. Pauli strings such as ZZ or Z_0Z_1, the names of gates and the names of program classes are not listed.

ADAPT-VQE	Adaptive derivative-assembled pseudo-Trotter VQE (a VQE that grows its own ansatz)
BFGS	Broyden–Fletcher–Goldfarb–Shanno quasi-Newton optimizer
CD	Contrastive divergence
CNOT	Controlled-NOT gate
ESR	Electron spin resonance
FCI	Full configuration interaction
HHL	Harrow–Hassidim–Lloyd algorithm for linear systems of equations
KKT	Karush–Kuhn–Tucker conditions
KL	Kullback–Leibler divergence
MLP	Multilayer perceptron (a fully connected neural network)
NISQ	Noisy intermediate-scale quantum (device)
PCD	Persistent contrastive divergence
PINN	Physics-informed neural network
PSD	Positive semidefinite
QAOA	Quantum approximate optimization algorithm
QBM	Quantum Boltzmann machine
QED	Quantum electrodynamics (circuit QED: superconducting circuits coupled to microwave resonators)
QFI	Quantum Fisher information
QML	Quantum machine learning
QNN	Quantum neural network
Q-PINN	Physics-informed quantum neural network
QSVM	Quantum support-vector machine
QUBO	Quadratic unconstrained binary optimization
RBF	Radial basis function (Gaussian kernel)
RBM	Restricted Boltzmann machine
SMO	Sequential minimal optimization
SR	Stochastic reconfiguration
SVM	Support-vector machine
TDVP	Time-dependent variational principle
TFD	Thermofield double (state)
VQE	Variational quantum eigensolver
VQT	Variational quantum thermalizer
XOR	Exclusive or (the parity classification problem)

Chapter 1
Introduction and Motivation

Abstract Machine learning and quantum computing can be combined in four ways, according to whether the data and the computer are classical or quantum; this book is mainly about classical data processed by a quantum computer, the setting of the quantum support-vector machine, the quantum neural network and the quantum approximate optimization algorithm. The chapter recalls what machine learning is, defines a quantum machine-learning model as a parametrized circuit with a data-dependent encoding and a measurement, and examines the three properties of quantum mechanics usually invoked in its favor, the size of the state space, interference and entanglement, together with the two kinds of speedup claim in the literature and the conditions they rest on. The challenges of near-term hardware, shot noise, data encoding, barren plateaus and fair benchmarking are set out, and the current reassessment of variational circuits is presented: their trainability limits, the classical surrogates that compete with them, the total cost of encoding and measurement, and the evidence that the clearest advantages concern quantum rather than classical data. A table maps the chapters of the companion volume, and of the author's machine-learning and many-body texts, to their use here. The basis, angle and amplitude encodings are introduced with a program on the book's own simulator, and the plan of the book, its conventions and its software are described.

1.1 Two fields and a question

Machine learning and quantum computing are the two computational developments of the last decades that have changed what physicists expect a computer to do. The first has turned the fitting of functions to data, an activity as old as least squares, into a technology that recognizes images, translates languages, predicts the folding of proteins and, closer to the subjects of this book, represents the ground states of many-body Hamiltonians by neural networks and emulates expensive simulations by surrogate models. The second has produced, after forty years of theory, machines of a few hundred qubits on which short circuits can be run, and a catalog of algorithms, among them phase estimation, Hamiltonian simulation and the variational quantum eigensolver, which the companion volume *Quantum Computing for Many-body Physics* [1] develops for the many-body problem. It was inevitable that the two would be combined, and *quantum machine learning* (QML) is the name of the combination [2–4]. This book is the second half of that story and was written to be read after the first: the companion volume is the material of the first half of a one-semester course on quantum computing and quantum machine learning, and the present chapters are the second half. They use its qubits, gates, measurements, Hamiltonians and variational algorithms as background without rederiving them. Section 1.7 says precisely which parts.

The combination can be made in four ways, and it is worth being clear from the start which of them this book is about. The data can be classical, such as the pixels of an image, the features of a flower or the measured properties of a material, or quantum, such as the state of a register prepared by a quantum simulation; and the computer that processes the data can be classical or quantum. Classical data on a classical computer is ordinary machine learning. Quantum data on a classical computer is what the neural-network quantum states and Monte Carlo methods of many-body physics have done for a decade, with the state represented classically. Quantum data on a quantum computer is the most natural pairing, and the one where the strongest arguments for an advantage exist, because the data are already in the form the machine works with and no loading step is needed. The bulk of this book, and all of Chapters 2, 3 and 4, concerns the remaining case, *classical data on a quantum computer*: the classical input $\boldsymbol{x}$ is written into the state of a register by a data-dependent circuit, a parametrized circuit processes it, and a measurement returns the prediction. This is the setting of the quantum support-vector machine, the quantum neural network and the quantum approximate optimization algorithm, it is the setting in which the first experiments on real hardware have been performed, and it is also the setting in which the claims of quantum advantage must be read most carefully. Chapter 5 on quantum Boltzmann machines, whose model is a Hamiltonian and whose samples come from its thermal state, is the point of the book where the quantum-data case is approached.

The question that runs through the book is a simple one. A quantum computer with n qubits has a state space of 2^n dimensions, which is more than any classical feature space a machine could afford to work in explicitly. Does this give a learning algorithm anything that a classical algorithm cannot have? The honest answer, developed from Section 1.4 onwards, is "sometimes, under conditions that are easy to state and hard to meet", and the purpose of the book is to put the reader in a position to judge for themselves, with the mathematics written out and with programs that run.

1.2 What machine learning is

Machine learning is the study of algorithms whose performance on a task improves with data. The data are a set of *samples*, each a vector $\boldsymbol{x} \in \mathbb{R}^d$ of d *features*, and the task is specified by what, if anything, accompanies each sample.

- In *supervised learning* every sample comes with a label y, a class ($y = \pm 1$ for the binary classification problems of Chapter 2) or a real number (regression), and the algorithm must predict the label of samples it has not seen.
- In *unsupervised learning* there are no labels, and the algorithm must find structure in the samples themselves: clusters, a low-dimensional description, or a probability distribution from which the samples could have been drawn. The Boltzmann machines of Chapter 5 are models of this kind.
- In *reinforcement learning* the algorithm is an agent that acts in an environment and receives rewards, and it must learn a policy that maximizes the reward over time. Chapter 6 is reserved for it and its quantum versions.

Whatever the task, the workflow is the same and is drawn in Fig. 1.1: a *model* $f(\boldsymbol{x};\boldsymbol{\theta})$ with adjustable parameters $\boldsymbol{\theta}$ is chosen, a *cost* (or loss) function $\mathscr{L}(\boldsymbol{\theta})$ measures how badly the model does on the training data, an *optimizer* moves the parameters so as to reduce the cost, and the trained model is finally judged on *test* data it was not trained on. The last step is the one that distinguishes learning from curve fitting: a model that reproduces the training data perfectly and the test data badly has *overfitted*, and most of the practical art of the field, from regularization to the choice of model class, is about avoiding that.

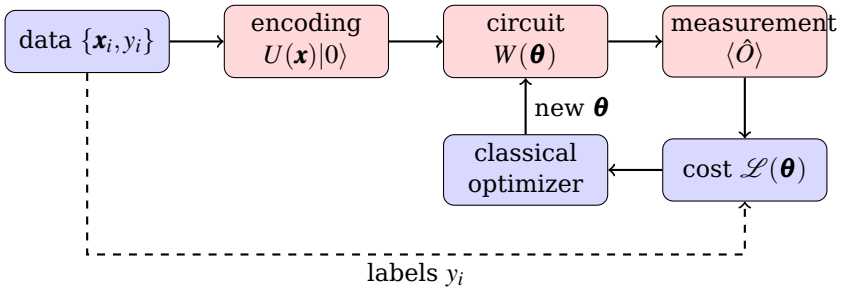


Fig. 1.1 The workflow of a hybrid quantum-classical learning model. The three red boxes run on the quantum computer: a data-dependent circuit $U(\boldsymbol{x})$ loads a sample, a parametrized circuit $W(\boldsymbol{\theta})$ processes it, and a measurement returns a number. Everything else, the cost that compares the output with the label and the optimizer that updates $\boldsymbol{\theta}$, is classical. In a kernel method (Chapter 2) the box $W(\boldsymbol{\theta})$ is absent and the measurement returns an overlap instead.

Two families of supervised models will matter for us. The first, *kernel methods*, of which the support-vector machine of Chapter 2 is the best-known member, never work with the features themselves but only with a similarity $K(\boldsymbol{x},\boldsymbol{x}')$ between pairs of samples; the model is a weighted sum of similarities to the training points, $f(\boldsymbol{x}) = \sum_i \lambda_i y_i K(\boldsymbol{x}_i,\boldsymbol{x}) + b$, and training is a convex optimization problem with a unique solution. The second, *neural networks*, compose simple parametrized maps into a deep function whose parameters are found by gradient descent on a non-convex cost. The quantum models of this book fall into the same two families, and one of the lessons of Chapter 2, due to Schuld [5], is that the division is less sharp than it looks: every quantum model of the form of Fig. 1.1 is a linear model in the space of density matrices, and can therefore be written as a kernel machine. The standard references for the classical material are the books of Hastie, Tibshirani and Friedman [6], Bishop [7] and Goodfellow, Bengio and Courville [8], and, closest to this book in notation, programs and outlook, the author's *Machine Learning and the Physical Sciences* [9], whose chapters on optimization, support-vector machines, neural networks, neural networks for differential equations, Boltzmann machines and reinforcement learning are the classical halves of Chapters 2, 4, 5 and 6; we assume the reader has met linear regression, gradient descent and the idea of a training and a test set, and we derive everything else that we need.

1.3 What quantum machine learning is

In the setting of this book a quantum machine-learning model is a function from a classical input to a real number, computed as the expectation value of an observable in a state prepared by a circuit that depends on the input and on parameters:

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \langle 0|\, U^\dagger(\boldsymbol{x})\, W^\dagger(\boldsymbol{\theta})\, \hat{O}\, W(\boldsymbol{\theta})\, U(\boldsymbol{x})\, |0\rangle. \tag{1.1}$$

Here $|0\rangle = |0\rangle^{\otimes n}$ is the initial state of n qubits, $U(\boldsymbol{x})$ is the *encoding* or *feature map* circuit that writes the sample into the register, $W(\boldsymbol{\theta})$ is a parametrized circuit, and $\hat{O}$ is a Hermitian operator, usually a single Pauli Z or a product of them, whose expectation value is estimated by repeating the circuit and averaging the measurement outcomes. Every model in Chapters 2 to 4 is a special case of Eq. (1.1).

- The *quantum support-vector machine* of Chapter 2 has no trainable W at all. Its output is the overlap of two encoded states, $K(\boldsymbol{x},\boldsymbol{x}') = |\langle 0|U^\dagger(\boldsymbol{x})U(\boldsymbol{x}')|0\rangle|^2$, which is Eq. (1.1) with

$\hat{O} = |0\rangle\langle 0|$ and $U^\dagger(\boldsymbol{x}')$ in the place of W; the quantum computer supplies a kernel and a classical support-vector machine does the learning.
- The *quantum approximate optimization algorithm* of Chapter 3 encodes the problem rather than a sample: the data, a graph or a cost matrix, define a diagonal Hamiltonian H_C, $W(\boldsymbol{\theta})$ alternates $e^{-i\gamma H_C}$ with a mixing term, and $\hat{O} = H_C$ itself.
- The *quantum neural network* or variational quantum classifier of Chapter 4 is Eq. (1.1) as it stands, with layers of rotations and entangling gates in $W(\boldsymbol{\theta})$ and the sign of f as the predicted class; its parameters are trained by gradient descent, with gradients that the quantum computer itself can evaluate exactly by the parameter-shift rule [10, 11].
- The *quantum Boltzmann machine* of Chapter 5 replaces the pure state by the thermal state $e^{-\beta H(\boldsymbol{\theta})}/Z$ of a parametrized Hamiltonian, and its training adjusts H so that the measurement statistics of the thermal state match those of the data.

The quantum computer, in all four, is used as a device that prepares a state in a 2^n-dimensional space and estimates expectation values in it; the learning, the comparison with the labels and the update of the parameters, is classical. This is the *hybrid* architecture of Fig. 1.1, the same architecture as the variational quantum eigensolver, and the reader who knows VQE from the companion volume [1] will recognize every piece of it: the ansatz is $W(\boldsymbol{\theta})$, the Hamiltonian is $\hat{O}$, and the only new element is the encoding $U(\boldsymbol{x})$, which makes the circuit depend on data. That element turns out to be decisive; Schuld's remark that "the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models" [5] could serve as the motto of the whole book.

1.4 Why quantum?

Three properties of quantum mechanics are usually invoked, and it pays to state exactly what each does and does not give.

The size of the state space The state of n qubits is a vector of 2^n complex amplitudes. An encoding circuit $U(\boldsymbol{x})$ therefore maps a d-dimensional sample into a 2^n-dimensional feature space, and the overlap of two encoded states is an inner product in that space: this is a kernel, and a kernel is exactly what a support-vector machine needs. The analogy with the classical kernel trick, where a polynomial or Gaussian kernel evaluates an inner product in a high- or infinite-dimensional space without ever constructing the feature vectors, is exact, and Chapter 2 makes it precise. What the size of the space does *not* give is a free lunch: a classical computer can also work in a 2^n-dimensional space implicitly, through the kernel, and the question is whether the particular inner product that the quantum circuit computes is useful for the data at hand and hard to compute classically. Both conditions are needed, and the second is only known to hold for feature maps built from problems, such as the discrete logarithm, that are believed to be classically intractable [12].

Interference Amplitudes add before probabilities are taken, so a circuit can arrange for wrong answers to cancel and right ones to reinforce. This is what makes quantum algorithms faster than classical ones when they are, and it is what the quantum Fourier transform exploits: the discrete Fourier transform of N numbers costs $\mathcal{O}(N^2)$ operations, the fast Fourier transform $\mathcal{O}(N \log N)$, and the quantum Fourier transform $\mathcal{O}(\log^2 N)$ gates on $\log_2 N$ qubits [13]. The last figure is the one usually quoted and the one that needs reading twice: the transform is applied to amplitudes, which must first be loaded and cannot afterwards be read out one by one, so the exponential saving is real only for problems in which the input is already a quantum state and the output is a single number or a sample. Phase estimation, which the

companion volume uses to compute eigenvalues, is such a problem. Transforming a classical signal is not.

Entanglement A product state of n qubits is described by $2n$ real parameters, an entangled one by up to $2^{n+1}-2$. Entangling gates in $U(\boldsymbol{x})$ therefore produce feature maps whose kernels do not factorize into one-dimensional pieces, and this is where a quantum kernel can differ from any classical kernel that is cheap to evaluate. The angle encoding of Section 1.8, which puts one feature on one qubit and entangles nothing, has a kernel that is a product of cosines (Exercise 2) and is simulated by a classical computer in $\mathcal{O}(d)$ operations; the entangling feature map of Havlíček *et al.* [14] has a kernel that, for enough qubits, no known classical method evaluates efficiently. Chapter 2 runs both.

The speedup claims Two kinds of claim appear in the literature, and they are made for different machines. The first, due to Rebentrost, Mohseni and Lloyd [15], applies the quantum linear-systems algorithm of Harrow, Hassidim and Lloyd [16] to the equations that train a least-squares support-vector machine, and obtains a running time polylogarithmic in the number of samples N and the number of features d. The result is correct and the assumptions are severe: the data must be available in a quantum random-access memory [17] that loads a vector of N entries in $\mathcal{O}(\log N)$ time, the kernel matrix must be well conditioned, and the output is a quantum state from which only a few numbers can be extracted. Aaronson's "read the fine print" [18] is the reference on these conditions, and Tang's dequantization of the recommendation-systems algorithm [19] showed that, once the same data-access assumptions are granted to a classical algorithm, the exponential advantage of several of these proposals disappears. The second kind of claim makes no promise about running time at all. It concerns the *near-term* or NISQ machines [20] of the present decade, estimates the kernel or runs the variational circuit on them, and asks only whether the resulting model can learn something that classical models cannot. The answer is known to be yes for carefully constructed problems [12], is known to depend on the data rather than on the model [21], and is open for the data sets one actually meets. Everything in Chapters 2 to 4 is of the second kind, and we try to say in each case what was gained and what was not.

1.5 Challenges

The reasons that quantum machine learning is a research field rather than a product are the same reasons that the companion volume gives for every near-term algorithm, with two additions, data encoding and fair benchmarking, that are specific to learning.

Hardware The qubits of today's machines decohere in microseconds to milliseconds, two-qubit gates fail at the level of 10^{-3} to 10^{-2}, and circuits must therefore be shallow. A model whose accuracy depends on a deep entangling feature map, or on many layers of $W(\boldsymbol{\theta})$, cannot be run as written. Every result in this book is obtained on a classical state-vector simulator of our own, in which the state never decoheres, and the chapters say where a real device would differ.

Shot noise Every number a quantum computer returns is an estimate from a finite number S of repetitions, with a statistical error that falls only as $1/\sqrt{S}$. For a kernel method this means that every entry of the $N \times N$ kernel matrix is noisy, and Section 2.9.1 shows what that does to the classifier; for a variational model it means that every gradient component is noisy, and the optimizer must be one that tolerates that.

Data encoding A quantum computer cannot read a file. Every sample has to be written into the register by a circuit, and the choice of that circuit, taken up in Section 1.8 and throughout

Chapter 2, is both the main design freedom of a quantum model and its main cost: an encoding that is cheap uses one qubit per feature and an encoding that is compact needs a circuit of depth exponential in the number of qubits.

Trainability The cost landscape of a generic parametrized circuit on n qubits is exponentially flat: the variance of any gradient component falls as 2^{-n} once the circuit is deep enough to approximate a random unitary [22]. These *barren plateaus*, familiar from VQE, are made worse in learning by the need to evaluate the circuit on every training sample, and the analogous phenomenon for kernels, exponential concentration of all kernel values towards a constant [23], limits which feature maps can be used at scale.

Scale and benchmarks The experiments that can be run today have tens of qubits and hundreds of samples, while the classical methods they are compared with handle millions of both. A fair comparison therefore has to be made on small problems, where classical kernels and small neural networks are hard to beat, and much of the early literature reported quantum models that matched a classical baseline rather than exceeded it [24]. We follow the same discipline: every quantum model in this book is run beside the classical model it replaces, on the same data, in the same program.

1.6 Variational circuits reconsidered

The models of Chapters 3 to 5 are all *variational*: a circuit with adjustable parameters, a cost measured on a quantum device, and a classical optimizer in a loop. More than a decade after the first proposals this architecture is not being abandoned, but its role is being reassessed, and a textbook written in 2026 owes the reader an account of why. Our reading of the literature is that confidence has weakened in *generic* parametrized circuits as replacements for classical neural networks, and has shifted towards uses of quantum hardware for a specific physical or information-processing task, with substantial classical learning around it. The reasons are three connected difficulties, each of which the later chapters meet in a concrete form.

Trainability The gradients of a generic parametrized circuit become exponentially small as the number of qubits grows (the barren plateaus of the previous section, reproduced numerically in Section 4.6), and noise and finite measurement statistics compound the problem: a gradient that is 10^{-6} in exact arithmetic cannot be resolved with 10^4 shots. The remedies, shallow circuits, local costs, structured ansätze and informed initializations, all restrict the circuit to a corner of the space of unitaries.

Classical competition The corner is often one that a classical computer can reach. The circuits that provably avoid barren plateaus are, in many of the architectures proposed so far, also circuits that admit efficient classical surrogates: Cerezo and coworkers [25] establish this connection case by case, sometimes at the price of an initial quantum data-acquisition stage, and Bermejo and coworkers [26] construct classical surrogates for the standard quantum convolutional neural network from classical shadows of its input. In the second case quantum measurements of the input may still be needed, but the repeated quantum training loop can be dispensed with under the conditions studied. The qualification matters: the absence of barren plateaus does not universally imply classical simulability, and these results are a serious challenge to particular designs rather than a theorem that rules out variational learning. They do mean that “it trains” is no longer evidence of “it needs a quantum computer”.

Total cost Encoding classical data, estimating observables to the precision a learning problem needs, and calling the hardware once per sample per gradient step can outweigh any

benefit of the quantum model. Hoefler, Häner and Troyer [27] make the general point for quantum computing: an algorithm whose input is large and whose quantum part is a modest polynomial speedup loses to the classical machine on input and output alone. For learning, where the input is the data set, this is the rule rather than the exception.

Classical data and quantum data The distinction that organizes the way out is the one made in Section 1.1: learning *classical data with a quantum computer*, and learning *about quantum systems with quantum resources*. For the second there is now concrete evidence of an advantage. Huang and coworkers [28] showed that an agent with coherent access to several copies of a state, and the ability to measure them jointly, needs exponentially fewer experiments than one that measures each copy separately and reasons classically, and a photonic experiment of Liu and coworkers [29] demonstrated such an advantage for the learning of bosonic displacement processes with entanglement. This is evidence for learning quantum processes, not for a general advantage on ordinary classification data sets; quantum inputs avoid the data-loading bottleneck, and coherent access and entangled measurements can offer sample advantages, but the advantages depend on the task and on the experimental access permitted. The classical-surrogate results point the same way from the other side: the essential quantum contribution may be *acquiring* information, with the useful learning done classically on the measurement record.

What remains for variational circuits Within this picture variational circuits keep a credible role, in three forms that the later chapters take up. Circuits that respect the symmetries, locality and conservation laws of a problem carry an inductive bias that generic circuits lack [30, 31], which for many-body physics means architectures built around particle number, spin and spatial symmetry; good trainability and good predictions do not by themselves establish an advantage over tensor networks or classical networks, but they make the quantum model a reasonable one. Fixed quantum feature maps and quantum reservoirs let quantum dynamics generate features and train mainly a classical readout [32, 33], which removes the optimization overhead and leaves the measurement cost and strong classical baselines as the decisive questions. And a Hamiltonian-based model trained on quantum data, the quantum Boltzmann machine of Chapter 5 in its state-learning form, is a variational model whose data the classical competitor cannot even represent.

We have therefore written the book to teach variational circuits alongside their limits. Every chapter runs the quantum model beside its classical counterpart on the same data, says what the classical surrogate would be where one is known, and separates the claims that hold for classical data from those that hold for quantum data. The concluding Chapter 7 returns to the reassessment with the evidence of the intervening chapters in hand and sets out, in the form of a priority list, where we think the field is going. The reader who wants the short version now: the central question to ask of any proposed quantum learning model is which quantum resource reduces the number of experiments, improves the accuracy of an estimate, or lowers the total computation at a fixed accuracy, and the models that answer it are, in our judgment, mostly models of quantum systems.

1.7 Relation to the companion volume

The companion volume [1] is assumed throughout, and Table 1.1 says which of its chapters each part of this book rests on, so that a reader who comes to the present chapters without it knows what to read first. Three of its chapters are used on every page. Its chapter on the fundamentals of quantum computation supplies the qubits, the Bloch sphere, measurements, density matrices, entanglement, gates and circuits in the notation we use (Section 1.10).

Its chapter on mapping many-body Hamiltonians to qubits supplies the Pauli strings, the Jordan–Wigner transformation and the measurement of a Pauli sum, which is how every expectation value in this book is estimated. And its chapter on variational quantum algorithms supplies the variational principle on a circuit, the hardware-efficient and physically motivated ansätze, the parameter-shift rule, shot noise in the optimization loop, barren plateaus, ADAPT-VQE and a first treatment of QAOA, whose closing section already reads variational circuits as machine-learning models: Chapters 3 and 4 begin where that section ends. The many-body primer, with its four model Hamiltonians (Lipkin, pairing, Hubbard and Heisenberg), full configuration interaction, Hartree–Fock and coupled-cluster theory, is the source of the Hamiltonians that reappear here as targets, in particular the transverse-field Ising and Heisenberg chains of Chapters 4 and 5. The chapter on the quantum Fourier transform, phase estimation and the linear-system algorithm is the background to the speedup claims of Section 1.4, and the chapter on the rodeo algorithm, with its section on circuits, depth and the bottlenecks of quantum computing, is the background to Sections 1.5 and 1.6. We do not use its chapters on Hamiltonian simulation, error correction and lattice gauge theories, except in the perspectives of Chapter 7.

Two further volumes by the author stand behind the two halves of the word "quantum machine learning", and the reader who wants more of either than the companion volume gives will find it there. *Machine Learning and the Physical Sciences* [9] is the classical machine-learning text this book is paired with: its chapters on linear regression and optimization, on support-vector machines, on neural networks and their training, on neural networks for differential equations, on Boltzmann machines and the other generative models, and its part on reinforcement learning, develop in full the classical models that Chapters 2 to 6 here summarize before replacing one of their components by a quantum circuit, with the same programming conventions and the same notebooks. *Quantum Mechanics for Many-particle Systems: A Computational Perspective* [34] is the many-body text behind the companion volume's primer: second quantization, the model Hamiltonians, configuration interaction, mean-field and coupled-cluster theory, variational and diffusion Monte Carlo with the stochastic-reconfiguration optimization that Section 4.5 identifies with the natural gradient, Boltzmann machines and neural quantum states as wave-function ansätze, and its own chapters on phase estimation and the variational quantum eigensolver. Table 1.1 lists the three volumes together.

1.8 Loading classical data into qubits

Since the encoding circuit is the element that every later chapter builds on, we introduce the three standard choices here, with the program `data_encodings.py` of `BookPrograms/chapter1/`. We write $\boldsymbol{x} = (x_1, \ldots, x_d)$ for a sample and use the conventions of Section 1.10.

Basis encoding A sample that is a bit string, or a real number rounded to m bits, becomes the computational basis state with that label,

$$\boldsymbol{x} = (b_1, \ldots, b_m) \in \{0,1\}^m \longmapsto |b_1 b_2 \cdots b_m\rangle. \tag{1.2}$$

It costs m qubits per feature and a circuit of X gates, and its only use in learning is in models that process superpositions of many samples at once, which require the quantum random-access memory of Section 1.4.

Angle encoding Each feature becomes the angle of a rotation of its own qubit,

Table 1.1 The chapters of the companion volume *Quantum Computing for Many-body Physics* [1] on which this book depends, and where they are used here; the last two rows give the chapters of the author's machine-learning [9] and many-body [34] texts that supply the classical and the many-body background.

Companion volume	Used in this book
Fundamentals of quantum computation (qubits, measurements, density matrices, entanglement, gates, circuits)	everywhere; the notation of Section 1.10 and the simulator of `common/`
Quantum many-body physics primer (second quantization, Lipkin, pairing, Hubbard and Heisenberg models, FCI, Hartree–Fock, coupled cluster)	the Hamiltonians used as targets in Chapters 4 and 5; the Boltzmann machine as statistical mechanics (Chapter 5)
Mapping many-body Hamiltonians to qubits (Pauli strings, Jordan–Wigner, measuring a Pauli sum, unitary coupled cluster)	cost Hamiltonians of Chapter 3; Pauli-string Hamiltonians of Chapter 5; every expectation value
Quantum Fourier transform, phase estimation, the HHL algorithm	the speedup claims and their fine print, Section 1.4
Variational quantum algorithms (VQE, ansätze, measuring the energy, parameter shift, barren plateaus, ADAPT-VQE, QAOA)	the variational loop of Chapters 3 to 5; Chapter 3 extends its QAOA section, Chapter 4 its gradient and barren-plateau sections
The rodeo algorithm; circuits, depth and the bottlenecks of quantum computing	the challenges of Sections 1.5 and 1.6
Conclusion and outlook	the perspectives of Chapter 7
Machine Learning and the Physical Sciences [9]: optimization; support-vector machines; neural networks; neural networks for differential equations; Boltzmann machines; reinforcement learning	the classical models of Chapters 2, 4, 5 and 6 in full
Quantum Mechanics for Many-particle Systems [34]: second quantization and model Hamiltonians; FCI, mean field, coupled cluster; variational Monte Carlo and stochastic reconfiguration; Boltzmann machines and neural quantum states; QPE and VQE	the many-body background of the companion volume; Section 4.5; Chapter 5

$$|\phi(\boldsymbol{x})\rangle = \bigotimes_{q=1}^{d} R_y(x_q)|0\rangle = \bigotimes_{q=1}^{d} \left(\cos\frac{x_q}{2}|0\rangle + \sin\frac{x_q}{2}|1\rangle\right). \tag{1.3}$$

It costs d qubits and a circuit of depth one, and it is the encoding most experiments use. Two of its properties will recur. The state is a product state, so the overlap of two encoded samples factorizes,

$$|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2 = \prod_{q=1}^{d} \cos^2\frac{x_q - x'_q}{2}, \tag{1.4}$$

which is a perfectly good classical kernel; and the state is periodic in every feature with period 4π (and the probabilities with period 2π), so a feature whose range exceeds π is no longer encoded faithfully: two samples that differ by more than π become more similar again, and two that differ by 2π are encoded into the same state (right panel of Fig. 1.2). The data are therefore rescaled, feature by feature, into $[0,\pi]$ before they are encoded; all programs of the book do this with the `Scaler` class of `BookPrograms/common/datasets.py`. Repeating the rotations and interleaving them with entangling gates turns the angle encoding into the entangling feature maps of Chapter 2.

Amplitude encoding The sample itself, normalized and padded with zeros to a length 2^n, becomes the amplitude vector,

$$|\phi(\boldsymbol{x})\rangle = \frac{1}{\|\boldsymbol{x}\|}\sum_{k=1}^{d} x_k |k-1\rangle, \qquad n = \lceil \log_2 d \rceil . \tag{1.5}$$

It is the most compact encoding, $\log_2 d$ qubits for d features (Fig. 1.2), and the one assumed by the linear-algebra algorithms of Section 1.4; its kernel is $(\boldsymbol{x}^T\boldsymbol{x}')^2/(\|\boldsymbol{x}\|^2\|\boldsymbol{x}'\|^2)$, the squared cosine of the angle between the samples. Its cost is the circuit that prepares an arbitrary state of n qubits, which in general needs $\mathcal{O}(2^n) = \mathcal{O}(d)$ gates, so that the saving in qubits is paid for in depth.

The program prints the three encodings of one small vector:

```
import numpy as np
from data_encodings import basis_encoding, angle_encoding, amplitude_encoding

x = np.array([0.3, 1.2, 2.0, 0.7])
print(basis_encoding(5, 3))      # |101>
print(angle_encoding(x))         # product state on 4 qubits
print(amplitude_encoding(x))     # x/||x|| on 2 qubits
overlap = abs(angle_encoding(x).overlap(angle_encoding(x + 0.4))) ** 2
print(overlap, np.prod(np.cos(0.4 / 2) ** 2 * np.ones(4))) # Eq. (1—anglekernel)
```

The two numbers in the last line agree, 0.8512: the kernel of the angle encoding is the product of cosines of Eq. (1.4).

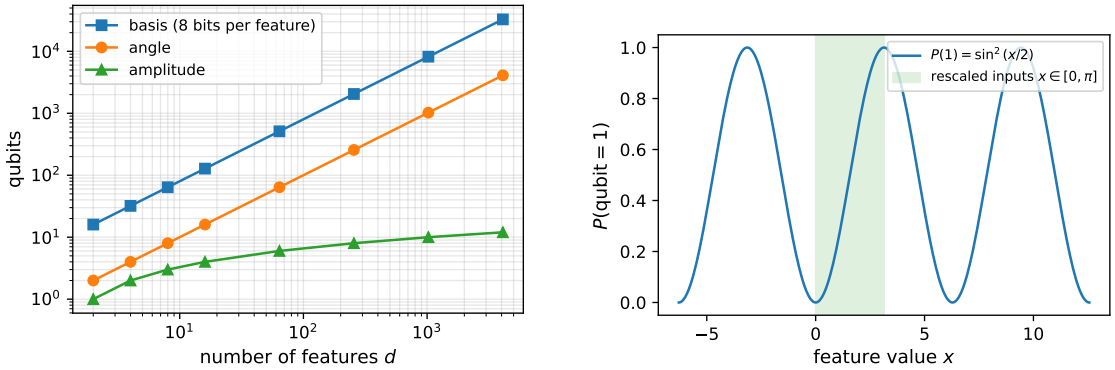


Fig. 1.2 Left: the number of qubits needed to encode d features in the basis encoding (eight bits per feature), the angle encoding and the amplitude encoding. Right: the probability of reading 1 from the qubit $R_y(x)|0\rangle$ as a function of the feature value x. The state is periodic, so features are rescaled into $[0,\pi]$ (shaded) before they are encoded.

1.9 The plan of the book

Table 1.2 lists the chapters. Each of Chapters 2 to 5 takes a classical model, derives it in enough detail that the reader could implement it, replaces one of its components by a quantum circuit, and runs both versions on the same data. They are meant to be read in order, since the kernel of Chapter 2 reappears as the unifying concept in Chapter 4, and the variational loop of Chapter 3 is the training loop of Chapters 4 and 5. Chapter 6 is a placeholder for the third kind of learning of Section 1.2, reinforcement learning and its quantum versions, which a later edition will fill; and Chapter 7 collects the perspectives of the book.

Table 1.2 The chapters of this book, the classical model each starts from, and the quantum element each adds.

Chapter	Classical model	Quantum element	Programs
2	support-vector machine	feature-map kernel $\lvert\langle\phi(\boldsymbol{x})\rvert\phi(\boldsymbol{x}')\rangle\rvert^2$	`chapter2/`
3	combinatorial optimization	alternating-operator ansatz	`chapter3/`
4	neural network	variational circuit, parameter-shift gradients	`chapter4/`
5	Boltzmann machine	transverse-field Hamiltonian, thermal state	`chapter5/`
6	reinforcement learning (placeholder)	policies and value functions on circuits	—
7	summary and perspectives		—

1.10 Conventions, notation and software

We use the conventions of the companion volume [1] throughout. Qubit 0 is the leftmost tensor factor and the most significant bit of a basis-state index, so that $|b_0 b_1 \cdots b_{n-1}\rangle$ has index $\sum_q b_q 2^{n-1-q}$. The Pauli matrices are X, Y, Z, the Hadamard gate H, and the rotations $R_x(\theta) = e^{-i\theta X/2}$, $R_y(\theta) = e^{-i\theta Y/2}$ and $R_z(\theta) = e^{-i\theta Z/2}$, so that $R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$. A sample is $\boldsymbol{x} \in \mathbb{R}^d$, a data set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ with labels $y_i = \pm 1$, a feature map $\boldsymbol{x} \mapsto |\phi(\boldsymbol{x})\rangle$ or, classically, $\boldsymbol{x} \mapsto \boldsymbol{\phi}(\boldsymbol{x})$, and a kernel $K(\boldsymbol{x}, \boldsymbol{x}')$. Trainable parameters are $\boldsymbol{\theta}$, Lagrange multipliers λ_i, the regularization constant C. Expectation values are written $\langle \hat{O} \rangle$ and estimated from S shots.

The programs are in `BookPrograms/`, one folder per chapter and a `common/` folder for the modules every chapter uses: the state-vector simulator and gate library (the same code as in the companion volume), the data sets, and the plotting conventions. They need only `numpy` and `matplotlib`. We simulate the circuits ourselves, with the mechanics visible, rather than through one of the quantum software frameworks; PennyLane [35] and Qiskit [36] appear only in clearly marked optional examples, which run when the package is installed and are skipped when it is not. Every number and every figure in the text is produced by a program named in the closing section of its chapter, and the notebooks in `Notebooks/chapterN/` are generated from the chapter text and those programs, so that the reader can execute the chapter.

1.11 Exercises

1. **The four quadrants.** Give one example from your own field of each of the four combinations of classical or quantum data with a classical or quantum computer, and say for each whether the data would have to be loaded by a circuit.
2. **The angle-encoding kernel.** (a) Derive Eq. (1.4) from Eq. (1.3). (b) Show that the kernel takes the same form for R_x in the place of R_y and also for the encoding $R_z(x_q)H|0\rangle$, and find what it becomes for $R_z(x_q)|0\rangle$ without the Hadamard. (c) Check the result with `data_encodings.py` for a random pair of samples. (d) Why does a classical computer evaluate this kernel in $\mathcal{O}(d)$ operations, and what does that imply about the use of the angle encoding on its own?
3. **Periodicity.** A feature takes values in $[0, 10]$ and is angle-encoded without rescaling. Find two values that are encoded into the same state, and two that are encoded into orthogonal states, and explain with the right panel of Fig. 1.2 why the rescaling into $[0, \pi]$ rather than $[0, 2\pi]$ is the usual choice.

4. **Amplitude encoding.** (a) Show that the kernel of Eq. (1.5) is $(\boldsymbol{x}^T\boldsymbol{x}')^2/(\|\boldsymbol{x}\|^2\|\boldsymbol{x}'\|^2)$ and that it is a Mercer kernel. (b) Write down the circuit that prepares $|\phi(\boldsymbol{x})\rangle$ for $d=2$ (one qubit) and for $d=4$ (two qubits, using one R_y and two controlled R_y rotations), and count the gates for general n.
5. **Reading the fine print.** Take the quantum support-vector machine of Rebentrost, Mohseni and Lloyd [15] and list the assumptions under which its running time is polylogarithmic in N. For each assumption, say whether it concerns the input, the kernel matrix, or the output, and whether a classical algorithm granted the same assumption could also profit from it [19].

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter1/`, which build on the shared modules of `BookPrograms/common/` (`statevector`, `gates`, `datasets`, `bookplot`) and are imported by the notebook `Notebooks/chapter1/chapter1.ipynb`.

`data_encodings.py` The basis, angle and amplitude encodings of Eqs. (1.2)–(1.5) on the shared state-vector simulator, the check of Eq. (1.4), and the two panels of Fig. 1.2. One second.

Chapter 2
Quantum Support Vector Machines

Abstract The support-vector machine looks at its data only through a kernel, and a quantum computer can supply one as the overlap of two encoded states. The chapter first derives the classical machine from the margin: the primal and dual problems, the Karush–Kuhn–Tucker conditions and the support vectors, the soft margin, the kernel trick and Mercer's theorem, and the sequential minimal optimization algorithm that solves the dual in a few lines of NumPy. It then defines quantum feature maps and the fidelity kernel, works out the angle, amplitude and entangling ZZ maps, shows how the kernel is estimated by the adjoint and SWAP tests with a finite number of shots, and assembles the quantum support-vector machine. The machine is run beside its classical counterpart on the two moons, two circles, XOR and Iris data, where it offers no advantage, and on a data set labeled by the ZZ map itself, where it is exact and the Gaussian kernel fails; the effects of shot noise and of the kernel bandwidth are measured, the feature map is trained by kernel-target alignment with parameter-shift gradients, and the chapter closes with the complexity of the near-term and fault-tolerant machines and with the observation that every hybrid quantum model is a kernel method.

2.1 Introduction

The support-vector machine (SVM) is the natural first model for a book on quantum machine learning, for a reason that will take the whole chapter to appreciate and can be stated in one sentence: it never looks at the data except through a similarity $K(\boldsymbol{x},\boldsymbol{x}')$ between pairs of samples, and a quantum computer can supply such a similarity as the overlap of two encoded states. Everything else, the notion of a margin, the convex optimization problem that finds it, the sparsity of the solution in terms of *support vectors*, stays exactly as Boser, Guyon, Vapnik and Cortes left it [37, 38]. The quantum support-vector machine (QSVM) is therefore not a new algorithm but a new kernel, and the questions it raises, whether the kernel can be estimated accurately enough, whether it fits the data better than a Gaussian, whether it could be computed classically after all, are questions about that kernel. The quantum side of the chapter needs only the fundamentals of the companion volume [1]: the overlap of two states, the measurement of a register in the computational basis, and the density matrix of a prepared state (its chapter on the fundamentals of quantum computation), together with its chapter on mapping Hamiltonians to qubits for the Pauli-string gates of the feature maps; the reader who has those in hand can read this chapter without any other preparation.

The chapter is in two halves. The first, Sections 2.2 to 2.6, derives the classical machine from scratch: the hyperplane and the margin, the Lagrangian and its dual, the Karush–Kuhn–Tucker conditions that single out the support vectors, the soft margin that tolerates over-

lapping classes, the kernel trick, Mercer's theorem, and the sequential minimal optimization algorithm that solves the dual problem in a few dozen lines of NumPy; the derivation is the one of the chapter on support-vector machines of the author's machine-learning text [9], which also treats the regression and multiclass versions not needed here. Readers who know this material can go directly to Section 2.7, but should at least note the notation of Eqs. (2.10) and (2.13), which is used throughout. The second half replaces the kernel by a quantum one: Section 2.7 defines quantum feature maps and the fidelity kernel and works out the angle, amplitude and entangling maps of Havlíček *et al.* [14]; Section 2.8 shows how the kernel is measured, with the adjoint test and the SWAP test, and what a finite number of shots does to it; Section 2.9 assembles the machine and runs it on four data sets, including one that was constructed so that the quantum kernel separates it and classical kernels do not; Section 2.10 trains the feature map itself by kernel-target alignment; and Section 2.11 weighs the result against the complexity of the classical machine and the fault-tolerant proposal of Rebentrost, Mohseni and Lloyd [15]. The programs are in `BookPrograms/chapter2/`: a `Kernel` hierarchy, an `SVM` class with its own solver, the feature maps and a `QuantumKernel` that estimates overlaps with or without shot noise, a `QSVM` class, and a `KernelAlignment` class. Every number quoted below is printed by one of them.

2.2 Hyperplanes and the margin

We have N training samples $\boldsymbol{x}_i \in \mathbb{R}^p$ with p features each, and a label $y_i \in \{-1,+1\}$ for each of them. A *hyperplane* in $\mathbb{R}^p$ is an affine subspace of dimension $p-1$, a line in the plane, a plane in space, and it is the set of points satisfying

$$f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b = 0, \tag{2.1}$$

for a vector $\boldsymbol{w}$ normal to the plane and an intercept b. In two dimensions this is $b + w_1x_1 + w_2x_2 = 0$. Two points $\boldsymbol{x}_1$, $\boldsymbol{x}_2$ on the plane satisfy $\boldsymbol{w}^T(\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$, which is what "normal" means, and the signed distance of an arbitrary point $\boldsymbol{x}$ from the plane is

$$\delta(\boldsymbol{x}) = \frac{1}{\|\boldsymbol{w}\|}\left(\boldsymbol{w}^T\boldsymbol{x} + b\right) = \frac{f(\boldsymbol{x})}{\|\boldsymbol{w}\|}, \tag{2.2}$$

positive on the side $\boldsymbol{w}$ points to and negative on the other. A hyperplane therefore defines a classifier, $\hat{y}(\boldsymbol{x}) = \operatorname{sign} f(\boldsymbol{x})$, and the hyperplane classifies the training set correctly if

$$y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i + b\right) > 0 \qquad \forall\, i = 1,\ldots,N. \tag{2.3}$$

If such a plane exists the classes are *linearly separable*.

A first attempt The most direct way of finding $\boldsymbol{w}$ and b is to penalize the misclassified points. With M the set of indices violating Eq. (2.3) one minimizes

$$C(\boldsymbol{w}, b) = -\sum_{i\in M} y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i + b\right), \tag{2.4}$$

whose derivatives are $\partial C/\partial b = -\sum_{i\in M} y_i$ and $\partial C/\partial \boldsymbol{w} = -\sum_{i\in M} y_i\boldsymbol{x}_i$, and updates $b \leftarrow b - \eta\,\partial C/\partial b$, $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\,\partial C/\partial \boldsymbol{w}$ with a learning rate η take the familiar form of the perceptron rule and of gradient descent in logistic regression. The approach is easy to code and has two defects. When the classes are separable there are infinitely many separating planes, Eq. (2.4) is zero on all of them, and which one the iteration stops at depends on the starting point and the

order of the data; and when the classes are not separable the iteration need not converge at all. Both defects are cured by asking for a particular plane: the one farthest from the data.

The margin Suppose the classes are separable. For any separating plane the smallest signed distance of a training point from it, $\min_i y_i \delta(\boldsymbol{x}_i)$, is positive; it is called the *margin* M of the plane, and we want the plane with the largest margin. That is the problem

$$\max_{\boldsymbol{w},b} M \qquad \text{subject to} \qquad \frac{1}{\|\boldsymbol{w}\|} y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b\right) \geq M \quad \forall i. \tag{2.5}$$

The constraint is unchanged if $\boldsymbol{w}$ and b are multiplied by the same positive constant, so we may fix the scale by $\|\boldsymbol{w}\| = 1/M$. Maximizing M is then minimizing $\|\boldsymbol{w}\|$, or more conveniently $\frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}$, and the problem becomes

$$\min_{\boldsymbol{w},b} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} \qquad \text{subject to} \qquad y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b\right) \geq 1 \quad \forall i. \tag{2.6}$$

This is the *primal* form of the hard-margin support-vector machine: a quadratic objective with N linear inequality constraints, a convex problem with a unique solution. The two planes $\boldsymbol{w}^T\boldsymbol{x} + b = \pm 1$ are the margin boundaries, at distance $1/\|\boldsymbol{w}\| = M$ on either side of the decision plane, and the training points that lie on them will turn out to be the only ones that matter.

2.3 Lagrange multipliers and the dual problem

A reminder on constrained extrema A function $f(x,y,z)$ has an extremum where $df = f_x dx + f_y dy + f_z dz = 0$, and if the three variables are independent this requires $f_x = f_y = f_z = 0$. If the variables are tied by a constraint $\phi(x,y,z) = 0$ they are no longer independent: only two of the differentials, say dx and dy, can be chosen freely, and dz is fixed by $d\phi = \phi_x dx + \phi_y dy + \phi_z dz = 0$. One could eliminate z and work with two variables, but it is usually more convenient to add a multiple $\lambda\, d\phi$ of the constraint to df,

$$df + \lambda\, d\phi = (f_x + \lambda\phi_x)\, dx + (f_y + \lambda\phi_y)\, dy + (f_z + \lambda\phi_z)\, dz,$$

to choose λ so that the coefficient of the dependent differential vanishes, $f_z + \lambda\phi_z = 0$, and to note that the two remaining differentials are arbitrary, so that their coefficients must vanish too. The extremum therefore satisfies $\partial f/\partial x_i + \lambda\, \partial\phi/\partial x_i = 0$ for every variable, together with $\phi = 0$: four equations for x, y, z, λ. With several constraints $\phi_k = 0$ one multiplier λ_k per constraint appears, $\partial f/\partial x_i + \sum_k \lambda_k\, \partial\phi_k/\partial x_i = 0$. The multipliers are Lagrange's "undetermined" multipliers; they are usually not wanted, but in the problem at hand they will be the quantities of interest. For *inequality* constraints $\phi_k \geq 0$ the same construction holds with the extra conditions that $\lambda_k \geq 0$ and that $\lambda_k\phi_k = 0$ at the solution: a constraint that is not active ($\phi_k > 0$) has multiplier zero, and a constraint with nonzero multiplier is active ($\phi_k = 0$). These are the Karush–Kuhn–Tucker (KKT) conditions, and Boyd and Vandenberghe [39] is the reference for everything we use about them.

The Lagrangian of the margin problem Applied to Eq. (2.6), with one multiplier $\lambda_i \geq 0$ per sample, the construction gives the Lagrangian

$$\mathscr{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_{i=1}^{N} \lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b\right) - 1 \right], \tag{2.7}$$

to be minimized over $\boldsymbol{w}$ and b and maximized over the multipliers. The derivatives with respect to b and $\boldsymbol{w}$ give

$$\frac{\partial \mathscr{L}}{\partial b} = -\sum_i \lambda_i y_i = 0, \tag{2.8}$$

$$\frac{\partial \mathscr{L}}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{x}_i = 0 \qquad \Longrightarrow \qquad \boldsymbol{w} = \sum_{i=1}^{N} \lambda_i y_i \boldsymbol{x}_i. \tag{2.9}$$

The second equation is the key structural fact about the machine: the normal vector is a linear combination of the training samples. Inserting both into Eq. (2.7), the terms linear in b vanish by Eq. (2.8), and the two quadratic terms combine into

$$\mathscr{L}_D(\boldsymbol{\lambda}) = \sum_{i=1}^{N} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{N} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j, \tag{2.10}$$

to be *maximized* subject to $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$. This is the *dual* problem. It has as many variables as there are samples rather than features, it is again a convex quadratic program, and, decisively for what follows, the samples enter it only through their inner products $\boldsymbol{x}_i^T \boldsymbol{x}_j$.

Support vectors The KKT conditions of the problem read

$$\lambda_i \left[y_i \left(\boldsymbol{w}^T \boldsymbol{x}_i + b \right) - 1 \right] = 0 \qquad \forall\, i. \tag{2.11}$$

Either $\lambda_i > 0$, and then $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$: the sample lies exactly on the margin boundary. Or $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 1$, the sample is strictly inside its half space, and then $\lambda_i = 0$. The samples with $\lambda_i > 0$ are the *support vectors*; they are the points closest to the decision plane, they alone define it through Eq. (2.9), and removing any other point from the training set changes nothing. From any support vector the intercept follows, $b = y_i - \boldsymbol{w}^T \boldsymbol{x}_i$ (using $1/y_i = y_i$), and in practice one averages over the set $\mathscr{S}$ of the N_s support vectors to reduce rounding errors,

$$b = \frac{1}{N_s} \sum_{j \in \mathscr{S}} \left(y_j - \sum_{i=1}^{N} \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_j \right). \tag{2.12}$$

The classifier for a new sample is then

$$\hat{y}(\boldsymbol{x}) = \operatorname{sign} \left(\sum_{i=1}^{N} \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x} + b \right), \tag{2.13}$$

with only the support vectors contributing to the sum.

The matrix form With the vectors $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_N)$ and $\boldsymbol{y} = (y_1, \ldots, y_N)$ and the matrix $P_{ij} = y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j$, maximizing Eq. (2.10) is minimizing

$$\frac{1}{2} \boldsymbol{\lambda}^T \boldsymbol{P} \boldsymbol{\lambda} - \mathbf{1}^T \boldsymbol{\lambda} \qquad \text{subject to} \qquad \boldsymbol{y}^T \boldsymbol{\lambda} = 0, \quad \lambda_i \geq 0, \tag{2.14}$$

where $\boldsymbol{P}$, being the Gram matrix of the vectors $y_i \boldsymbol{x}_i$, is positive semidefinite. We return to how this is solved in Section 2.6; for the moment we let the program `svm.py` do it.

A first example Figure 2.1 shows the hard-margin machine trained on two Gaussian clouds of twenty points each. The solver finds $\boldsymbol{w} = (1.2494, -0.0909)$, $b = -0.0248$ and a margin $1/\|\boldsymbol{w}\| = 0.7983$, with three support vectors, two of one class and one of the other, which sit exactly on the dashed margin lines; every other point has $\lambda_i = 0$ and $y_i f(\boldsymbol{x}_i) > 1$, as the program checks. Fifty-seven two-variable updates of the algorithm of Section 2.6 were needed.

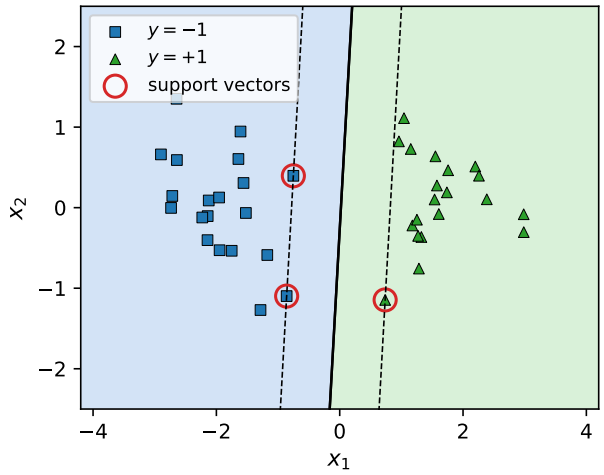


Fig. 2.1 The hard-margin support-vector machine on two separable Gaussian clouds. The solid line is the decision plane $\boldsymbol{w}^T\boldsymbol{x}+b=0$, the dashed lines are the margin boundaries $\boldsymbol{w}^T\boldsymbol{x}+b=\pm 1$, and the circled points are the three support vectors, the only samples with $\lambda_i>0$. From `svm.py`.

2.4 The soft margin

When the classes overlap, no plane satisfies all the constraints of Eq. (2.6) and the dual problem is unbounded. The remedy of Cortes and Vapnik [38] is to allow each sample to violate its margin by a *slack* $\xi_i \geq 0$ and to pay for the total violation:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_{i=1}^{N}\xi_i \qquad \text{subject to} \qquad y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i+b\right) \geq 1-\xi_i, \quad \xi_i \geq 0. \tag{2.15}$$

A sample with $0<\xi_i<1$ is on the correct side of the decision plane but inside the margin, a sample with $\xi_i>1$ is misclassified, and $\sum_i \xi_i$ bounds the number of misclassifications. The constant $C>0$ sets the price of a violation: a large C approaches the hard margin, a small C accepts violations in exchange for a wider margin, and C is the regularization parameter of the method, to be chosen by cross-validation. The Lagrangian acquires a second set of multipliers $\gamma_i \geq 0$ for the constraints $\xi_i \geq 0$,

$$\mathscr{L} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \xi_i - \sum_i \lambda_i\left[y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i+b\right)-1+\xi_i\right] - \sum_i \gamma_i\xi_i, \tag{2.16}$$

and the stationarity conditions are Eqs. (2.8) and (2.9) as before, plus one from $\partial\mathscr{L}/\partial\xi_i=0$,

$$C-\lambda_i-\gamma_i=0 \qquad \Longrightarrow \qquad 0\leq\lambda_i\leq C. \tag{2.17}$$

Inserting them, the slacks and the γ_i drop out, and the dual is the same function $\mathscr{L}_D(\boldsymbol{\lambda})$ of Eq. (2.10), now maximized in the box $0\leq\lambda_i\leq C$ with $\sum_i \lambda_i y_i = 0$. The KKT conditions become

$$\lambda_i\left[y_i\left(\boldsymbol{w}^T\boldsymbol{x}_i+b\right)-1+\xi_i\right]=0, \qquad \gamma_i\xi_i=(C-\lambda_i)\xi_i=0, \tag{2.18}$$

and they sort the samples into three groups: $\lambda_i=0$, correctly classified outside the margin; $0<\lambda_i<C$, on the margin boundary with $\xi_i=0$; and $\lambda_i=C$, inside the margin or misclassified with $\xi_i>0$. The intercept of Eq. (2.12) must now be averaged over the second group only, which is what `svm.py` does. Figure 2.2 shows the effect of C on two overlapping clouds: for

$C = 0.05$ the margin is 1.09, 28 of the 60 points are support vectors and 26 of them violate the margin; for $C = 1$ the margin is 0.56 with 11 support vectors and 9 violations; for $C = 100$ it is 0.25 with 7 and 6. The training accuracy is 0.95, 0.97 and 0.97: a narrower margin fits the training set no better, and the wide-margin solution is the one that will generalize.

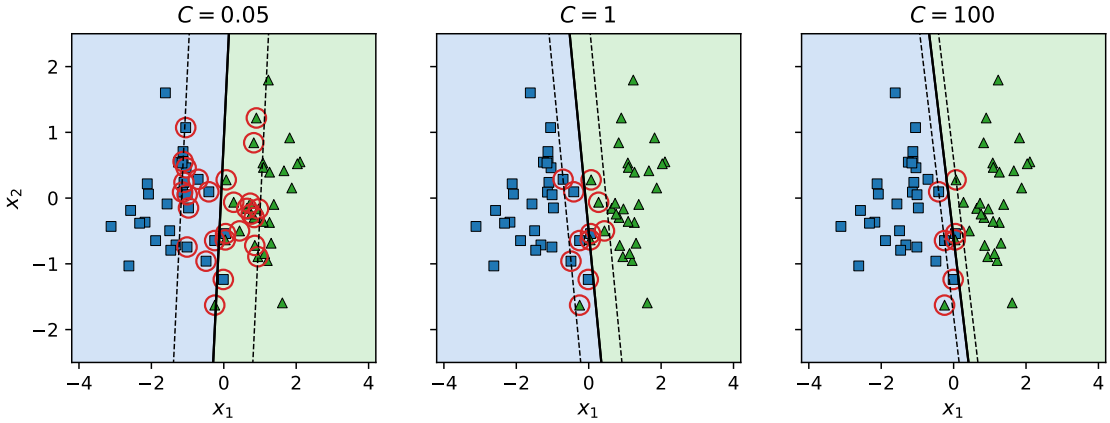


Fig. 2.2 The soft-margin machine on two overlapping clouds for three values of the regularization constant C. The circled points are the support vectors, $\lambda_i > 0$; as C grows the margin narrows and the solution is carried by fewer points. From `svm.py`.

2.5 Kernels

Non-linear feature maps Everything so far finds a *linear* boundary in the space of the features. Most data are not linearly separable there, and the classical way out, in regression as in classification, is to map the features into a larger space, $\boldsymbol{x} \mapsto \boldsymbol{z} = \boldsymbol{\phi}(\boldsymbol{x})$, with polynomials, splines or any other basis functions, and to fit the linear model in that space. The top row of Fig. 2.3 shows the simplest case: nine points on a line with the middle five in one class, which no threshold separates, become separable by a line after the map $x \mapsto (x, x^2)$. In two dimensions the quadratic map is

$$\boldsymbol{\phi}(x_1, x_2) = \left(x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2\right), \tag{2.19}$$

and the dual problem of Eq. (2.10) in the new space is the same problem with $\boldsymbol{x}_i^T \boldsymbol{x}_j$ replaced by $\boldsymbol{z}_i^T \boldsymbol{z}_j = \boldsymbol{\phi}(\boldsymbol{x}_i)^T \boldsymbol{\phi}(\boldsymbol{x}_j)$.

The kernel trick Multiply out the inner product of Eq. (2.19):

$$\boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\phi}(\boldsymbol{x}') = x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1x_2x_1'x_2' = \left(\boldsymbol{x}^T \boldsymbol{x}'\right)^2. \tag{2.20}$$

The inner product in the three-dimensional feature space is a function of the inner product in the original space, and can be evaluated without ever forming $\boldsymbol{\phi}$. This is the *kernel trick* [37]: define the *kernel* $K(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\phi}(\boldsymbol{x}')$, and write the dual problem and the classifier in terms of it,

$$\max_{\boldsymbol{\lambda}} \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j) \quad \text{s.t.} \quad \sum_i \lambda_i y_i = 0,\ 0 \le \lambda_i \le C, \tag{2.21}$$

$$\hat{y}(\boldsymbol{x}) = \operatorname{sign}\left(\sum_{i=1}^{N} \lambda_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b\right). \tag{2.22}$$

Neither the training nor the prediction ever touches $\boldsymbol{\phi}$. The feature space can therefore be as large as one likes, including infinite-dimensional, as long as the kernel is cheap, and this is the observation on which the quantum machine rests: the feature space will be the Hilbert space of n qubits, of dimension 2^n, and the kernel will be measured.

Common kernels Apart from the linear kernel $K = \boldsymbol{x}^T\boldsymbol{x}'$ the standard choices are

$$K(\boldsymbol{x}, \boldsymbol{x}') = \left(\gamma \boldsymbol{x}^T\boldsymbol{x}' + r\right)^d \qquad \text{(polynomial of degree } d\text{)}, \tag{2.23}$$

$$K(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\gamma \|\boldsymbol{x} - \boldsymbol{x}'\|^2\right) \qquad \text{(Gaussian radial basis function, RBF)}, \tag{2.24}$$

and the sigmoid kernel $K = \tanh(\gamma \boldsymbol{x}^T\boldsymbol{x}' + r)$. The RBF kernel is the most used; its feature space is infinite-dimensional, its parameter γ sets the width $1/\sqrt{\gamma}$ over which two samples are considered similar, and the bottom row of Fig. 2.3 gives the picture behind it: a sample is described by its similarity to a few *landmarks*, and in those coordinates the nine points of the left panel become linearly separable again.

Mercer's theorem Which functions K are inner products in *some* feature space? Mercer's theorem [40] answers: a symmetric, continuous K is the inner product $\boldsymbol{\phi}(\boldsymbol{x})^T\boldsymbol{\phi}(\boldsymbol{x}')$ of some map $\boldsymbol{\phi}$, in general into an infinite-dimensional space, if and only if for every finite set of points the Gram matrix

$$K_{ij} = K(\boldsymbol{x}_i, \boldsymbol{x}_j) \qquad \text{is positive semidefinite.} \tag{2.25}$$

One may then use K without knowing $\boldsymbol{\phi}$, and the dual problem, whose matrix $P_{ij} = y_i y_j K_{ij}$ inherits the positive semidefiniteness, stays convex. The program `kernels.py` checks the condition numerically on the hundred points of the two-moons data below: the linear, polynomial and RBF kernels pass, and the sigmoid kernel fails for both parameter settings tried, with negative eigenvalues in its Gram matrix. The sigmoid kernel is nevertheless used in practice; the quantum kernels of Section 2.7 will turn out to satisfy Mercer's condition automatically, and the *estimated* ones of Section 2.8 to violate it.

The two moons Figure 2.4 applies four kernels to the standard "two moons" data, two interleaved half circles with Gaussian noise, with $C = 5$ and the features standardized to zero mean and unit variance. The linear kernel reaches a training accuracy of 0.88 with 34 support vectors; the cubic polynomial 0.98 with 12; the RBF kernel 0.96 with 20 for $\gamma = 0.5$ and 0.98 with 43 for $\gamma = 5$. The last panel shows what a too-narrow kernel does: the boundary wraps around individual points, nearly half the data become support vectors, and the model has begun to memorize the noise.

2.6 Solving the dual problem

Quadratic programming The dual of Eq. (2.21) is an instance of the standard quadratic program

$$\min_{\boldsymbol{\lambda}} \ \frac{1}{2}\boldsymbol{\lambda}^T\boldsymbol{P}\boldsymbol{\lambda} + \boldsymbol{q}^T\boldsymbol{\lambda} \qquad \text{subject to} \qquad \boldsymbol{G}\boldsymbol{\lambda} \preceq \boldsymbol{h}, \quad \boldsymbol{A}\boldsymbol{\lambda} = \boldsymbol{f}, \tag{2.26}$$

with $P_{ij} = y_i y_j K(\boldsymbol{x}_i, \boldsymbol{x}_j)$, $\boldsymbol{q} = -\mathbf{1}$, the equality constraint $\boldsymbol{A} = \boldsymbol{y}^T$, $\boldsymbol{f} = 0$, and the box $0 \le \lambda_i \le C$ written as the $2N$ inequalities $-\lambda_i \le 0$ and $\lambda_i \le C$, which fill the rows of $\boldsymbol{G}$ and $\boldsymbol{h}$. Since $\boldsymbol{P}$ is positive semidefinite the problem is convex, and any of the interior-point solvers of convex optimization [39] solves it; the method `SVM.qp_matrices` of `svm.py` builds the six objects, and

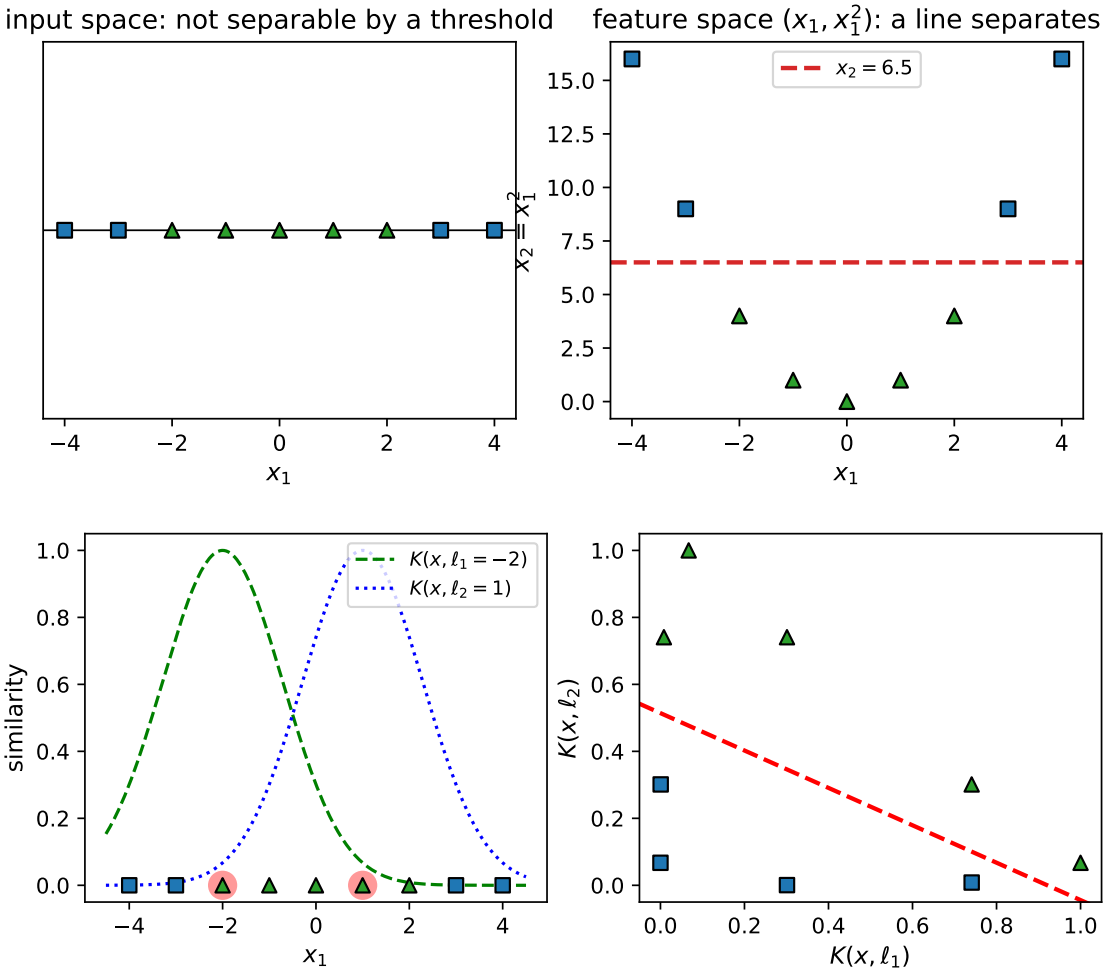


Fig. 2.3 Top: nine samples on a line, not separable by a threshold (left), become linearly separable after the map $x \mapsto (x, x^2)$ (right). Bottom: the same samples described by their Gaussian similarity $\exp(-\gamma(x-\ell)^2)$ to two landmarks $\ell_1 = -2$, $\ell_2 = 1$ (left), and in the coordinates $(K(x,\ell_1), K(x,\ell_2))$, where a line separates them (right). From `kernels.py`.

`solve_with_cvxopt` hands them to the CVXOPT library when it is installed, for comparison with the solver described next. Gradient methods on Eq. (2.21) also work but converge slowly, because the equality constraint couples all the multipliers.

Sequential minimal optimization The solver we use throughout is Platt's sequential minimal optimization (SMO) [41], which needs nothing beyond NumPy and exposes the structure of the problem. The idea is that the constraint $\sum_i \lambda_i y_i = 0$ forbids changing one multiplier alone, but allows changing two, λ_i and λ_j, along the line $\lambda_i y_i + \lambda_j y_j = \text{const}$, and that along that line the objective is a parabola whose maximum is known in closed form. With the errors $E_i = f(\boldsymbol{x}_i) - y_i$ of the current model and the curvature $\eta = K_{ii} + K_{jj} - 2K_{ij} \geq 0$, the unconstrained optimum along the line is

$$\lambda_j^{\text{new}} = \lambda_j + \frac{y_j\,(E_i - E_j)}{\eta}, \tag{2.27}$$

which is then clipped to the segment $[L, H]$ of the line that stays inside the box $0 \leq \lambda \leq C$ ($L = \max(0, \lambda_j - \lambda_i)$, $H = \min(C, C + \lambda_j - \lambda_i)$ when $y_i \neq y_j$, and $L = \max(0, \lambda_i + \lambda_j - C)$, $H = \min(C, \lambda_i + \lambda_j)$ when $y_i = y_j$), and λ_i is moved by $y_i y_j$ times the opposite amount. The intercept is updated so

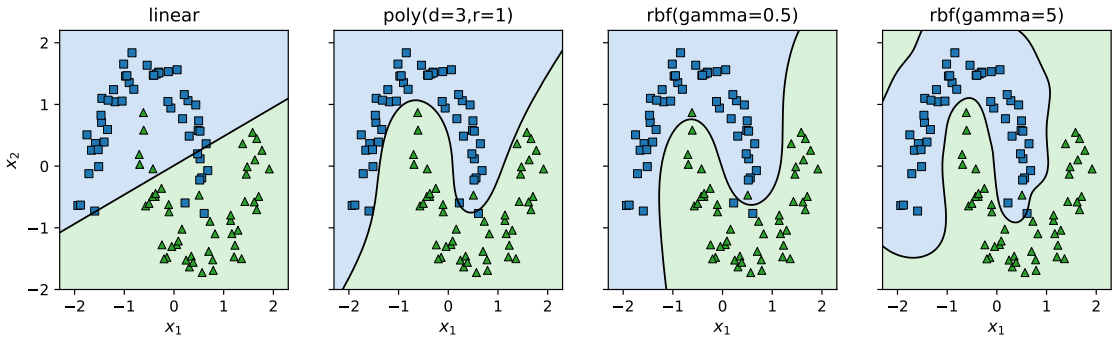


Fig. 2.4 The two-moons data with a linear, a cubic polynomial and two Gaussian kernels, $C = 5$, features standardized. Shading shows the predicted class and the black curve the decision boundary. From `svm.py`.

that the KKT condition of the moved multiplier holds, and the pair is chosen by scanning the samples for a KKT violation and pairing the violator with the sample of the largest $|E_i - E_j|$. The core of the update in `svm.py`, with `Q` the matrix $y_i y_j K_{ij}$, is

```
import numpy as np
from svm import SVM
# the two-multiplier step of SMO, Eq. (2-smo), as svm.py implements it
def take_step(K, y, lam, b, i, j, C):
    E = (y[:, None] * y[None, :] * K) @ lam * y - y + b # E_i = f(x_i) - y_i
    eta = K[i, i] + K[j, j] - 2 * K[i, j]
    if y[i] != y[j]:
        L, H = max(0.0, lam[j] - lam[i]), min(C, C + lam[j] - lam[i])
    else:
        L, H = max(0.0, lam[i] + lam[j] - C), min(C, lam[i] + lam[j])
    lam_j = np.clip(lam[j] + y[j] * (E[i] - E[j]) / eta, L, H)
    lam_i = lam[i] + y[i] * y[j] * (lam[j] - lam_j)
    return lam_i, lam_j
```

Each step is $\mathcal{O}(N)$ once the Gram matrix is stored, the number of steps is typically a small multiple of N (57 for the three-support-vector problem of Fig. 2.1, a few hundred for the moons), and the iteration stops when no sample violates its KKT condition beyond a tolerance. The class `SVM` accepts any kernel object of `kernels.py` or, through its argument `K`, a precomputed Gram matrix; the latter is how the quantum kernel will enter. On the RBF problem of Fig. 2.4 its dual objective agrees with that of the LIBSVM solver inside scikit-learn to six digits (`svm.py` prints both when scikit-learn is installed).

2.7 Quantum feature maps and quantum kernels

The definition A classical feature map sends a sample to a vector in a large space. A *quantum feature map* sends it to the state of a register of n qubits, prepared from $|0\rangle^{\otimes n}$ by a circuit whose gates depend on the sample,

$$\boldsymbol{x} \in \mathbb{R}^p \;\longmapsto\; |\phi(\boldsymbol{x})\rangle = U(\boldsymbol{x})|0\rangle^{\otimes n} \in \mathbb{C}^{2^n}, \tag{2.28}$$

and the *quantum kernel* is the overlap of two such states,

$$K(\boldsymbol{x},\boldsymbol{x}') = \big|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle\big|^2 = \big|\langle 0|U^\dagger(\boldsymbol{x})\,U(\boldsymbol{x}')|0\rangle\big|^2. \tag{2.29}$$

The squared modulus rather than the amplitude itself is used because the amplitude $\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle$ is complex and not directly measurable, while its squared modulus is a probability, the *fidelity* of the two states, and Section 2.8 shows how it is measured. Equation (2.29) is the kernel of Havlíček *et al.* [14] and of Schuld and Killoran [42], who introduced the view of an encoding circuit as a feature map, and it is a Mercer kernel without any further condition: writing $\rho(\boldsymbol{x}) = |\phi(\boldsymbol{x})\rangle\langle\phi(\boldsymbol{x})|$ for the density matrix of the encoded state,

$$K(\boldsymbol{x},\boldsymbol{x}') = \operatorname{Tr}\left[\rho(\boldsymbol{x})\,\rho(\boldsymbol{x}')\right], \tag{2.30}$$

which is the Hilbert–Schmidt inner product of two vectors in the real vector space of Hermitian $2^n \times 2^n$ matrices. The feature map whose existence Mercer's theorem guarantees is therefore known explicitly: it is $\boldsymbol{x} \mapsto \rho(\boldsymbol{x})$, into a space of dimension 4^n, and the kernel matrix of any data set is positive semidefinite. Equation (2.30) also makes precise the sense in which the quantum machine works in an exponentially large feature space, and it will return in Section 2.11 as the reason why every quantum model of the type of Chapter 1 is a kernel method.

Angle encoding The angle encoding of Chapter 1,

$$U(\boldsymbol{x}) = \bigotimes_{q=1}^{p} R_y(x_q), \qquad n = p, \tag{2.31}$$

with one qubit per feature, gives the kernel of Eq. (1.4),

$$K(\boldsymbol{x},\boldsymbol{x}') = \prod_{q=1}^{p} \cos^2\frac{x_q - x_q'}{2}, \tag{2.32}$$

a product of one-dimensional kernels, each a smooth periodic function of the difference of one feature. It is a stationary kernel like the RBF, with a fixed width, and a classical computer evaluates it in $\mathcal{O}(p)$ operations. The same holds for R_x rotations and for the encoding $R_z(x_q)H$, whose kernel is again $\prod_q \cos^2((x_q - x_q')/2)$ (Exercise 2). The program `feature_maps.py` checks Eq. (2.32) for a random pair of samples; the value it prints, 0.7019, agrees with the product of cosines.

Amplitude encoding With the sample as the amplitude vector, Eq. (1.5),

$$K(\boldsymbol{x},\boldsymbol{x}') = \frac{\left(\boldsymbol{x}^T\boldsymbol{x}'\right)^2}{\|\boldsymbol{x}\|^2\|\boldsymbol{x}'\|^2}, \tag{2.33}$$

the square of the cosine of the angle between the samples, which is the quadratic polynomial kernel of Eq. (2.20) on normalized data. The compact encoding, $\lceil \log_2 p \rceil$ qubits, buys nothing here: the kernel is classical, and the state-preparation circuit is expensive.

Entangling feature maps Neither of the two maps above can give a quantum advantage, because their kernels are evaluated classically in polynomial time; the remark of Havlíček *et al.* that a feature map that is easy to simulate classically makes the quantum computer unnecessary [14] is the starting point for everything that follows. Their proposal, the *second-order Pauli-Z feature map* or ZZ feature map, is

$$U(\boldsymbol{x}) = \left[U_\Phi(\boldsymbol{x})\,H^{\otimes n}\right]^r, \qquad U_\Phi(\boldsymbol{x}) = \exp\Big(i\sum_q \phi_q(\boldsymbol{x})\,Z_q + i\sum_{q<q'} \phi_{qq'}(\boldsymbol{x})\,Z_qZ_{q'}\Big), \tag{2.34}$$

with one qubit per feature, r repetitions (two in the original work), and the functions

$$\phi_q(\boldsymbol{x}) = x_q, \qquad \phi_{qq'}(\boldsymbol{x}) = (\pi - x_q)(\pi - x_{q'}), \tag{2.35}$$

with the pairs (q, q') running over nearest neighbors ("linear" entanglement) or over all pairs ("full"). The Hadamards put every qubit into $(|0\rangle + |1\rangle)/\sqrt{2}$, the single-$Z$ terms write the features into the phases, and the $Z_q Z_{q'}$ terms write products of features into the phases of the two-qubit states, which is where the entanglement and the non-linearity come from. All the terms in the exponent commute, being diagonal, so the exponential is a product of one-qubit phase rotations $e^{i\phi_q Z_q} = R_z(-2\phi_q)$ and two-qubit rotations $e^{i\phi_{qq'} Z_q Z_{q'}}$, each of which is a CNOT, an $R_z(-2\phi_{qq'})$ on the target, and a second CNOT, Fig. 2.5. The depth is $\mathcal{O}(rn)$ for linear and $\mathcal{O}(rn^2)$ for full entanglement. The general form, with $Z_q Z_{q'}$ replaced by arbitrary Pauli strings and $\phi_{qq'}$ by other functions, is the Pauli feature map; the ZZ map is the one for which classical hardness is argued, through its relation to sampling from the output of such circuits, and the one every quantum machine-learning library implements.

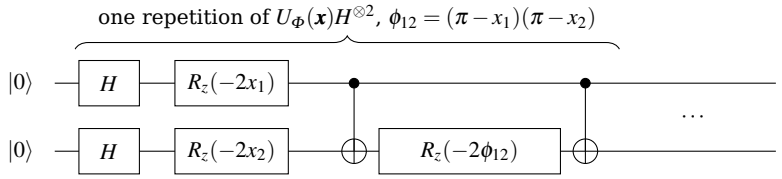


Fig. 2.5 One repetition of the ZZ feature map of Eq. (2.34) on two qubits. The two-qubit rotation $e^{i\phi_{12} Z_1 Z_2}$ is realized as CNOT, $R_z(-2\phi_{12})$ on the target, CNOT. The circuit is repeated r times, and $U^\dagger(\boldsymbol{x}')$ is the same circuit read backwards with the angles of $\boldsymbol{x}'$ and their signs reversed.

The program implements the map as the class `ZZFeatureMap` and checks it, on two qubits, against the matrix exponential of the diagonal generator; the two agree up to a global phase. Its argument `alpha` multiplies the features before they enter Eq. (2.35), a *bandwidth* that plays the role of γ in the RBF kernel and whose importance was pointed out by Shaydulin and Wild [43]: a kernel whose phases vary too fast with the features becomes nearly diagonal, every sample is similar only to itself, and the machine memorizes. We shall see this happen in Table 2.1.

Where the kernel lives It is worth pausing on what has and has not changed relative to Section 2.5. The feature space is explicit, $\mathbb{C}^{2^n}$ or, by Eq. (2.30), the 4^n-dimensional space of density matrices, and the kernel is its inner product; a classical computer could in principle construct the same feature vectors by simulating the circuit, at a cost exponential in n, which is why the quantum computer is used instead. For a product map such as Eq. (2.31) the simulation costs $\mathcal{O}(n)$ and the quantum computer is not needed; for the ZZ map with a dozen or more qubits and full entanglement no efficient simulation is known. The machine itself, Eqs. (2.21) and (2.22), is the classical one of Sections 2.3–2.6 and is trained on a classical computer.

2.8 Estimating the kernel on a quantum computer

A quantum computer does not return an overlap; it returns measurement outcomes, and the fidelity of Eq. (2.29) has to be extracted from their statistics. Two circuits do this.

The adjoint test Prepare $|\phi(\boldsymbol{x}')\rangle = U(\boldsymbol{x}')|0\rangle$, apply the inverse of the circuit of the other sample, $U^\dagger(\boldsymbol{x})$, and measure every qubit in the computational basis (Fig. 2.6). The probability of reading all zeros is

$$P(0\cdots0) = \left|\langle 0|U^\dagger(\boldsymbol{x})U(\boldsymbol{x}')|0\rangle\right|^2 = K(\boldsymbol{x},\boldsymbol{x}'), \tag{2.36}$$

exactly the kernel. The test uses n qubits and twice the depth of the feature map, and the inverse circuit is the feature-map circuit run backwards with the signs of all rotation angles reversed, so that nothing new has to be compiled. The estimate from S shots is the fraction $\hat{K} = S_0/S$ of all-zero outcomes, a binomial variable with

$$\operatorname{Var}\hat{K} = \frac{K(1-K)}{S} \le \frac{1}{4S}. \tag{2.37}$$

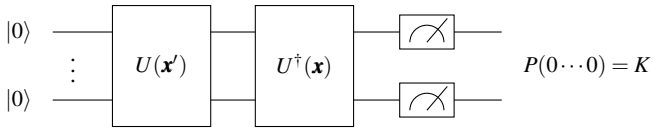


Fig. 2.6 The adjoint (inversion) test for the quantum kernel: the feature-map circuit of one sample followed by the inverse circuit of the other. The fraction of shots in which every qubit reads 0 estimates $K(\boldsymbol{x},\boldsymbol{x}')$, Eq. (2.36).

The SWAP test The older alternative [44] prepares both states in two registers of n qubits and uses an ancilla (Fig. 2.7): a Hadamard on the ancilla, a swap of the two registers controlled by the ancilla, a second Hadamard, and a measurement of the ancilla. Writing $|a\rangle = |\phi(\boldsymbol{x})\rangle$ and $|b\rangle = |\phi(\boldsymbol{x}')\rangle$, the state before the measurement is

$$\frac{1}{2}|0\rangle\big(|a\rangle|b\rangle + |b\rangle|a\rangle\big) + \frac{1}{2}|1\rangle\big(|a\rangle|b\rangle - |b\rangle|a\rangle\big),$$

and the probability of reading 0 on the ancilla is

$$P(0) = \frac{1}{2}\left(1 + |\langle a|b\rangle|^2\right) = \frac{1+K}{2}. \tag{2.38}$$

The estimate $\hat{K} = 2\hat{P}(0) - 1$ has variance

$$\operatorname{Var}\hat{K} = 4\,\frac{P(0)(1-P(0))}{S} = \frac{1-K^2}{S}, \tag{2.39}$$

which is $(1+K)/K$ times that of the adjoint test: twice as large near $K = 1$ and without bound for small kernel values. The SWAP test needs $2n+1$ qubits and n controlled-swap (Fredkin) gates, each of which costs several two-qubit gates on hardware; its one advantage is that it does not require the inverse circuit, so that it also works when the states come from somewhere other than a known circuit, for example from a quantum simulation. For kernels of encoded classical data the adjoint test is used.

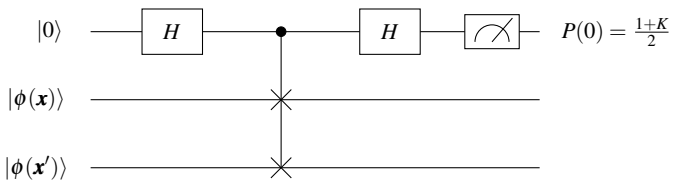


Fig. 2.7 The SWAP test: the two encoded states in two registers (each line stands for n qubits), a controlled swap of the registers, and a measurement of the ancilla, Eq. (2.38).

On the simulator The class `QuantumKernel` of `feature_maps.py` evaluates Eq. (2.29) exactly from the two state vectors, which is what a noiseless simulator can do and a quantum computer cannot, and also builds the two test circuits on the simulator of `common/statevector.py` and samples them:

```
import numpy as np
from feature_maps import ZZFeatureMap, QuantumKernel
rng = np.random.default_rng(1)              # the pair of the program's own test
x, xp = rng.uniform(0, np.pi, 2), rng.uniform(0, np.pi, 2)
kernel = QuantumKernel(ZZFeatureMap(2, reps=1))
K = kernel([x], [xp])[0, 0]                 # exact fidelity
adj = kernel.adjoint_circuit(x, xp)         # U(x)^dagger U(x')|0>
print(K, adj.probabilities()[0])            # P(00) = K
print(adj.sample(1000, np.random.default_rng(3)))
swap = kernel.swap_test_circuit(x, xp)      # ancilla + two registers
print((1 + K) / 2, swap.marginal(0)[0])     # P(ancilla = 0)
```

For the pair of samples in the program's own test the fidelity is 0.1570, the adjoint circuit returns all zeros with that probability, the ancilla of the SWAP circuit reads 0 with probability $0.5785 = (1+K)/2$, and a thousand shots of the adjoint test give 164 all-zero outcomes, an estimate $\hat{K} = 0.164$ with the standard error $\sqrt{K(1-K)/S} = 0.012$ of Eq. (2.37). With a finite `shots` argument the class returns, for every kernel entry, a binomial sample of the exact probability; this is statistically identical to running the circuit S times and counting, and it is how all shot-noise results below are produced.

The cost, and a defect of estimated kernels Training needs the $N(N-1)/2$ distinct off-diagonal entries of the Gram matrix (the diagonal is 1), each from its own circuit run S times, and prediction needs N_s more entries per test sample; the quantum part of the machine therefore costs $\mathcal{O}(N^2 S)$ circuit executions, which for the $N = 90$ training points and $S = 10^4$ shots of Section 2.9.1 is 4×10^7 shots, of the order of an hour on a present-day device. The estimated matrix $\hat{\boldsymbol{K}}$ (which the program samples entry by entry, $\hat{K}_{ij}$ and $\hat{K}_{ji}$ independently) is symmetric only on average and, more seriously, is not positive semidefinite: the sampling errors of $\mathcal{O}(1/\sqrt{S})$ in every entry shift the small eigenvalues of $\boldsymbol{K}$ below zero, and the dual problem of Eq. (2.21) is then not concave, so that SMO can cycle and an interior-point solver can fail. The remedy used by `qsvm.py`, and by the libraries, is to replace $\hat{\boldsymbol{K}}$ by the nearest positive-semidefinite matrix: symmetrize, diagonalize, and set the negative eigenvalues to zero,

$$\frac{1}{2}\big(\hat{\boldsymbol{K}} + \hat{\boldsymbol{K}}^T\big) = \sum_k \omega_k \boldsymbol{v}_k \boldsymbol{v}_k^T \;\longrightarrow\; \hat{\boldsymbol{K}}_{\mathrm{psd}} = \sum_{k:\,\omega_k > 0} \omega_k \boldsymbol{v}_k \boldsymbol{v}_k^T . \tag{2.40}$$

Table 2.3 will show how negative the smallest eigenvalue gets.

2.9 The quantum support-vector machine

The algorithm Everything is now in place, and the QSVM is the following four steps [14,42].

1. Choose a feature map $U(\boldsymbol{x})$ and rescale the features into the range the map expects, $[0, \pi]$ for rotation angles.
2. Estimate the training Gram matrix $K_{ij} = |\langle \phi(\boldsymbol{x}_i) | \phi(\boldsymbol{x}_j) \rangle|^2$ on the quantum computer, one circuit per pair, and project it onto the positive-semidefinite cone if it was estimated with shots.
3. Solve the dual problem

$$\max_{\boldsymbol{\lambda}} \sum_i \lambda_i - \frac{1}{2}\sum_{i,j} \lambda_i \lambda_j y_i y_j K_{ij} \quad \text{s.t.} \quad \sum_i \lambda_i y_i = 0,\ 0 \le \lambda_i \le C, \tag{2.41}$$

on the classical computer, which is Eq. (2.21) with the quantum Gram matrix, and compute b from the margin support vectors by Eq. (2.12).

4. Classify a new sample by estimating its kernel with each support vector and evaluating

$$\hat{y}(\boldsymbol{x}) = \operatorname{sign}\left(\sum_{i \in \mathscr{S}} \lambda_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b \right). \tag{2.42}$$

The class `QSVM` of `qsvm.py` is these four steps, and is short enough to show in full (the program adds only a switch `psd` that turns the projection off):

```
import numpy as np
from svm import SVM
from feature_maps import QuantumKernel, ZZFeatureMap

class QSVM:
    """A support-vector machine with a quantum kernel."""
    def __init__(self, feature_map, C=1.0, shots=None, method="adjoint", rng=0):
        self.kernel = QuantumKernel(feature_map, shots=shots, method=method, rng=rng)
        self.svm = SVM(C=C)
    def fit(self, X, y):
        self.X_train = np.asarray(X, float)
        K = self.kernel.gram(self.X_train)          # the quantum part
        if self.kernel.shots:                       # Eq. (2-psdfix)
            w, V = np.linalg.eigh((K + K.T) / 2)
            K = (V * np.maximum(w, 0)) @ V.T
        self.svm.fit(self.X_train, y, K=K)          # the classical part
        return self
    def decision_function(self, X):
        K = self.kernel(np.asarray(X, float), self.X_train)
        return self.svm.decision_function(X, K=K)
    def predict(self, X):
        return np.sign(self.decision_function(X))
```

The solver never learns that the kernel came from a quantum circuit, and the same class runs with any `FeatureMap`.

Four data sets Table 2.1 trains six machines on four two-class problems, each split 70/30 into training and test sets: the two moons and two circles of Fig. 2.4, the XOR problem (four Gaussian clouds at the corners of a square with diagonally opposite corners in the same class, which no linear classifier can learn), and the two overlapping species, versicolor and virginica, of Fisher's Iris flowers [45] described by petal length and width. The classical machines use standardized features and the quantum ones features rescaled into $[0, \pi]$; $C = 1$ throughout.

Table 2.1 Training/test accuracy of classical and quantum kernels on four two-class problems, $C = 1$, 70/30 split (two features, two qubits in every case). The angle encoding is Eq. (2.31); "ent" adds a second layer of rotations with a CNOT between them; the ZZ map is Eq. (2.34) with $r = 2$, with and without the bandwidth $\alpha = 0.5$. From `qsvm.py`.

Data set	linear	RBF ($\gamma = 1$)	angle	angle, ent.	ZZ	ZZ, $\alpha = 0.5$
two moons	0.89/0.83	0.96/0.97	0.89/0.77	0.89/0.83	0.77/0.83	0.94/0.90
two circles	0.67/0.37	1.00/1.00	1.00/1.00	0.99/1.00	0.80/0.73	0.96/0.97
XOR	0.81/0.60	1.00/1.00	1.00/1.00	0.97/0.90	0.81/0.83	1.00/1.00
Iris (versicolor/virginica)	0.96/0.90	0.97/0.90	0.96/0.90	0.97/0.90	0.83/0.70	0.93/0.87

Three things are visible in the table and in Fig. 2.8, which draws the decision regions of three of the kernels. First, the plain angle encoding does what its kernel, Eq. (2.32), says it should: it is a stationary kernel of fixed width, and it performs like the RBF kernel on the circles and the XOR data, where a width of order one is right, and worse on the moons, where the RBF kernel with its tunable γ wins. Second, the ZZ map with the features scaled into $[0,\pi]$ fits the training set *worse* than every other non-linear kernel in the table, on every data set: its phases $\phi_{qq'} = (\pi - x_q)(\pi - x_{q'})$ vary so quickly across the data that distinct samples are nearly orthogonal in the feature space, the Gram matrix is nearly diagonal, and the machine cannot generalize, which is the bandwidth problem of Shaydulin and Wild [43]. Halving the features, $\alpha = 0.5$, repairs it, and the ZZ kernel then matches the RBF kernel on the XOR data and comes close on the others. Third, and most important, nothing in the table is *better* than the Gaussian kernel. These are two-dimensional problems; the two-qubit feature space is four-dimensional, its density matrices span a sixteen-dimensional real space, and a classical computer evaluates all the quantum kernels of the table exactly in microseconds, as the simulator just did. There was no reason to expect an advantage and there is none.

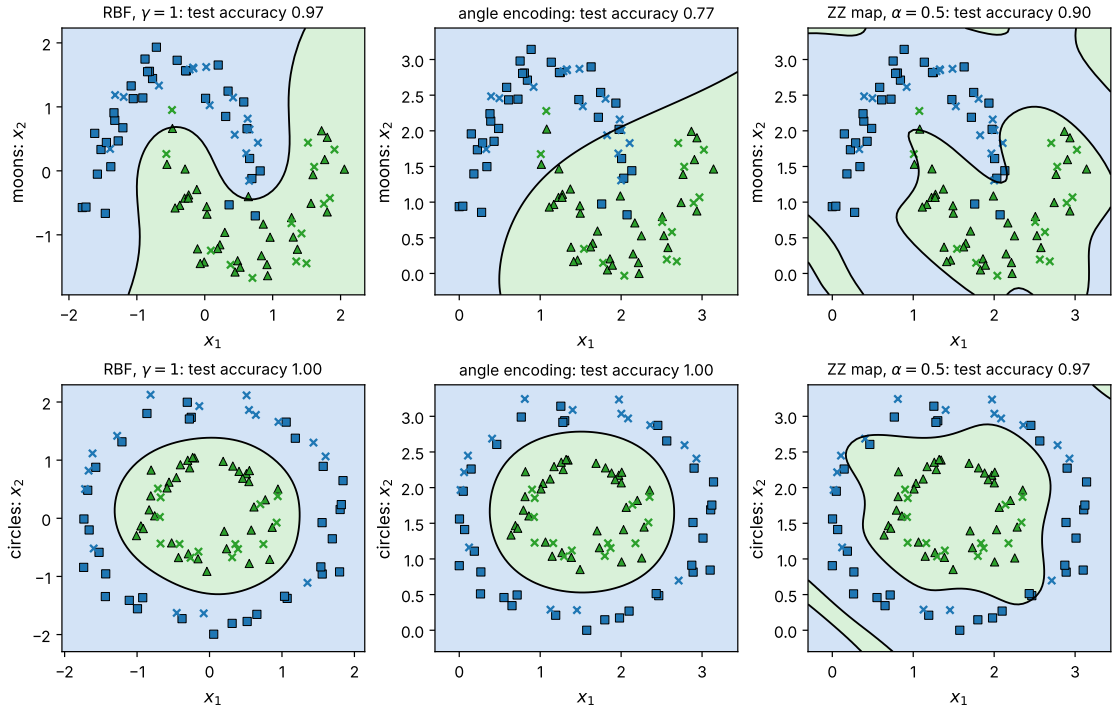


Fig. 2.8 Decision regions of the RBF kernel, the angle-encoding kernel and the ZZ kernel with bandwidth $\alpha = 0.5$ on the two moons (top) and the two circles (bottom); $C = 1$. Filled markers are training samples, crosses test samples. The classical kernel sees standardized features, the quantum ones features rescaled into $[0,\pi]$. From `qsvm.py`.

A data set made for the quantum kernel Havlíček *et al.* demonstrated their machine on data that were constructed to be separable by their feature map [14], and the construction is instructive. Draw samples $\boldsymbol{x}$ uniformly from $[0,2\pi)^2$, encode each with the ZZ map, $|\phi(\boldsymbol{x})\rangle$, and label it by the sign of the expectation value of a fixed observable in that state,

$$y(\boldsymbol{x}) = \operatorname{sign} f(\boldsymbol{x}), \qquad f(\boldsymbol{x}) = \langle \phi(\boldsymbol{x}) | V^\dagger Z \otimes Z V | \phi(\boldsymbol{x}) \rangle, \tag{2.43}$$

with V a fixed random unitary; samples with $|f(\boldsymbol{x})| < \Delta$ are discarded so that the classes are separated by a gap $\Delta = 0.3$. By construction the labels are a linear function of the density matrix $\rho(\boldsymbol{x})$, hence linearly separable in the feature space of Eq. (2.30), and a support-vector machine with exactly this kernel should find a separating hyperplane; a classical kernel has no such guarantee. Table 2.2 and Fig. 2.9 show the outcome for 120 samples, 90 of them for training. The linear kernel and the angle encoding are at chance level on the test set; the RBF kernel reaches a test accuracy of 0.57 to 0.63 whatever its width, and with $\gamma = 5$ fits the training set to 0.97 by memorizing it, with 80 of 90 points as support vectors; the ZZ kernel with the right number of repetitions classifies training and test sets perfectly, with 27 support vectors, or 13 with a harder margin. The boundary it draws, in Fig. 2.9, is a complicated, fine-grained function of the features that no kernel of fixed width can reproduce, and that the ZZ kernel reproduces because it is the function the labels were made from.

Table 2.2 The data set of Eq. (2.43): 120 samples, 65 with $y = +1$, 90 for training and 30 for testing, $C = 1$ unless stated. From `qsvm.py`.

Kernel	Training accuracy	Test accuracy	Support vectors
linear	0.64	0.57	72
RBF, $\gamma = 0.2$	0.67	0.57	76
RBF, $\gamma = 1$	0.80	0.57	75
RBF, $\gamma = 5$	0.97	0.63	80
angle encoding	0.66	0.43	78
ZZ, $r = 1$	0.90	0.97	46
ZZ, $r = 2$	1.00	1.00	27
ZZ, $r = 2$, $C = 100$	1.00	1.00	13

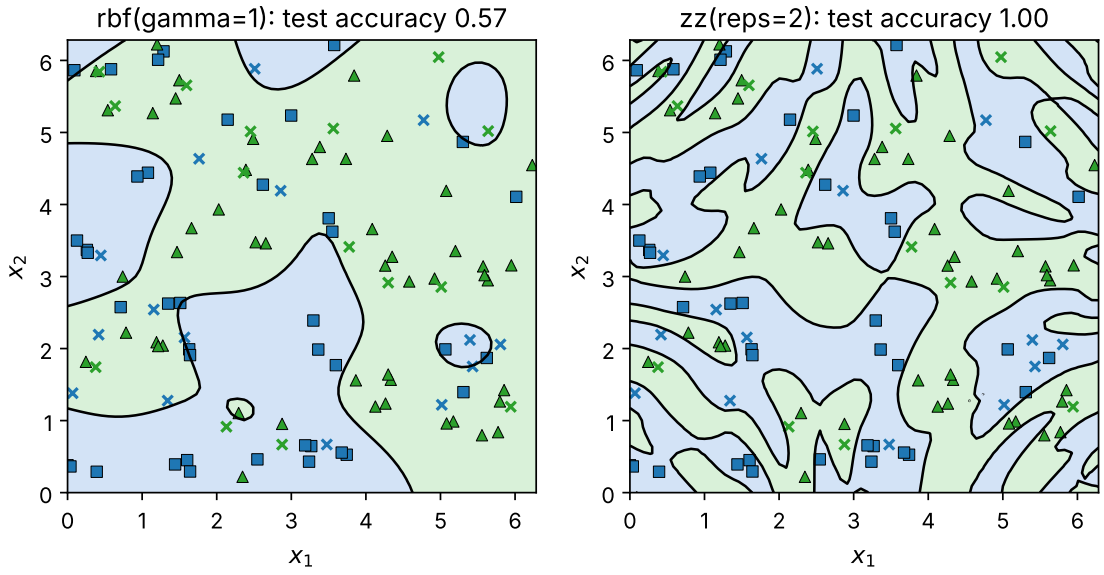


Fig. 2.9 Decision regions of the RBF kernel ($\gamma = 1$) and of the ZZ kernel ($r = 2$) on the data of Eq. (2.43), which were labeled by the ZZ map itself. Filled markers are training samples, crosses test samples. From `qsvm.py`.

This is the demonstration one can give today, and it must be read for what it is. It shows that a quantum kernel is a kernel like any other, with its own inductive bias [46], and that

data whose structure matches that bias are learned by it and not by others. It does not show that natural data have that structure, and the random unitary V that generated the labels is not something one meets in a flower or a material. Whether there exist practically relevant problems whose labels are, in this sense, a function of a classically intractable quantum state is the open question of the field; Liu, Arunachalam and Temme [12] constructed one from the discrete-logarithm problem, with a provable advantage, and Huang *et al.* [21] gave a test, computable from the two Gram matrices, of whether a given data set could profit from a given quantum kernel at all.

2.9.1 Kernels from a finite number of shots

All the quantum results above used the exact fidelities. A quantum computer gives estimates, and Table 2.3 and Fig. 2.10 repeat the ZZ experiment of Table 2.2 with the training Gram matrix estimated from S shots per entry, by each of the two tests, and projected onto the positive-semidefinite cone before training. The root-mean-square error of the entries falls as $1/\sqrt{S}$, with the prefactor of Eq. (2.37) for the adjoint test and a prefactor about 2.4 times larger for the SWAP test, as Eqs. (2.37) and (2.39) predict for the small kernel values that dominate this Gram matrix. The smallest eigenvalue of the raw estimate is negative at every shot count, down to -1.4 (adjoint) and -3.6 (SWAP) at ten shots for a matrix whose exact eigenvalues lie in $[0, N]$, which is why the projection of Eq. (2.40) is not optional. The classifier is remarkably robust: with the adjoint test ten shots per entry already give a perfect test accuracy, and with the SWAP test a thousand do. The reason is that the support-vector machine is a maximum-margin method, and a margin of $\Delta = 0.3$ in the label function tolerates a good deal of noise in the kernel; for data with overlapping classes the shot count would matter more, and Exercise 7 explores this.

Table 2.3 The ZZ kernel ($r = 2$) of the training set of Table 2.2 estimated from S shots per entry: the largest and the root-mean-square error of the entries, the smallest eigenvalue of the (symmetrized) estimate before projection, and the test accuracy of the machine trained on the projected estimate. From `qsvm.py`.

Test	S	$\max \lvert\hat{K}_{ij} - K_{ij}\rvert$	rms error	$\omega_{\min}$	Test accuracy
adjoint	10	0.576	0.119	-1.447	1.00
adjoint	100	0.167	0.038	-0.431	1.00
adjoint	1000	0.066	0.012	-0.149	1.00
adjoint	10000	0.024	0.004	-0.044	1.00
adjoint	100000	0.006	0.001	-0.014	1.00
SWAP	10	1.225	0.290	-3.626	0.70
SWAP	100	0.370	0.092	-1.125	0.93
SWAP	1000	0.126	0.030	-0.335	1.00
SWAP	10000	0.038	0.009	-0.104	1.00
SWAP	100000	0.012	0.003	-0.036	1.00

The same with a library The feature map, the adjoint test and the precomputed-kernel machine are what the quantum machine-learning libraries package. In PennyLane [35] the adjoint test is written by applying the template of the feature map and then its adjoint, and the kernel is handed to scikit-learn's `SVC` with `kernel='precomputed'`:

```
try:
    import pennylane as qml
```

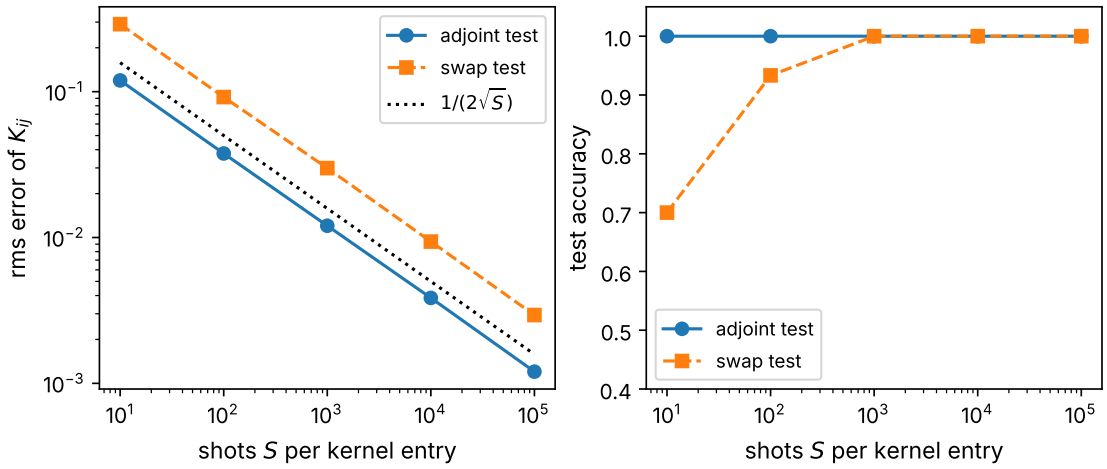


Fig. 2.10 Left: root-mean-square error of the estimated kernel entries against the number of shots per entry, for the adjoint and the SWAP test, with the bound $1/(2\sqrt{S})$ of Eq. (2.37). Right: test accuracy of the machine trained on the projected estimate. From `qsvm.py`.

```
    from pennylane import numpy as pnp
    from sklearn.svm import SVC
    dev = qml.device("default.qubit", wires=2)
    @qml.qnode(dev)
    def adjoint_test(x1, x2):
        qml.AngleEmbedding(x1, wires=[0, 1], rotation="Y")
        qml.adjoint(qml.AngleEmbedding)(x2, wires=[0, 1], rotation="Y")
        return qml.probs(wires=[0, 1])
    kernel = lambda a, b: adjoint_test(a, b)[0]      # P(00)
    X_train = pnp.array([[0.1, 0.2], [1.1, 2.4], [0.0, 2.9], [3.0, 0.5]])
    y_train = pnp.array([+1, +1, −1, −1])
    K_train = qml.kernels.kernel_matrix(X_train, X_train, kernel)
    clf = SVC(kernel="precomputed").fit(K_train, y_train)
    print(clf.predict(qml.kernels.kernel_matrix(X_train, X_train, kernel)))
except ImportError:
    print("PennyLane or scikit−learn not installed; the book's own QSVM does the same")
```

Qiskit [36] provides the ZZ map as `ZZFeatureMap` and the whole machine as `QSVC` in its machine-learning module:

```
try:
    from qiskit.circuit.library import ZZFeatureMap
    from qiskit_machine_learning.kernels import FidelityQuantumKernel
    from qiskit_machine_learning.algorithms import QSVC
    import numpy as np
    feature_map = ZZFeatureMap(feature_dimension=2, reps=2, entanglement="linear")
    qkernel = FidelityQuantumKernel(feature_map=feature_map)
    X_train = np.array([[0.1, 0.2], [1.1, 2.4], [0.0, 2.9], [3.0, 0.5]])
    y_train = np.array([+1, +1, −1, −1])
    qsvc = QSVC(quantum_kernel=qkernel, C=1.0).fit(X_train, y_train)
    print(qsvc.predict(X_train))
except ImportError:
    print("Qiskit machine learning not installed; the book's own QSVM does the same")
```

Both snippets run when the package is installed and print a message when it is not; the results of this chapter do not depend on either.

2.10 Training the feature map

The feature map has so far been fixed by hand, and the bandwidth problem of Table 2.1 showed how much depends on that choice. A feature map can instead carry parameters of its own, $U(\boldsymbol{x};\boldsymbol{\theta})$, and these can be trained *before* the support-vector machine is, on a criterion that measures how well the kernel suits the labels. The standard criterion is the *kernel-target alignment* of Cristianini *et al.* [47],

$$A(\boldsymbol{\theta}) = \frac{\sum_{i,j} K_{ij}(\boldsymbol{\theta})\, y_i y_j}{\|\boldsymbol{K}(\boldsymbol{\theta})\|_F \, \|\boldsymbol{y}\boldsymbol{y}^T\|_F} = \frac{\langle \boldsymbol{K}, \boldsymbol{y}\boldsymbol{y}^T \rangle_F}{\|\boldsymbol{K}\|_F \, N}, \tag{2.44}$$

the cosine of the angle between the Gram matrix and the ideal kernel $\boldsymbol{y}\boldsymbol{y}^T$, which is $+1$ for two samples of the same class and -1 otherwise. A kernel with $A = 1$ would be proportional to $\boldsymbol{y}\boldsymbol{y}^T$ and would separate the classes with a single support vector per class. Hubregtsen *et al.* [48] proposed to maximize A over the parameters of a quantum embedding by gradient ascent, with the gradient evaluated on the quantum computer by the parameter-shift rule.

The gradient Every gate of the embedding is a rotation $e^{-ia\sigma/2}$ with σ a Pauli operator, and for such a gate the derivative of any expectation value with respect to its angle a is the difference of two shifted evaluations, $\partial_a g = \frac{1}{2}[g(a+\pi/2) - g(a-\pi/2)]$ [10, 11]. A kernel entry $K_{ij} = |\langle 0|U^\dagger(\boldsymbol{x}_i;\boldsymbol{\theta})U(\boldsymbol{x}_j;\boldsymbol{\theta})|0\rangle|^2$ is an expectation value of the projector $|0\rangle\langle 0|$, but a parameter θ_k appears twice in it, in the bra circuit and in the ket circuit, and the rule is applied to each occurrence separately:

$$\frac{\partial K_{ij}}{\partial \theta_k} = \frac{1}{2}\left[K_{ij}(\theta_k^+, \theta_k) - K_{ij}(\theta_k^-, \theta_k)\right] + \frac{1}{2}\left[K_{ij}(\theta_k, \theta_k^+) - K_{ij}(\theta_k, \theta_k^-)\right], \tag{2.45}$$

where $K_{ij}(\theta_k^\pm, \theta_k)$ is the fidelity with the shift $\pm\pi/2$ applied in the bra circuit only, and the second bracket is the transpose of the first, since $K_{ij}(\theta,\theta') = K_{ji}(\theta',\theta)$. If the parameter multiplies a feature, as in the gate $R_y(w_q x_q)$, the chain rule adds a factor x_q of the shifted sample. The gradient of A then follows from Eq. (2.44) by the chain rule, and costs two Gram matrices per parameter.

The embedding and the experiment The class `TrainableEmbedding` of `feature_maps.py` is a *data re-uploading* circuit [49, 50]: r repetitions of a data layer $R_y(w_q x_q)$ on every qubit followed by a trainable layer $R_z(\theta)R_y(\theta')$ on every qubit and a ring of CNOTs,

$$U(\boldsymbol{x};\boldsymbol{\theta}) = \prod_{\ell=1}^{r}\Big[\mathrm{CNOT}_{\mathrm{ring}} \bigotimes_q R_y(\theta'_{\ell q}) R_z(\theta_{\ell q}) \bigotimes_q R_y(w_{\ell q} x_q)\Big], \tag{2.46}$$

with the input weights w among the trainable parameters, so that the alignment can also learn the bandwidth. The program `kernel_alignment.py` trains it on the two moons, with the weights deliberately started at $w = 2.5$, a badly scaled embedding, and the angles at random. The gradient of Eq. (2.45) is first checked against finite differences (they agree to 10^{-11}), then forty steps of gradient ascent with step size one are taken, and every five steps a QSVM is trained on the current kernel. Figure 2.11 shows the result: the alignment rises from 0.14 to 0.37, and the test accuracy of the machine from 0.67 at the badly scaled start to 0.97 after ten steps and 1.00 at the end, better than any fixed kernel of Table 2.1 on these data. Interestingly, the weights do not shrink back towards one; the alignment instead reshapes the trainable rotations so that the large-angle data layers become useful. The procedure costs a full set of Gram matrices per parameter and per step, so it is expensive in circuit executions, but the circuits are the same shallow ones as before, and the machine that is finally trained is still the convex problem of Eq. (2.41).

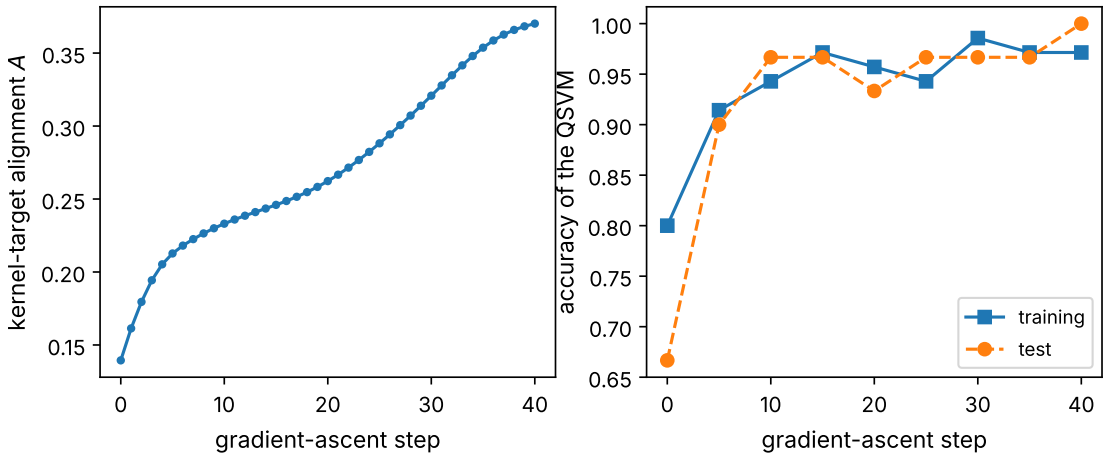


Fig. 2.11 Kernel-target alignment of the trainable embedding of Eq. (2.46) on the two moons during gradient ascent (left) and the training and test accuracy of the quantum support-vector machine with the kernel of the current parameters (right). From `kernel_alignment.py`.

2.11 Perspectives

Complexity A classical kernel machine on N samples spends $\mathcal{O}(N^2)$ kernel evaluations and, with a general-purpose solver, $\mathcal{O}(N^3)$ time on the dual problem; prediction costs N_s kernel evaluations per sample. The near-term quantum machine of this chapter spends $\mathcal{O}(N^2 S)$ circuit executions on the Gram matrix and the same $\mathcal{O}(N^3)$ classical time on the dual. It offers no speedup at all, and makes no claim to one; what it offers is a different kernel, which may or may not suit the data. The fault-tolerant machine of Rebentrost, Mohseni and Lloyd [15] is a different proposal: it replaces the inequality constraints by equalities (the least-squares SVM), so that training becomes the solution of an $(N+1)$-dimensional linear system with the kernel matrix as its principal block, and solves that system with the HHL algorithm [16] in time $\mathcal{O}(\log(Np))$ under the assumptions of quantum random-access memory for the data, a well-conditioned, efficiently simulable kernel matrix, and an output that is the state $\sum_i \lambda_i |i\rangle$ rather than the numbers λ_i. The companion volume treats HHL in its phase-estimation chapter, and the reader who has followed it will see that each assumption is substantive; Tang's dequantization results [19] show that for low-rank kernel matrices and sampling access to the data a classical algorithm matches the polylogarithmic scaling, so that the exponential claim survives only under conditions that have not been met in practice.

Every quantum model is a kernel method Return to the general model of Eq. (1.1), $f(\boldsymbol{x};\boldsymbol{\theta}) = \mathrm{Tr}[\rho(\boldsymbol{x}) W^\dagger \hat{O} W]$. For fixed $\boldsymbol{\theta}$ the operator $M = W^\dagger(\boldsymbol{\theta})\hat{O}W(\boldsymbol{\theta})$ is a fixed Hermitian matrix, and f is *linear* in the density matrix $\rho(\boldsymbol{x})$, the feature vector of Eq. (2.30). Every variational quantum classifier, whatever its trainable circuit, is therefore a linear model in the same feature space in which the quantum support-vector machine with kernel $\mathrm{Tr}[\rho(\boldsymbol{x})\rho(\boldsymbol{x}')]$ is linear, and the representer theorem of kernel methods implies that the minimizer of any regularized loss over such linear models is of the form $\sum_i \alpha_i K(\boldsymbol{x}_i, \boldsymbol{x})$. This is Schuld's observation [5] that supervised quantum models are kernel methods: the trainable circuit W can only choose a direction M in the feature space, and the support-vector machine, which optimizes over all directions, is at least as expressive. Two qualifications matter. The two models are trained on different losses, and the variational model, restricted to a family of M, may generalize better precisely because it is restricted [51]; and the kernel machine needs $\mathcal{O}(N^2)$ circuits where the variational model needs $\mathcal{O}(N)$ per epoch. Chapter 4 takes up the variational side.

Exponential concentration The feature maps that cannot be simulated classically are, for the same reason, the ones whose kernels are hard to *measure*: for a map that is expressive enough to produce near-random states, all kernel values between distinct samples concentrate exponentially fast in n around a constant, $K_{ij} \approx 2^{-n}$, and the number of shots needed to distinguish them grows exponentially [23]. The bandwidth problem that made the ZZ map fail in Table 2.1 is a small-n instance of this: the problem is not the circuit but the fact that the circuit is too expressive for the data. The design of feature maps that are classically hard, that match the structure of some class of data, and that keep their kernel values measurable is where the research is.

2.12 Summary

A support-vector machine finds the maximum-margin hyperplane between two classes. Its dual problem, Eq. (2.10), involves the samples only through inner products, its solution is carried by the support vectors singled out by the KKT conditions, and the soft margin with its constant C extends it to overlapping classes. Replacing the inner product by a kernel, Eq. (2.21), moves the machine into any feature space whose inner product is computable, and Mercer's condition says which functions qualify. A quantum feature map, Eq. (2.28), is such a space, of dimension 2^n, and its kernel is the fidelity of two encoded states, Eq. (2.29), which is a Mercer kernel automatically and is measured on a quantum computer by the adjoint test with variance $K(1-K)/S$. The quantum support-vector machine is the classical machine trained on this measured Gram matrix, projected onto the positive-semidefinite cone. Product feature maps give classical kernels; entangling maps such as the ZZ map give kernels that are not efficiently simulable for enough qubits, that require a bandwidth to be chosen or learned, and that separate data whose labels were made by the same map. The feature map can be trained by kernel-target alignment with parameter-shift gradients. There is no running-time advantage in any of this, and the advantage in accuracy exists for constructed data and is open for natural data; and every quantum model of the hybrid kind, variational classifiers included, is a linear model in the space of density matrices and therefore a kernel method.

2.13 Exercises

1. **The margin.** (a) Show that the distance between the two planes $\boldsymbol{w}^T\boldsymbol{x} + b = \pm 1$ is $2/\|\boldsymbol{w}\|$. (b) Derive Eq. (2.10) from Eq. (2.7) by inserting Eqs. (2.8) and (2.9), keeping track of the factor $\frac{1}{2}$. (c) Show that at the solution the dual objective equals $\frac{1}{2}\|\boldsymbol{w}\|^2$, and check this with `svm.py` on the data of Fig. 2.1 (`SVM.dual_objective` and `SVM.margin`).
2. **Soft margin.** (a) Derive Eq. (2.17) and show that $\lambda_i = C$ for every sample with $\xi_i > 0$. (b) Explain why the intercept must be computed from the samples with $0 < \lambda_i < C$ only. (c) Reproduce Fig. 2.2, add $C = 10^{-3}$ and $C = 10^{4}$, and plot the margin and the number of support vectors against C on a logarithmic axis. (d) Split the data into training and test sets and find the C that maximizes the test accuracy.
3. **Kernels.** (a) Show that the polynomial kernel $(\boldsymbol{x}^T\boldsymbol{x}' + 1)^2$ in two dimensions corresponds to a six-dimensional feature map, and write it down. (b) Show that the RBF kernel's feature space is infinite-dimensional by expanding $\exp(2\gamma\boldsymbol{x}^T\boldsymbol{x}')$ in a power series. (c) Prove that the sum and the product of two Mercer kernels are Mercer kernels, and that $|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$ is one for any feature map, using Eq. (2.30). (d) Find a set of three points on which the sigmoid kernel $\tanh(\boldsymbol{x}^T\boldsymbol{x}' - 1)$ has a Gram matrix with a negative eigenvalue.

4. **SMO.** (a) Show that the dual objective restricted to the line $\lambda_i y_i + \lambda_j y_j = \text{const}$ is a parabola with curvature $-\eta$, $\eta = K_{ii} + K_{jj} - 2K_{ij}$, and that its maximum is at Eq. (2.27). (b) Show that $\eta \geq 0$ for a Mercer kernel and that $\eta = 0$ if and only if $\boldsymbol{\phi}(\boldsymbol{x}_i) = \boldsymbol{\phi}(\boldsymbol{x}_j)$. (c) Derive the bounds L and H for the two cases $y_i = y_j$ and $y_i \neq y_j$. (d) Install CVXOPT, set up the matrices of Eq. (2.26) with `SVM.qp_matrices`, and compare the multipliers with those of SMO on the data of Fig. 2.4.
5. **Feature maps.** (a) Verify that $e^{i\phi Z\otimes Z}$ equals $\mathrm{CNOT}\,(I\otimes R_z(-2\phi))\,\mathrm{CNOT}$ exactly, with $R_z(\theta) = e^{-i\theta Z/2}$. (b) Write out $|\phi(\boldsymbol{x})\rangle$ for the ZZ map with $r = 1$ on two qubits and compute its kernel in closed form. Is it a function of $\boldsymbol{x} - \boldsymbol{x}'$ only? (c) Compute the kernel of the entangled angle encoding ("angle, ent." of Table 2.1) on two qubits and compare it with Eq. (2.32). (d) With `feature_maps.py`, plot $K(\boldsymbol{x}, \boldsymbol{x}')$ against $\|\boldsymbol{x} - \boldsymbol{x}'\|$ for a thousand random pairs and the ZZ map with $\alpha = 1$, 0.5 and 0.2; relate what you see to the bandwidth discussion of Section 2.9.
6. **Kernel estimation.** (a) Derive Eqs. (2.38) and (2.39). (b) Show that the SWAP-test variance exceeds the adjoint-test variance by the factor $(1 + K)/K$ and compute the factor for the mean off-diagonal kernel value of the Gram matrix of Table 2.3, available as the attribute `K_raw` of `QSVM`. (c) Prove that Eq. (2.40) gives the nearest positive-semidefinite matrix in the Frobenius norm. (d) How many circuit executions does the whole experiment of Table 2.3 require at $S = 10^4$, training and testing included?
7. **Shots and overlapping classes.** Repeat the shot-noise study of Section 2.9.1 for the two moons with the ZZ kernel, $\alpha = 0.5$, and for the Iris data, and plot the test accuracy against S. At what shot count does the accuracy reach its noiseless value, and how does that compare with Table 2.3? Try also training on the raw estimate without the projection of Eq. (2.40) (argument `psd=False` of `QSVM`) and report what the solver does.
8. **Kernel-target alignment.** (a) Show that $A \leq 1$ with equality if and only if $\boldsymbol{K} \propto \boldsymbol{y}\boldsymbol{y}^T$. (b) Derive Eq. (2.45) and the chain-rule factor for a parameter that multiplies a feature. (c) Rerun `kernel_alignment.py` from the weights $w = 1$ and from $w = 5$, and with the Adam optimizer in the place of plain gradient ascent. Does the alignment find the same optimum? (d) Replace the alignment by the test accuracy of the QSVM as the criterion to be maximized. Why is that a worse idea, and why can it not be done by the parameter-shift rule?
9. **Every model is a kernel method.** (a) Show that $f(\boldsymbol{x}) = \mathrm{Tr}[\rho(\boldsymbol{x})M]$ is linear in $\rho(\boldsymbol{x})$ and that the set $\{\rho(\boldsymbol{x})\}$ spans at most a 4^n-dimensional real space. (b) Show that a one-qubit variational classifier with $\hat{O} = Z$ and any $W(\boldsymbol{\theta})$ computes $f(\boldsymbol{x}) = \boldsymbol{n}(\boldsymbol{\theta}) \cdot \boldsymbol{r}(\boldsymbol{x})$, with $\boldsymbol{r}$ the Bloch vector of $\rho(\boldsymbol{x})$ and $\boldsymbol{n}$ a unit vector, and that a QSVM with the fidelity kernel can represent every such f. (c) Construct the data of Eq. (2.43) with a *product* observable and a product unitary V, and show with `qsvm.py` that the RBF kernel then learns it as well as the ZZ kernel does.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter2/`, which build on the shared modules of `BookPrograms/common/` (`statevector`, `gates`, `datasets`, `bookplot`) and are imported by the notebook `Notebooks/chapter2/chapter2.ipynb`. All run on `numpy` and `matplotlib` alone; CVXOPT, PennyLane, Qiskit and scikit-learn are used only where the text says so and only if installed.

`kernels.py` The `Kernel` classes `LinearKernel`, `PolynomialKernel`, `RBFKernel`, `TanhKernel` and `PrecomputedKernel`, Eqs. (2.23)–(2.24), the quadratic map of Eq. (2.19), a numerical Mercer check, and the two rows of Fig. 2.3. Three seconds.

`svm.py` The class `SVM`: the dual problem of Eq. (2.21) solved by sequential minimal optimization, Eq. (2.27), for any kernel or a precomputed Gram matrix; the intercept of Eq. (2.12), the decision function of Eq. (2.22), the matrices of Eq. (2.26) and an optional CVXOPT solution. As a script it produces Figs. 2.1, 2.2 and 2.4 and the numbers quoted with them. Ten seconds.

`feature_maps.py` The feature maps `AngleEncoding`, `ZZFeatureMap`, `AmplitudeEncoding` and `TrainableEmbedding`, Eqs. (2.31), (2.34), (2.33) and (2.46), and the class `QuantumKernel` with exact fidelities, shot-sampled estimates by the adjoint or the SWAP test, and the two test circuits on the simulator. As a script it checks the kernels of Eqs. (2.32) and (2.33), the ZZ map against its matrix exponential, and Eqs. (2.36) and (2.38). One second.

`qsvm.py` The class `QSVM` with the projection of Eq. (2.40), the data set of Eq. (2.43), Tables 2.1, 2.2 and 2.3, and Figs. 2.8, 2.9 and 2.10. Half a minute.

`kernel_alignment.py` The class `KernelAlignment`: the alignment of Eq. (2.44), its parameter-shift gradient, Eq. (2.45), checked against finite differences, and gradient ascent; Fig. 2.11 and the numbers of Section 2.10. Forty seconds.

Chapter 3
The Quantum Approximate Optimization Algorithm

Abstract The quantum approximate optimization algorithm is the simplest variational quantum circuit and the one whose purpose is easiest to state: find the bit string that minimizes a cost. The chapter builds the pipeline from a combinatorial problem to a quadratic unconstrained binary optimization, from there to an Ising model and a diagonal cost Hamiltonian, and from the Hamiltonian to the alternating circuit of phase separators and transverse-field mixers, derived from adiabatic evolution by Trotterization and written out gate by gate. The single-edge instance is solved by hand, the algorithm is run on a ring and on a six-vertex graph, and its performance, landscape and gate cost are discussed. Gradients require the parameter-shift rule applied term by term, since the generators have many eigenvalues. Two electron spins in a double quantum dot, coupled by the exchange interaction, implement the one-edge instance natively, and the selection of a portfolio of stocks from real price data shows the pipeline on a constrained problem, with the penalty method, its tuning and the connection to thermal states. The chapter closes by reading QAOA as a machine-learning model: a clustering algorithm, a quantum neural network with a problem-fixed architecture, a feature map and a sampler.

3.1 Introduction

The quantum support-vector machine of Chapter 2 used the quantum computer for one thing only, to evaluate a kernel; the learning was classical and convex, and, the trainable kernels of its last section apart, there were no parameters on the quantum side. This chapter introduces the other way of using the machine, which the rest of the book will follow: a circuit with adjustable angles, an expectation value that depends on them, and a classical optimizer that moves the angles downhill. The *quantum approximate optimization algorithm* (QAOA) of Farhi, Goldstone and Gutmann [52] is the simplest and the most structured example of such a variational circuit, and it is also the one whose purpose is easiest to state: it looks for the bit string that minimizes a cost function, by preparing a quantum state whose measurement outcomes are concentrated on good bit strings.

The algorithm is a bridge in two directions. Towards physics, it is a variational eigensolver for an Ising Hamiltonian, the problem is written as a classical spin glass, and its ansatz is the Trotterized form of adiabatic quantum evolution, so that the reader of the companion volume [1] will recognize every ingredient: its chapter on variational quantum algorithms introduces QAOA in a section of its own, after the variational principle, the ansätze, the measurement of a Pauli sum and the parameter-shift rule, and the present chapter continues from there, with the relation between its sign convention for the cost Hamiltonian and ours made explicit in Section 3.2. Towards machine learning, it is a parametrized circuit trained

by gradient descent on a cost, the data of a problem enter through the Hamiltonian, and the variational classifiers and generative models of Chapters 4 and 5 differ from it only in where the data and the parameters sit. We develop both directions.

The chapter is organized as follows. Section 3.2 sets up the problem: combinatorial optimization over bit strings, its quadratic unconstrained binary form (QUBO), the change of variables that turns it into an Ising model, and the cost Hamiltonian, with MaxCut as the running example. Section 3.3 defines the algorithm, derives its ansatz from adiabatic evolution by Trotterization, and gives the circuit gate by gate. Section 3.4 solves the smallest instance, a single edge, by hand, runs the algorithm on a four-vertex ring and a six-vertex graph, and discusses what is known about its performance, its landscape and its cost in gates. Section 3.5 is about gradients, where the parameter-shift rule of Chapter 2 meets a complication, generators with many eigenvalues, and where a simple per-term rule resolves it. Section 3.6 leaves the abstract register for a physical one: two electrons in a double quantum well whose exchange interaction implements the cost unitary, following a recent two-qubit experiment [53]. Section 3.7 applies the pipeline to a problem outside physics, the selection of a portfolio of stocks from real price data, and uses it to discuss constraints, penalties and the connection to the statistical mechanics of spin glasses. Section 3.8 returns to the question of the book and reads QAOA as a machine-learning model, with clustering as the example. The programs are in `BookPrograms/chapter3/`: a `QUBO` and an `Ising` class, a `QAOA` class on the simulator of the earlier chapters with the three gradient rules, and the scripts of the experiments. Every number quoted below is printed by one of them.

3.2 From a cost function to a Hamiltonian

Binary optimization A combinatorial optimization problem asks for the bit string $\boldsymbol{x} = (x_1, \ldots, x_n) \in \{0,1\}^n$ that minimizes a cost function $C : \{0,1\}^n \to \mathbb{R}$,

$$\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}\in\{0,1\}^n} C(\boldsymbol{x}). \tag{3.1}$$

The search space has 2^n elements, and for the problems of interest, the *NP-hard* ones, no algorithm is known that finds $\boldsymbol{x}^*$ in time polynomial in n for every instance. One settles for approximations, and measures their quality by the *approximation ratio*, the value achieved divided by the optimum; for a minimization with negative costs, which is how MaxCut will appear below, the ratio $\alpha = C(\boldsymbol{x})/C(\boldsymbol{x}^*) \leq 1$, and the closer to one the better.

QUBO A large class of such problems can be written with a cost that is quadratic in the bits,

$$C(\boldsymbol{x}) = \boldsymbol{x}^T \boldsymbol{Q}\boldsymbol{x} + c_0 = \sum_i Q_{ii} x_i + \sum_{i<j} 2Q_{ij}\, x_i x_j + c_0, \tag{3.2}$$

with a real symmetric matrix $\boldsymbol{Q}$ and a constant c_0; since $x_i^2 = x_i$ for a bit, the diagonal of $\boldsymbol{Q}$ carries the linear terms. This is *quadratic unconstrained binary optimization*, QUBO [54], and Lucas's catalog [55] shows how partitioning, covering, coloring, scheduling and satisfiability problems are brought into this form, constraints included, by adding penalty terms. QUBO is itself NP-hard, so nothing is lost in generality, and everything is gained in uniformity: a single algorithm for Eq. (3.2) is an algorithm for all of them.

From bits to spins The substitution

$$x_i = \frac{1 - z_i}{2}, \qquad z_i = 1 - 2x_i \in \{-1, +1\}, \tag{3.3}$$

turns a bit into a classical spin, with $x_i = 0$ the spin $z_i = +1$ and $x_i = 1$ the spin $z_i = -1$. Inserting it into Eq. (3.2) and collecting the terms by their degree in the spins gives an *Ising model*,

$$C(\boldsymbol{x}) = H(\boldsymbol{z}) = \sum_i h_i z_i + \sum_{i<j} J_{ij} z_i z_j + c, \tag{3.4}$$

with, for symmetric $\boldsymbol{Q}$,

$$J_{ij} = \frac{Q_{ij}}{2}, \qquad h_i = -\frac{Q_{ii}}{2} - \sum_{j\neq i} \frac{Q_{ij}}{2}, \qquad c = c_0 + \sum_i \frac{Q_{ii}}{2} + \sum_{i<j} \frac{Q_{ij}}{2}. \tag{3.5}$$

The derivation is a few lines (Exercise 1): the product $x_i x_j = (1-z_i)(1-z_j)/4$ contributes $z_i z_j/4$, $-z_i/4$, $-z_j/4$ and a constant, each appearing twice in the double sum of Eq. (3.2), and the linear term $Q_{ii}x_i$ contributes $-Q_{ii}z_i/2$ and $Q_{ii}/2$. Minimizing C over bit strings is finding the ground state of the Ising model, a problem that statistical physics has studied for a century, and whose hardness, for couplings J_{ij} of both signs that cannot all be satisfied at once, is the *frustration* of a spin glass. The class `QUBO` of `qubo.py` carries out the substitution and the class `Ising` lists the energies of all bit strings:

```
import numpy as np
from qubo import QUBO, bitstrings
q = QUBO([[1, −2], [−2, 1]])              # C(x) = x1 + x2 − 4 x1 x2, Table 3−qubo
ising = q.to_ising()                      # Eq. (3−qubotoising)
print("J_12 =", ising.J[0, 1], " h =", ising.h, " const =", ising.const)
for x, c, e in zip(bitstrings(2), q.costs(), ising.energies()):
    print(x, 1 − 2 ∗ x, c, e)             # bits, spins, C(x), H(z) + const
ring = QUBO.maxcut(4, [(0, 1), (1, 2), (2, 3), (3, 0)]).to_ising()
print("ring: ground energy", ring.ground_state()[0], "= minus", 4, "cut edges")
```

Table 3.1 The QUBO with $\boldsymbol{Q} = \left(\begin{smallmatrix} 1 & -2 \\ -2 & 1 \end{smallmatrix}\right)$, $C(\boldsymbol{x}) = x_1 + x_2 - 4x_1x_2$, and its Ising form $H(\boldsymbol{z}) = -z_1z_2 + \frac{1}{2}(z_1+z_2)$, $c = 0$, by Eq. (3.5). From `qubo.py`.

$\boldsymbol{x}$	$\boldsymbol{z}$	$C(\boldsymbol{x})$	$H(\boldsymbol{z})+c$
$(0,0)$	$(+1,+1)$	0	0
$(0,1)$	$(+1,-1)$	1	1
$(1,0)$	$(-1,+1)$	1	1
$(1,1)$	$(-1,-1)$	-2	-2

From spins to qubits A classical spin $z_i = \pm 1$ is the eigenvalue of the Pauli operator Z_i on the computational basis states $|0\rangle$ and $|1\rangle$ of qubit i, and the substitution $z_i \to Z_i$ turns Eq. (3.4) into the *cost Hamiltonian*

$$H_C = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j + cI, \qquad H_C|\boldsymbol{x}\rangle = C(\boldsymbol{x})|\boldsymbol{x}\rangle. \tag{3.6}$$

It is diagonal in the computational basis, every bit string is an eigenstate with its cost as the eigenvalue, and the optimal bit string is the ground state. The whole of the chapter is about preparing a state of n qubits that has a large overlap with that ground state. We use the conventions of Chapter 1: qubit 0 is the leftmost factor, $|0\rangle \leftrightarrow z = +1 \leftrightarrow x = 0$.

MaxCut The example that every discussion of QAOA starts from is *MaxCut*. Given a graph $G = (V, E)$ with weights w_{ij} on its edges, split the vertices into two sets so that the total weight of the edges running between the sets, the *cut*, is as large as possible. With $x_i = 1$ for the vertices of one set, an edge is cut when $x_i \neq x_j$, which the quadratic expression $x_i + x_j - 2x_ix_j$

detects, so the cut and the cost to be minimized are

$$\mathrm{Cut}(\boldsymbol{x}) = \sum_{(i,j)\in E} w_{ij}\,(x_i + x_j - 2x_i x_j)\,, \qquad C(\boldsymbol{x}) = -\mathrm{Cut}(\boldsymbol{x}). \tag{3.7}$$

The substitution of Eq. (3.3) gives $x_i + x_j - 2x_i x_j = (1 - z_i z_j)/2$ and therefore

$$H_C = -\frac{1}{2} \sum_{(i,j)\in E} w_{ij}\,(I - Z_i Z_j)\,, \tag{3.8}$$

an antiferromagnetic Ising model, $J_{ij} = w_{ij}/2$, without fields and with the constant $c = -\frac{1}{2}\sum w_{ij}$; the ground-state energy is minus the maximum cut. MaxCut is NP-hard; the best classical algorithm with a guarantee, the semidefinite relaxation of Goemans and Williamson [56], achieves an approximation ratio of 0.878 on every graph, and it is widely believed that no polynomial algorithm does better in the worst case. Our running example is the ring of four vertices, C_4, with edges $(0,1)$, $(1,2)$, $(2,3)$, $(3,0)$ and unit weights, whose Hamiltonian $H_C = \frac{1}{2}(Z_0Z_1 + Z_1Z_2 + Z_2Z_3 + Z_3Z_0) - 2$ has the ground energy -4, all four edges cut, at the two alternating strings 0101 and 1010; the program `qubo.py` checks this against the cut of every bit string. We follow the sign convention of Eq. (3.8), minimizing $\langle H_C\rangle$, throughout the chapter, since it is the one in which QUBO, the double well and the portfolio all appear; the original paper and the companion volume maximize the cut operator $-H_C$ instead, which amounts to the sign change $\gamma \to -\gamma$ in everything that follows.

3.3 The algorithm

The variational principle For any state, $\langle\psi|H_C|\psi\rangle \geq E_0$, with equality only in the ground-state eigenspace, and the expectation value of a diagonal operator is the average cost over the measurement distribution,

$$E = \langle\psi|H_C|\psi\rangle = \sum_{\boldsymbol{x}} C(\boldsymbol{x})\,|\langle\boldsymbol{x}|\psi\rangle|^2. \tag{3.9}$$

A variational algorithm restricts $|\psi\rangle$ to a family $|\psi(\boldsymbol{\theta})\rangle$ prepared by a circuit, minimizes Eq. (3.9) over $\boldsymbol{\theta}$ with a classical optimizer, and reads out bit strings from the optimized state. The variational quantum eigensolver of the companion volume [1] does this for molecular and nuclear Hamiltonians with chemistry-inspired or hardware-efficient circuits. QAOA is a variational quantum eigensolver whose circuit is dictated by the problem.

The mixer and the ansatz The second ingredient is the *mixing Hamiltonian*

$$H_M = \sum_i X_i, \tag{3.10}$$

a transverse field. It does not commute with H_C, so that alternating the two generates transitions between bit strings, and the ground state of $-H_M$ is the uniform superposition $|+\rangle^{\otimes n} = H^{\otimes n}|0\rangle^{\otimes n} = 2^{-n/2}\sum_{\boldsymbol{x}}|\boldsymbol{x}\rangle$, which is also the state of complete ignorance about the answer. The depth-p QAOA state is

$$|\psi_p(\boldsymbol{\gamma},\boldsymbol{\beta})\rangle = \prod_{k=1}^{p} e^{-i\beta_k H_M}\, e^{-i\gamma_k H_C}\,|+\rangle^{\otimes n}, \tag{3.11}$$

with the factor $k = 1$ acting first, and the $2p$ angles $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$, $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$ are the variational parameters. Each layer first applies the *phase separator* $U_C(\gamma) = e^{-i\gamma H_C}$, which is diagonal and writes the cost of every bit string into its phase, $U_C(\gamma)|\boldsymbol{x}\rangle = e^{-i\gamma C(\boldsymbol{x})}|\boldsymbol{x}\rangle$, and then the *mixer* $U_M(\beta) = e^{-i\beta H_M} = \bigotimes_i R_x(2\beta)$, a product of single-qubit rotations, which turns the phases into amplitudes by interference. The algorithm minimizes $E_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \psi_p | H_C | \psi_p \rangle$, and the claim of Farhi *et al.* is that $\min E_p \to E_0$ as $p \to \infty$. The landscape is 2π-periodic in every γ_k when the costs are integers, as in unweighted MaxCut, and π-periodic in every β_k, since $e^{-i\pi H_M} = (-1)^n I$; for a cost without field terms, such as Eq. (3.8), the period in β is even $\pi/2$, since $e^{-i(\pi/2)H_M} = (-i)^n X^{\otimes n}$ is a global phase times the flip of all bits, which leaves such an H_C invariant.

Where the ansatz comes from The form of Eq. (3.11) is not a guess. Adiabatic quantum computation [57, 58] prepares the ground state of H_C by starting in the ground state of H_M and evolving under the interpolating Hamiltonian

$$H(s) = (1-s)\,(-H_M) + s\,H_C, \qquad s = t/T \in [0,1], \tag{3.12}$$

for a time T long compared with the inverse square of the smallest gap of $H(s)$; the adiabatic theorem then keeps the system in the instantaneous ground state, and the final state is the answer. (The sign of H_M is chosen so that $|+\rangle^{\otimes n}$ is its ground state; it is a convention, since $\beta \to -\beta$ undoes it.) Discretize the evolution into p steps of length $\Delta t = T/p$ at $s_k = k/p$, and split each step by the first-order Trotter formula of the companion volume,

$$e^{-i\Delta t\, H(s_k)} = e^{+i\Delta t\,(1-s_k)H_M}\, e^{-i\Delta t\, s_k H_C} + \mathcal{O}\big(\Delta t^2\,[H_C, H_M]\big)\,. \tag{3.13}$$

The product over k is exactly Eq. (3.11) with the *schedule* $\gamma_k = s_k \Delta t$, $\beta_k = -(1-s_k)\Delta t$. QAOA takes this circuit and frees the angles: instead of following the schedule, which is only guaranteed to work for a very long T and therefore a very large p, it lets a classical optimizer choose $\boldsymbol{\gamma}, \boldsymbol{\beta}$ for the p at hand. At $p \to \infty$ the adiabatic schedule is among the admissible parameters and the minimum of E_p reaches E_0; at small p the optimized angles are usually nothing like a schedule, and the algorithm is a different object, closer to a learned control protocol [59, 60] than to an approximation of adiabatic evolution. The commutator in Eq. (3.13) is what makes the circuit non-trivial: for MaxCut, with $[Z_a, X_b] = 2i\delta_{ab} Y_a$, $[H_C, H_M] = i\sum_{(i,j)\in E} w_{ij}(Y_i Z_j + Z_i Y_j)$, and it is this operator that generates entanglement between the qubits of an edge.

The circuit Both unitaries have an exact, short circuit. Each term of H_C is a Pauli string that is diagonal in the computational basis, and the terms commute, so $U_C(\gamma)$ is a product of one gate per term: $e^{-i\gamma h_i Z_i} = R_z(2\gamma h_i)$ for a field, and for a coupling

$$e^{-i\gamma J_{ij} Z_i Z_j} = \mathrm{CNOT}_{ij}\, R_z^{(j)}(2\gamma J_{ij})\, \mathrm{CNOT}_{ij}, \tag{3.14}$$

the same gadget as the ZZ feature map of Chapter 2: the first CNOT writes the parity $z_i z_j$ into qubit j, the rotation applies the phase, and the second CNOT restores the qubit. The mixer is $R_x(2\beta)$ on every qubit. Figure 3.1 draws one layer for a path of three vertices, and the gate count of a layer is

$$N_{\mathrm{CNOT}} = 2\,|E|, \qquad N_{R_z} = |E| + n_h, \qquad N_{R_x} = n, \tag{3.15}$$

with n_h the number of nonzero fields, plus the n Hadamards of the initial state. Every gate of the phase separator can be applied in parallel with the gates of the edges that share no vertex, so that the depth of a layer is set by the degree of the graph rather than by its size. On the simulator, we exploit a different structure: the phase separator is an elementwise phase on the 2^n amplitudes, and the mixer is diagonal in the X basis,

$$e^{-i\beta \sum_q X_q} = H^{\otimes n}\, e^{-i\beta \sum_q Z_q}\, H^{\otimes n}, \tag{3.16}$$

so that one layer costs two Hadamard transforms and two phases. This is how the class `QAOA` of `qaoa.py` prepares Eq. (3.11); stripped of the bookkeeping for the parameter shifts of Section 3.5, its core is

```
import numpy as np
from qubo import QUBO, bitstrings
ising = QUBO.maxcut(4, [(0, 1), (1, 2), (2, 3), (3, 0)]).to_ising()
n, cost = ising.n, ising.energies()          # C(z) for every bit string
z = 1 - 2 * bitstrings(n)                    # z_q = +-1 per bit string and qubit
H1 = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
Hn = H1
for _ in range(n - 1):
    Hn = np.kron(Hn, H1)                     # H^{(x)n}

def qaoa_state(gammas, betas):
    psi = np.full(2 ** n, 2 ** (-n / 2), dtype=complex)   # |+>^n
    for gamma, beta in zip(gammas, betas):
        psi = np.exp(-1j * gamma * cost) * psi            # phase separator
        psi = Hn @ (np.exp(-1j * beta * z.sum(1)) * (Hn @ psi)) # mixer, Eq. (3-mixerwht)
    return psi

psi = qaoa_state([np.pi / 4], [3 * np.pi / 8])
print("E =", np.real(np.vdot(psi, cost * psi))) # -3: three of four edges cut
```

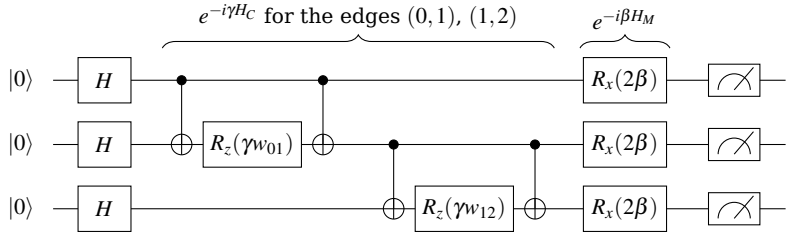


Fig. 3.1 One layer of the QAOA circuit for MaxCut on the path $0-1-2$ (qubit 0 on the top wire). After the Hadamards that prepare $|+\rangle^{\otimes 3}$, each edge contributes the gadget of Eq. (3.14), CNOT, $R_z(2\gamma J_{ij}) = R_z(\gamma w_{ij})$ on the target, CNOT, the constant of H_C being a global phase; the mixer is one $R_x(2\beta)$ per qubit. Deeper circuits repeat the two blocks with new angles before the measurement.

The loop The classical optimizer sees only the function $E_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$, estimated on a quantum computer by preparing the state, measuring all qubits, and averaging the cost of the outcomes over S shots, with the statistical error $\sigma_C/\sqrt{S}$ of Chapter 1. The optimizer may be gradient-free (Nelder–Mead, COBYLA, or the stochastic approximation of the companion volume) or use the gradients of Section 3.5; since the landscape has many local minima, it is run from several random starting points, and since the optimal angles at depth p are a good start for depth $p+1$ [60], we also start the depth-$(p+1)$ search from the depth-p optimum with one layer appended (the *warm start*). The output is not the energy but a bit string: the optimized state is sampled, and the best bit string among the samples is returned. Since the state is a superposition, this best string is usually better than the average, and the probability of sampling an optimal string is a more honest figure of merit than the approximation ratio of the energy.

3.4 The algorithm at work

3.4.1 One edge, by hand

The smallest instance, two vertices and one edge, can be solved completely in closed form, and the calculation shows how the two layers of the circuit cooperate. With $H_C = -\frac{1}{2}(I - Z_0Z_1)$ and $H_M = X_0 + X_1$ the $p = 1$ state is $|\psi\rangle = e^{-i\beta(X_0+X_1)}e^{-i\gamma H_C}|{+}{+}\rangle$, and we need $\langle Z_0Z_1\rangle$. Rather than computing the state, we move the operator through the circuit, the Heisenberg picture. The mixer rotates each Z about the x axis,

$$e^{i\beta X} Z e^{-i\beta X} = Z\, e^{-2i\beta X} = \cos(2\beta)\, Z + \sin(2\beta)\, Y, \tag{3.17}$$

(using $ZX = iY$ and that Z anticommutes with X), so that

$$U_M^\dagger Z_0Z_1 U_M = \cos^2(2\beta)\, Z_0Z_1 + \sin(2\beta)\cos(2\beta)\,(Z_0Y_1 + Y_0Z_1) + \sin^2(2\beta)\, Y_0Y_1.$$

The phase separator, $e^{-i\gamma H_C} = e^{i\gamma/2}e^{-i\gamma Z_0Z_1/2}$, commutes with Z_0Z_1 and with Y_0Y_1 and leaves them unchanged; its generator Z_0Z_1 anticommutes with Z_0Y_1, for which

$$U_C^\dagger Z_0Y_1\, U_C = Z_0Y_1\, e^{-i\gamma Z_0Z_1} = \cos\gamma\, Z_0Y_1 - i\sin\gamma\, Z_0Y_1Z_0Z_1 = \cos\gamma\, Z_0Y_1 + \sin\gamma\, X_1,$$

since $Y_1Z_1 = iX_1$, and likewise $Y_0Z_1 \to \cos\gamma\, Y_0Z_1 + \sin\gamma\, X_0$. In $|{+}{+}\rangle$ every operator containing a Y or a Z has expectation zero and $\langle X_0\rangle = \langle X_1\rangle = 1$, so that $\langle Z_0Z_1\rangle = 2\sin(2\beta)\cos(2\beta)\sin\gamma = \sin(4\beta)\sin\gamma$, and

$$E_1(\gamma,\beta) = -\frac{1}{2}\,(1 - \langle Z_0Z_1\rangle) = -\frac{1}{2} + \frac{1}{2}\sin(4\beta)\sin\gamma. \tag{3.18}$$

The minimum $E_1 = -1$, the exact ground energy, is reached at $\sin(4\beta)\sin\gamma = -1$, for instance $\beta^* = \pi/8$, $\gamma^* = 3\pi/2$ (equivalently $-\pi/2$). The program `qaoa.py` checks Eq. (3.18) against the simulator on a grid, to 10^{-15}, and prints the optimal state, $\frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$ up to a phase: the equal superposition of the two optimal cuts, a Bell state, with a single measurement giving an optimal bit string with certainty. One layer suffices because the problem has one edge; we shall return to this state in Section 3.6, where the two qubits are two electron spins.

The same Heisenberg calculation on a general graph gives the $p = 1$ energy edge by edge [52]: the expectation of Z_iZ_j depends only on the edges adjacent to (i, j), and for a triangle-free graph in which i and j have degrees d_i and d_j, $\langle Z_iZ_j\rangle = \frac{1}{2}\sin(4\beta)\sin\gamma(\cos^{d_i-1}\gamma + \cos^{d_j-1}\gamma)$, which for the ring, $d = 2$, gives

$$E_1^{\text{ring}}(\gamma,\beta) = -2 + \sin(4\beta)\sin(2\gamma) \qquad \text{for } C_4, \tag{3.19}$$

also verified by `maxcut.py`. Its minimum is -3: one layer cuts three of the four edges, an approximation ratio of $3/4$. This *locality* is general. At depth p the expectation of an edge term depends only on the subgraph within distance p of the edge, its light cone, because each layer of the circuit spreads the operator by one edge; the QAOA energy at fixed p is therefore a sum of local contributions, and the algorithm cannot see global structure, such as whether the graph is bipartite, until p is comparable to the diameter of the graph [61, 62].

3.4.2 The ring and a six-vertex graph

Table 3.2 and Fig. 3.2 show the algorithm on the ring C_4, with eight random starts of the BFGS optimizer per depth and the warm start from the previous depth. At $p = 1$ the optimizer finds

$\gamma = 3\pi/4$, $\beta = \pi/8$ (mod the periods), one of the two minima of Eq. (3.19), with $E = -3$ and a probability 0.53 of reading one of the two optimal strings; the landscape in the first panel has four extrema, two equivalent minima and two equivalent maxima, related by the symmetries $(\gamma,\beta) \to (\gamma+\pi/2, \beta+\pi/4)$ of the ring. At $p = 2$ the state is exactly the superposition of the two optimal cuts, $E = -4$ and $P_{\text{opt}} = 1$: the light cone of two layers covers the whole ring, and two layers are enough to prepare the ground state. This is the general pattern on small bipartite graphs and should not be mistaken for a property of the algorithm.

Table 3.2 QAOA on MaxCut of the ring C_4 (maximum cut 4) and of the six-vertex graph with edges $(0,1),(1,2),(2,3),(3,4),(4,5),(5,0),(0,2),(1,4)$ (maximum cut 7 of 8). Energy $\langle H_C \rangle$, approximation ratio $\langle H_C \rangle / E_0$ and probability of an optimal string at the optimized angles, eight random starts plus the warm start. From `maxcut.py`.

	ring C_4			six-vertex graph		
p	$\langle H_C \rangle$	ratio	P_{opt}	$\langle H_C \rangle$	ratio	P_{opt}
1	-3.000	0.750	0.531	-5.469	0.781	0.187
2	-4.000	1.000	1.000	-6.150	0.879	0.364
3	-4.000	1.000	1.000	-6.455	0.922	0.542
4				-6.680	0.954	0.768

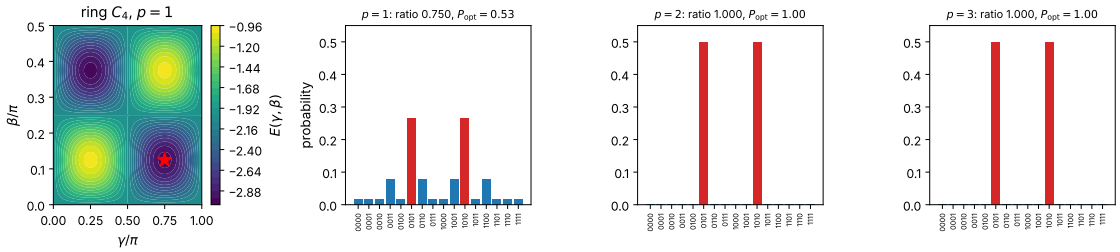


Fig. 3.2 MaxCut on the ring C_4. Left: the $p = 1$ landscape $E_1(\gamma,\beta)$ of Eq. (3.19) with the optimum found by the optimizer (star). Right: the output distributions of the optimized states for $p = 1,2,3$; the two optimal strings are in red. From `maxcut.py`.

The second half of Table 3.2 is a graph with a triangle, the six-ring with the chords $(0,2)$ and $(1,4)$, which is not bipartite, so that no cut takes all eight edges; the maximum is seven, reached by two strings. Here the ratio climbs from 0.78 at $p = 1$ to 0.95 at $p = 4$, and the probability of sampling an optimal string from 0.19 to 0.77, with no sign of the exact solution at any depth we tried. The light-cone argument says why: already a two-layer circuit on a graph of diameter two can see everything, but seeing is not solving, and the optimized angles at small p balance the terms of Eq. (3.9) rather than satisfying them. Each layer on this graph costs 16 CNOTs, 8 R_z and 6 R_x gates, by Eq. (3.15).

3.4.3 Performance, landscape and cost

What is known in general is less than one might hope. At $p = 1$ the ratio for unweighted 3-regular graphs is at least 0.6924 [52], below the 0.878 of Goemans and Williamson, and the bounds improve slowly with p; for worst-case instances the locality argument shows that a constant p cannot beat the classical guarantee, and that p must grow with the size of the

graph [62]. On typical instances the algorithm does much better than its worst-case bounds, and the p needed for a given ratio has been studied numerically up to tens of qubits [60]; an early large hardware demonstration ran MaxCut on up to 23 qubits [63], with the noise of the device visible in a performance that peaked at $p = 3$ and decreased beyond. Whether QAOA at any feasible depth offers an advantage over the best classical heuristics for optimization is an open question, and the case for it rests on the facts that the circuit is shallow, that its output cannot in general be sampled classically even at $p = 1$ [61], and that it is a natural fit for the hardware.

The landscape $E_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ is a trigonometric polynomial in the angles, smooth and periodic. At $p = 1$ it is simple enough to scan, as Fig. 3.2 did, and often to write down. As p grows it acquires the many local minima of a glassy function, and the optimization becomes the hard part of the algorithm; the warm start and the observation that the optimal angles vary smoothly with p and transfer between instances of the same class [60, 64] are the practical answers. The barren plateaus of Chapter 1 are milder here than for a generic circuit: the cost is a sum of two-body terms, the circuit has only two parameters per layer, and for constant p the gradient of an edge term involves only its light cone, so that the variance of the gradient decays polynomially rather than exponentially with n. They do return when p grows with n.

Finally, the cost in gates. A layer costs the $2|E| + |E| + n$ gates of Eq. (3.15), and with a two-qubit error rate ε the fidelity of the whole circuit decays as $(1-\varepsilon)^{2p|E|}$. For a graph with $n = 100$ vertices and $|E| = 500$ edges and $\varepsilon = 10^{-3}$, a single layer already has a thousand CNOTs and a fidelity of e^{-1}; the depths of Table 3.2 are affordable only on small graphs or with much better gates, and the demonstrations on hardware have accordingly used small p and error mitigation. This arithmetic, more than any theorem, sets the near-term scope of the algorithm.

3.5 Gradients

The optimizers themselves, gradient descent with and without momentum, Adam and BFGS, are those of the optimization chapter of the machine-learning text [9], implemented in `common/optimize.py`; what is specific to a circuit is how the gradient is obtained. The optimizer needs $\partial E/\partial\gamma_k$ and $\partial E/\partial\beta_k$, and the parameter-shift rule of Chapter 2 seems to supply them: for a gate $e^{-i\theta P}$ with $P^2 = I$, $\partial_\theta f = f(\theta + \pi/4) - f(\theta - \pi/4)$ (the rule of Chapter 2 written for $e^{-i\theta P}$ rather than $e^{-i\theta P/2}$, which halves the shift and removes the factor $\frac{1}{2}$). The generators here are H_C and H_M, and neither squares to the identity. The naive application of the rule to the whole angle,

$$\frac{\partial E}{\partial \gamma_k} \stackrel{?}{=} E(\gamma_k + \pi/4) - E(\gamma_k - \pi/4), \tag{3.20}$$

is therefore not a derivative but a finite difference with a large step, and Fig. 3.3 shows how wrong it is on the six-vertex graph: for γ it has the right sign most of the time and the wrong magnitude everywhere, and for β it returns zero along the whole line, because the two shifted mixers differ by $e^{-i(\pi/2)\sum_q X_q} = (-i)^n X^{\otimes n}$, the flip of all bits, which leaves a field-free H_C invariant, so that $E(\beta + \pi/4) = E(\beta - \pi/4)$ identically.

The correct rule follows from the structure of the unitaries. The phase separator is a product of commuting gates, one per Pauli term P_a of $H_C = \sum_a c_a P_a$, each with its own angle $\theta_a = \gamma c_a$, and the mixer is a product of one gate per qubit with angle β. The chain rule gives

$$\frac{\partial E}{\partial \gamma_k} = \sum_a c_a \left[E(\theta_a^{(k)} + \tfrac{\pi}{4}) - E(\theta_a^{(k)} - \tfrac{\pi}{4}) \right], \qquad \frac{\partial E}{\partial \beta_k} = \sum_q \left[E(\beta_q^{(k)} + \tfrac{\pi}{4}) - E(\beta_q^{(k)} - \tfrac{\pi}{4}) \right], \tag{3.21}$$

where the shift is applied to the angle of one term, or of one qubit, in layer k only, with all other angles at their values. Each evaluation is a circuit that differs from the original by one gate angle, so the rule is exact and hardware-compatible [11, 65, 66], and it costs $2(K+n)$ energy evaluations per layer, with K the number of terms; Fig. 3.3 shows it agreeing with central finite differences to 10^{-9}. (The finite-difference gradient is exact on a noiseless simulator and useless on hardware, where a step small enough to approximate the derivative is swamped by shot noise; the shift rule uses steps of order one.) Terms that commute and act on disjoint qubits can share a shift, which brings the cost down to the number of commuting groups, and for the two angles per layer of QAOA the whole gradient is cheap compared with the gradient of a generic circuit with hundreds of parameters. In `qaoa.py` the rule reads, with `self.terms` the list of terms $(c_a, P_a, \dots)$ and `energy(params, shift)` evaluating the circuit with one angle shifted:

```
import numpy as np
from qubo import QUBO
from qaoa import QAOA
q = QAOA(QUBO.maxcut(4, [(0, 1), (1, 2), (2, 3), (3, 0)]).to_ising(), p=1)
x, s = np.array([0.8, 0.5]), np.pi / 4

def gradient_shift(q, x):                    # Eq. (3—pertermshift)
    g = np.zeros(2 * q.p)
    for k in range(q.p):
        for a, (c, d, _) in enumerate(q.terms):
            g[k] += c * (q.energy(x, (k, "cost", a, +s)) − q.energy(x, (k, "cost", a, −s)))
        for qubit in range(q.n):
            g[q.p + k] += q.energy(x, (k, "mixer", qubit, +s)) − q.energy(x, (k, "mixer", qubit, −s))
    return g

print(gradient_shift(q, x), q.gradient_fd(x), q.gradient_global_shift(x))
```

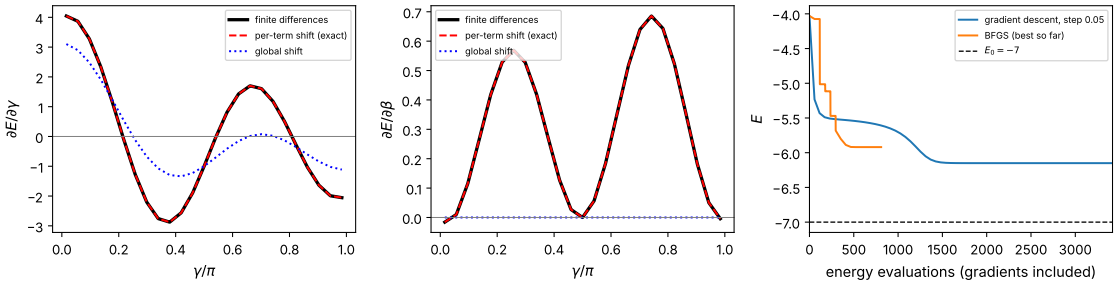


Fig. 3.3 Gradients of the $p=1$ energy of the six-vertex graph along the line $\beta=0.4$: central finite differences, the per-term parameter-shift rule of Eq. (3.21), and the naive global shift of Eq. (3.20), for γ (left) and β (middle). Right: plain gradient descent with the shift-rule gradient against BFGS from the same random start at $p=2$, with the cost counted in energy evaluations (a gradient costs $2p(K+n)=56$ of them). From `maxcut.py`.

The third panel of Fig. 3.3 compares two optimizers fed with the same gradient from the same start on the six-vertex graph at $p=2$. Plain gradient descent with step 0.05 reaches -6.15, the best value of Table 3.2, after sixty steps and 3420 energy evaluations; BFGS converges in fifteen iterations and 865 evaluations, but to a local minimum at -5.92, which a second start would have fixed. Both behaviors are typical: the landscape rewards a cheap optimizer with many restarts over an expensive one with few, and on hardware, where every evaluation costs thousands of shots, the count of evaluations is the figure to watch.

3.6 A physical register: two electrons in a double quantum well

Everything so far was simulated on abstract qubits. The smallest QAOA instance, one edge, needs two of them, and there is a physical system in which the two qubits and the cost unitary are both native: two electrons, one in each well of a double quantum dot, coupled by the exchange interaction. This is the spin-qubit platform of Loss and DiVincenzo [67], in which the first exchange gates were demonstrated by Petta *et al.* [68] and which is reviewed in [69]; the following is a translation to it of a recent two-qubit QAOA experiment on a nitrogen-vacancy center [53].

The system Two spin-$\frac{1}{2}$ electrons are confined in the left and right wells L, R of a double-well potential, in the regime where the tunnel coupling t between the wells is small compared with the on-site Coulomb energy U and the detuning keeps one electron in each well (the $(1,1)$ charge configuration). The orbital degrees of freedom are then frozen and the low-energy Hilbert space is that of two spins, $\mathbb{C}^2 \otimes \mathbb{C}^2$, with the qubit encoding $|\uparrow\rangle = |0\rangle$, $|\downarrow\rangle = |1\rangle$ on each electron. Second-order tunneling, allowed by the Pauli principle only for antiparallel spins, produces the effective *exchange* Hamiltonian

$$H_{\mathrm{ex}} = J\,\boldsymbol{S}_L \cdot \boldsymbol{S}_R = \frac{J}{4}\left(X_L X_R + Y_L Y_R + Z_L Z_R\right), \qquad J = \frac{4t^2}{U} > 0, \tag{3.22}$$

whose spectrum, printed by `double_well.py`, is the singlet $|S\rangle = \frac{1}{\sqrt{2}}(|\uparrow\downarrow\rangle - |\downarrow\uparrow\rangle)$ at $-3J/4$ and the three triplets $|T_+\rangle = |\uparrow\uparrow\rangle$, $|T_0\rangle = \frac{1}{\sqrt{2}}(|\uparrow\downarrow\rangle + |\downarrow\uparrow\rangle)$, $|T_-\rangle = |\downarrow\downarrow\rangle$ at $J/4$. The coupling J is set by the gate voltages that control t and the detuning, and can be switched on and off in nanoseconds.

The cost unitary from the exchange The MaxCut Hamiltonian of the single edge, $H_C = -\frac{1}{2}(I - Z_L Z_R)$, has the cost 0 on $|\uparrow\uparrow\rangle$ and $|\downarrow\downarrow\rangle$ and -1 on the two antiparallel configurations: the two cuts are the two ways of having one spin up and one down. Up to a constant and the factor $2/J$, H_C is the Ising part of Eq. (3.22), $\frac{J}{4} Z_L Z_R$, and if the flip-flop part $X_L X_R + Y_L Y_R$ is suppressed, which a Zeeman splitting between the two electrons larger than J does, free evolution under the exchange for a time t is the phase separator,

$$e^{-it\,(J/4) Z_L Z_R} = e^{-i\gamma H_C} \text{ (up to a phase)} \qquad \text{for} \quad t = \frac{2\gamma}{J}. \tag{3.23}$$

The mixer $R_x(2\beta)$ on each spin is a microwave pulse in electron spin resonance (ESR), the initial Hadamards are two $\pi/2$ pulses, the initial state $|\uparrow\uparrow\rangle$ is prepared by spin polarization, and the readout is spin-to-charge conversion followed by charge sensing, one measurement per electron. Table 3.3 lists the correspondence. Every step of Eq. (3.11) is a standard operation on the platform, and the whole $p = 1$ circuit is a pulse sequence of a few hundred nanoseconds.

Table 3.3 The $p = 1$ QAOA circuit for MaxCut on one edge and its implementation on a two-electron double quantum dot.

Circuit step	Double-dot operation
prepare $\lvert 00\rangle = \lvert\uparrow\uparrow\rangle$	spin-selective tunneling or optical pumping
$H \otimes H$	two simultaneous $\pi/2$ ESR pulses
$e^{-i\gamma H_C}$, Eq. (3.14)	exchange pulse of duration $t = 2\gamma/J$, Eq. (3.23)
$R_x(2\beta) \otimes R_x(2\beta)$	two individual ESR rotations
measure Z_L, Z_R	spin-to-charge conversion, charge sensing

What the experiment measures The landscape of the $p = 1$ energy is the closed form of Eq. (3.18), and Fig. 3.4 shows it with the simulator on top: the minimum $E = -1$ at $(\gamma, \beta) = (3\pi/2, \pi/8)$, the maximum 0 at $(\pi/2, \pi/8)$, and the two sinusoidal slices through the optimum. At the optimum the state is the triplet $|T_0\rangle$, with the overlap $|\langle T_0|\psi^*\rangle|^2 = 1.000000$ printed by the program; it is not the singlet, the ground state of the full exchange Hamiltonian, because the flip-flop terms that single out the singlet are precisely the ones that the circuit does not use. Figure 3.5 shows the output distribution at the optimum and at a point off it, $(3\pi/2, \pi/16)$, where $E = -0.854$ and the two cut states carry 0.427 each, together with 100 simulated single-shot spin readouts at the optimum: 50 of $|\uparrow\downarrow\rangle$, 50 of $|\downarrow\uparrow\rangle$, none of the parallel configurations. On a real device the two cut states would be read with a probability below one by the readout fidelity and the decoherence during the exchange pulse, and the landscape would be measured as a noisy surface over a grid of pulse parameters; the exact result is the reference against which such a measurement is compared. Deeper circuits change nothing here, as the last lines of the program confirm for $p = 2, 3$: one edge is solved by one layer.

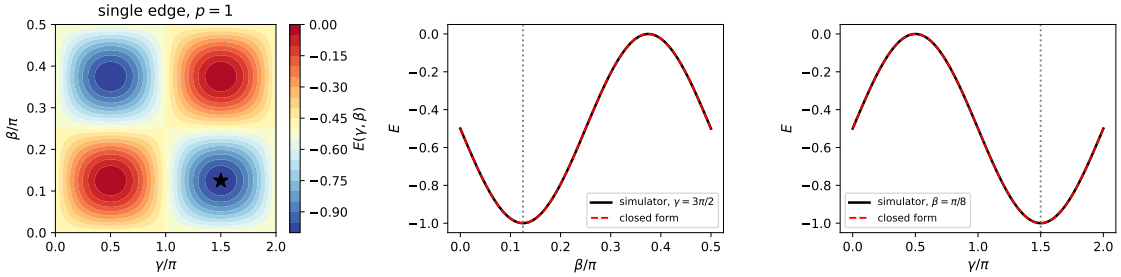


Fig. 3.4 The $p = 1$ energy of the two-electron (single-edge) problem, $E_1 = -\frac{1}{2} + \frac{1}{2}\sin(4\beta)\sin\gamma$, Eq. (3.18): the landscape from the simulator with the optimum marked (left), and the slices through the optimum at fixed $\gamma = 3\pi/2$ (middle) and fixed $\beta = \pi/8$ (right) against the closed form. From `double_well.py`.

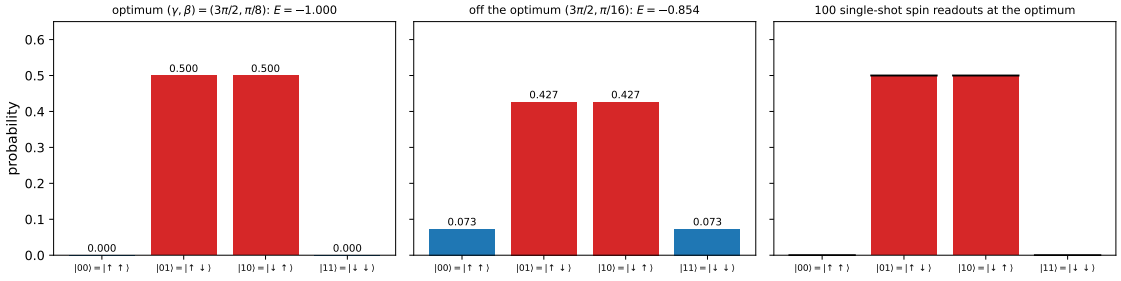


Fig. 3.5 Output distributions of the two-electron QAOA state at the optimum (left), off the optimum (middle), and 100 simulated single-shot readouts of the two spins at the optimum with the exact probabilities as bars (right). The cut states, antiparallel spins, are in red. From `double_well.py`.

3.7 An application: selecting a portfolio

The pipeline cost function → QUBO → Ising → cost Hamiltonian → QAOA is the same whatever the problem, and the domain enters only through the matrix $\boldsymbol{Q}$. To see it on a problem far from graphs, we take the oldest problem of quantitative finance, Markowitz's mean-variance

portfolio selection [70], in the combinatorial version that quantum-computing proposals for finance [71, 72] usually consider.

The problem There are n assets with expected returns μ_i over some period and a covariance matrix Σ_{ij} of their returns; both are estimated from price histories. A portfolio that holds asset i or not is a bit string, and the investor wants a high return and a low variance,

$$\min_{\boldsymbol{x}\in\{0,1\}^n} \; -\boldsymbol{\mu}^T\boldsymbol{x} + \lambda\, \boldsymbol{x}^T\boldsymbol{\Sigma}\boldsymbol{x} \qquad \text{subject to} \qquad \sum_i x_i = K, \tag{3.24}$$

with a risk-aversion parameter $\lambda > 0$ and a budget of exactly K assets. The constraint is the new element. QUBO has no constraints, and the standard device [55] is to add the squared violation as a penalty,

$$C(\boldsymbol{x}) = -\boldsymbol{\mu}^T\boldsymbol{x} + \lambda\, \boldsymbol{x}^T\boldsymbol{\Sigma}\boldsymbol{x} + A\Big(\sum_i x_i - K\Big)^2, \tag{3.25}$$

with a strength A large enough that no infeasible portfolio has a lower cost than the best feasible one. Expanding the square and using $x_i^2 = x_i$ gives the QUBO of Eq. (3.2) with

$$Q_{ii} = -\mu_i + \lambda\Sigma_{ii} + A(1-2K), \qquad Q_{ij} = \lambda\Sigma_{ij} + A \;\; (i \neq j), \qquad c_0 = AK^2, \tag{3.26}$$

and Eq. (3.5) turns it into an Ising model whose couplings $J_{ij} = (\lambda\Sigma_{ij} + A)/2$ are all positive (the penalty and the covariances both discourage holding two assets at once) and whose fields are set by the returns. The couplings, being an empirical covariance matrix, are as good as random: the portfolio problem is a small spin glass, and the frustration that makes spin glasses hard is here the fact that the assets that return most are correlated with each other.

Two data sets The program `portfolio.py` uses two data sets of four assets each, so that $n = 4$ qubits, $K = 2$, and all 2^4 portfolios can be listed. The first is synthetic, with annualized returns $(0.12, 0.10, 0.08, 0.15)$ and a covariance matrix with variances 0.06–0.10 and small positive covariances. The second is real: 164 weekly closing prices of Apple, Microsoft, Amazon and Alphabet from January 2020 to December 2023, stored in `sp500_weekly.csv` and drawn in Fig. 3.6, from which the weekly log returns give annualized mean returns of 30.9, 24.6, 26.3 and 30.7 percent, volatilities of 30.8, 24.4, 30.7 and 29.2 percent, and correlations between 0.80 and 0.94: four large technology stocks that moved together through the pandemic, the 2022 correction and the 2023 recovery. With $\lambda = 2$ the six feasible pairs have the costs of Table 3.4; Microsoft and Alphabet, the two with the lowest volatility and the second-highest return respectively, are the best pair, by a margin of 0.02 over Apple and Microsoft, and the worst pair, Apple and Amazon, is 0.11 away. The penalty $A = 0.3$ exceeds every such difference and the ground state of the cost Hamiltonian is the feasible optimum, as the program verifies by brute force.

QAOA on the stocks Table 3.5 and Fig. 3.7 give the result for $p = 1$ to 4. Since the energies are no longer integers the approximation ratio is not a useful measure, and we report instead the fraction, closed by the optimized state, of the gap between the energy of the initial state $|+\rangle^{\otimes n}$ (the average cost over all sixteen portfolios) and the ground energy,

$$g_p = \frac{E_{\text{uniform}} - E_p}{E_{\text{uniform}} - E_0}, \tag{3.27}$$

together with the probability of sampling a feasible portfolio and the probability of sampling the optimal one. One layer closes 70 percent of the gap and concentrates 65 percent of the probability on the six feasible strings; four layers reach 90 percent and 97 percent, and the optimal pair, Microsoft and Alphabet, is the most probable outcome with 0.26. The distribu-

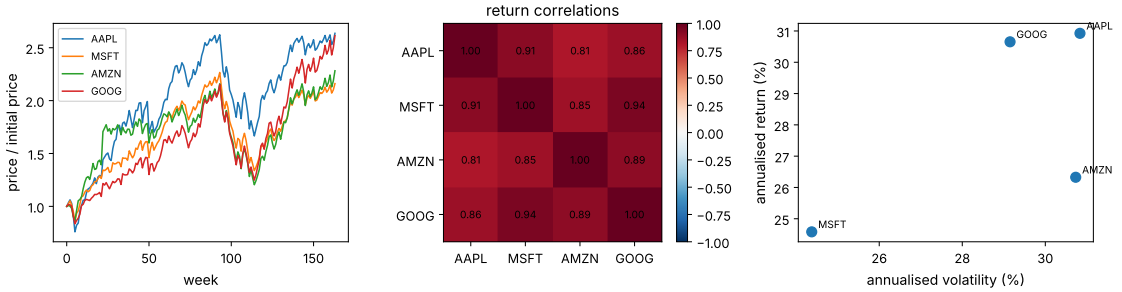


Fig. 3.6 The data of the portfolio problem: weekly closing prices of the four stocks relative to January 2020 (left), the correlation matrix of their weekly log returns (middle), and their annualized return against volatility (right). From `portfolio.py`.

Table 3.4 The six feasible portfolios ($K = 2$) of the two data sets and their mean-variance cost $-\boldsymbol{\mu}^T\boldsymbol{x} + \lambda \boldsymbol{x}^T\boldsymbol{\Sigma}\boldsymbol{x}$, with $\lambda = 1$ (synthetic) and $\lambda = 2$ (stocks). From `portfolio.py`.

synthetic assets			stocks		
portfolio	$\boldsymbol{x}$	cost	portfolio	$\boldsymbol{x}$	cost
B+D	0101	−0.060	MSFT+GOOG	0101	0.003
C+D	0011	−0.040	AAPL+MSFT	1100	0.026
A+D	1001	−0.020	AAPL+GOOG	1001	0.053
A+C	1010	−0.020	MSFT+AMZN	0110	0.054
B+C	0110	0.000	AMZN+GOOG	0011	0.108
A+B	1100	0.000	AAPL+AMZN	1010	0.112

tions in Fig. 3.7 show what the algorithm does first: it learns the constraint. The penalty is the largest energy scale of the problem, the feasible strings are separated from the others by $A = 0.3$ while they differ among themselves by at most 0.11, and the first layers push the amplitude onto the feasible subspace, within which the ordering of the pairs is resolved only at larger p. This is the generic weakness of penalty methods: the constraint consumes the expressive power of the circuit, and the alternative, a mixer that preserves the number of ones, the XY mixer of the quantum alternating operator ansatz [73], keeps the state inside the feasible subspace from the start and needs no penalty at all (Exercise 7).

Table 3.5 QAOA for the two portfolio problems of Table 3.4 with the penalty $A = 0.3$: the energy, the fraction of the gap closed, Eq. (3.27), and the probabilities of sampling a feasible and the optimal portfolio from the optimized state (six random starts plus the warm start). From `portfolio.py`.

	synthetic assets ($E_0 = -0.060$)				stocks ($E_0 = 0.003$)			
p	E_p	g_p	P_{feas}	P_{opt}	E_p	g_p	P_{feas}	P_{opt}
1	0.072	0.63	0.74	0.13	0.151	0.70	0.65	0.11
2	−0.021	0.89	0.99	0.17	0.083	0.84	0.95	0.15
3	−0.023	0.90	1.00	0.18	0.073	0.86	0.95	0.15
4	−0.025	0.90	1.00	0.20	0.052	0.90	0.97	0.26

The strength of the penalty Table 3.6 varies A at $p = 2$. For $A = 0.03$ and 0.1 the ground state of H_C is the single stock Alphabet, a portfolio of the wrong size, because the gain from dropping a correlated asset exceeds the penalty, and the algorithm faithfully finds that wrong answer. For $A = 0.3$ the ground state is the right one and the feasible probability is 0.95. For

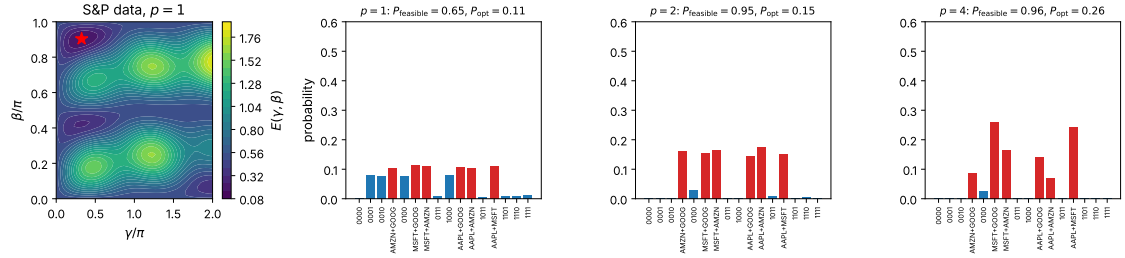


Fig. 3.7 The portfolio problem on the stock data. Left: the $p = 1$ landscape with the optimum found. Right: output distributions of the optimized states for $p = 1, 2, 4$; the six feasible portfolios are in red and labeled by their stocks. From `portfolio.py`.

$A = 1$ it is still the right one, but the feasible probability falls to 0.66 and the probability of the optimum hardly moves, because the penalty now dominates the landscape so completely that the angles are spent on it and the cost differences within the feasible subspace become invisible to two layers. The penalty must be large enough and no larger, and the right value is found by exactly the brute-force check that is impossible on the instances for which one would want a quantum computer; this is a known weakness of the method and one of the reasons for the constraint-preserving mixers mentioned above.

Table 3.6 The effect of the penalty strength A on the stock data at $p = 2$: the ground state of H_C and its budget, and the probabilities of a feasible portfolio, of the ground state, and of the best feasible pair MSFT+GOOG in the optimized QAOA state. From `portfolio.py`.

A	ground state	budget	P_{feas}	P(ground state)	P(MSFT+GOOG)
0.03	0001 (GOOG)	1	0.04	0.23	0.00
0.1	0001 (GOOG)	1	0.14	0.22	0.04
0.3	0101 (MSFT+GOOG)	2	0.95	0.15	0.15
1.0	0101 (MSFT+GOOG)	2	0.66	0.15	0.15

Thermal states and the variational state The Ising energies of the sixteen portfolios define a classical partition function and a thermal average,

$$\langle H \rangle_{\beta_T} = \frac{\sum_{\boldsymbol{z}} H(\boldsymbol{z})\, e^{-\beta_T H(\boldsymbol{z})}}{\sum_{\boldsymbol{z}} e^{-\beta_T H(\boldsymbol{z})}}, \tag{3.28}$$

which decreases from the mean cost at infinite temperature to the ground energy as $\beta_T \to \infty$; this is the energy of simulated annealing at temperature $1/\beta_T$. Figure 3.8 draws it with the QAOA energies on top: $p = 1$ matches the thermal state at $\beta_T \approx 2.4$, $p = 2$ at 6.4, $p = 4$ at 14.5. The comparison is suggestive rather than deep. A QAOA state is pure, prepared by a unitary, and its distribution over bit strings is not a Boltzmann distribution; but the effective temperature it corresponds to is a convenient way of stating how far a given depth has cooled the problem, and it makes the point that the mixer plays the role that thermal fluctuations play in annealing, with real-time interference in the place of imaginary-time damping. The quantum Boltzmann machines of Chapter 5 make the thermal state itself the object of the computation.

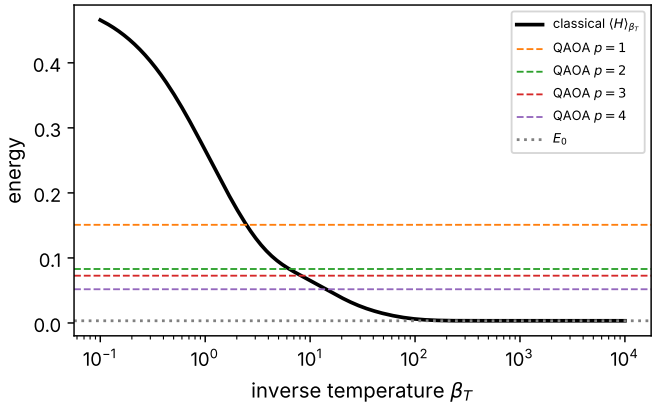


Fig. 3.8 The classical thermal energy of Eq. (3.28) of the stock-portfolio Ising model as a function of the inverse temperature, with the QAOA energies of Table 3.5 as horizontal lines. From `portfolio.py`.

3.8 QAOA as a machine-learning model

Return to the general model of Eq. (1.1), $f(\boldsymbol{x};\boldsymbol{\theta}) = \langle 0|U^\dagger W^\dagger \hat{O} W U|0\rangle$. In QAOA the observable is $\hat{O} = H_C$, the trainable circuit is the alternation of Eq. (3.11), and the data enter not through an encoding $U(\boldsymbol{x})$ of individual samples but through the Hamiltonian: H_C is built from a graph, a covariance matrix, a set of distances, and the circuit $e^{-i\gamma H_C(\text{data})}$ is a feature map of the whole data set at once. Three consequences follow.

Clustering is MaxCut The most direct learning task for the algorithm is unsupervised. Given N points $\boldsymbol{x}_i$, split them into two groups so that points in different groups are far apart: with the similarity graph of all pairs and the weights

$$w_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2, \tag{3.29}$$

the partition that maximizes the weight of the cut is the two-way clustering, a discrete relative of the problem that spectral clustering relaxes [74] (Exercise 9), and a qubit per point turns it into QAOA on a weighted complete graph. The program `clustering.py` does this for eight points drawn from two overlapping Gaussian clouds, Fig. 3.9: the maximum cut, found by brute force, coincides with the partition of the classical two-means algorithm, and QAOA with $p = 3$ returns that partition with probability 0.47, its two strings (a partition and its complement are the same clustering) being the most probable outcomes by nearly a factor of two. The approximation ratio is 0.60, 0.77 and 0.90 for $p = 1, 2, 3$; a dense graph on eight vertices has 28 edges, each layer 56 CNOTs, and the depth at which the optimal partition dominates the output is, as for the six-vertex graph, not small. The whole pipeline, from the points to the partition, is a few lines (with fewer restarts than the program and without its warm start, so the ratio printed here is lower than the 0.77 above):

```
import numpy as np
from datasets import two_blobs
from qubo import QUBO, bitstrings
from qaoa import QAOA
X, _ = two_blobs(n=8, separation=3.0, noise=0.7, rng=2)
edges = [(i, j) for i in range(8) for j in range(i + 1, 8)]
weights = [float(np.sum((X[i] - X[j]) ** 2)) for i, j in edges] # Eq. (3—clustering)
ising = QUBO.maxcut(8, edges, weights).to_ising()
ising.h, ising.J, ising.const = (t / np.mean(weights) for t in (ising.h, ising.J, ising.const))
q = QAOA(ising, p=2)
```

```
x, E, _ = q.optimize(restarts=4, rng=0, grad="fd")
best = np.argmax(q.probabilities(x))
print("partition", bitstrings(8)[best], "ratio", round(q.approximation_ratio(x), 3))
```

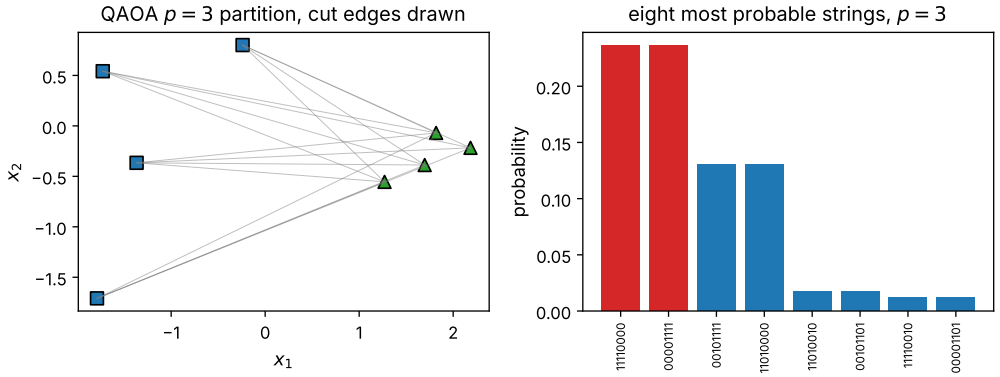


Fig. 3.9 Two-way clustering as MaxCut on the similarity graph of Eq. (3.29). Left: eight points colored by the most probable QAOA partition at $p = 3$, with the cut edges drawn. Right: the eight most probable output strings; the two optimal ones (a partition and its complement) are in red. From `clustering.py`.

QAOA as a structured neural network Written out, the QAOA state is $|\psi_p\rangle = \prod_k W_k(\beta_k) S_k(\gamma_k; \text{data})|+\rangle^{\otimes n}$ with a data-dependent layer $S_k = e^{-i\gamma_k H_C(\text{data})}$ followed by a trainable layer $W_k = e^{-i\beta_k H_M}$: this is a quantum neural network of the kind that Chapter 4 studies, with the data re-uploaded in every layer, two parameters per layer, and an architecture fixed by the problem rather than chosen. Its training is the same loop as that of Chapter 4, with the energy as the loss, the parameter-shift rule of Eq. (3.21) as the gradient, and the same restarts, warm starts and barren-plateau considerations. One can also encode a labeled data set in the cost, $H_C = -\sum_i y_i \hat{f}(\boldsymbol{x}_i)$ with $\hat{f}$ a sum of Pauli strings whose coefficients are functions of the sample, and train the alternation to classify; the variational classifier of Chapter 4 is the better-studied form of this idea.

Kernels and samplers Finally, the data-dependent state $|\psi_p(\boldsymbol{x})\rangle$ obtained by encoding one sample in $H_C(\boldsymbol{x})$ defines a quantum feature map in the sense of Chapter 2, with the fidelity kernel $|\langle\psi_p(\boldsymbol{x})|\psi_p(\boldsymbol{x}')\rangle|^2$ of Eq. (2.29), and the kernel-method theorem of Section 2.11 applies to QAOA as to any other model of the form of Eq. (1.1). And the optimized state, sampled in the computational basis, is a generative model over bit strings: a *Born machine* whose angles can be trained so that the samples follow a target distribution rather than concentrate on a ground state. Both views lead out of this chapter, the first back to kernels and the second forward to the Boltzmann machines of Chapter 5.

3.9 Summary

A combinatorial problem over bit strings with a quadratic cost is a QUBO, Eq. (3.2); the substitution $x_i = (1 - z_i)/2$ makes it an Ising model, Eq. (3.5), and $z_i \to Z_i$ a diagonal cost Hamiltonian whose ground state is the answer, Eq. (3.6). QAOA prepares the state of Eq. (3.11), p alternations of the phase separator $e^{-i\gamma H_C}$ and the transverse-field mixer $e^{-i\beta H_M}$ on the uniform superposition, and minimizes $\langle H_C\rangle$ over the $2p$ angles classically; the ansatz is Trotterized adiabatic evolution with the schedule freed, its circuit is a CNOT–R_z–CNOT gadget per coupling and an R_x per qubit, and the $p = 1$ energy of a single edge is $-\frac{1}{2} + \frac{1}{2}\sin(4\beta)\sin\gamma$, reaching

the exact Bell-state answer at $\beta = \pi/8$, $\gamma = 3\pi/2$. On a ring two layers are exact; on a graph with a triangle the ratio climbs slowly with p, and the light cone of the circuit explains both. The gradient requires the parameter-shift rule term by term, Eq. (3.21), because H_C and H_M have many eigenvalues; the naive rule is wrong. Two electron spins in a double quantum dot implement the one-edge instance natively, with the exchange interaction as the cost unitary. On a portfolio problem with real data the same pipeline works, learns the budget constraint before it learns the costs, and shows how the penalty strength must be tuned. Read as a learning model, QAOA is a quantum neural network with a problem-fixed architecture, a clustering algorithm, a feature map and a sampler, and it is the point from which Chapters 4 and 5 depart.

3.10 Exercises

1. **QUBO to Ising.** (a) Derive Eq. (3.5) from Eq. (3.2) and check it on Table 3.1. (b) Show that for MaxCut with unit weights the Ising constant is $-|E|/2$ and the couplings are $1/2$, and that the fields vanish. (c) Write the graph three-coloring problem of Lucas [55] as a QUBO with $3|V|$ bits and a penalty for vertices with no or several colors, and give its Ising coefficients.
2. **The adiabatic origin.** (a) Verify Eq. (3.13) to second order in Δt with the Baker–Campbell–Hausdorff formula and compute $[H_C, H_M]$ for a general Ising cost with fields. (b) Show that $|+\rangle^{\otimes n}$ is the ground state of $-H_M$ and that the adiabatic schedule corresponds to $\gamma_k = k\Delta t/p$, $\beta_k = -(1-k/p)\Delta t$. (c) With `qaoa.py`, evaluate the energy of the ring C_4 along the schedule for $p = 10, 20, 50$ and $T = p\Delta t$ with $\Delta t = 0.1, 0.3, 0.5$, and compare with the optimized energies of Table 3.2.
3. **The one-edge formula.** (a) Verify Eq. (3.17) by expanding $e^{-2i\beta X}$ and show $e^{i\beta X} Y e^{-i\beta X} = \cos(2\beta) Y - \sin(2\beta) Z$. (b) Complete the derivation of Eq. (3.18), including the vanishing of the $Y_0 Y_1$ and $Z_0 Z_1$ terms. (c) Derive the triangle-free formula quoted after Eq. (3.18) for an edge whose endpoints have degrees d_i and d_j, and from it Eq. (3.19). (d) Write the optimal one-edge state explicitly, including its phase, and show that it is $|T_0\rangle$ up to a global phase.
4. **Symmetries of the landscape.** (a) Show that for an integer-valued cost E_p is 2π-periodic in every γ_k, and that for a field-free cost it is $\pi/2$-periodic in every β_k. (b) Show $E_p(-\boldsymbol{\gamma}, -\boldsymbol{\beta}) = E_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ by time reversal (complex conjugation of the state). (c) Explain the four extrema of the ring landscape in Fig. 3.2 from (a) and (b).
5. **Gradients.** (a) Show that the two-point rule $\partial_\theta f = f(\theta + \pi/4) - f(\theta - \pi/4)$ holds for any expectation value of a gate $e^{-i\theta P}$ with $P^2 = I$, by writing f as $a + b\cos 2\theta + c \sin 2\theta$. (b) Derive Eq. (3.21) by the chain rule. (c) The global rule of Eq. (3.20) is exact for a generator with eigenvalues ± 1 only; show that H_M on n qubits has the eigenvalues $-n, -n+2, \ldots, n$ and explain why the global rule for β returned zero in Fig. 3.3. (d) Group the terms of the six-vertex cost into sets acting on disjoint qubits and determine the smallest number of energy evaluations that gives the exact gradient with respect to γ.
6. **The double well.** (a) Diagonalize Eq. (3.22) by hand and identify the singlet and the triplets. (b) Show that the singlet is an eigenstate of $Z_L Z_R$ with eigenvalue -1, hence a cut state, and explain why QAOA nevertheless prepares $|T_0\rangle$ rather than $|S\rangle$. (c) Add the flip-flop terms to the cost unitary, $e^{-itH_{\mathrm{ex}}}$ in the place of Eq. (3.23), and recompute the $p = 1$ landscape with `double_well.py`: is $E = -1$ still reached? (d) Model a readout error that flips each measured spin with probability ε and compute the probability of reading a cut state at the optimum as a function of ε.
7. **Constraints without penalties.** The XY mixer $H_{XY} = \sum_{i<j} (X_i X_j + Y_i Y_j)$ commutes with the number operator $\sum_i (I - Z_i)/2$. (a) Show this, and show that starting from a uniform super-

position of the strings with exactly K ones (a Dicke state) the QAOA state stays feasible for any angles. (b) Implement the XY mixer in `qaoa.py` (as a $2^n \times 2^n$ matrix exponential, since it is no longer a product of single-qubit gates) and rerun the stock-portfolio problem without penalty, comparing P_{opt} with Table 3.5. (c) How would the Dicke state be prepared on a quantum computer?

8. **Portfolios.** (a) Derive Eq. (3.26). (b) Give a lower bound on the penalty A in terms of $\boldsymbol{\mu}$, λ and $\boldsymbol{\Sigma}$ that guarantees a feasible ground state, and compare it with Table 3.6. (c) Repeat the stock experiment with $\lambda = 0.5$ and $\lambda = 5$ and explain the change in the optimal pair from the data of Fig. 3.6. (d) Estimate $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ from the first half of the price history only, optimize, and evaluate the chosen portfolio on the second half: how much of the in-sample advantage survives?
9. **Clustering.** (a) Show that maximizing the cut of Eq. (3.29) is equivalent to minimizing the sum of squared distances of the points to their cluster centroids (the two-means objective) when the clusters have equal size, and that it is not equivalent in general. (b) Add a penalty that enforces equal cluster sizes and rerun `clustering.py`. (c) Replace the squared distances by the Gaussian similarity $e^{-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/2\sigma^2}$, which is what spectral clustering uses, and show that the minimum cut of this graph, rather than the maximum, is then the clustering objective; adapt the sign of H_C accordingly.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter3/`, which build on the shared modules of `BookPrograms/common/` (`statevector`, `gates`, `optimize`, `datasets`, `bookplot`) and are imported by the notebook `Notebooks/chapter3/chapter3.ipynb`. All run on `numpy` and `matplotlib` alone.

`qubo.py` The classes `QUBO` (with the builders `maxcut` and `portfolio`, Eqs. (3.7) and (3.26), and `to_ising`, Eq. (3.5)) and `Ising` (energies of all bit strings, the Pauli terms of H_C, the brute-force ground state). As a script it checks Table 3.1 and the ring. Under a second.

`qaoa.py` The class `QAOA`: the state of Eq. (3.11) on the simulator by phases and Hadamard transforms, Eq. (3.16); the energy, the probabilities, the samples; the three gradients of Section 3.5; BFGS and gradient-descent optimization with restarts and the warm start; the gate count of Eq. (3.15). As a script it checks Eq. (3.18) and the gradient rules. One second.

`maxcut.py` The ring and the six-vertex graph: Table 3.2, the closed form of Eq. (3.19), Figs. 3.2 and 3.3. About a minute.

`double_well.py` The exchange Hamiltonian, the cost table, the closed form, Figs. 3.4 and 3.5, the sampled readouts and the depths $p = 1, 2, 3$. Five seconds.

`portfolio.py` The two portfolio problems: the data of `sp500_weekly.csv` and Fig. 3.6, Tables 3.4, 3.5 and 3.6, Figs. 3.7 and 3.8. About forty seconds.

`clustering.py` Two-way clustering as weighted MaxCut on eight points, Fig. 3.9, against brute force and two-means. Ten seconds.

Chapter 4
Quantum Neural Networks

Abstract A quantum neural network is the full hybrid model of this book: a data-encoding circuit, a trainable circuit of rotations and entangling gates, and a measured expectation value as the output, trained by a classical optimizer on a classical loss. This chapter defines the model and its data re-uploading form, shows that its output is a trigonometric polynomial in the encoded data whose frequencies are fixed by the encoding and whose coefficients are set by the trainable angles, and uses this single fact to read every experiment that follows. Training rests on the parameter-shift rule, which we derive together with its second-order and general forms; the same shift rules give the quantum Fisher information, the metric of the manifold of circuit states, from which the natural gradient, imaginary-time evolution, the time-dependent variational principle and the stochastic reconfiguration of variational Monte Carlo follow as one equation. Barren plateaus, the concentration of measure that flattens the loss of random deep circuits, are reproduced numerically with their exponential laws for local and global costs. The model is then trained on regression and classification problems against the kernel machines of Chapter 2 and a classical network, and as a scoring function for the portfolio problem of Chapter 3. The last part of the chapter turns to differential equations: the physics-informed quantum neural network, whose loss is the residual of an equation and whose derivatives with respect to the input come from the same shift rules, with boundary and initial conditions built into the trial function, applied to the Poisson equation in strong and Ritz form, to an initial-value problem and to the wave equation.

4.1 Introduction

The two preceding chapters used a quantum computer in two different ways. The quantum support-vector machine of Chapter 2 evaluated a kernel with a fixed circuit and left the learning to a convex classical program; the QAOA of Chapter 3 put all of its parameters into the circuit and used a classical optimizer to move them, but its input was a single Hamiltonian rather than a training set. The *quantum neural network* (QNN) of this chapter combines the two: a data-dependent encoding circuit, a trainable circuit with adjustable rotation angles, and a measured expectation value that plays the role of the network output. It is the general hybrid model of Eq. (1.1) with nothing left out, and it is the model that most of the current literature on quantum machine learning has in mind when it speaks of “training a quantum circuit.” It is also the model closest to the variational quantum eigensolver, and the reader of the companion volume [1] will find that its chapter on variational quantum algorithms, with the hardware-efficient ansatz, the parameter-shift rule, shot noise in the optimization loop and barren plateaus, has already introduced half of what follows; this chapter adds the data,

the Fourier picture that the data bring with them, the geometry of the parameter space, and the applications.

The name is a convenient misnomer. A classical neural network is a composition of affine maps and nonlinear activation functions, trained by backpropagation; a QNN is a product of unitary matrices, which is linear on the state and nonlinear in the parameters only through the cosines and sines of the rotation angles, and its "backpropagation" is a set of extra circuit runs at shifted angles. What the two share is the workflow of Fig. 1.1: a parametrized function, a loss on data, a gradient, an optimizer. What they do not share is the function class. The output of a QNN is a trigonometric polynomial in the encoded data whose frequencies are fixed by the encoding and whose coefficients are set by the trainable angles. This simple fact, made precise in Section 4.3, explains most of what a QNN can and cannot do, and we will return to it in every application.

The chapter is organized as follows. Section 4.2 defines the model, circuit by circuit, and introduces the data re-uploading architecture. Section 4.3 derives the Fourier picture. Section 4.4 is about training: loss functions, the parameter-shift rule and its generalizations, and the classical optimizers. Section 4.5 introduces the quantum Fisher information, the natural metric on the manifold of circuit states, and the natural-gradient and imaginary-time methods that follow from it, with a bridge to the stochastic reconfiguration of variational Monte Carlo. Section 4.6 treats the central obstacle to deep quantum circuits, barren plateaus, numerically and analytically. Section 4.7 trains QNNs on the regression and classification problems of the earlier chapters and on the portfolio problem of Chapter 3. Section 4.8 turns to differential equations: the physics-informed quantum neural network, whose loss is the residual of an equation rather than a distance to data, and whose derivatives with respect to the input are again obtained by evaluating the circuit at shifted angles. The programs are in `BookPrograms/chapter4/`: a `QNN` class on top of the simulator of the earlier chapters, the Fisher-information and barren-plateau experiments, the regression and classification models, and a small `QPINN` class in which every quantity of a differential equation is a linear functional of the circuit output. Every number quoted below is printed by one of them.

4.2 The model

Three ingredients A QNN on n qubits is specified by an encoding unitary $S(\boldsymbol{x})$, a trainable unitary $W(\boldsymbol{\theta})$ and an observable O,

$$|\psi(\boldsymbol{x};\boldsymbol{\theta})\rangle = W(\boldsymbol{\theta})\,S(\boldsymbol{x})\,|0\rangle^{\otimes n}, \qquad f(\boldsymbol{x};\boldsymbol{\theta}) = \langle\psi(\boldsymbol{x};\boldsymbol{\theta})|\,O\,|\psi(\boldsymbol{x};\boldsymbol{\theta})\rangle. \tag{4.1}$$

The encoding is one of the feature maps of Chapters 1 and 2; throughout this chapter we use the angle encoding of Eq. (1.3), one rotation per qubit,

$$S(\boldsymbol{x}) = \bigotimes_{q=0}^{n-1} R_y(\phi_q), \qquad \phi_q = \phi_q(\boldsymbol{x}), \tag{4.2}$$

with the angles ϕ_q linear in the features: $\phi_q = \pi x_q$ for a feature rescaled into $[0,1]$, or $\phi_q = s\,x$ for a single scalar input shared by all qubits, the case of the differential equations of Section 4.8. The trainable part is a product of L *layers*, each a rotation on every qubit followed by a fixed entangling pattern,

$$W_l(\boldsymbol{\theta}_l) = \Big[\prod_{q=0}^{n-2} \mathrm{CNOT}_{q\to q+1}\Big] \prod_{q=0}^{n-1} R_z(\theta_{lq}^{z})\,R_y(\theta_{lq}^{y}), \tag{4.3}$$

the CNOTs forming a *ladder* from qubit 0 down to qubit $n-1$, and the observable is $O = Z_{n-1}$, the Pauli Z of the last qubit, the one at the end of the ladder. Each layer has $2n$ parameters, so a circuit of L layers has $P = 2nL$ of them. The output of Eq. (4.1) is a number in $[-1,1]$; a classical scale and offset, $af+b$, are added when the target is not in that interval.

Data re-uploading In the simplest architecture the data enter once, before the first layer, $W = W_L \cdots W_1$. Pérez-Salinas et al. [49] observed that repeating the encoding before every layer,

$$|\psi(\boldsymbol{x};\boldsymbol{\theta})\rangle = W_L(\boldsymbol{\theta}_L)\, S(\boldsymbol{x}) \cdots W_2(\boldsymbol{\theta}_2)\, S(\boldsymbol{x})\, W_1(\boldsymbol{\theta}_1)\, S(\boldsymbol{x})\, |0\rangle^{\otimes n}, \tag{4.4}$$

makes the model considerably more expressive at no cost in qubits, and we will see in Section 4.3 exactly why: every re-upload adds a band of frequencies. We use the re-uploading form throughout; the single-upload model is the special case in which the later copies of S are omitted (`reupload=False` in the program). Figure 4.1 draws two layers on two qubits. The class `QNN` of `qnn.py` implements Eqs. (4.2)–(4.4) on batches of inputs, with the parameter order θ_{lrq}, layer first, then the rotation axis (y, then z), then the qubit.

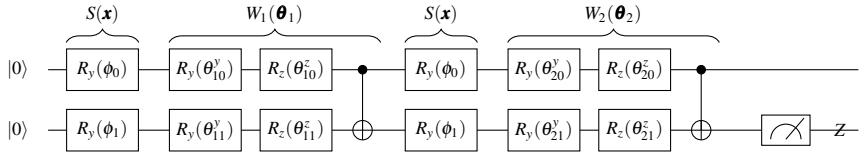


Fig. 4.1 The two-qubit, two-layer QNN of Eqs. (4.3) and (4.4) (qubit 0 on the top wire). The data enter through the angles ϕ_0, ϕ_1 of the encoding rotations, which are re-uploaded before every layer; each layer rotates every qubit by trainable angles and entangles the qubits with the CNOT ladder; the output $f = \langle Z_1 \rangle$ is the expectation value of Z on the last qubit. The circuit has $P = 8$ parameters.

What the circuit computes It is worth asking, before any training, what function of the data a circuit of this kind can produce. The Heisenberg picture answers the question layer by layer. The output is $\langle 0|S^\dagger W^\dagger O W S|0\rangle$, so one may conjugate the observable backwards through the circuit, $O \to W^\dagger O W$, and ask how the measured operator looks when it reaches the encoding. For a layer generated by a Hamiltonian, $W = e^{-iH}$, the Baker–Campbell–Hausdorff expansion

$$W^\dagger O W = O + i[H,O] + \frac{i^2}{2!}[H,[H,O]] + \cdots \tag{4.5}$$

shows that each commutator spreads the observable over more qubits and more Pauli strings: depth builds a hierarchy of correlations, in the same way as the cluster operator of coupled-cluster theory builds excitations [1]. For the Clifford part of our layers the conjugation is exact and simple. A CNOT maps Z on its target to the product of Z on control and target, so the ladder of Eq. (4.3) maps the measured Z_{n-1} to the full string,

$$\Big[\prod_q \mathrm{CNOT}_{q\to q+1}\Big]^\dagger Z_{n-1} \Big[\prod_q \mathrm{CNOT}_{q\to q+1}\Big] = Z_0 Z_1 \cdots Z_{n-1}, \tag{4.6}$$

which is why we measure the last qubit: every qubit, and therefore every rotation of the last layer, is seen by the measurement. (A ring of CNOTs closed by $\mathrm{CNOT}_{n-1\to 0}$ with Z_0 measured, a common choice, maps Z_0 to a string that omits qubit 0 itself, and the rotations of qubit 0 in the last layer are then invisible.) The R_z rotations of the last layer commute with the Z string and are likewise invisible; they are redundant parameters, of which Section 4.5 will say more. The R_y rotations turn each factor into $\cos\theta^y_{Lq} Z_q - \sin\theta^y_{Lq} X_q$, and for a single layer the expectation value in the product state $S(\boldsymbol{x})|0\rangle$ factorizes,

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \prod_{q=0}^{n-1} \cos\left(\phi_q + \theta_{1q}^{y}\right) \qquad (L=1). \tag{4.7}$$

A single layer, whatever its entangler, is a product of cosines, one per qubit, with the trainable angles as phases; this is the first and simplest statement of the limited function class of these circuits, and `timedependent.py` checks it numerically to machine precision. With two or more layers the rotations of the earlier layers act on an entangled operator and the product structure is lost; what survives is the Fourier structure of the next section.

4.3 The output as a Fourier series

The theorem Consider a scalar input x encoded through G gates $e^{-i\phi P_g/2}$, $\phi = sx$, with Pauli generators P_g (in our circuits the n rotations $R_y(sx)$ of each of the n_{enc} encoding blocks, $G = n n_{\text{enc}}$). Each gate multiplies the two eigencomponents of its generator by $e^{\mp i\phi/2}$, so every amplitude of $|\psi(x;\boldsymbol{\theta})\rangle$ is a sum of terms $e^{ik\phi/2}$ with integer k, $|k| \le G$, and the expectation value, a quadratic form in the amplitudes, is a sum of terms $e^{ik\phi}$ with integer k and $|k| \le G$. Hence [75]

$$f(x;\boldsymbol{\theta}) = \sum_{k=-G}^{G} c_k(\boldsymbol{\theta})\, e^{iksx}, \qquad c_{-k} = c_k^*, \tag{4.8}$$

a real trigonometric polynomial of degree G in $\phi = sx$. The frequencies ks are fixed by the encoding, the coefficients by the trainable part; the period of the model is $2\pi/s$. The same argument with several inputs gives a multivariate trigonometric polynomial of degree G_i in each variable x_i, G_i counting the gates that carry x_i. The statement is general: whatever the trainable layers do, the model never leaves the span of the $2G+1$ functions $1, \cos ksx, \sin ksx$, and a target with a frequency outside this band cannot be represented. Conversely the $2G+1$ coefficients follow exactly from $2G+1$ evaluations of the circuit at equidistant angles, a discrete Fourier transform, which is how `QNN.fourier_coefficients` computes them.

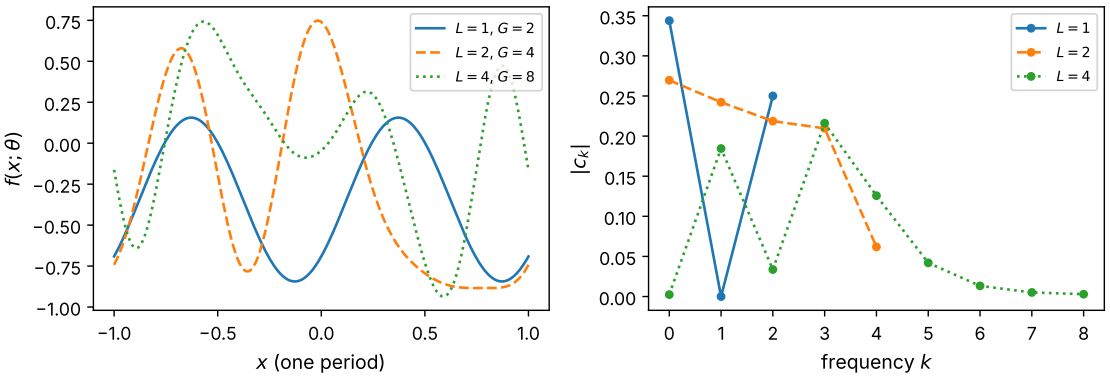


Fig. 4.2 Left: the output of two-qubit circuits with random parameters and $L = 1,2,4$ re-uploading layers over one period of the input, $\phi = \pi x$. Right: the magnitudes of their Fourier coefficients c_k, Eq. (4.8); the highest frequency is $G = nL = 2L$ in every case. The single layer has no $k = 1$ component, a consequence of the product form Eq. (4.7). From `qnn.py`.

What this means for design Figure 4.2 shows the spectra of random two-qubit circuits with $L = 1,2,4$ re-uploads: the band widens with L as Eq. (4.8) predicts, $G = 2L$. Three consequences will recur in the applications. First, the number of frequencies is set by the number

of encoding gates, $G = n n_{\text{enc}}$, while the number of parameters, $P = 2nL$, is at best equal to the number of real coefficients $2G+1$ for a scalar input ($P = 2G$) and far smaller than the $(2G+1)^d$ coefficients of d inputs; in either case the model is a *constrained* Fourier series whose coefficients are not free but functions of the angles, and the set of reachable coefficient vectors is a curved submanifold of low dimension. Second, the period of the model is a design choice through the scale s. A target that is not periodic on the data interval must be given room: if the data cover the whole period the model is forced to return at the end of the interval to its value at the beginning, and the usual choice is to let the data cover half a period or less (Section 4.7). Third, the families of different depth are not nested. A circuit with L re-uploads contains L copies of $S(x)$ that no choice of the trainable angles can cancel, so the function class of depth $L+1$ does not contain that of depth L; more layers reach new functions but may lose old ones, and Section 4.8 has a striking example. The single layer of Eq. (4.7), finally, has a spectrum with gaps, the $k = 1$ coefficient of Fig. 4.2 being identically zero for $n = 2$, since a product of two cosines contains only the sum and the difference of its arguments.

4.4 Training

Loss functions Given training pairs $(\boldsymbol{x}_i, y_i)$, $i = 1, \ldots, N$, a regression model is trained on the mean squared error

$$\mathscr{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \big[a f(\boldsymbol{x}_i; \boldsymbol{\theta}) + b - y_i \big]^2, \tag{4.9}$$

and a two-class classifier with labels $y_i = \pm 1$ either on the same loss, the predicted class being the sign of $af + b$, or on the cross-entropy of the probabilities $p_i = (1 + f_i)/2$. For a model whose output is an expectation value the square loss has a physical reading: with $O = Z_{n-1}$ it is the squared distance between the measured magnetization of the last qubit and the label. Whatever the loss, its gradient with respect to the circuit parameters is obtained from the gradient of f by the chain rule, so the quantum question is how to differentiate an expectation value with respect to a rotation angle.

The parameter-shift rule Let θ be the angle of one gate $R(\theta) = e^{-i\theta P/2}$ with a Pauli generator P, $P^2 = I$, and write the circuit as $U = A\,R(\theta)\,B$ with A and B independent of θ. Since $R(\theta) = \cos(\theta/2) I - i \sin(\theta/2) P$, the state $R(\theta)B|0\rangle$ is a linear combination of two fixed vectors with coefficients $\cos(\theta/2)$ and $\sin(\theta/2)$, and the expectation value, a quadratic form in these coefficients, has the form

$$f(\theta) = \alpha + \beta \cos\theta + \gamma \sin\theta, \tag{4.10}$$

with α, β, γ independent of θ. This is Eq. (4.8) with $G = 1$. Differentiating, $f'(\theta) = -\beta \sin\theta + \gamma \cos\theta$, and evaluating Eq. (4.10) at $\theta \pm \pi/2$,

$$\frac{\partial f}{\partial \theta} = \frac{1}{2} \Big[f\big(\theta + \tfrac{\pi}{2}\big) - f\big(\theta - \tfrac{\pi}{2}\big) \Big], \tag{4.11}$$

the *parameter-shift rule* of Mitarai et al. [10] and Schuld et al. [11], which Chapter 2 used for the kernel alignment and Chapter 3 generalized to generators with many eigenvalues. It is exact, not a finite difference: the two shifted circuits are the same circuit with one angle changed by a quarter turn, and no small parameter enters. The same identity applied twice gives the second derivative,

$$\frac{\partial^2 f}{\partial \theta^2} = \frac{1}{4} \Big[f(\theta + \pi) - 2 f(\theta) + f(\theta - \pi) \Big] = \frac{1}{2} \Big[f(\theta + \pi) - f(\theta) \Big], \tag{4.12}$$

the second form using the periodicity $f(\theta+\pi)=f(\theta-\pi)$, and the mixed derivative with respect to two angles is the product of the two first-order rules, four circuits at $\theta_j \pm \pi/2$, $\theta_k \pm \pi/2$. (Several lecture notes, including the slides this chapter grew out of, quote the second derivative without the factor $1/4$; Eq. (4.12) is checked against finite differences in `qnn.py`.) The derivative is also a *quantum response function*: writing $K = P/2$ for the generator, $\partial_\theta f = i\langle[K, O_{\text{eff}}]\rangle$ with $O_{\text{eff}} = A^\dagger O A$ the observable conjugated back to the gate, which is the linear-response formula of many-body physics with the gate's generator as the perturbation.

A gradient of P parameters costs $2P$ circuit evaluations, each of them in practice an estimate from S shots. The shift rule is a difference of two unbiased estimates, so its variance is half the variance of a single expectation value, $\sim 1/(2S)$, independently of the gradient's size; a finite difference with step h would have variance $\sim 1/(h^2 S)$, and could not be made accurate by making h small. The listing below is the rule as the program implements it on a batch of inputs; `QNN.gradient` evaluates all $2P$ shifted parameter sets in a single call.

```
import numpy as np
from qnn import QNN

qnn = QNN(n_qubits=2, n_layers=2)  # R_y, R_z per qubit and layer; CNOT ladder; <Z_1>
theta = qnn.random_parameters(rng=0)
angles = np.pi * np.array([[0.3, 0.7]]) # one sample: one encoding angle per qubit

def gradient(angles, theta, shift=np.pi / 2):
    """Parameter-shift gradient of f(x; theta), one pair of circuits per angle."""
    g = np.zeros(len(theta))
    for mu in range(len(theta)):
        tp, tm = theta.copy(), theta.copy()
        tp[mu] += shift; tm[mu] -= shift
        g[mu] = 0.5 * (qnn.forward(angles, tp)[0] - qnn.forward(angles, tm)[0])
    return g

print(np.round(gradient(angles, theta), 5))
print(np.round(qnn.gradient(angles, theta)[0], 5)) # the vectorized version of qnn.py
```

Both lines print the same eight numbers; the last two, the R_z angles of the second layer, are zero, as Eq. (4.6) predicts.

The general shift rule When the same angle enters G gates, as x does in a re-uploading circuit, f is the trigonometric polynomial of degree G of Eq. (4.8), and the two-point rule no longer applies. Two exact routes remain. The *product rule* differentiates one gate at a time: with ϕ_g the angle of gate g,

$$\frac{\partial f}{\partial x} = \sum_{g=1}^{G} \frac{\partial \phi_g}{\partial x}\,\frac{\partial f}{\partial \phi_g} = \frac{s}{2}\sum_{g=1}^{G}\Big[f\big(\phi_g+\tfrac{\pi}{2}\big) - f\big(\phi_g-\tfrac{\pi}{2}\big)\Big], \tag{4.13}$$

where $f(\phi_g \pm \pi/2)$ means that only the g-th gate is shifted; $2G$ circuits for the first derivative, and of order G^2 for the second. The *general shift rule* of Wierichs et al. [66] uses the structure of Eq. (4.8) instead: a trigonometric polynomial of degree G is determined by its values at $2G+1$ distinct angles, so each of its derivatives at ϕ is a fixed linear combination of $2G+1$ evaluations,

$$\frac{\partial^r f}{\partial \phi^r} = \sum_{m=-G}^{G} w_m^{(r)}\, f\Big(\phi + \frac{2\pi m}{2G+1}\Big), \tag{4.14}$$

with weights that depend only on G and r: writing the polynomial in the basis $1, \cos k\phi, \sin k\phi$, the $2G+1$ samples determine the coefficients through the inverse of the matrix of basis functions at the sample angles, and the derivative at the base point is the r-th derivative of the basis applied to these coefficients; the function `trig_weights` of `qnn.py` solves this linear

system. For $G=1$ and $r=1$ the weights are $\pm 1/\sqrt{3}$ at $\pm 2\pi/3$, a variant of Eq. (4.11) on a different pair of nodes; for $r=2$ and $G=1$ one obtains $\frac{1}{3}[f(\phi+2\pi/3)-2f(\phi)+f(\phi-2\pi/3)]$, the analog of Eq. (4.12) on the same three nodes. The general rule costs $2G+1$ circuits for every derivative order at once, the product rule $2G$ for the first and $4G^2-G$ for the second (14 and 138 in Table 4.6), and the two agree to machine precision (Table 4.6 of Section 4.8).

Optimizers With the gradient in hand, the classical optimizer is any of the standard ones. Plain gradient descent, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathscr{L}$, is the reference. The method of choice in the QML literature is *Adam* [76], gradient descent with a running mean $\boldsymbol{m}$ of the gradient and a running mean $\boldsymbol{v}$ of its square,

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}}+\varepsilon}, \qquad \hat{\boldsymbol{m}} = \frac{\boldsymbol{m}}{1-\beta_1^t}, \quad \hat{\boldsymbol{v}} = \frac{\boldsymbol{v}}{1-\beta_2^t}, \tag{4.15}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$: each parameter gets its own step size, which makes the method robust to the very different scales of the gradient components and to shot noise. On a simulator, where the loss and its gradient are exact and cheap, the quasi-Newton method BFGS of `common/optimize.py`, which Chapter 3 used for the QAOA angles, converges in far fewer iterations and is what we use for the differential equations of Section 4.8; the class `Adam` of the same module serves the data-driven models. Neither method sees the geometry of the circuit states, which is the subject of the next section.

4.5 Geometry: the quantum Fisher information

Classical Fisher information Let $p(\boldsymbol{x}|\boldsymbol{\theta})$ be a family of probability distributions and $s_j(\boldsymbol{x}) = \partial_j \log p(\boldsymbol{x}|\boldsymbol{\theta})$ the *score*, the relative sensitivity $\partial_j p/p$ of the distribution to the parameter. Differentiating the normalization $\int p = 1$ gives $\mathbb{E}[s_j] = 0$, and differentiating once more gives the two equivalent forms of the *Fisher information matrix*,

$$F^{\mathrm{cl}}_{jk} = \mathbb{E}_p\big[s_j s_k\big] = -\mathbb{E}_p\Big[\frac{\partial^2 \log p}{\partial\theta_j \partial\theta_k}\Big], \tag{4.16}$$

the covariance of the score and the mean curvature of the log-likelihood. Its meaning is distinguishability: for a small change of the parameters the Kullback–Leibler divergence between the two distributions is

$$D_{\mathrm{KL}}\big(p_{\boldsymbol{\theta}} \,\|\, p_{\boldsymbol{\theta}+d\boldsymbol{\theta}}\big) = \frac{1}{2}\, d\boldsymbol{\theta}^{T} F^{\mathrm{cl}}\, d\boldsymbol{\theta} + O(d\theta^3), \tag{4.17}$$

so F^{cl} is a Riemannian metric on parameter space, the Fisher–Rao metric, and the Cramér–Rao bound $\mathrm{Cov}(\hat{\boldsymbol{\theta}}) \succeq (N_s F^{\mathrm{cl}})^{-1}$ says that no unbiased estimator from N_s samples can resolve a direction of parameter space better than the metric allows. Directions of small Fisher information are directions the data cannot see.

Quantum Fisher information For a pure variational state $|\psi(\boldsymbol{\theta})\rangle$ the analog is obtained by asking how distinguishable two neighboring states are by the best possible measurement. The answer is the *quantum Fisher information* (QFI) [77, 78],

$$F_{jk} = 4\,\mathrm{Re}\Big(\langle\partial_j\psi|\partial_k\psi\rangle - \langle\partial_j\psi|\psi\rangle\langle\psi|\partial_k\psi\rangle\Big), \tag{4.18}$$

four times the real part of the quantum geometric tensor $\chi_{jk} = \langle\partial_j\psi|(I-|\psi\rangle\langle\psi|)|\partial_k\psi\rangle$, whose imaginary part is the Berry curvature. It is, up to a conventional factor 4, the Fubini–Study

metric on the manifold of circuit states, $ds^2 = \frac{1}{4}\sum_{jk} F_{jk} d\theta_j d\theta_k$ being the squared distance between $|\psi(\boldsymbol{\theta})\rangle$ and $|\psi(\boldsymbol{\theta}+d\boldsymbol{\theta})\rangle$ in projective Hilbert space, where states differing by a phase coincide, and it bounds every classical Fisher information obtained from a measurement of the state. For a state generated by a single Hermitian generator, $|\psi(\theta)\rangle = e^{-i\theta K}|\psi_0\rangle$, one has $\partial_\theta \psi = -iK\psi$ and

$$F = 4\,\mathrm{Var}_\psi(K) = 4\big(\langle K^2\rangle - \langle K\rangle^2\big), \tag{4.19}$$

the fluctuations of the generator: for a single qubit rotated by $R_y(\theta)$ from $|0\rangle$, $K = Y/2$ and $F = 1$ for every θ, which is the first check of `qfi.py`. For a product of rotations in a single layer the same calculation gives $F_{jk} = 4\,\mathrm{Cov}_\psi(K_j, K_k)$; for deeper circuits the generators of different layers are conjugated by the layers in between and Eq. (4.18) must be evaluated as it stands.

Computing the QFI On a simulator Eq. (4.18) is cheap, because the derivative of the state with respect to a rotation angle is itself a circuit state: from $dR(\theta)/d\theta = \frac{1}{2}R(\theta+\pi)$ for $R = e^{-i\theta P/2}$,

$$\frac{\partial}{\partial\theta_\mu}|\psi(\boldsymbol{\theta})\rangle = \frac{1}{2}\,|\psi(\boldsymbol{\theta}+\pi\boldsymbol{e}_\mu)\rangle, \tag{4.20}$$

so the P derivatives cost P extra state preparations (the global phase matters here, and the identity holds because it is the same circuit). On hardware one measures overlaps instead. With $\mathcal{O}(\boldsymbol{\theta}') = |\langle\psi(\boldsymbol{\theta})|\psi(\boldsymbol{\theta}')\rangle|^2$, which has its maximum 1 at $\boldsymbol{\theta}' = \boldsymbol{\theta}$, expanding to second order gives $\mathcal{O} = 1 - \frac{1}{4}\,d\boldsymbol{\theta}^T F\, d\boldsymbol{\theta}$, hence $F_{jk} = -2\,\partial'_j\partial'_k\mathcal{O}$, and the mixed second derivative follows from the shift rules of Section 4.4 [79],

$$F_{jk} = -\frac{1}{2}\Big[\mathcal{O}_{++} - \mathcal{O}_{+-} - \mathcal{O}_{-+} + \mathcal{O}_{--}\Big]\ (j\neq k), \qquad F_{jj} = 1 - \mathcal{O}(\theta_j+\pi), \tag{4.21}$$

where $\mathcal{O}_{\pm\pm}$ is the overlap with θ_j and θ_k shifted by $\pm\pi/2$; the overlaps are measured with the adjoint test of Fig. 2.6, which is the circuit followed by the inverse circuit at the other parameters. The sign in front is easy to get wrong: the overlap has a maximum, so its second derivative is negative and F positive. Table 4.1 compares the two formulas and shows what the matrix knows about a circuit.

```
import numpy as np
from qnn import QNN

def qfi_matrix(qnn, angles, theta):
    """F_jk = 4 Re(<d_j psi|d_k psi> − <d_j psi|psi><psi|d_k psi>), Eq. (4−qfi),
    from d|psi>/d theta_mu = |psi(theta + pi e_mu)>/2, averaged over the inputs."""
    psi = qnn.state(angles, theta)                  # (N, 2^n)
    D = qnn.state_derivatives(angles, theta)        # (P, N, 2^n)
    G = np.einsum("jna,kna->njk", D.conj(), D) # <d_j psi|d_k psi>
    v = np.einsum("jna,na->nj", D.conj(), psi)   # <d_j psi|psi>
    chi = G − np.einsum("nj,nk->njk", v, v.conj()) # quantum geometric tensor
    return 4 * chi.real.mean(axis=0)

qnn = QNN(2, 2); theta = qnn.random_parameters(1)
F = qfi_matrix(qnn, np.pi * np.random.default_rng(0).uniform(0, 1, (8, 2)), theta)
print("rank", np.linalg.matrix_rank(F), "of", qnn.n_params, "; eigenvalues", np.round(np.linalg.eigvalsh(F),
    3))
```

Two lessons are in the table. The circuit of Fig. 4.1 has a QFI of full rank: all eight angles move the state. Yet two of them, the R_z of the last layer, have zero gradient for every input, because they commute with the measured Z string, Eq. (4.6). The metric is a property of the state, the gradient of the loss also of the observable, and the two kinds of redundancy must be told apart: a flat direction of F is a parameter that does nothing at all, and the fourth row has two of them, two consecutive R_y rotations about the same axis with nothing in be-

Table 4.1 The quantum Fisher information of two-qubit circuits at random parameters, averaged over eight random inputs. The rank of F counts the directions in parameter space along which the state changes; the next column counts the parameters whose output gradient vanishes identically, which is a property of the observable rather than of the state; the last column is the difference between Eq. (4.18) and the overlap formula Eq. (4.21). From `qfi.py`.

Circuit	P	rank F	zero gradients	$\max\lvert F - F_{\text{shift}}\rvert$
R_y only, $L=1$	2	2	0	2×10^{-16}
R_y only, $L=2$	4	4	0	3×10^{-16}
R_y, R_z, $L=2$ (Fig. 4.1)	8	8	2	2×10^{-16}
R_y only, $L=2$, no re-upload, no CNOT	4	2	2	2×10^{-16}
R_y, R_z, $L=2$, no CNOT	8	8	5	3×10^{-16}

tween, of which only the sum can be trained; a zero gradient with full-rank F is a parameter the measurement does not see, which a different observable would. The Cauchy–Schwarz inequality on the Fubini–Study metric ties the two together, $|\partial_j f|^2 \le F_{jj}\,\mathrm{Var}_\psi(O)$: a small diagonal QFI entry forces a small gradient of every observable, and Section 4.6 will use this as a diagnostic.

The natural gradient Gradient descent takes the step that decreases the loss most for a fixed Euclidean length $|d\boldsymbol{\theta}|$, a notion that depends on how the circuit happens to be parametrized. Amari's *natural gradient* [80] asks instead for the step that decreases the loss most for a fixed distance between the *states*, $d\boldsymbol{\theta}^T F\, d\boldsymbol{\theta} = \varepsilon^2$; a Lagrange multiplier gives

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\big(F + \lambda I\big)^{-1}\nabla\mathscr{L}, \tag{4.22}$$

with a small regulator λ for the flat directions [81]. The update is covariant: reparametrizing the circuit changes F by the Jacobian in exactly the way that leaves the step on the manifold unchanged, which plain gradient descent cannot claim. Its cost is the QFI, $P+1$ state preparations on the simulator or $O(P^2)$ overlaps on hardware, plus a $P\times P$ solve.

Imaginary time, TDVP and stochastic reconfiguration The natural gradient has a second origin, in dynamics. Projecting the Schrödinger equation $i\partial_t|\psi\rangle = H|\psi\rangle$ onto the tangent space of the variational manifold, spanned by the $|\partial_j\psi\rangle$, is the Dirac–Frenkel or *time-dependent variational principle* (TDVP), and it gives equations of motion for the parameters,

$$\sum_k F_{jk}\,\dot\theta_k = 4\,\mathrm{Im}\,\langle\partial_j\psi|H|\psi\rangle, \tag{4.23}$$

with the QFI as the "mass matrix" [82]. In imaginary time, $t\to -i\tau$, the evolution $e^{-\tau H}$ filters out the ground state, and the projected equation becomes

$$\sum_k F_{jk}\,\dot\theta_k = -4\,\mathrm{Re}\,\langle\partial_j\psi|H|\psi\rangle = -2\,\frac{\partial E}{\partial\theta_j}, \qquad \dot{\boldsymbol{\theta}} = -2F^{-1}\nabla E, \tag{4.24}$$

the natural gradient flow of the energy [81, 83]. Readers of variational Monte Carlo (the chapters on variational Monte Carlo and on optimization and resampling of the many-body text [34]) will recognize Sorella's *stochastic reconfiguration* [84]: there the state is a wave function $\Psi(\boldsymbol{R};\boldsymbol{\alpha})$ sampled from $p=|\Psi|^2/Z$, the score of this distribution is $s_k = 2(O_k - \langle O_k\rangle)$ with $O_k = \partial_k \ln\Psi$ the logarithmic derivative, and the SR matrix $S_{jk} = \langle O_j O_k\rangle - \langle O_j\rangle\langle O_k\rangle$ is one quarter of the Fisher information of the Born distribution,

$$F_{jk} = 4S_{jk}, \tag{4.25}$$

so that the SR update $S\,\delta\boldsymbol{\alpha} = -\delta\tau\,\boldsymbol{g}$ is the natural gradient, is projected imaginary-time evolution, and is the TDVP in imaginary time, three names for one equation. The QFI of a circuit and the SR matrix of a neural-network wave function are the same object computed in two bases.

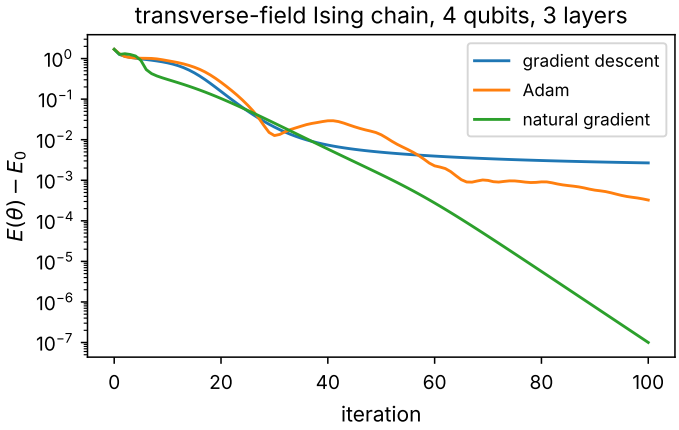


Fig. 4.3 Variational ground-state search for the transverse-field Ising chain $H = -\sum_i Z_i Z_{i+1} - \sum_i X_i$ on four qubits with a three-layer R_y circuit (12 parameters) and the parameter-shift gradient: energy error against iteration for gradient descent ($\eta = 0.2$), Adam ($\eta = 0.1$) and the natural gradient Eq. (4.22) ($\eta = 0.2$, $\lambda = 10^{-3}$) from the same start. From `qfi.py`.

Figure 4.3 shows the three optimizers on the problem for which the natural gradient was designed, the ground state of the transverse-field Ising chain on four qubits, $E_0 = -4.758770$, with a three-layer circuit of R_y rotations and the parameter-shift gradient of the energy. From the same start, after 100 iterations, gradient descent is 2.7×10^{-3} above the ground state, Adam 3.3×10^{-4}, the natural gradient 1.0×10^{-7} and still converging exponentially. The eigenvalues of F at the start span three orders of magnitude, from 0.002 to 2.9, which is the anisotropy the metric corrects for. On the data-driven losses of Section 4.7 the advantage is smaller: there the metric of the state is not the metric of the loss, which also depends on the observable and the data, and the Gauss–Newton approximation of the loss Hessian (the *neural tangent kernel* below) is the more relevant geometry.

Effective dimension Abbas et al. [85] use the Fisher information to compare the capacity of quantum and classical models. Their *effective dimension*, essentially the logarithm of the volume $\int \sqrt{|(|I + cF)}\, d\boldsymbol{\theta}$ of parameter space measured with the Fisher metric and normalized by the number of training samples, counts the parameters that the data can actually resolve; redundant and flat directions do not count. Some QNNs reach a higher effective dimension than classical networks with the same number of parameters, that is, their Fisher spectrum is flatter, with fewer nearly-zero eigenvalues. The same flat spectrum is, however, what a random deep circuit loses entirely, as we now show.

4.6 Barren plateaus

The phenomenon McClean et al. [22] discovered that for random parametrized circuits of sufficient depth the gradient of any cost $C(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|O|\psi(\boldsymbol{\theta})\rangle$ vanishes exponentially in the number of qubits: averaged over the parameters,

$$\mathbb{E}_{\boldsymbol{\theta}}\left[\partial_j C\right] = 0, \qquad \mathrm{Var}_{\boldsymbol{\theta}}\left[\partial_j C\right] = O\left(b^{-n}\right), \quad b > 1. \tag{4.26}$$

The landscape is then flat almost everywhere, a *barren plateau*: not a landscape with many minima, but one with no slope, on which a gradient estimated from a polynomial number of shots is pure noise. The origin is concentration of measure. A circuit whose parameters are random enough samples the unitary group approximately uniformly (it forms a *unitary 2-design*, a set of unitaries whose first two moments match the Haar measure), and for a Haar-random state the expectation value of any observable is, with overwhelming probability, within $O(2^{-n/2})$ of its average $\mathrm{Tr}\, O/2^n$ (Lévy's lemma). Explicitly, for the derivative with respect to a gate in the middle of the circuit, with 2-designs on both sides of it,

$$\mathrm{Var}\left[\partial_j C\right] \approx \frac{\mathrm{Tr}\, O^2}{2^{2n}} \times O(1), \tag{4.27}$$

which is 2^{-n} for a Pauli string and 4^{-n} for a rank-one projector. Cerezo et al. [86] sharpened the picture by distinguishing *global* costs, which measure all qubits at once, such as $O_G = |0\cdots0\rangle\langle 0\cdots0|$, from *local* costs, sums of few-qubit terms such as $O_L = \frac{1}{n}\sum_i |0\rangle\langle 0|_i$: a global cost has a barren plateau for any depth, including constant depth, while a local cost on a circuit of depth $O(\log n)$ has gradients that vanish only polynomially in n. The exponential decay returns for local costs once the depth is linear in n.

Table 4.2 Variance over 200 random parameter sets of the gradient component $\partial C/\partial\theta_1$ (the first R_y of the first layer) for the hardware-efficient circuit of Eq. (4.3) with R_y, R_z rotations, n qubits and $L = n$ layers, for the local cost O_L and the global cost O_G. From `barren.py`.

n	3	4	5	6	7	8	9	10
local, $\mathrm{Var} \times 10^4$	92	28	12	5.0	2.6	1.1	0.56	0.30
global, $\mathrm{Var} \times 10^4$	87	19	3.5	1.3	0.34	0.069	0.018	0.0047

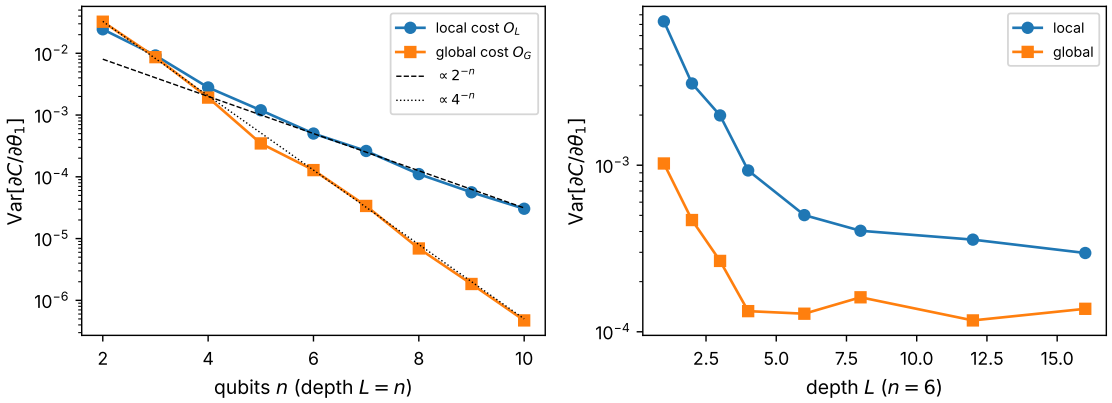


Fig. 4.4 Barren plateaus. Left: the variance of Table 4.2 against the number of qubits at depth $L = n$, with the lines $\propto 2^{-n}$ and $\propto 4^{-n}$ through the $n = 6$ points. Right: the variance against the depth at $n = 6$, saturating once the circuit is random enough. From `barren.py`.

The experiment Table 4.2 and Fig. 4.4 reproduce both statements with the circuit of Eq. (4.3) at depth $L = n$ and uniformly random angles, 200 samples per point, the gradient component being that of the first rotation of the first layer. A fit over $n = 6, \ldots, 10$ gives $\mathrm{Var} \propto b^{-n}$ with $b = 2.04$ for the local and $b = 4.10$ for the global cost, the 2^{-n} and 4^{-n} of Eq. (4.27). At

ten qubits the local gradient has a standard deviation of 0.0055 and the global one of 0.0007; since a shift-rule estimate from S shots has a variance of about $1/(2S)$, resolving a typical component to ten per cent would take some 2×10^6 shots for the local and 10^8 for the global cost, and the numbers double or quadruple with every added qubit. The right panel shows the depth dependence at $n=6$: the variance falls over the first few layers and then saturates, the circuit having become a 2-design, at the level set by the width. Two remarks on the experiment. Depth $L=n$ is already deep enough for the saturation, and the circuits of the applications below, two or three qubits and two to four layers, are far from it; the plateau is a statement about scaling up. And the variance is of the gradient at *random* parameters; a good initialization, near the identity or from a smaller problem, starts the optimizer off the plateau, and whether it stays off it is the practical question.

Noise, mitigation and the trade-off Hardware noise adds a plateau of its own. Wang et al. [87] showed that depolarizing noise of rate ε per gate shrinks the expectation values towards $\mathrm{Tr}\,O/2^n$ as $(1-\varepsilon)^D$ with D the number of gates, and the gradient variance as the square of it, which does not depend on the initialization and affects shallow circuits with many gates as much as deep ones. The strategies that help against the unitary plateau are listed in Table 4.3; the theme common to all of them is *structure*. A circuit that is not a 2-design, because it is shallow, because its entanglement is limited, or because it is built from the symmetries of the problem, has no reason to concentrate; the QAOA circuits of Chapter 3, whose layers are generated by the cost Hamiltonian, and the physics-informed circuits of Section 4.8 are examples. The price is expressivity. A circuit that can reach every state is a circuit that concentrates; Holmes et al. [88] make the relation quantitative, the gradient variance being bounded by the distance of the circuit ensemble from a 2-design. Entanglement tells the same story: a state with volume-law entanglement entropy across every cut is generic and untrainable, a product state is trainable and classically simulable, and the useful regime lies between, with area-law entanglement and a depth that grows at most logarithmically with the system.

Table 4.3 Strategies against barren plateaus and the mechanism behind each.

Strategy	Mechanism
Local cost functions	polynomial decay at logarithmic depth [86]
Problem-inspired ansatz	the circuit is not a 2-design (QAOA, Hamiltonian variational)
Shallow circuits, limited entanglement	area-law states do not concentrate
Layer-wise training	each stage is shallow
Identity or small-angle initialization	starts off the plateau; may return to it
Natural gradient, QFI monitoring	flat directions are detected before they are entered

The quantum neural tangent kernel The gradients of the model at the training points define a kernel,

$$\Theta(\boldsymbol{x},\boldsymbol{x}') = \sum_{j=1}^{P} \frac{\partial f(\boldsymbol{x};\boldsymbol{\theta})}{\partial\theta_j}\,\frac{\partial f(\boldsymbol{x}';\boldsymbol{\theta})}{\partial\theta_j}, \tag{4.28}$$

the *quantum neural tangent kernel*. In the regime where the parameters move little during training, the model is linear in the parameter change and gradient descent on the square loss becomes kernel regression with Θ: the eigenvalues of the kernel matrix $\Theta(\boldsymbol{x}_i,\boldsymbol{x}_k)$ set the rates at which the components of the target are learned, and a barren plateau is the statement that $\Theta\approx0$ for all pairs. This closes the circle with Chapter 2: a variational model trained by gradient descent is, in the linearized regime, a kernel method with a kernel it does not choose, and Schuld's observation [5] that a kernel machine with the fidelity kernel of the

same encoding reaches at least the same training loss as the variational model is the exact version of the same statement.

4.7 Quantum neural networks at work

4.7.1 Regression

The first test is the one that Section 4.3 prepared: fit a function of one variable and see how the Fourier band grows with the depth. The target is the cubic $g(x) = x^3 - 2x^2 + x/2 + 1$ on $[-2, 2]$, sampled at 150 training and 100 test points with Gaussian noise of standard deviation 0.15, so that the best achievable test error is the noise floor 0.0225. The quantum models are two-qubit circuits with $L = 1, 2, 4, 6$ re-uploading layers, the input mapped to the angle $\phi = \pi x/4$ on both qubits and a trained scale and offset on the output; the targets are standardized before training and the errors reported in the original units. The classical reference is the network $1{-}16{-}16{-}1$ with hyperbolic-tangent units of `common/neuralnet.py`, 321 parameters, trained by backpropagation. Both are trained with Adam.

Table 4.4 Regression of the noisy cubic: mean squared error on the training and the test set, in the original units, for two-qubit QNNs of growing depth (the parameter count includes the output scale and offset) and for the classical network. The noise floor is 0.0225. From `regression.py`.

Model	parameters	train MSE	test MSE
QNN, $n = 2$, $L = 1$	6	7.67	9.07
QNN, $n = 2$, $L = 2$	10	0.055	0.061
QNN, $n = 2$, $L = 4$	18	0.029	0.024
QNN, $n = 2$, $L = 6$	26	0.026	0.026
MLP 1–16–16–1	321	0.024	0.022

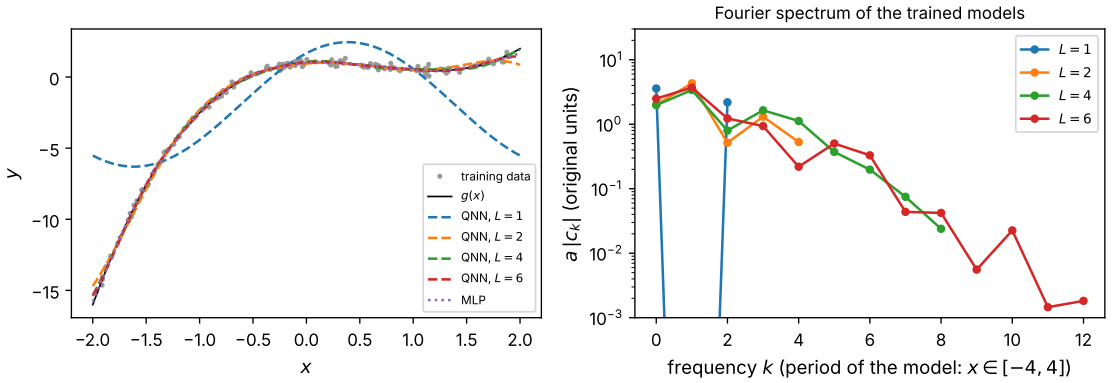


Fig. 4.5 Left: the training data, the target $g(x)$, the trained QNNs of Table 4.4 and the classical network. Right: the Fourier coefficients $a|c_k|$ of the trained quantum models, in the original units, against the frequency index k (the period of the model is the interval $[-4, 4]$); the single layer has no $k = 1$ component and fails. From `regression.py`.

Table 4.4 and Fig. 4.5 tell the story of Section 4.3 in numbers. The single layer cannot fit the cubic at all: its spectrum, by Eq. (4.7), has no $k = 1$ term, and a cubic on a half period is mostly $k = 1$. Two layers reach a test error of 0.061, four layers 0.024, essentially the noise

floor, with 18 parameters where the classical network uses 321; six layers bring nothing more. The right panel shows why: the trained four- and six-layer models put their weight into the first few frequencies and suppress the rest, and the cubic needs no more than that. The choice $\phi = \pi x/4$, which makes the data cover half the period of the model, is essential; with $\phi = \pi x/2$ the data cover the whole period, the model is forced to make $g(-2)$ equal to $g(2)$, and the four-layer test error rises to above 2 (Exercise 3). Two things the table does not say should be said. The classical network trains in a fraction of a second, the four-layer QNN in a few seconds on the simulator, and every QNN gradient would cost $2P$ circuit evaluations per training point on hardware. And the comparison is between a model with a bias towards smooth periodic functions and a universal approximator on a task that suits the former; the QNN wins on parameter count because the target is close to its function class, not because the Hilbert space is large.

4.7.2 Classification

The second test uses the four two-class problems of Table 2.1, with the same 70/30 splits, so that the variational classifier can be compared with the kernel machines of Chapter 2. Each feature is rescaled into $[0, \pi]$ and encoded on its own qubit, and the model is a two-qubit circuit with $L = 1$ or $L = 3$ layers, trained for 300 Adam steps on the square loss of Eq. (4.9) with the labels ± 1; the class is the sign of $af + b$, and the best of three random starts is kept. The class `QNNClassifier` of `training.py` is a dozen lines on top of `QNN`:

```
import numpy as np
from training import QNNClassifier
from datasets import two_moons, train_test_split, Scaler

X, y = two_moons(100, 0.15)
Xtr, Xte, ytr, yte = train_test_split(X, y, 0.3, rng=0)
sc = Scaler("angle").fit(Xtr)                    # features -> angles in [0, pi]
clf = QNNClassifier(n_qubits=2, n_layers=3).fit(sc.transform(Xtr), ytr, steps=300, lr=0.1, rng=0)
print(f"{clf.n_params} parameters, loss {clf.history[0]:.3f} -> {clf.history[-1]:.3f}, "
      f"accuracy train {clf.score(sc.transform(Xtr), ytr):.2f} test {clf.score(sc.transform(Xte), yte):.2f}")
```

Table 4.5 Training/test accuracy of the variational classifier on the four problems of Table 2.1, two qubits, $L = 1$ and $L = 3$ re-uploading layers (parameter counts in parentheses), against the RBF-kernel SVM and the angle-encoding QSVM of Chapter 2 rerun on the same splits. From `classifier.py`.

Data set	RBF SVM	angle QSVM	QNN $L = 1$ (6)	QNN $L = 3$ (14)
two moons	0.96/0.97	0.89/0.77	0.87/0.80	0.96/0.97
two circles	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
XOR	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
Iris (versicolor/virginica)	0.97/0.90	0.96/0.90	0.94/0.93	0.99/0.90

Table 4.5 and Fig. 4.6 should be read next to the kernel results of Chapter 2. The single layer reproduces, within a few points, the angle-encoding QSVM: the same encoding, with the same product kernel of Eq. (2.32) behind it, and a decision function that by Eq. (4.7) is a product of two cosines, which is enough for the circles and the XOR problem and not for the moons. Three re-uploading layers bring the moons to 0.97, the level of the RBF kernel, with 14 parameters trained by gradient descent instead of a kernel matrix. On the Iris data all models but the single-layer QNN (0.93) sit at the 0.90 test accuracy that the overlap of the two species allows. The decision regions show the two models as what they are, trigonometric

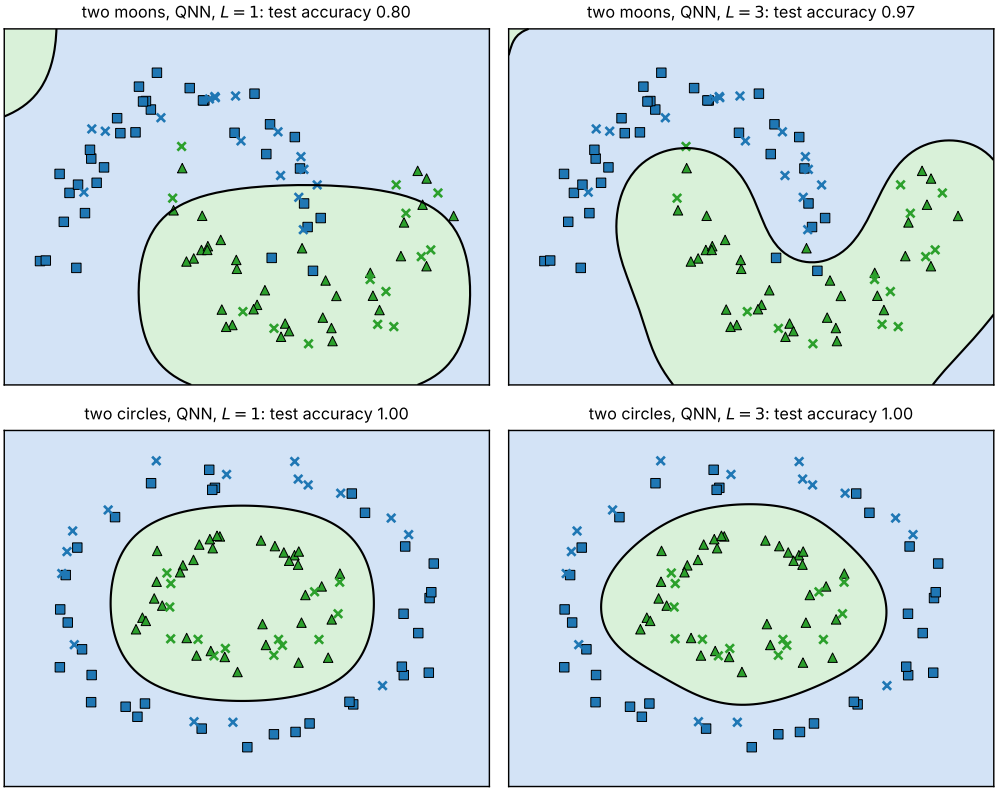


Fig. 4.6 Decision regions of the variational classifier with $L = 1$ (left) and $L = 3$ (right) re-uploading layers on the two moons (top) and the two circles (bottom); training points as squares and triangles, test points as crosses. The single layer draws the boundary of a product of cosines; the three-layer model has the frequencies to follow the moons. From `classifier.py`.

polynomials of low degree in the two angles: smooth, periodic and with no sharp corners. We do not claim an advantage; the point of the exercise is that the variational and the kernel route from the same encoding arrive at the same place, as Section 4.6 anticipated.

4.7.3 A QNN as a scoring function: portfolio selection

Chapter 3 selected K of n assets by searching the 2^n bit strings with QAOA. A QNN offers a different route, which scales linearly in the number of assets: learn a *score* for each asset from its features and select the K best. The single-qubit circuit of Pérez-Salinas et al. [49] is enough. With the feature vector $\boldsymbol{\varphi}_i = (\mu_i, \sigma_i, 1)$ of asset i, the standardized expected return, the standardized volatility and a bias, and weight vectors $\boldsymbol{w}_l^y, \boldsymbol{w}_l^z \in \mathbb{R}^3$ shared between all assets,

$$|\psi_i\rangle = \prod_{l=1}^{L} R_z\big(\boldsymbol{w}_l^z \cdot \boldsymbol{\varphi}_i\big)\, R_y\big(\boldsymbol{w}_l^y \cdot \boldsymbol{\varphi}_i\big)\, |0\rangle, \qquad s_i = \langle \psi_i | Z | \psi_i \rangle \in [-1, 1], \tag{4.29}$$

a re-uploading circuit in which the encoding angles are themselves trainable linear functions of the features. The scores are turned into selection probabilities by a sigmoid with a common threshold τ fixed by the budget,

$$p_i = \sigma\big(\alpha(s_i - \tau)\big), \qquad \sum_{i=1}^{n} p_i = K, \tag{4.30}$$

and the weights are trained on the mean-variance cost of Eq. (3.25) evaluated at the soft selection,

$$\mathscr{L}(\boldsymbol{w}) = -\boldsymbol{\mu}\cdot\boldsymbol{p} + \lambda\, \boldsymbol{p}^T \boldsymbol{\Sigma}\, \boldsymbol{p} + A\Big(\sum_i p_i - K\Big)^2. \tag{4.31}$$

The gradient is exact: the shift rule gives $\partial s_i/\partial(\boldsymbol{w}_l^y \cdot \boldsymbol{\varphi}_i)$ and the chain rule multiplies it by $\boldsymbol{\varphi}_i$, the implicit function theorem differentiates the threshold through the constraint in Eq. (4.30), and Eq. (4.31) is a quadratic form. Trained with Adam and $L = 3$ (18 weights, sharpness $\alpha = 5$) on the two problems of Section 3.7, `portfolio_qnn.py` scores the synthetic assets $(-0.89, -0.79, -0.86, -0.62)$ and selects B and D, and scores the stocks $(-0.25, +1.00, -1.00, +0.12)$ and selects MSFT and GOOG, in both cases the portfolio that Table 3.4 found to be the best. The soft cost at the end of training, -0.091 and -0.0004, lies below the cost of the best hard portfolio, -0.060 and $+0.0034$: the relaxation is not tight, a selection probability of 0.5 buys half an asset, and the hard portfolio is recovered by ranking. For four assets this is a sledgehammer; for a thousand, where QAOA would need a thousand qubits, it is a model with 18 parameters and a thousand single-qubit circuits.

4.8 Physics-informed quantum neural networks

4.8.1 From data to equations

Everything so far fitted a model to pairs $(\boldsymbol{x}_i, y_i)$. A *physics-informed neural network* (PINN) [89] fits a model to an equation; the classical method is developed in the chapter on neural networks for differential equations of the machine-learning text [9] and, for the Schrödinger equation of many-body systems, in the chapter on neural quantum states and physics-informed networks of the many-body text [34], and this section builds its quantum version. Let

$$\mathscr{D}[u](x) = f(x) \ \text{ on } \Omega, \qquad \mathscr{B}[u](x) = g(x) \ \text{ on } \partial\Omega, \tag{4.32}$$

be a differential equation with its boundary or initial conditions, $\mathscr{D}$ a differential operator. A PINN approximates u by a network $u_\theta(x)$ and trains it on the residuals at a set of *collocation points* $x_k \in \Omega$ and boundary points $x_k^b \in \partial\Omega$,

$$\mathscr{L}(\boldsymbol{\theta}) = \frac{1}{N_r}\sum_{k=1}^{N_r}\big[\mathscr{D}[u_\theta](x_k) - f(x_k)\big]^2 + \lambda_b \frac{1}{N_b}\sum_{k=1}^{N_b}\big[\mathscr{B}[u_\theta](x_k^b) - g(x_k^b)\big]^2, \tag{4.33}$$

the derivatives in $\mathscr{D}$ being computed by automatic differentiation of the network with respect to its input. No solution is ever sampled; the physics enters through the loss. The method is attractive where grids are expensive, in high dimension or for inverse problems, and it has well-documented failure modes [90]: the two terms of Eq. (4.33) compete, their relative weight λ_b must be tuned, and classical networks learn low frequencies first and high frequencies slowly, the *spectral bias*, so that stiff or oscillatory solutions converge badly.

A *physics-informed quantum neural network* (Q-PINN) [91, 92] replaces the network by a QNN, $u_\theta(x) = f(x;\boldsymbol{\theta})$ of Eq. (4.1) with the coordinate encoded in the angles. Two features of the quantum model make this natural. Its derivatives with respect to the input are exact and come from the same shift rules as the parameter gradient, since the input enters through rotation gates like any other angle. And its spectrum is explicit, Eq. (4.8): a Q-PINN has

no spectral bias, it has a spectral *band*, which must be chosen to contain the solution. The competition between residual and boundary terms can be removed altogether by a device older than PINNs [93]: build the boundary conditions into the trial function. For Dirichlet conditions $u(0)=g_0$, $u(1)=g_1$ on $[0,1]$,

$$\hat{u}_\theta(x) = u_\theta(x) - (1-x)\,u_\theta(0) - x\,u_\theta(1) + (1-x)g_0 + x g_1 \tag{4.34}$$

satisfies them identically for every $\boldsymbol{\theta}$; since the correction is linear in x, $\hat{u}_\theta$ has the same second derivative as u_θ, and the loss reduces to the residual term alone. For an initial-value problem with $u(0)=u_0$ and $u'(0)=v_0$,

$$\hat{u}_\theta(t) = u_\theta(t) - u_\theta(0) + u_0 + t\left[v_0 - u'_\theta(0)\right] \tag{4.35}$$

does the same. Both corrections are additive and linear in circuit evaluations, which is the property that makes the quantum version simple.

4.8.2 Derivatives with respect to the input

Let the coordinate x enter the angles of G encoding gates with a common scale, $\phi_g = sx$. If $G=1$, Eq. (4.11) applied to the encoding gate gives the *input-shift rule*

$$\frac{\partial u_\theta}{\partial x} = \frac{s}{2}\left[u_\theta\left(x+\tfrac{\pi}{2s}\right) - u_\theta\left(x-\tfrac{\pi}{2s}\right)\right], \qquad \frac{\partial^2 u_\theta}{\partial x^2} = \frac{s^2}{4}\left[u_\theta\left(x+\tfrac{\pi}{s}\right) - 2u_\theta(x) + u_\theta\left(x-\tfrac{\pi}{s}\right)\right], \tag{4.36}$$

the shifts being quarter and half turns *of the angle*; with $\phi=\pi x$ the shift in x is $1/2$, not $\pi/2$, a point on which the literature is not always careful. With several encoding gates, the case of every circuit in this chapter, one uses the product rule Eq. (4.13) or, more economically, the general shift rule Eq. (4.14) with $G = n\,n_{\text{enc}}$: all derivatives at x from $2G+1$ evaluations at $x + 2\pi m/((2G+1)s)$. Table 4.6 checks both against finite differences. For several coordinates, each encoded on its own set of qubits, the output is a trigonometric polynomial in each coordinate separately, and a derivative with respect to x_i needs $2G_i+1$ evaluations shifted in x_i alone; mixed derivatives take products of the one-dimensional stencils.

Table 4.6 Derivatives of the output of random circuits with respect to the input x (angle πx on every qubit) at $x=0.37$ by the general shift rule Eq. (4.14), the per-gate product rule Eq. (4.13) and central finite differences ($h=10^{-4}$), with the number of circuit evaluations of the two exact rules. From `qpinn.py`.

Circuit	derivative	general rule	per-gate rule	finite diff.	evals (general)	evals (gates)
$n=2$, $L=1$ ($G=2$)	u'	2.938503	2.938503	2.938502	4	4
$n=2$, $L=1$ ($G=2$)	u''	6.981992	6.981992	6.981992	5	14
$n=2$, $L=3$ ($G=6$)	u'	0.720406	0.720406	0.720406	12	12
$n=2$, $L=3$ ($G=6$)	u''	27.833288	27.833288	27.833288	13	138
$n=3$, $L=2$ ($G=6$)	u'	3.804347	3.804347	3.804346	12	12
$n=3$, $L=2$ ($G=6$)	u''	−29.664718	−29.664718	−29.664716	13	138

Everything is a linear functional The observation that organizes the implementation is that every quantity in Eq. (4.33), the solution at a collocation point, its derivatives, its boundary values, the corrections of Eqs. (4.34) and (4.35), is a *linear functional* of the circuit output, a weighted sum of evaluations at shifted inputs plus a known constant,

$$\mathscr{F}[u_\theta](x_k) = \sum_j C_{kj}\, u_\theta(\boldsymbol{\phi}_j) + c_k, \tag{4.37}$$

and therefore that its gradient with respect to the parameters is the same weighted sum of parameter-shift gradients, $\partial_\mu \mathscr{F}[u_\theta](x_k) = \sum_j C_{kj}\, \partial_\mu u_\theta(\boldsymbol{\phi}_j)$. Linear differential operators commute with the parameter shift. The class `Functional` of `qpinn.py` holds the points $\boldsymbol{\phi}_j$, the coefficient matrix C and the constants, can be added, subtracted and multiplied by point-dependent weights, and evaluates the value and the Jacobian; `QPINN.functional(X, deriv)` builds the derivative functionals from Eq. (4.14); and a residual loss is a few lines. For the Poisson problem below:

```
import numpy as np
from qnn import QNN
from qpinn import QPINN, residual_loss, train

qnn = QNN(n_qubits=2, n_layers=2)
qp = QPINN(qnn, scales=np.full((2, 1), np.pi))  # angle pi x on both qubits
x = (np.arange(20) + 0.5) / 20                  # collocation points in (0, 1)
u0, u1 = qp.functional([[0.0]]), qp.functional([[1.0]])
u_hat = qp.functional(x[:, None]) - (1 - x) * u0 - x * u1 # hard Dirichlet conditions, Eq. (4-hardbc)
d2u = qp.functional(x[:, None], deriv=(0, 0))   # u'' by the general shift rule (unchanged by the correction)
residual = -1.0 * d2u - np.pi ** 2 * np.sin(np.pi * x)    # -u'' - f at the collocation points
theta, history = train(residual_loss(residual), qnn.random_parameters(0)) # BFGS, shift-rule gradients
xs = np.linspace(0, 1, 101)
u_sol = (qp.functional(xs[:, None]) - (1 - xs) * u0 - xs * u1).value(theta)
print(f"loss {history[-1]:.1e} after {len(history) - 1} iterations; max error {np.max(np.abs(u_sol -
    np.sin(np.pi * xs))):.1e}")
```

The cost of one gradient is $2P$ circuit evaluations per stencil point; for 20 collocation points, a stencil of $2G+1 = 9$ points and $P = 8$ this is 2880 evaluations, the last column of Table 4.7, to be compared with the $N_r \times (2G+1)$ evaluations of the loss itself.

4.8.3 The Poisson equation

The first problem is the textbook example,

$$-u''(x) = \pi^2 \sin \pi x, \quad x \in (0,1), \qquad u(0) = u(1) = 0, \qquad u^\star(x) = \sin \pi x, \tag{4.38}$$

with the two-qubit circuit, the angle πx on both qubits, the trial function Eq. (4.34), twenty collocation points at the midpoints of equal cells and BFGS from the best of three random starts. Table 4.7 and Fig. 4.7 give the result for $L = 1, 2, 3$.

Table 4.7 The strong-form Q-PINN on the Poisson problem Eq. (4.38): number of parameters, BFGS iterations, final residual loss, L^2 and maximum error of $\hat{u}_\theta$ against $\sin \pi x$ on 201 points, and circuit evaluations per gradient. From `poisson.py`.

L	P	iterations	loss	L^2 error	max error	evaluations/gradient
1	4	4	1.6×10^{2}	1.6×10^{-1}	2.5×10^{-1}	800
2	8	63	7.6×10^{-17}	8.8×10^{-10}	1.2×10^{-9}	2880
3	12	108	3.7×10^{-1}	1.2×10^{-2}	1.8×10^{-2}	6240

The table has three different outcomes, and each has a reason. The single layer cannot represent the solution, whatever the optimizer does: by Eq. (4.7) its output has no $k = 1$ component, $\sin \pi x$ is nothing but a $k = 1$ component, and the optimizer stops after four iterations

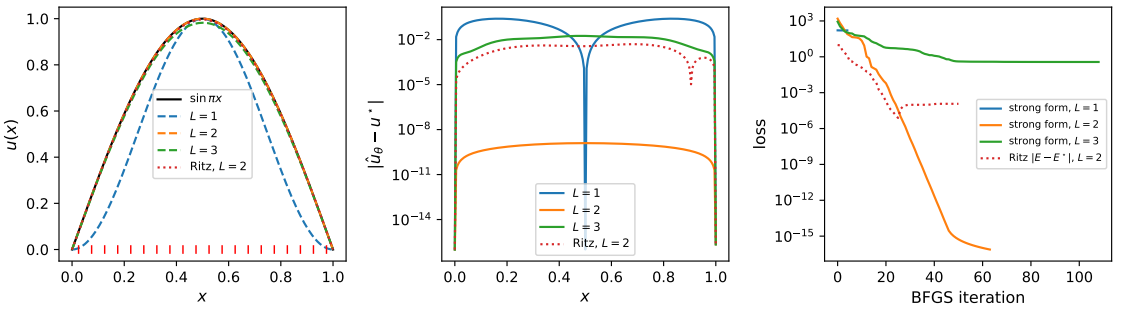


Fig. 4.7 The Poisson problem Eq. (4.38). Left: the exact solution, the strong-form Q-PINNs with $L = 1, 2, 3$ and the Ritz Q-PINN with $L = 2$; the red ticks are the collocation points. Middle: the pointwise error. Right: the loss against the BFGS iteration, for the Ritz form the distance of the energy from its exact value $-\pi^2/4$. From `poisson.py`.

at a loss of 160 and an error of 0.16. Two layers represent it exactly, and BFGS finds the exact parameters in 63 iterations, the residual loss falling to 10^{-16} and the maximum error to 10^{-9}: the solution lies in the function class and the landscape, from this start, leads to it. Three layers do not do better but worse. The function class of $L = 3$ does not contain that of $L = 2$, as Section 4.3 explained, it contains $\sin \pi x$ only approximately, and the residual landscape is rougher: all three starts end in local minima with losses between 0.4 and 1.4 and errors of a few per cent, and more iterations or Adam instead of BFGS (Exercise 7) do not change this. The general lesson, that depth is not a free parameter and that a Q-PINN must be sized to the spectrum of its solution, is worth more than the particular numbers. The residual loss, finally, is a hard loss: it weighs a frequency-k error of the solution by k^4, so that the $L = 3$ loss of 0.37 corresponds to a solution error of one per cent, and the first iterations in the right panel of Fig. 4.7 start from losses of 10^3.

The Ritz form For a self-adjoint operator the strong form is not the only option. The Poisson problem is the Euler–Lagrange equation of the energy $E[u] = \int_0^1 [\frac{1}{2}u'^2 - fu]\,dx$, whose minimum over functions with the boundary conditions is the solution, and a Q-PINN can minimize the quadrature of the energy instead of the squared residual,

$$E(\boldsymbol{\theta}) = \sum_k w_k \left[\tfrac{1}{2}\, \hat{u}'_{\boldsymbol{\theta}}(x_k)^2 - f(x_k)\, \hat{u}_{\boldsymbol{\theta}}(x_k) \right], \tag{4.39}$$

the Ritz method with the circuit as the trial space. It needs first derivatives only, its landscape is that of a quadratic functional, and it converges in 50 iterations in Fig. 4.7. Its accuracy is limited by the quadrature: with the midpoint rule on 20 cells the discrete energy of the exact solution is -2.46740, the Q-PINN reaches -2.46752, slightly *below* it, because the discretized functional has its minimum at a function that is not the exact solution, and the L^2 error settles at 3×10^{-3}. The strong form, when the solution is in the class, is exact; the Ritz form is robust. The analogy with the finite-element method is close, the circuit replacing the piecewise polynomials, with the difference that the trial space here is a curved manifold of dimension P rather than a linear space.

4.8.4 An initial-value problem and a wave equation

The harmonic oscillator With the trial function Eq. (4.35) the same machinery solves initial-value problems. For $u'' + \omega^2 u = 0$, $u(0) = 0$, $u'(0) = 1$ on $[0, 1]$ with $\omega = 2$, whose solu-

tion $\sin(\omega t)/\omega$ is not a mode of the model, the two-qubit, two-layer circuit with the angle $\pi t/2$ (the interval is a quarter of the period of the model, which gives the Fourier series room to approximate a non-periodic function) converges in 101 BFGS iterations to a residual loss of 4.5×10^{-7}, an L^2 error of 4.6×10^{-6} and a maximum error of 8×10^{-6}, with $\hat{u}(0)=0$ and $\hat{u}'(0)=1$ exactly by construction (Fig. 4.8, left). The correction $u'_\theta(0)$ in Eq. (4.35) is itself a derivative functional, and the program treats it like any other.

The wave equation For a partial differential equation in two variables each coordinate is encoded on its own qubits. The wave equation

$$\frac{\partial^2 u}{\partial t^2}=\frac{\partial^2 u}{\partial x^2},\quad u(x,0)=\sin\pi x,\quad \partial_t u(x,0)=0,\quad u(0,t)=u(1,t)=0, \tag{4.40}$$

on $(0,1)\times(0,1]$ has the solution $u^\star=\sin\pi x\cos\pi t$. We encode x on qubit 0 and t on qubit 1, both with the angle $\pi\times$coordinate, and build all four conditions into the trial function by two additive corrections,

$$\hat{u}=u_\theta-(1-x)u_\theta(0,t)-x\,u_\theta(1,t),\qquad \tilde{u}=\hat{u}(x,t)-\hat{u}(x,0)-t\,\partial_t\hat{u}(x,0)+\sin\pi x, \tag{4.41}$$

the first for the boundary, the second for the two initial conditions (it leaves the boundary values at zero because $\hat{u}(0,t)=\hat{u}(0,0)=0$). The residual $\partial_{tt}\tilde{u}-\partial_{xx}\tilde{u}$ involves, besides the second derivatives of u_θ in each variable, the mixed third derivative $\partial_{xx}\partial_t u_\theta(x,0)$, which the general shift rule provides at no extra effort, and the 8×8 collocation grid leads to a functional with 2880 stencil points for the two-layer circuit. Figure 4.8 shows the outcome, and Section 4.2 predicted it. With a *single* layer the output is the product $\cos(\pi x+\theta_0)\cos(\pi t+\theta_1)$ of Eq. (4.7), the solution is this product at $\theta_0=-\pi/2$, $\theta_1=0$, and BFGS finds it in 14 iterations with a loss of 10^{-23} and a maximum error of 2×10^{-13}: a four-parameter circuit solving a partial differential equation to machine precision, because the separable solution is one term of the separable ansatz. With two layers the model is richer and worse: the product is no longer in the class, the best of three starts ends at a loss of 8 and a maximum error of 0.4, and the dotted curves in the figure miss the exact ones by that much. A Q-PINN is a Galerkin method with a peculiar, curved trial space, and the first question to ask of it is whether the solution is in the space.

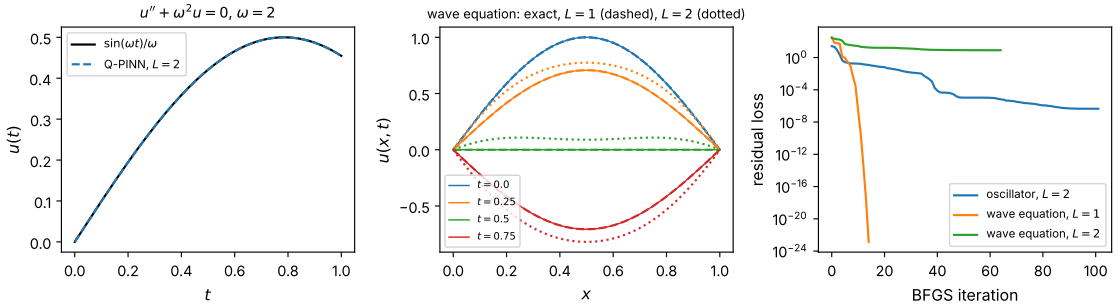


Fig. 4.8 Left: the harmonic oscillator $u''+4u=0$, $u(0)=0$, $u'(0)=1$, exact and Q-PINN. Middle: the wave equation Eq. (4.40) at four times, exact (solid), the single-layer Q-PINN (dashed, on top of the exact curves) and the two-layer Q-PINN (dotted). Right: the residual losses against the BFGS iteration. From `timedependent.py`.

4.8.5 What the method can and cannot do

The heat equation and the band The reader will have noticed that the heat equation $\partial_t u = \kappa \partial_{xx} u$, the standard example of the Q-PINN literature, is not among the examples. It is left as Exercise 9, with a warning. Its solution $e^{-\kappa\pi^2 t}\sin \pi x$ is a mode in x but an exponential in t, and an exponential is not in the Fourier band of any small circuit: with one qubit for t and L layers the model is a trigonometric polynomial of degree L in $\pi t/T$, which can approximate $e^{-\kappa\pi^2 t}$ on $[0,T]$ only roughly, and the product with the x-dependence is subject to the rigidity of Eqs. (4.7) and (4.8). Our two- and three-qubit circuits reach errors of a few per cent at best, and the published Q-PINN solutions of parabolic problems use six to ten qubits, trainable encoding scales and output scales, and still report errors of that order. The error of a Q-PINN decomposes as

$$\|\hat{u}_{\theta^\star} - u^\star\| \le \varepsilon_{\text{approx}} + \varepsilon_{\text{opt}} + \varepsilon_{\text{stat}}, \tag{4.42}$$

the distance of the solution from the function class, the distance of the optimizer's end point from the best point of the class, and the sampling error of a finite number of collocation points and, on hardware, of shots; the examples of this section exhibit each of the three in turn (the single layer on the Poisson problem, the three layers, the Ritz quadrature).

Comparison with classical PINNs Table 4.8 sets the two methods side by side. The quantum version has exact derivatives and an explicit spectrum, and no spectral bias in the classical sense; what it has instead is a spectral band of fixed width, and a trainable part whose parameters are no more numerous than the coefficients of the band, and far fewer in several variables. The classical version is a universal approximator with automatic differentiation and production-grade software, and it is what one should use for any problem one actually wants solved today. The arguments for the quantum version are the usual ones, the exponential dimension of the state space for high-dimensional problems and the possibility of running on hardware what is otherwise simulated, and they come with the usual caveats: the output is a single expectation value, read out one point at a time with shots, so that the exponential memory of the state is never exposed, and the barren plateaus of Section 4.6 limit the depth that the extra qubits could use.

Table 4.8 Classical and quantum physics-informed networks.

	classical PINN	Q-PINN
ansatz	deep network $u_\theta(x)$	circuit output $\langle O\rangle_{x,\theta}$
spatial derivatives	automatic differentiation	shift rules, Eqs. (4.36), (4.14)
parameter gradient	backpropagation	parameter-shift rule
function class	universal; spectral bias	trigonometric polynomials; fixed band
boundary conditions	penalty or trial function	penalty or trial function
cost of a gradient	one backward pass	$2P\times$(stencil points) circuits
accuracy reached here	$\sim$ noise floor	exact when the solution is in the class
maturity	production software	research, simulators

The Schrödinger equation The equation that a quantum computer should be able to solve is its own. The time-dependent Schrödinger equation $i\partial_t \psi = H\psi$ with $H = -\frac{1}{2}\partial_{xx} + V(x)$ splits into two real equations for $\psi = \psi_R + i\psi_I$, $\partial_t \psi_R = -\frac{1}{2}\partial_{xx}\psi_I + V\psi_I$ and $\partial_t \psi_I = \frac{1}{2}\partial_{xx}\psi_R - V\psi_R$, each a Q-PINN residual in the two variables (x,t) with the potential evaluated classically; two circuits, or one circuit with two observables, approximate the two parts. This is the classical simulation of a quantum evolution by a quantum circuit, and the comparison with the Hamiltonian-simulation algorithms of the companion volume [1], which implement e^{-iHt} directly on a register that *is* the wave function, is not in its favor. The time-independent equa-

tion is a different matter: minimizing the Rayleigh quotient of H over a circuit ansatz is the variational quantum eigensolver, and Fig. 4.3 was an example; a Q-PINN for $H\psi = E\psi$ on a grid is its continuous cousin, with the energy as an additional parameter and the orthogonality to the states already found as a penalty for the excited states.

4.9 Summary and perspectives

A quantum neural network is the hybrid model of Chapter 1 with every part in place: an encoding, a trainable circuit, an observable, a classical loss and a classical optimizer. We have seen what the model is, a trigonometric polynomial in the encoded data of degree set by the number of encoding gates and with coefficients set by the trainable angles, and we have used this one fact to read every experiment: the frequency band that re-uploading widens (Fig. 4.2), the cubic that one layer cannot fit and four layers fit to the noise floor (Table 4.4), the moons that a product of cosines cannot separate (Fig. 4.6), the Poisson solution that two layers represent exactly and one layer not at all (Table 4.7), the wave equation solved to machine precision by four parameters because the separable solution is a term of the separable ansatz (Fig. 4.8). Training rests on the parameter-shift rule and its generalizations, which make the gradient and all input derivatives of the model exact sums of circuit evaluations; the same rules give the quantum Fisher information, the metric of the state manifold, from which the natural gradient, imaginary-time evolution and stochastic reconfiguration follow as one equation, and which detects the redundant parameters of a circuit. The obstacle to scaling the model up is the barren plateau, the concentration of measure of random circuits, which we reproduced numerically with its 2^{-n} and 4^{-n} laws for local and global costs, and against which the only known defense is structure.

Where does this leave the question of the book? On the problems of this chapter the quantum models matched the classical ones with fewer parameters when the target was close to their function class, and lost to them when it was not, at a cost in circuit evaluations that no parameter count makes up for. The regime in which a QNN could do something a classical network cannot is the one where the data or the equation are themselves quantum, where the state space of the circuit is the natural representation and the read-out bottleneck does not apply, and the variational eigensolver of Fig. 4.3 is the prototype of that regime. The next chapter turns to generative models, where the circuit's output is not an expectation value but the distribution it samples from, and where the quantum state has room to be more than a function approximator.

4.10 Exercises

1. **The parameter-shift rule by hand.** For $f(\theta) = \langle 0|R_y(\theta)^\dagger Z R_y(\theta)|0\rangle$ compute f, apply Eq. (4.11) and Eq. (4.12), and verify both by direct differentiation. Then take two gates, $f(\theta) = \langle 0|R_y(\theta)^\dagger R_x(\pi/3)^\dagger Z R_x(\pi/3) R_y(\theta)|0\rangle$, and show that the two-point rule fails when the *same* θ is put into both gates, while Eq. (4.14) with $G = 2$ succeeds.
2. **The product form.** Prove Eq. (4.7) for the circuit of Eq. (4.3) with $L = 1$ and any number of qubits, starting from Eq. (4.6). Which Fourier coefficients of Eq. (4.8) vanish identically for $n = 2$ and for $n = 3$? Check with `QNN.fourier_coefficients`.
3. **The period of the model.** Repeat the regression of Table 4.4 with the encoding angles $\pi x/2$ and $\pi x/8$ instead of $\pi x/4$, and explain the two outcomes with Eq. (4.8). Then make the

scale s a trainable parameter, derive its gradient from the product rule Eq. (4.13), and see whether training finds the good value.

4. **Cross-entropy.** Replace the square loss of `QNNClassifier` by the cross-entropy of $p_i = (1 + af_i + b)/2$ (clipped away from 0 and 1), derive the chain rule for the gradient, and compare the accuracies of Table 4.5.
5. **Quantum Fisher information of a single qubit.** For $|\psi(\theta,\varphi)\rangle = R_z(\varphi)R_y(\theta)|0\rangle$ compute the 2×2 QFI matrix from Eq. (4.18) and interpret it as the metric of the Bloch sphere. Where is it singular, and why? Verify with `qfi_matrix` and with the overlap formula Eq. (4.21).
6. **Natural gradient on the Ising chain.** Rerun `qfi.py` with six qubits and with the transverse field $h = 0.5$ and $h = 2$. How does the condition number of F at the start depend on h, and does the advantage of the natural gradient follow it? Add a diagonal approximation, $F \to \mathrm{diag}(F)$, which costs P overlaps instead of P^2, and compare.
7. **Depth and local minima.** Train the three-layer Poisson Q-PINN of Table 4.7 with Adam for 1000 steps and with BFGS from ten random starts. Does any start reach the loss of the two-layer model? Then initialize the three-layer circuit from the two-layer solution with the third layer at random small angles, and explain what you observe with the non-nesting argument of Section 4.3.
8. **Neumann conditions.** Solve $-u'' = \pi^2 \cos \pi x$ on $(0,1)$ with $u'(0) = u'(1) = 0$, exact solution $\cos \pi x$ up to a constant, with a trial function that builds the two derivative conditions in additively (the correction is quadratic in x), and fix the constant by a penalty on $\int u\,dx$.
9. **The heat equation.** Solve $\partial_t u = \partial_{xx} u$ on $(0,1)\times(0,1/4]$ with $u(x,0) = \sin \pi x$ and $u(0,t) = u(1,t) = 0$, exact solution $e^{-\pi^2 t}\sin \pi x$, with x on two qubits and t on one, and with the trial function $\tilde{u} = \hat{u}(x,t) - \hat{u}(x,0) + \sin \pi x$, $\hat{u}$ as in Eq. (4.41). Measure the error for $L = 1,2,3$, then encode t on two qubits with the angles $\pi t/T$ and $2\pi t/T$ (the general shift rule then needs the gates' frequencies as integer multiples of a base frequency; extend `QPINN` accordingly). Which of the three terms of Eq. (4.42) dominates?
10. **Shot noise in the gradient.** Replace the exact expectation values of `QNN.forward` by binomial estimates from S shots, as `QuantumKernel` did in Chapter 2, and compare the variance of the shift-rule gradient with that of a central finite difference with step h, for $S = 100, 1000$ and $h = 0.1, 0.01$. Then train the two-moons classifier with $S = 200$ shots per evaluation and Adam, and comment on the accuracy and on the number of circuit runs.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter4/` (NumPy and matplotlib only), which import `common/gates.py`, `statevector.py`, `optimize.py` (BFGS and Adam), `neuralnet.py` (the classical network), `datasets.py` and `bookplot.py`, and are imported by the notebook `Notebooks/chapter4/chapter4.ipynb`.

`qnn.py` The class `QNN` of Eqs. (4.1)–(4.4) on batches of inputs and of parameter sets: state, output, parameter-shift gradient, state derivatives Eq. (4.20), Fourier coefficients; `trig_weights` for Eq. (4.14). Self-test and Fig. 4.2. 2 s.

`training.py` `QNNRegressor` and `QNNClassifier`: Adam training on the square loss with the output scale and offset.

`qfi.py` `qfi_matrix` (Eq. (4.18)), `qfi_overlap` (Eq. (4.21)), the natural gradient and the transverse-field Ising ground state: Table 4.1, Fig. 4.3. 11 s.

`barren.py` Gradient variances of random circuits for the local and the global cost: Table 4.2, Fig. 4.4. 65 s.

`regression.py` The noisy cubic against the classical network: Table 4.4, Fig. 4.5. 17 s.

`classifier.py` The four two-class problems: Table 4.5, Fig. 4.6. 17 s.

`portfolio_qnn.py` The single-qubit scoring circuit Eq. (4.29) on the two portfolio problems of Chapter 3. 5 s.

`qpinn.py` `Functional`, `QPINN` (general and per-gate derivative functionals), `residual_loss`, `ritz_loss`, `train`; Table 4.6. 1 s.

`poisson.py` The Poisson problem in strong and Ritz form: Table 4.7, Fig. 4.7. 11 s.

`timedependent.py` The harmonic oscillator and the wave equation: Fig. 4.8. 30 s.

Chapter 5
Quantum Boltzmann Machines

Abstract The Boltzmann machine is statistical mechanics turned into a learning algorithm: a Boltzmann distribution over binary units whose energy function is shaped until its samples reproduce the data, trained by comparing correlation functions of the data with those of the model. This chapter develops the classical theory, the restricted model with its conditional independence, the exact likelihood gradient and the contrastive-divergence approximation whose bias we measure against the exact gradient, and then replaces the energy by a Hamiltonian with non-commuting terms and the distribution by its Gibbs state. The quantum Boltzmann machine inherits the negative phase of the classical model unchanged, through the cyclicity of the trace, but its positive phase becomes an imaginary-time correlation function by Duhamel's formula, and the training rule of the literature is shown to be the gradient of a Golden–Thompson bound that cannot see the visible transverse fields. Exact computations on small models show that on classical data the likelihood drives the transverse fields to zero, so that the model falls back on its classical limit, while on quantum data the quantum relative entropy is a convex loss with a gradient of expectation values, Hamiltonians are recovered exactly from their thermal states, and the operator content of the model decides what can be learned. The last part treats the quantum computer's share of the work: the free-energy variational principle, the thermofield double, and the variational quantum thermalizer, which prepares a Gibbs state with the circuits of Chapter 4 and a classical latent distribution whose entropy is known in closed form. The chapter closes with the connections between the two trainable quantum models of the book.

5.1 Introduction

Every model of the preceding chapters was a function. The support-vector machine of Chapter 2 and the quantum neural network of Chapter 4 map an input $\boldsymbol{x}$ to a label or a number, the QAOA of Chapter 3 maps a set of angles to an energy, and training meant adjusting parameters until the function agreed with the data at the training points. The *Boltzmann machine* of this chapter is a different kind of object: a probability distribution. It is trained not to reproduce a function but to reproduce the statistics of a set of samples, so that new samples drawn from the model are indistinguishable, as far as the model can tell, from the data. Models of this kind are called *generative*, and the Boltzmann machine is their oldest member, older than the neural networks of Chapter 4 by a few years [94, 95]. The classical machine is developed at length in the chapter on Boltzmann machines of the machine-learning text [9], and its use as a wave-function ansatz, the neural quantum state, in the chapter on Boltzmann machines and generative models of the many-body text [34]; Section 5.2 summarizes what this chapter needs from the first.

The reason it belongs in a book on quantum machine learning for physicists is that the Boltzmann machine *is* statistical mechanics, and the reader of the companion volume [1] will meet here, as learning problems, the objects of its many-body primer and of its chapter on mapping Hamiltonians to qubits: Pauli-string Hamiltonians, the transverse-field Ising and Heisenberg chains, thermal states and partial traces. Its distribution is the Boltzmann distribution of an energy function of binary variables, its partition function is the obstacle to training, its training rule compares correlation functions of the data with correlation functions of the model, and the sampling methods used to estimate the latter are the Monte Carlo methods of spin systems. The *quantum Boltzmann machine* (QBM) [96] takes the last step that a physicist would take: it replaces the classical energy by a Hamiltonian with non-commuting terms, the Boltzmann distribution by the thermal (Gibbs) state $\rho = e^{-\beta H}/Z$, and the probability of a configuration by the Born rule. The model is then a quantum many-body system at finite temperature, and the questions of this book acquire a new form. What does the non-commutativity buy when the data are classical bit strings? What can be learned when the data are themselves quantum states? And how does one prepare a Gibbs state, rather than a ground state, on a quantum computer?

The chapter answers these questions in order, and the answers are mixed in a way that is instructive. Section 5.2 develops the classical theory: energy-based models, the restricted Boltzmann machine, the exact gradient of the likelihood, and the contrastive divergence approximation that made the model practical, with its bias measured against the exact gradient on problems small enough to enumerate. Section 5.3 defines the quantum model and derives its gradient, which is where the non-commutativity bites: the derivative of $e^{-\beta H}$ with respect to a coupling is no longer an expectation value but an integral over imaginary time (Duhamel's formula), and the practical training rule of the literature is the gradient of an upper bound on the loss rather than of the loss itself. Section 5.4 trains both models on classical data and finds, as the exact computation must, that the transverse fields which make the model quantum are driven to zero: a classical distribution is best fit by a classical model. Section 5.5 turns to quantum data, where the picture reverses. When the target is a density matrix, the quantum relative entropy is a convex loss whose gradient is a difference of expectation values, a Hamiltonian in the model family is recovered exactly from its Gibbs state, and a Hamiltonian outside it is approximated only as well as the operator content of the model allows. Section 5.6 is about the quantum computer's part of the job: the variational principle for the free energy, the thermofield double, and the variational quantum thermalizer, which prepares a Gibbs state with the circuits of Chapter 4 and a classical latent distribution. Section 5.7 places the QBM next to the QNN and next to the transverse-field Ising model that has accompanied us since Chapter 3, and Section 5.8 closes the chapter.

As in the previous chapters, every number is computed by a program in `BookPrograms/chapter5/`, and all models are small enough (at most seven qubits) for every quantity, including the partition function and the exact gradient, to be evaluated by dense linear algebra. This is deliberate. The approximations that make Boltzmann machines usable at scale, classical and quantum, are approximations to quantities we can here compute exactly, and the point of the chapter is to see what they cost.

5.2 Energy-based models and the Boltzmann machine

5.2.1 Learning as the minimization of a divergence

The data are N samples $\boldsymbol{v}^{(1)}, \ldots, \boldsymbol{v}^{(N)}$ of a vector of n_v binary variables, $\boldsymbol{v} \in \{0,1\}^{n_v}$, drawn from an unknown distribution $\pi(\boldsymbol{v})$; the model is a family of distributions $p_{\boldsymbol{\theta}}(\boldsymbol{v})$ over the same

2^{n_v} configurations. The natural measure of the distance from the data to the model is the Kullback–Leibler divergence

$$\mathrm{KL}(\pi \,\|\, p_{\boldsymbol{\theta}}) = \sum_{\boldsymbol{v}} \pi(\boldsymbol{v}) \ln \frac{\pi(\boldsymbol{v})}{p_{\boldsymbol{\theta}}(\boldsymbol{v})} = -S(\pi) - \sum_{\boldsymbol{v}} \pi(\boldsymbol{v}) \ln p_{\boldsymbol{\theta}}(\boldsymbol{v}), \tag{5.1}$$

which is non-negative and zero only when the two distributions coincide. The first term, minus the entropy of the data, does not depend on $\boldsymbol{\theta}$; the second is the expected negative log-likelihood of the model under the data,

$$\mathscr{L}(\boldsymbol{\theta}) = -\sum_{\boldsymbol{v}} \pi(\boldsymbol{v}) \ln p_{\boldsymbol{\theta}}(\boldsymbol{v}) \approx -\frac{1}{N} \sum_{k=1}^{N} \ln p_{\boldsymbol{\theta}}(\boldsymbol{v}^{(k)}), \tag{5.2}$$

where the second form replaces the unknown π by the empirical distribution of the samples. Minimizing the divergence and maximizing the likelihood are the same thing, and $\mathscr{L}(\boldsymbol{\theta}) - S(\pi) = \mathrm{KL}(\pi \| p_{\boldsymbol{\theta}}) \geq 0$ is the quantity we tabulate below: it is zero exactly when the model has learned the data. On the toy distributions of this chapter π is known, so we can use Eq. (5.2) with the exact π and reserve the sampled form for the methods that need samples.

5.2.2 Energy-based models

An energy-based model assigns to every configuration an energy $E_{\boldsymbol{\theta}}(\boldsymbol{v})$ and defines the probabilities by the Boltzmann distribution at unit temperature,

$$p_{\boldsymbol{\theta}}(\boldsymbol{v}) = \frac{e^{-E_{\boldsymbol{\theta}}(\boldsymbol{v})}}{Z(\boldsymbol{\theta})}, \qquad Z(\boldsymbol{\theta}) = \sum_{\boldsymbol{v}} e^{-E_{\boldsymbol{\theta}}(\boldsymbol{v})}. \tag{5.3}$$

The temperature is not a separate parameter because it can be absorbed into $\boldsymbol{\theta}$. Low-energy configurations are probable, and learning means shaping the energy landscape so that its valleys sit on the data. The gradient of the negative log-likelihood has a form that every later formula of this chapter will echo:

$$\frac{\partial \mathscr{L}}{\partial \theta_\mu} = \sum_{\boldsymbol{v}} \pi(\boldsymbol{v}) \frac{\partial E_{\boldsymbol{\theta}}(\boldsymbol{v})}{\partial \theta_\mu} + \frac{\partial \ln Z}{\partial \theta_\mu} = \left\langle \frac{\partial E}{\partial \theta_\mu} \right\rangle_{\text{data}} - \left\langle \frac{\partial E}{\partial \theta_\mu} \right\rangle_{\text{model}}, \tag{5.4}$$

since $\partial_\mu \ln Z = -\sum_{\boldsymbol{v}} p_{\boldsymbol{\theta}}(\boldsymbol{v}) \partial_\mu E(\boldsymbol{v})$. Gradient descent lowers the energy of the data (the *positive phase*) and raises the energy of what the model currently believes in (the *negative phase*), and stops when the two averages agree. The positive phase is a sum over the samples and costs nothing. The negative phase is an expectation value under the model, which for n_v variables is a sum over 2^{n_v} configurations: it is the partition function in disguise, and everything that is hard about Boltzmann machines is in this term.

5.2.3 The Boltzmann machine and its restricted form

A Boltzmann machine [94] is an energy-based model with the quadratic energy of an Ising model in the variables $\{0,1\}$, $E(\boldsymbol{s}) = -\sum_i a_i s_i - \sum_{i<j} W_{ij} s_i s_j$, in which only a subset of the units, the *visible* units $\boldsymbol{v}$, carries data. The remaining *hidden* units $\boldsymbol{h} \in \{0,1\}^{n_h}$ are summed over, so that the model distribution of the visible units is the marginal

$$p_{\boldsymbol{\theta}}(\boldsymbol{v}) = \sum_{\boldsymbol{h}} p_{\boldsymbol{\theta}}(\boldsymbol{v},\boldsymbol{h}) = \frac{1}{Z}\sum_{\boldsymbol{h}} e^{-E(\boldsymbol{v},\boldsymbol{h})}. \tag{5.5}$$

Hidden units matter because a quadratic energy in the visible units alone can only represent distributions with pairwise structure; the parity distribution of two bits, which is the XOR problem of Chapter 2 read as a distribution, is the standard example of one it cannot. With hidden units the marginal of a quadratic energy is a non-quadratic function of $\boldsymbol{v}$, and with enough hidden units any distribution over $\{0,1\}^{n_v}$ can be approximated [97].

The *restricted* Boltzmann machine (RBM) [95, 98] keeps only the couplings between the two layers,

$$E(\boldsymbol{v},\boldsymbol{h}) = -\boldsymbol{a}\cdot\boldsymbol{v} - \boldsymbol{b}\cdot\boldsymbol{h} - \boldsymbol{v}^T\boldsymbol{W}\boldsymbol{h}, \tag{5.6}$$

with visible biases $\boldsymbol{a}\in\mathbb{R}^{n_v}$, hidden biases $\boldsymbol{b}\in\mathbb{R}^{n_h}$ and weights $\boldsymbol{W}\in\mathbb{R}^{n_v\times n_h}$, so that $\boldsymbol{\theta}=(\boldsymbol{a},\boldsymbol{b},\boldsymbol{W})$ has $n_v+n_h+n_vn_h$ components. The restriction is what makes the model usable. Because no two hidden units interact, the hidden units are conditionally independent given the visible ones, and vice versa,

$$p(h_j=1\,|\,\boldsymbol{v}) = \sigma\big(b_j + (\boldsymbol{W}^T\boldsymbol{v})_j\big), \qquad p(v_i=1\,|\,\boldsymbol{h}) = \sigma\big(a_i + (\boldsymbol{W}\boldsymbol{h})_i\big), \tag{5.7}$$

with the logistic function $\sigma(x)=1/(1+e^{-x})$: the conditional distribution of a layer factorizes, and a whole layer can be sampled at once. The same independence lets the sum over the hidden units in Eq. (5.5) be done in closed form,

$$p_{\boldsymbol{\theta}}(\boldsymbol{v}) = \frac{1}{Z}e^{-F(\boldsymbol{v})}, \qquad F(\boldsymbol{v}) = -\boldsymbol{a}\cdot\boldsymbol{v} - \sum_{j=1}^{n_h}\ln\Big(1+e^{\,b_j+(\boldsymbol{W}^T\boldsymbol{v})_j}\Big), \tag{5.8}$$

where $F(\boldsymbol{v})$ is called the free energy of the visible configuration and $Z=\sum_{\boldsymbol{v}}e^{-F(\boldsymbol{v})}$. The model of the visible units is therefore an energy-based model with the non-quadratic energy $F(\boldsymbol{v})$; each hidden unit contributes a soft-plus function of a linear combination of the visible bits, which is the sense in which an RBM is a one-hidden-layer network. The gradient Eq. (5.4) applied to Eq. (5.6) and averaged over the hidden units gives the celebrated rule

$$\begin{aligned}
\frac{\partial \mathscr{L}}{\partial W_{ij}} &= \langle v_i\, p(h_j=1|\boldsymbol{v})\rangle_{\text{model}} - \langle v_i\, p(h_j=1|\boldsymbol{v})\rangle_{\text{data}},\\
\frac{\partial \mathscr{L}}{\partial a_i} &= \langle v_i\rangle_{\text{model}} - \langle v_i\rangle_{\text{data}}, \qquad \frac{\partial \mathscr{L}}{\partial b_j} = \langle p(h_j=1|\boldsymbol{v})\rangle_{\text{model}} - \langle p(h_j=1|\boldsymbol{v})\rangle_{\text{data}},
\end{aligned} \tag{5.9}$$

in which the hidden units have been replaced by their conditional means, which is exact. The three quantities inside the averages are the sufficient statistics of the model; the data average runs over the samples, the model average over $p_{\boldsymbol{\theta}}(\boldsymbol{v})$. For $n_v\le 20$ or so the model average can be computed by enumeration from Eq. (5.8), and the class `RBM` of `rbm.py` does exactly this: it stores $\boldsymbol{\theta}$ as one flat vector, evaluates $\ln p(\boldsymbol{v})$ for all 2^{n_v} patterns with a stable log-sum-exp, and returns the gradient Eq. (5.9) as the difference of two weighted statistics vectors. The program checks it against finite differences to 4×10^{-10}.

5.2.4 Gibbs sampling and contrastive divergence

When n_v is large the model average must be estimated by sampling, and Eq. (5.7) gives the sampler: a *block Gibbs* step draws all hidden units from $p(\boldsymbol{h}|\boldsymbol{v})$ and then all visible units from $p(\boldsymbol{v}|\boldsymbol{h})$,

$$\boldsymbol{v}_0 \;\to\; \boldsymbol{h}_0 \;\to\; \boldsymbol{v}_1 \;\to\; \boldsymbol{h}_1 \;\to\; \cdots \;\to\; \boldsymbol{v}_k, \tag{5.10}$$

and the chain converges to the model distribution $p_{\boldsymbol{\theta}}(\boldsymbol{v})$ as $k \to \infty$ (the transition kernel has $p_{\boldsymbol{\theta}}$ as its stationary distribution and is ergodic because every configuration has non-zero probability). Running the chain to equilibrium at every gradient step is expensive, and the mixing time grows as the model learns and its energy valleys deepen.

Hinton's *contrastive divergence* (CD) [98] is the observation that one does not need equilibrium. Start the chain at the data, $\boldsymbol{v}_0 \sim \pi$, run it for k steps, and use the distribution p_k of $\boldsymbol{v}_k$ in place of the model distribution in the negative phase:

$$\frac{\partial \mathscr{L}}{\partial \theta_\mu} \approx \left\langle \frac{\partial E}{\partial \theta_\mu} \right\rangle_{p_0} - \left\langle \frac{\partial E}{\partial \theta_\mu} \right\rangle_{p_k} \qquad \text{(CD-}k\text{)}, \tag{5.11}$$

with the hidden units again replaced by their conditional means. For $k = 1$ each gradient step costs one reconstruction of the data, and the rule has an appealing reading: the model is pushed to make the data more probable and its own one-step reconstruction of the data less probable, so that the chain started at the data goes nowhere. The name comes from the objective this rule approximately descends,

$$\mathrm{CD}_k = \mathrm{KL}(p_0 \,\|\, p_\infty) - \mathrm{KL}(p_k \,\|\, p_\infty), \tag{5.12}$$

the difference of two divergences from the model: the first is the likelihood objective Eq. (5.1) up to a constant, the second is the divergence of the partially relaxed chain, and the difference is non-negative and vanishes when the data distribution is stationary under the chain, that is, when the model has learned the data. The exact gradient of Eq. (5.12) contains, besides the two averages of Eq. (5.11), a term from the dependence of p_k on $\boldsymbol{\theta}$ through the transition kernel, and CD drops it. This is the approximation: CD-k is not the gradient of any function, and its fixed points are not the maximum of the likelihood [99, 100]. The bias decreases with k, and it is small when the model is close to the data (the chain then barely moves and both neglected quantities are small), which is why CD works far better than it has any right to.

Persistent contrastive divergence (PCD) [101] removes the restart: a set of chains is initialized once and advanced one Gibbs step per parameter update, the negative phase being averaged over the current chain states. If the parameters change slowly the chains remain close to equilibrium, and the estimate approaches the true gradient at the cost of a small learning rate and many chains. Both estimators are implemented in `rbm.py` as `gradient_cd` and `gradient_pcd`, on the same statistics function as the exact gradient, so that the only difference between the three is where the negative-phase configurations come from.

Table 5.1 Classical RBMs on two toy distributions: the divergence $\mathrm{KL}(\pi \| p_{\boldsymbol{\theta}}) = \mathscr{L} - S(\pi)$ after 3000 steps of plain gradient descent (rate 0.5) with the exact gradient, contrastive divergence with $k = 1$ and $k = 10$ steps, and persistent CD (rate 0.2, 50 chains); CD and PCD see a data set of 200 samples of π. XOR is the even-parity distribution on two bits, bars and stripes the uniform distribution over the six 2×2 images whose rows or columns are constant (patterns 0000, 0011, 1100, 1111, 0101, 1010). From `rbm.py`.

Target	n_h	exact	CD-1	CD-10	PCD
XOR ($n_v = 2$)	2	0.0044	0.0967	0.0228	0.0331
bars and stripes ($n_v = 4$)	4	0.0230	0.0823	0.0531	0.4814

Table 5.1 and Fig. 5.1 measure the estimators on the two distributions that will be used throughout the chapter, with the exact gradient as the reference. The exact gradient descends smoothly and is still descending after 3000 steps (the RBM likelihood has no finite maximum on a target with zero-probability patterns: the weights must grow without bound to drive those probabilities to zero, and the divergence of 0.02 for bars and stripes is a matter of patience, not of capacity). CD-1 stalls at a divergence four times larger and its learned

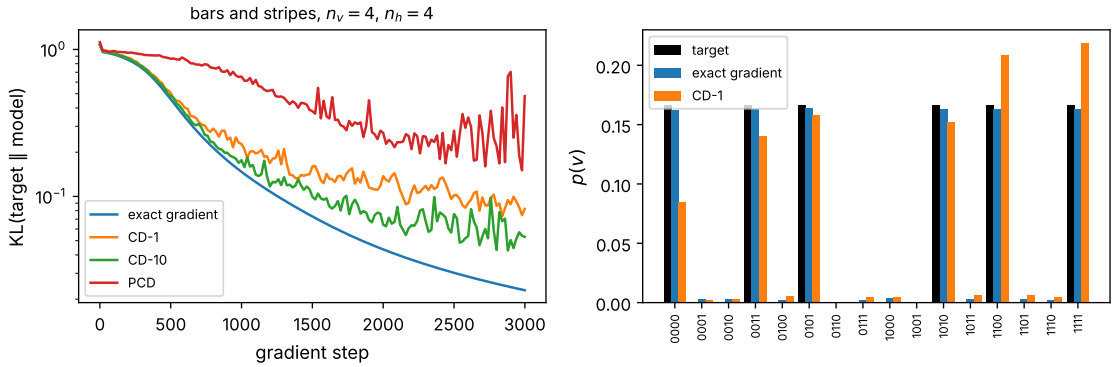


Fig. 5.1 Left: the divergence of Table 5.1 against the gradient step for the bars-and-stripes RBM with the four estimators of the negative phase. Right: the learned distributions with the exact gradient and with CD-1 against the target. From `rbm.py`.

distribution, on the right of Fig. 5.1, is visibly tilted: the bias of Eq. (5.11) moves probability between the six patterns and leaks some onto the ten others. CD-10 halves the gap, as the decreasing bias with k predicts. PCD with the learning rate and the 50 chains of the table is the worst of the four and still fluctuating at the end, a reminder that the persistent chains track the model only when the parameters move slowly; with a tenth of the rate and ten times the chains it overtakes CD-1 (Exercise 3). The point to retain is not that one estimator is best but that all of them are approximations to a quantity, the negative phase, that the quantum model of the next section inherits unchanged, together with one new difficulty of its own.

The listing below trains the XOR machine with the exact gradient and with CD-1 from the same start, and prints the two divergences.

```
import numpy as np
from rbm import RBM, parity, kl_divergence

target = parity(2)                    # p(00) = p(11) = 1/2: the XOR distribution
for method in ["exact", "cd"]:
    rbm = RBM(n_visible=2, n_hidden=2, rng=0)
    hist = rbm.fit(target, method=method, k=1, steps=3000, lr=0.5)
    print(f"{method:>5}: KL(target || model) = {hist[-1]:.4f}, "
          f"p(v) = {np.round(rbm.probabilities(), 3)}")
```

5.3 The quantum Boltzmann machine

5.3.1 Definition

A Boltzmann machine written in spin language has the energy $E(\boldsymbol{z}) = -\sum_i a_i z_i - \sum_{i<j} W_{ij} z_i z_j$ with $z_i = \pm 1$, and this energy is the diagonal of the Hamiltonian $H = -\sum_i a_i Z_i - \sum_{i<j} W_{ij} Z_i Z_j$ in the computational basis. The Boltzmann distribution Eq. (5.3) is the diagonal of the Gibbs state e^{-H}/Z of this Hamiltonian, and because H is diagonal the Gibbs state has nothing but a diagonal. The quantum Boltzmann machine of Amin and coworkers [96] adds terms that do not commute with the Z's. In the form used in this chapter,

$$H(\boldsymbol{\theta}) = -\sum_{\mu=1}^{P} \theta_\mu \Lambda_\mu, \qquad \rho(\boldsymbol{\theta}) = \frac{e^{-\beta H(\boldsymbol{\theta})}}{Z(\boldsymbol{\theta})}, \qquad Z(\boldsymbol{\theta}) = \operatorname{Tr} e^{-\beta H(\boldsymbol{\theta})}, \tag{5.13}$$

where the Λ_μ are Pauli strings on $n = n_v + n_h$ qubits chosen once and for all, the θ_μ are the trainable couplings and β is a fixed inverse temperature. The *restricted* QBM, the analog of Eq. (5.6) drawn in Fig. 5.2, takes

$$H = -\sum_{i\in v} a_i Z_i - \sum_{j\in h} b_j Z_j - \sum_{i\in v}\sum_{j\in h} W_{ij} Z_i Z_j - \sum_{q\in v\cup h} \Gamma_q X_q, \tag{5.14}$$

and the *transverse fields* Γ_q are the only quantum ingredient: with all $\Gamma_q = 0$ the model is the classical RBM in spin variables, $z_i = 1 - 2v_i$. The data are measured in the computational basis, so the probability the model assigns to a visible configuration $\boldsymbol{v}$ is the Born-rule probability of the projector onto that configuration of the visible qubits, with the hidden qubits traced out:

$$p_{\boldsymbol{\theta}}(\boldsymbol{v}) = \mathrm{Tr}\big[\Pi_{\boldsymbol{v}}\, \rho(\boldsymbol{\theta})\big], \qquad \Pi_{\boldsymbol{v}} = |\boldsymbol{v}\rangle\langle\boldsymbol{v}| \otimes I_h. \tag{5.15}$$

Equation (5.15) is the counterpart of the marginal Eq. (5.5): summing the diagonal of ρ over the hidden labels is the partial trace. Two things are new. The state ρ is not diagonal, so the model also assigns probabilities to measurements in other bases, and a QBM with hidden units can leave the visible qubits in a mixed state $\rho_v = \mathrm{Tr}_h \rho$ with coherences; and the diagonal of $e^{-\beta H}$ is no longer $e^{-\beta E(\boldsymbol{z})}$ for the quadratic energy of the model, or for any energy built from its terms, because the matrix exponential of a non-diagonal H mixes configurations. The model distribution over the visible bits is still a positive function on 2^{n_v} points, and the question of this chapter is whether it is a more useful family of such functions than the classical one. The loss is the same negative log-likelihood Eq. (5.2) with Eq. (5.15) inserted, and we keep calling it $\mathscr{L}(\boldsymbol{\theta})$.

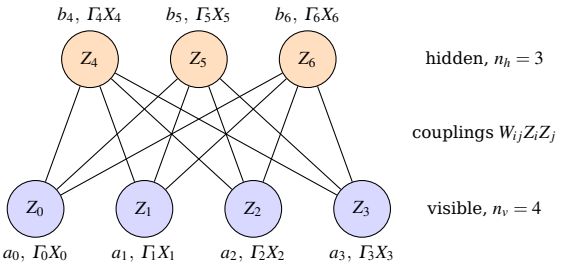


Fig. 5.2 The restricted quantum Boltzmann machine of Eq. (5.14) used for the bars-and-stripes data in Section 5.4: four visible and three hidden qubits, ZZ couplings W_{ij} between the layers only, longitudinal fields a_i, b_j and transverse fields Γ_q on every qubit. With $\Gamma_q = 0$ it is the classical RBM of Eq. (5.6) in spin variables. The model has $P = 2(n_v + n_h) + n_v n_h = 26$ parameters.

The class `QBM` of `qbm.py` represents a model by a list of Pauli strings (as dictionaries `{qubit: "X"/"Y"/"Z"}`), builds $H(\boldsymbol{\theta})$ and $\rho(\boldsymbol{\theta})$ as dense matrices, and reads off $p_{\boldsymbol{\theta}}(\boldsymbol{v})$ from the diagonal of ρ with a $2^{n_v} \times 2^n$ summation matrix for the partial trace. The helper `rqbm_terms` produces the terms of Eq. (5.14); `ising_terms` produces fully visible chains with chosen fields and couplings, which Section 5.5 needs. The thermal state, the partial trace, the entropies and the other tools of finite-temperature quantum mechanics live in `gibbs.py`.

5.3.2 The gradient: Duhamel's formula

Differentiating the loss requires the derivative of Eq. (5.15) with respect to a coupling,

$$\frac{\partial \ln p_{\boldsymbol{\theta}}(\boldsymbol{v})}{\partial \theta_\mu} = \frac{\mathrm{Tr}\big[\Pi_{\boldsymbol{v}}\, \partial_\mu e^{-\beta H}\big]}{Z\, p_{\boldsymbol{\theta}}(\boldsymbol{v})} - \frac{\partial \ln Z}{\partial \theta_\mu}, \tag{5.16}$$

and therefore the derivative of a matrix exponential with respect to a parameter of its exponent. When H is diagonal, or more generally when $\partial_\mu H$ commutes with H, the exponential differentiates like a number, $\partial_\mu e^{-\beta H} = -\beta\, \partial_\mu H\, e^{-\beta H} = \beta \Lambda_\mu e^{-\beta H}$, and Eq. (5.16) becomes

$$\frac{\partial \mathscr{L}}{\partial \theta_\mu} = \beta \Big(\langle \Lambda_\mu \rangle_\rho - \sum_{\boldsymbol{v}} \pi(\boldsymbol{v})\, \langle \Lambda_\mu \rangle_{\rho|\boldsymbol{v}} \Big), \qquad \langle \Lambda_\mu \rangle_{\rho|\boldsymbol{v}} = \frac{\mathrm{Tr}[\Pi_{\boldsymbol{v}} \Lambda_\mu \rho]}{\mathrm{Tr}[\Pi_{\boldsymbol{v}} \rho]}, \tag{5.17}$$

the classical rule Eq. (5.4) once more: the model expectation of the operator conjugate to θ_μ minus its expectation in the state *clamped* to the data, the visible qubits projected onto $\boldsymbol{v}$ and the hidden ones in the conditional thermal state. Two commuting operators have a common eigenbasis and the quantum model is, in that basis, a classical one.

When $[\Lambda_\mu, H] \neq 0$ this fails, because $e^{-\beta(H+\varepsilon\Lambda)} \neq e^{-\beta H} e^{-\beta \varepsilon \Lambda}$. The correct derivative is *Duhamel's formula*

$$\frac{\partial}{\partial \theta_\mu} e^{-\beta H(\boldsymbol{\theta})} = -\int_0^\beta ds\, e^{-(\beta - s)H}\, \frac{\partial H}{\partial \theta_\mu}\, e^{-sH} = \int_0^\beta ds\, e^{-(\beta-s)H}\, \Lambda_\mu\, e^{-sH}, \tag{5.18}$$

which follows from differentiating the identity $e^{-\beta H} = e^{-(\beta-s)H} e^{-sH}$ with respect to s under the perturbation and integrating (Exercise 4). The perturbing operator is inserted at every imaginary time s between 0 and β and the insertions are averaged: this is the imaginary-time correlation function of many-body perturbation theory, and it is what replaces the simple product of the commuting case. In the eigenbasis of H, $H = \sum_k e_k |k\rangle\langle k|$, the integral is elementary and gives the matrix elements

$$\langle k|\, \partial_\mu e^{-\beta H}\, |l\rangle = \langle k|\Lambda_\mu|l\rangle\, \frac{e^{-\beta e_l} - e^{-\beta e_k}}{e_k - e_l} \qquad (k \neq l), \qquad \langle k|\, \partial_\mu e^{-\beta H}\, |k\rangle = \beta\, e^{-\beta e_k} \langle k|\Lambda_\mu|k\rangle, \tag{5.19}$$

divided differences of the exponential between pairs of eigenvalues, which reduce to the diagonal rule as $e_l \to e_k$. The function `exp_derivative` of `gibbs.py` evaluates Eq. (5.19) from one diagonalization of H, and `gibbs.py` checks it against a central finite difference on a random three-qubit Hamiltonian to 9×10^{-11}. This is the exact gradient of the QBM, available whenever H can be diagonalized, and it is what the programs of this chapter use as the reference.

Inserting Eq. (5.18) into Eq. (5.16) gives the two phases of the quantum model. The negative phase, the derivative of the partition function, simplifies because the trace is cyclic:

$$\frac{\partial \ln Z}{\partial \theta_\mu} = \frac{1}{Z} \int_0^\beta ds\, \mathrm{Tr}\big[e^{-(\beta-s)H} \Lambda_\mu e^{-sH}\big] = \frac{1}{Z} \int_0^\beta ds\, \mathrm{Tr}\big[\Lambda_\mu e^{-\beta H}\big] = \beta\, \langle \Lambda_\mu \rangle_\rho. \tag{5.20}$$

The imaginary-time structure disappears under the trace, and the negative phase of the QBM is the same thermal expectation value as in the classical model: an observable of the Gibbs state, measurable on any device that prepares it. The positive phase does not simplify. The projector $\Pi_{\boldsymbol{v}}$ does not commute with H and sits inside the trace next to the integrand, so that

$$\frac{\partial \mathscr{L}}{\partial \theta_\mu} = \beta \Big(\langle \Lambda_\mu \rangle_\rho - \sum_{\boldsymbol{v}} \pi(\boldsymbol{v})\, \langle \Lambda_\mu \rangle_{\boldsymbol{v}}^{(\tau)} \Big), \qquad \langle \Lambda_\mu \rangle_{\boldsymbol{v}}^{(\tau)} = \frac{1}{\beta} \int_0^\beta ds\, \frac{\mathrm{Tr}\big[\Pi_{\boldsymbol{v}}\, e^{-(\beta-s)H} \Lambda_\mu e^{-sH}\big]}{Z\, p_{\boldsymbol{\theta}}(\boldsymbol{v})}, \tag{5.21}$$

with an imaginary-time-averaged clamped expectation in the place of the clamped expectation of Eq. (5.17), to which it reduces when $[\Pi_{\boldsymbol{v}}, H] = 0$. The quantity $\langle \Lambda_\mu \rangle_{\boldsymbol{v}}^{(\tau)}$ is not an expectation value of any state and there is no known efficient way to measure it on a quantum device;

it can be computed only when H can be diagonalized. This is the specific obstacle to training a quantum Boltzmann machine on classical data, and it is where the literature resorts to a bound.

5.3.3 The clamped-state bound

Amin and coworkers [96] replace the likelihood by a quantity that can be differentiated with expectation values alone. The Golden–Thompson inequality states that $\mathrm{Tr}[e^{A}e^{B}] \geq \mathrm{Tr}[e^{A+B}]$ for Hermitian A and B, with equality when they commute. Writing the projector as the limit of an exponential, $\Pi_{\boldsymbol{v}} = \lim_{\lambda\to\infty} e^{-\lambda(I-\Pi_{\boldsymbol{v}})}$, one obtains

$$p_{\boldsymbol{\theta}}(\boldsymbol{v}) = \frac{\mathrm{Tr}[\Pi_{\boldsymbol{v}} e^{-\beta H}]}{Z} \;\geq\; \frac{\mathrm{Tr}\big[\Pi_{\boldsymbol{v}}\, e^{-\beta H_{\boldsymbol{v}}}\big]}{Z} = \frac{Z_{\boldsymbol{v}}}{Z}, \qquad H_{\boldsymbol{v}} = \Pi_{\boldsymbol{v}} H \Pi_{\boldsymbol{v}}, \tag{5.22}$$

where $H_{\boldsymbol{v}}$ is the *clamped Hamiltonian*: the model Hamiltonian with the visible qubits frozen in the configuration $\boldsymbol{v}$, which for the Pauli strings of Eq. (5.14) means

$$Z_i \;\to\; 1 - 2v_i, \qquad X_i \;\to\; 0 \qquad (i \in v), \tag{5.23}$$

an operator on the hidden qubits only, and $Z_{\boldsymbol{v}} = \mathrm{Tr}_h\, e^{-\beta H_{\boldsymbol{v}}}$ its partition function. The visible transverse fields drop out of $H_{\boldsymbol{v}}$ because X_i has no diagonal elements in the computational basis of qubit i. Taking the logarithm and averaging over the data,

$$\mathscr{L}(\boldsymbol{\theta}) \;\leq\; \widetilde{\mathscr{L}}(\boldsymbol{\theta}) = \ln Z(\boldsymbol{\theta}) - \sum_{\boldsymbol{v}} \pi(\boldsymbol{v}) \ln Z_{\boldsymbol{v}}(\boldsymbol{\theta}), \tag{5.24}$$

an upper bound on the loss whose two terms are both logarithms of partition functions, of the full model and of the model clamped to each data point. Its gradient follows from Eq. (5.20) applied to each,

$$\frac{\partial \widetilde{\mathscr{L}}}{\partial \theta_\mu} = \beta \Big(\langle \Lambda_\mu \rangle_\rho - \sum_{\boldsymbol{v}} \pi(\boldsymbol{v})\, \langle \Lambda_\mu^{(\boldsymbol{v})} \rangle_{\rho_{\boldsymbol{v}}} \Big), \qquad \rho_{\boldsymbol{v}} = \frac{e^{-\beta H_{\boldsymbol{v}}}}{Z_{\boldsymbol{v}}}, \tag{5.25}$$

with $\Lambda_\mu^{(\boldsymbol{v})}$ the clamped version of Λ_μ by the rule Eq. (5.23). This is the formula that is usually quoted as "the gradient of the quantum Boltzmann machine," model expectation minus clamped expectation, and it has exactly the shape of the classical rule; both terms are expectation values of thermal states, of H and of $H_{\boldsymbol{v}}$, and could be measured on a device that prepares them. But it is the gradient of the bound Eq. (5.24), not of the likelihood, and the two differ in a way that matters. For a visible transverse field the clamped operator vanishes, so Eq. (5.25) gives $\partial\widetilde{\mathscr{L}}/\partial\Gamma_i = \beta\langle X_i\rangle_\rho$ whatever the data: descending the bound always reduces $|\Gamma_i|$, since $\langle X_i\rangle_\rho$ has the sign of Γ_i, and the bound cannot learn the visible transverse fields at all. Amin and coworkers therefore keep them fixed; the program `qbm.py` trains them anyway, so that the effect can be seen.

Table 5.2 compares the three gradients at a random point of a small model. The agreement of the Duhamel gradient with the finite difference confirms Eqs. (5.18)–(5.21); the bound gradient tracks the exact one to a few percent for the longitudinal fields and for all but the smallest coupling (Z_0Z_3 is off by a factor of three), is already off by a factor of two for a hidden transverse field, and for the visible transverse fields is twenty to thirty times too large: it is $\beta\langle X_i\rangle_\rho$, which knows nothing of the data. The program also checks that with all $\Gamma_q = 0$ the two gradients coincide to 10^{-16}, as Eq. (5.17) requires. The listing reproduces the comparison.

```
import numpy as np
from qbm import QBM, rqbm_terms
```

Table 5.2 The three gradients of the restricted QBM of Fig. 5.2 with two visible and two hidden qubits at random couplings and $\beta = 1$, for the XOR target: the exact gradient Eq. (5.21) by Duhamel's formula, a central finite difference of the loss, and the clamped-state gradient Eq. (5.25). The exact and finite-difference columns agree to 10^{-9}; the bound gradient is close for most Z terms and the hidden X terms and wrong by more than an order of magnitude for the visible transverse fields Γ_0, Γ_1. From `qbm.py`.

Term	exact (Duhamel)	finite difference	bound Eq. (5.25)
Z_0	−0.060300	−0.060300	−0.061133
Z_1	−0.141716	−0.141716	−0.143026
Z_2	0.081718	0.081718	0.083694
Z_3	0.029148	0.029148	0.029801
X_0	−0.009442	−0.009442	−0.227435
X_1	0.005532	0.005532	0.173332
X_2	0.017987	0.017987	0.019094
X_3	−0.002691	−0.002691	−0.001743
Z_0Z_2	0.196026	0.196026	0.206809
Z_0Z_3	−0.013928	−0.013928	−0.004916
Z_1Z_2	0.202632	0.202632	0.211565
Z_1Z_3	0.465294	0.465294	0.479711

```
from rbm import parity
from optimize import numerical_gradient

terms, names = rqbm_terms(n_v=2, n_h=2)      # Z_q, X_q, Z_i Z_j (visible–hidden)
model = QBM(n_visible=2, n_hidden=2, terms=terms, beta=1.0)
theta = np.random.default_rng(0).normal(0, 0.5, model.n_params)
target = parity(2)
g_exact = model.gradient_exact(theta, target)    # Duhamel, Eq. (5–exactgrad)
g_fd = numerical_gradient(lambda t: model.nll(t, target), theta)
g_bound = model.gradient_bound(theta, target) # clamped states, Eq. (5–boundgrad)
for nm, a, b, c in zip(names, g_exact, g_fd, g_bound):
    print(f"{nm:>5} exact {a:9.6f} finite diff. {b:9.6f} bound {c:9.6f}")
```

5.4 Boltzmann machines on classical data

We now train the models of the two preceding sections on the same two distributions, with the exact gradients throughout so that nothing in the comparison is due to sampling. Four models are trained on each target: the classical RBM with binary units of Section 5.2.3; the restricted QBM without transverse fields, which is the same model in spin variables; the restricted QBM with transverse fields on every qubit and the exact Duhamel gradient Eq. (5.21); and the same quantum model trained with the bound gradient Eq. (5.25). All use Adam [76] with the same learning rate and number of steps for the three spin models and the same random start for the two quantum ones; the RBM, whose $\{0,1\}$ units make its landscape differently scaled, is given 1000 steps. Table 5.3 and Fig. 5.3 summarize the result.

Table 5.3 Classical and quantum Boltzmann machines on classical data, trained with exact gradients and Adam: the divergence $\mathrm{KL}(\pi \| p_{\boldsymbol{\theta}})$ at the end of training for the binary-unit RBM (1000 steps, rate 0.1), the restricted QBM without transverse fields, the restricted QBM with transverse fields and the Duhamel gradient, and the same model with the clamped-state gradient; $\beta = 1$ throughout. From `classical_data.py`.

Target	n_v	n_h	steps (rate)	RBM	QBM, $\Gamma = 0$	QBM, exact	QBM, bound
XOR	2	2	300 (0.1)	0.0004	0.0003	0.0002	0.0002
bars and stripes	4	3	200 (0.05)	0.0028	0.0074	0.0082	0.0099

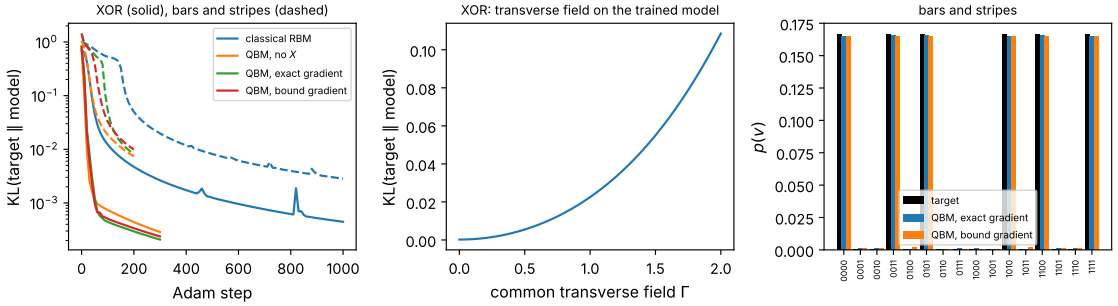


Fig. 5.3 Left: the divergence against the Adam step for the four models of Table 5.3 on XOR (solid) and bars and stripes (dashed). Middle: the divergence of the trained XOR model when a common transverse field Γ is applied to all four qubits on top of the learned couplings. Right: the bars-and-stripes distribution learned by the quantum model with the exact and with the bound gradient against the target. From `classical_data.py`.

Three observations. First, every model learns both distributions: the divergences are a few times 10^{-4} for XOR and a few times 10^{-3} for bars and stripes, and the right panel of Fig. 5.3 shows the six patterns reproduced to within the width of the bars. The XOR distribution, which the course material that this chapter grew out of used as an example of a distribution on which the classical RBM fails and the quantum one succeeds, is learned by the RBM as well as by any of the others; the failure that is sometimes shown is an artifact of a symmetric start (zero biases and small symmetric weights place the RBM on a saddle of its loss, from which plain gradient descent moves very slowly) and not of the model. On the small differences in the table, the binary-unit RBM reaches the lowest divergence on bars and stripes simply because it was given more steps; the left panel shows all curves still descending.

Second, the transverse fields go away. The quantum model trained with the exact gradient ends with $\Gamma_q = (0.000, 0.000, 0.018, 0.000)$ on its four qubits for XOR: the fields on both visible qubits are zero to three decimals, and the one surviving hidden field is small. The middle panel of Fig. 5.3 shows why: a common transverse field Γ applied to the trained couplings raises the divergence monotonically, from 0.0002 at $\Gamma = 0$ to 0.1085 at $\Gamma = 2$, with the minimum at $\Gamma = 0$. The visible state of the trained model, $\rho_v = \mathrm{Tr}_h \rho$, has the diagonal $(0.4999, 0.0001, 0.0001, 0.4999)$ and off-diagonal elements that sum to 0.0000 in absolute value: it is a classical mixture. This is not an accident of the start but what the likelihood must do. The data are a distribution over computational-basis strings and the loss sees only the diagonal of ρ_v; a transverse field moves weight off the diagonal, into coherences that no term of the loss rewards, and the diagonal it leaves behind is, for these couplings, a worse fit. Amin and coworkers [96] report instances in which a fixed transverse field helps, but the mechanism there is the optimization (the field smooths the landscape of the bound) rather than the representation, and the exact optimum on classical data lies, in all examples we have tried, at $\Gamma_q \approx 0$. The model is at best as good as its classical limit on classical data, and the extra parameters are, in the end, unused.

Third, the bound gradient works about as well as the exact one on these problems, reaching 0.0002 against 0.0002 for XOR and 0.0099 against 0.0082 for bars and stripes, and it drives the transverse fields to the same values, $(0.000, 0.000, 0.017, 0.000)$. This is less surprising than it looks after Table 5.2. The bound is loose only where the transverse fields are large, and the bound gradient's own tendency to shrink them, which there looked like a defect, is here the right thing to do: the exact likelihood also wants them gone. On classical data the two gradients agree where it matters, and a device that could prepare the thermal states of H and of the clamped $H_{\mathbf{v}}$ could train the model with expectation values alone. What it would obtain is a classical Boltzmann machine sampled by a quantum thermal process, which may be a

useful thing to have if the device thermalizes faster than a Gibbs chain mixes (Section 5.7), but it is not a richer model.

5.5 Learning quantum states

The conclusion of the previous section is sharp because the data were classical. If the data are quantum states, the coherences that the likelihood ignored are information, and the QBM becomes the natural model for them. Suppose the target is a density matrix ρ_t on n qubits, of which we can measure expectation values, and the model is the Gibbs state $\rho_{\boldsymbol{\theta}}$ of Eq. (5.13) with all qubits visible. The quantum relative entropy

$$S(\rho_t \,\|\, \rho_{\boldsymbol{\theta}}) = \operatorname{Tr}\big[\rho_t \ln \rho_t\big] - \operatorname{Tr}\big[\rho_t \ln \rho_{\boldsymbol{\theta}}\big] = -S(\rho_t) + \beta \operatorname{Tr}\big[\rho_t H(\boldsymbol{\theta})\big] + \ln Z(\boldsymbol{\theta}) \tag{5.26}$$

is the quantum counterpart of Eq. (5.1): it is non-negative by Klein's inequality and zero only for $\rho_{\boldsymbol{\theta}} = \rho_t$ [13], and it reduces to the classical divergence when both states are diagonal. The second equality uses the fact that the logarithm of a Gibbs state is $-\beta H - \ln Z$, and it makes the objective remarkably simple: the first term is a constant, the second is linear in $\boldsymbol{\theta}$ because H is, and the third is the log-partition function, which is a convex function of the couplings. The gradient is [102, 103]

$$\frac{\partial S(\rho_t \| \rho_{\boldsymbol{\theta}})}{\partial \theta_\mu} = -\beta \langle \Lambda_\mu \rangle_{\rho_t} + \frac{\partial \ln Z}{\partial \theta_\mu} = \beta \Big(\langle \Lambda_\mu \rangle_{\rho_{\boldsymbol{\theta}}} - \langle \Lambda_\mu \rangle_{\rho_t} \Big), \tag{5.27}$$

by Eq. (5.20). Everything that was hard in Section 5.3.2 has vanished. There is no projector inside the trace, no imaginary-time integral and no bound: the gradient is the difference between the expectation values of the model's own operators in the model and in the target, both measurable, and the loss is convex, so gradient descent converges to the global minimum. If the target is itself a Gibbs state of a Hamiltonian in the family, that minimum is the target's couplings, and the problem is one of *Hamiltonian learning* [104]: reconstruct H from measurements on its thermal state. The expectation values on the target side are the only data needed, which for P Pauli strings means P numbers however many copies of ρ_t it took to measure them.

Table 5.4 Hamiltonian learning. The target is the Gibbs state at $\beta = 1$ of a three-qubit transverse-field Ising chain with random couplings; the model is a fully visible QBM with the same eight operators, trained from $\boldsymbol{\theta} = 0$ by 400 Adam steps (rate 0.05) on the relative entropy Eq. (5.26) with the gradient Eq. (5.27). The relative entropy falls from 1.1288 to machine precision, the fidelity reaches 1 to eight decimals and the couplings are recovered to 2×10^{-9}, in 0.2 s. From `quantum_data.py`.

Term	target $\theta^\star_\mu$	learned θ_μ	Term	target $\theta^\star_\mu$	learned θ_μ
Z_0	-0.5214	-0.5214	X_1	-1.3131	-1.3131
Z_1	-0.1398	-0.1398	X_2	-0.0042	-0.0042
Z_2	1.3310	1.3310	Z_0Z_1	-0.4988	-0.4988
X_0	0.5273	0.5273	Z_1Z_2	0.1189	0.1189

Table 5.4 shows the recovery of a random transverse-field Ising chain from its Gibbs state: the loss falls exponentially (left panel of Fig. 5.4, the solid curve, which reaches the floor of double precision after about 300 steps), and the learned couplings reproduce the target ones on the diagonal of the right panel. The listing below is the whole computation: the relative-entropy gradient is one line.

```
import numpy as np
from qbm import QBM, ising_terms
from optimize import Adam

terms, names = ising_terms(3)                 # Z_q, X_q, Z_q Z_{q+1} on a 3-qubit chain
model = QBM(n_visible=3, n_hidden=0, terms=terms, beta=1.0)
theta_star = np.random.default_rng(4).normal(0, 0.8, model.n_params)
rho_target = model.gibbs(theta_star)          # the data: a Gibbs state in the family
target_expectations = model.expectations(rho_target) # P numbers, all the training needs
theta, opt = np.zeros(model.n_params), Adam(lr=0.05)
for step in range(400):
    grad = model.beta * (model.expectations(model.gibbs(theta)) - target_expectations)
    theta = opt.step(theta, grad)             # Eq. (5-relentgrad)
print("max |theta - theta*| =", np.max(np.abs(theta - theta_star)))
print("relative entropy S(rho_t || rho_theta) =", model.relative_entropy(theta, rho_target))
```

The convexity has a flip side: the model can only reach the distributions its operators span. The second experiment of `quantum_data.py` takes as target the Gibbs state of a Heisenberg XXZ chain in a tilted field with a next-nearest-neighbor coupling,

$$H_t = \sum_{q=0}^{1} \left(X_q X_{q+1} + Y_q Y_{q+1} + \tfrac{1}{2} Z_q Z_{q+1} \right) + \sum_{q=0}^{2} \left(0.3\, Z_q + 0.4\, X_q \right) + \tfrac{1}{2} Z_0 Z_2, \tag{5.28}$$

which lies outside every family we train, and fits it with three models of growing operator content: Z fields with nearest-neighbor ZZ couplings (a classical model, $P = 5$), the same with transverse fields (the transverse-field Ising family of Table 5.4, $P = 8$), and the family with XX and YY couplings added ($P = 12$). Table 5.5 gives the outcome. The classical family reaches a fidelity of 0.54; adding the transverse fields, the whole quantum content of Section 5.4, improves it to 0.57; adding the exchange couplings that the target actually contains raises it to 0.99, with the remaining 0.02 of relative entropy due to the next-nearest-neighbor term that none of the families has. The last column, the classical divergence between the diagonals of target and model, shows that the first two families do not even fit the measurement statistics in the computational basis, because the coherent terms of H_t shape its diagonal too. The lesson is the one of every variational method in this book: what can be learned is fixed by the ansatz, and for a Hamiltonian-based model the ansatz is the set of operators. The quantum advantage of the QBM, where there is one, is that the family can contain non-commuting operators at all, which no classical energy function can.

Table 5.5 The XXZ target Eq. (5.28) at $\beta = 1$ fitted by three fully visible families of growing operator content, 400 Adam steps on the relative entropy: the final relative entropy, the fidelity with the target and the classical divergence of the diagonals. From `quantum_data.py`.

Model operators	P	$S(\rho_t \Vert \rho_{\boldsymbol{\theta}})$	fidelity	KL of diagonals
Z, ZZ (classical)	5	0.9410	0.5398	0.0752
Z, X, ZZ	8	0.8608	0.5718	0.0758
Z, X, ZZ, XX, YY	12	0.0203	0.9906	0.0129

Hidden qubits can be added to this setting too, with the model $\rho_v = \mathrm{Tr}_h \rho_{\boldsymbol{\theta}}$ and the loss $S(\rho_t \Vert \rho_v)$, but the simplicity is then lost: the logarithm of a partial trace is not linear in H, and the gradient needs the derivative of $\ln \mathrm{Tr}_h e^{-\beta H}$, for which the literature again uses bounds [102, 105]. A second practical difference from the classical case concerns the data. A classical sample is a bit string that can be stored and reused; a quantum sample is a copy of ρ_t that is consumed by the measurement, and the P expectation values of Eq. (5.27) must be estimated from a finite number of copies with the shot noise of Chapter 2 (Exercise 7). The model side

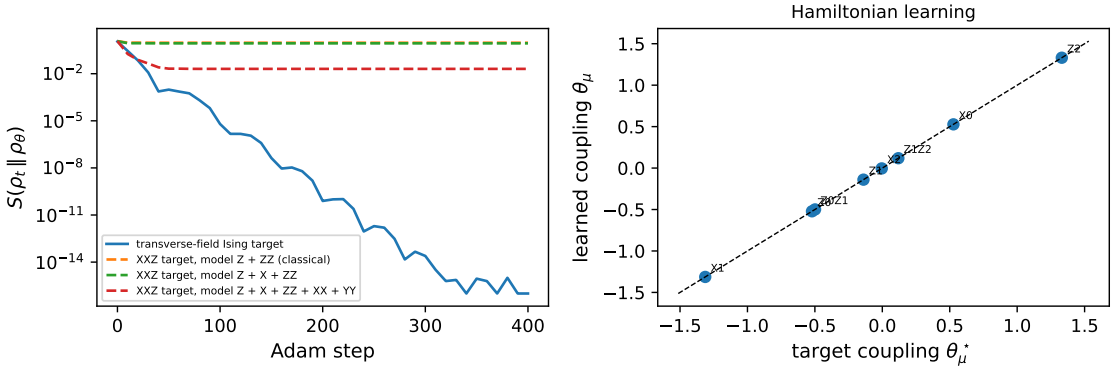


Fig. 5.4 Left: the relative entropy against the Adam step for the Hamiltonian-learning problem of Table 5.4 (solid) and for the three families fitted to the XXZ target of Table 5.5 (dashed). Right: learned against target couplings for the Hamiltonian-learning problem. From `quantum_data.py`.

has the same cost: every gradient step needs P expectation values of the current Gibbs state, which brings us to the question of how that state is prepared.

5.6 Preparing Gibbs states on a quantum computer

Every gradient of this chapter is an expectation value in a Gibbs state, Eq. (5.20), and the premise of the quantum Boltzmann machine is that a quantum device supplies it. A quantum annealer does so by construction, approximately, when it is run at finite temperature and read out before it has reached its ground state [96, 106]; the open question is at what effective β. On a gate-based computer the thermal state must be prepared by a circuit, and since a circuit acting on $|0\rangle^{\otimes n}$ produces a pure state, a mixed state requires either a classical source of randomness or ancilla qubits that are traced out. This section gives the variational principle that governs any such preparation, the purification that makes a thermal state the reduced state of a pure one, and the variational algorithm that puts the two together with the circuits of Chapter 4.

5.6.1 The free-energy principle

The Gibbs state is the solution of a variational problem. For any density matrix ρ define the free energy

$$F(\rho) = \mathrm{Tr}[H\rho] - \frac{1}{\beta} S(\rho), \qquad S(\rho) = -\mathrm{Tr}[\rho \ln \rho], \tag{5.29}$$

the energy minus the temperature times the von Neumann entropy. For the Gibbs state $\rho_G = e^{-\beta H}/Z$ the free energy is $F(\rho_G) = -\beta^{-1} \ln Z$, the thermodynamic free energy, and for any other state

$$F(\rho) - F(\rho_G) = \frac{1}{\beta} S(\rho \,\|\, \rho_G) \;\geq\; 0, \tag{5.30}$$

which is Eq. (5.26) read backwards: inserting $\ln \rho_G = -\beta H - \ln Z$ into the relative entropy gives $S(\rho \| \rho_G) = -S(\rho) + \beta \mathrm{Tr}[H\rho] + \ln Z = \beta [F(\rho) - F(\rho_G)]$, and Klein's inequality does the rest. The Gibbs state is therefore the unique minimizer of the free energy over all states, and the excess free energy of a trial state is, up to the factor β, its relative entropy to the target: minimizing

F over a family of states is the finite-temperature counterpart of the Rayleigh–Ritz principle on which the VQE of the QC book and the ground-state search of Section 4.5 rest, with the entropy as the new ingredient. As $\beta \to \infty$ the entropy term is switched off and the principle becomes the Ritz principle for the ground state; as $\beta \to 0$ the energy is switched off and the maximally mixed state wins. Figure 5.5 shows the landscape for one qubit with $H = -Z - X/2$ at $\beta = 1.2$, over the xz-plane of the Bloch ball, parametrized by the radius r and the polar angle ϑ: the energy prefers the surface of the ball in the direction of the field, the entropy prefers the center, and the minimum of F sits inside at $r = \tanh(\beta|\boldsymbol{h}|) = 0.872$, the Gibbs state, with Bloch vector $(0.390, 0, 0.780)$, entropy 0.238 and free energy -1.1731; the minimum over the plotted grid is -1.1731 as well. `gibbs.py` also checks Eq. (5.30) at a random state to 10^{-10}.

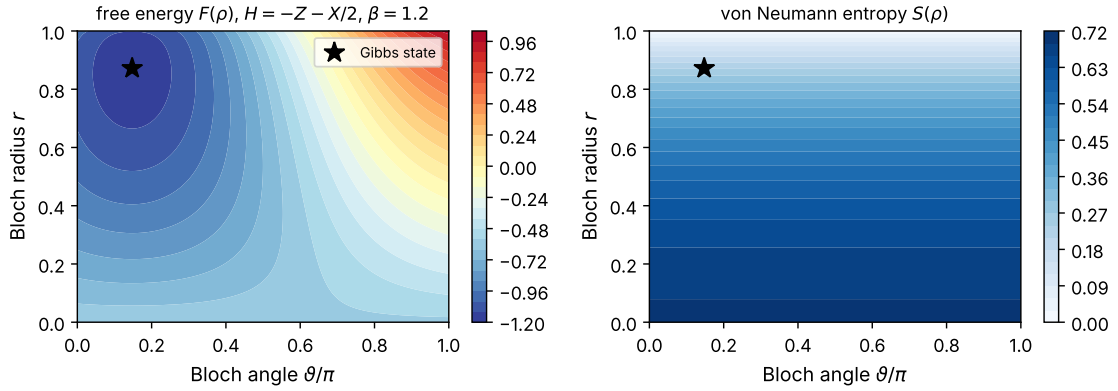


Fig. 5.5 The free-energy principle for one qubit, $H = -Z - X/2$, $\beta = 1.2$. Left: $F(\rho)$ over the states $\rho = \frac{1}{2}(I + r\sin\vartheta\, X + r\cos\vartheta\, Z)$ of the xz-plane of the Bloch ball; the star is the Gibbs state, the unique minimum. Right: the von Neumann entropy of the same states, which depends on r only. From `gibbs.py`.

5.6.2 The thermofield double

Any mixed state is the reduced state of a pure state on a larger system, and for a Gibbs state the purification has a name. Let $H = \sum_k e_k |k\rangle\langle k|$ act on n system qubits and take n ancilla qubits with the same basis. The state

$$|\mathrm{TFD}\rangle = \frac{1}{\sqrt{Z}} \sum_k e^{-\beta e_k/2} \, |k\rangle_S \otimes |k^*\rangle_A, \qquad \mathrm{Tr}_A |\mathrm{TFD}\rangle\langle\mathrm{TFD}| = \frac{e^{-\beta H}}{Z}, \tag{5.31}$$

the *thermofield double*, is a pure, entangled state of system and ancilla whose reduced state on the system is exactly the Gibbs state (the complex conjugate on the ancilla side is immaterial for a real Hamiltonian). The entanglement entropy between the two halves equals the thermal entropy of the system, and the thermal fluctuations of ρ_G are the quantum correlations of a pure state. At $\beta = 0$ the TFD is a product of n Bell pairs, at $\beta \to \infty$ it is the product of the two ground states; `gibbs.py` builds it for the random three-qubit Hamiltonian of Section 5.3.2 and verifies that it is normalized, that its partial trace reproduces ρ_G to 10^{-16} and that its entanglement entropy, 1.7146, equals the thermal entropy. Preparing the TFD on $2n$ qubits is thus one route to the Gibbs state on n, and the free-energy principle tells one how to do it variationally: a circuit $U(\boldsymbol{\phi})$ on $2n$ qubits is trained to minimize $F(\mathrm{Tr}_A U|0\rangle\langle 0|U^\dagger)$ [107, 108]. The obstacle is the entropy, which is not an observable: $S(\rho_S)$ of the reduced state has no efficient measurement in general, and these algorithms either esti-

mate it from the spectrum (by swap tests on several copies) or replace it by a quantity that is easier to measure. The algorithm of the next subsection sidesteps the problem by putting the randomness where its entropy is known in closed form.

5.6.3 The variational quantum thermalizer

The variational quantum thermalizer (VQT) of Verdon and coworkers [109] prepares a mixed state in two stages. A classical *latent* distribution $p(i;\boldsymbol{w})$ over the 2^n computational-basis strings i is sampled, the basis state $|i\rangle$ is prepared, and a parametrized unitary $U(\boldsymbol{\phi})$ is applied to it. The state produced on average is

$$\rho(\boldsymbol{w},\boldsymbol{\phi}) = \sum_i p(i;\boldsymbol{w})\, U(\boldsymbol{\phi})|i\rangle\langle i|U^\dagger(\boldsymbol{\phi}), \tag{5.32}$$

a mixture of orthogonal pure states, which is a density matrix with eigenvalues $p(i;\boldsymbol{w})$ and eigenvectors $U|i\rangle$: the unitary cannot change the spectrum, so the entropy of Eq. (5.32) is the Shannon entropy of the latent distribution, $S(\rho) = -\sum_i p_i \ln p_i$, a classical quantity computable without any measurement. The free energy Eq. (5.29) becomes

$$F(\boldsymbol{w},\boldsymbol{\phi}) = \sum_i p(i;\boldsymbol{w})\, E_i(\boldsymbol{\phi}) - \frac{1}{\beta} S(\boldsymbol{w}), \qquad E_i(\boldsymbol{\phi}) = \langle i|U^\dagger(\boldsymbol{\phi}) H U(\boldsymbol{\phi})|i\rangle, \tag{5.33}$$

a weighted average of the energies of the circuit started from the different basis states, each of which is a VQE energy, minus the entropy of the weights over β. The quantum computer evaluates the E_i (or, in the sampled version, the energy of a randomly chosen i), the classical computer does the rest, and the gradient splits accordingly: $\partial F/\partial\phi_\mu = \sum_i p_i\, \partial E_i/\partial\phi_\mu$ is the parameter-shift rule of Chapter 4 applied to each energy, and $\partial F/\partial \boldsymbol{w}$ is the derivative of a classical expression. By Eq. (5.30), $F(\boldsymbol{w},\boldsymbol{\phi}) - F(\rho_G) = \beta^{-1} S(\rho(\boldsymbol{w},\boldsymbol{\phi})\|\rho_G)$ measures how far the preparation is from the Gibbs state, and the excess free energy is the training curve of Fig. 5.6.

The latent distribution decides what the ansatz can represent. The scalable choice is a product of Bernoulli distributions, $p(i;\boldsymbol{w}) = \prod_q s_q^{i_q}(1-s_q)^{1-i_q}$ with $s_q = \sigma(w_q)$, one parameter per qubit, whose entropy is a sum of binary entropies; it can produce any product spectrum but not, for instance, a spectrum with three equal eigenvalues and all others zero. For the small systems of this chapter we can also afford the *full* latent distribution, a softmax over all 2^n strings, which can produce any spectrum and makes Eq. (5.32) exact if U can diagonalize H. The class `VQT` of `vqt.py` implements both, with the circuit of Chapter 4 (`QNN` with R_y, R_z layers and the CNOT ladder, without data re-uploading) as $U(\boldsymbol{\phi})$; the basis state $|i\rangle$ is prepared by the encoding rotations $R_y(\pi i_q)$, so that the whole machinery of batched states and shift-rule gradients of `qnn.py` carries over, and the program checks the gradient against finite differences to 3×10^{-10}.

Table 5.6 and Fig. 5.6 show the thermalizer on the transverse-field Ising chain of Chapter 4, here on three qubits, at two temperatures. Three layers of the full-latent ansatz reach a fidelity of 0.987 at $\beta = 1$ and 0.996 at $\beta = 3$; the product latent, with the same circuit, stops at 0.968 and 0.972. The column of entropies explains the gap and is the most instructive one. At $\beta = 1$ the exact thermal entropy is 0.995, almost half the maximum $3\ln 2 = 2.08$, and every variational state is purer than it should be: the product latent cannot shape its spectrum to match the thermal one and sacrifices entropy to lower the energy, and even the full latent with one layer underestimates S by 0.14 because its circuit cannot rotate the product basis into the eigenbasis of H, so that the latent probabilities are forced to a compromise between

Table 5.6 The variational quantum thermalizer for the transverse-field Ising chain $H = -\sum_q Z_q Z_{q+1} - \sum_q X_q$ on three qubits at $\beta = 1$ and $\beta = 3$, with the product (Bernoulli) and the full (softmax) latent distribution and $L = 1, 2, 3$ circuit layers (P is the total number of parameters). Each entry is the better of two random starts after 300 Adam steps (rate 0.05): the excess free energy $F - F_{\text{exact}}$, the fidelity with the exact Gibbs state and the entropy of the prepared state against the exact thermal entropy. From `vqt.py`.

β	latent	L	P	F_{exact}	$F - F_{\text{exact}}$	fidelity	S / S_{exact}
1	product	1	9	−3.9442	0.256	0.857	0.678 / 0.995
1	product	2	15	−3.9442	0.089	0.954	0.845 / 0.995
1	product	3	21	−3.9442	0.076	0.968	0.887 / 0.995
1	full	1	14	−3.9442	0.185	0.912	0.858 / 0.995
1	full	2	20	−3.9442	0.055	0.977	0.918 / 0.995
1	full	3	26	−3.9442	0.031	0.987	0.948 / 0.995
3	product	1	9	−3.5165	0.052	0.947	0.104 / 0.245
3	product	2	15	−3.5165	0.042	0.955	0.116 / 0.245
3	product	3	21	−3.5165	0.031	0.972	0.185 / 0.245
3	full	1	14	−3.5165	0.034	0.958	0.074 / 0.245
3	full	2	20	−3.5165	0.013	0.991	0.196 / 0.245
3	full	3	26	−3.5165	0.0089	0.996	0.236 / 0.245

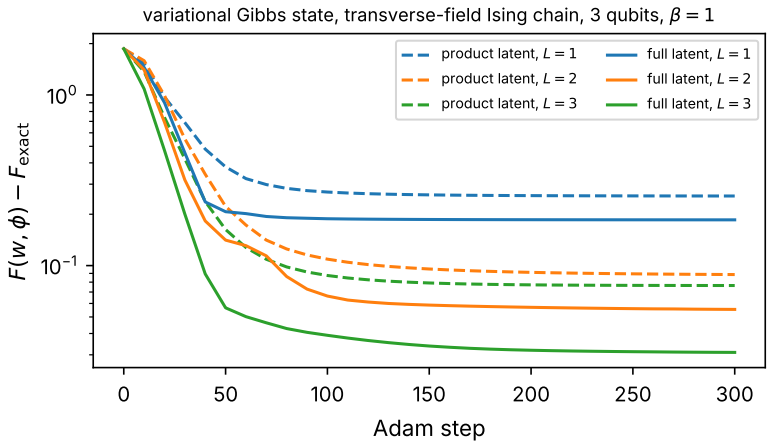


Fig. 5.6 The excess free energy $F(\boldsymbol{w}, \boldsymbol{\phi}) - F_{\text{exact}}$, equal to $S(\rho \| \rho_G)/\beta$ by Eq. (5.30), against the Adam step for the $\beta = 1$ runs of Table 5.6: product latent (dashed) and full latent (solid) with one, two and three circuit layers. From `vqt.py`.

the right spectrum and the right energies. Depth helps the circuit, and the latent then follows. At $\beta = 3$ the state is nearly pure ($S_{\text{exact}} = 0.245$), the task is close to a VQE, and all variants do well on the fidelity; the remaining entropy deficit is again the signature of the ansatz. The listing below runs one case.

```
import numpy as np
from vqt import VQT, transverse_ising
from gibbs import gibbs_state, fidelity, log_partition

n, beta = 3, 1.0
H = transverse_ising(n, h=1.0)
vqt = VQT(H, n_qubits=n, n_layers=2, beta=beta, latent="full")
x0 = np.concatenate([np.zeros(vqt.n_latent), vqt.qnn.random_parameters(0)])
params, history = vqt.fit(x0, steps=300, lr=0.05) # Adam on F(w, phi), Eq. (5—vqtfree)
F_exact = —log_partition(H, beta) / beta
print(f"F — F_exact = {history[—1] — F_exact:.4f}, "
      f"fidelity = {fidelity(gibbs_state(H, beta), vqt.state(params)):.4f}")
```

Two remarks close the section. The optimization problem of the VQT has the same landscape as that of the QNN, including the barren plateaus of Section 4.6 for deep random circuits, and the natural gradient of Section 4.5 has a finite-temperature counterpart in which the quantum Fisher information is replaced by the Bures metric of mixed states [109]. And once the Gibbs state of $H(\boldsymbol{\theta})$ is prepared, training the QBM is a loop within a loop: the inner loop prepares $\rho_{\boldsymbol{\theta}}$ variationally for the current couplings, the outer loop measures the $\langle \Lambda_\mu \rangle_\rho$ and updates $\boldsymbol{\theta}$ by Eq. (5.27) or Eq. (5.25). The *variational quantum Boltzmann machine* of Zoufal and coworkers [110] is this algorithm with the Gibbs state prepared by variational imaginary-time evolution, the McLachlan principle of Section 4.5 run in imaginary time from the maximally mixed state to β, which is the other general route to a thermal state and is where the two halves of Chapter 4 meet this one.

5.7 Connections

5.7.1 Quantum neural networks and quantum Boltzmann machines

The two trainable quantum models of this book, the QNN of Chapter 4 and the QBM of this chapter, are complementary in almost every respect, and Table 5.7 sets them side by side. The QNN is a unitary acting on a pure state; its parameters are rotation angles, its output is an expectation value, its natural task is to approximate a function and its training signal is the parameter-shift rule, which the quantum computer evaluates by running shifted circuits. The QBM is a Hamiltonian; its parameters are couplings, its output is a distribution or a density matrix, its natural task is to model data generatively and its training signal is a difference of expectation values, which the quantum computer evaluates by preparing a thermal state. The one has the structure of a feed-forward network and the other of a statistical-mechanics model, and the classical pair they correspond to, the multilayer perceptron and the restricted Boltzmann machine, has the same relationship. What they share is the variational loop: a classical optimizer moves parameters on the strength of numbers measured on a quantum device, with the landscape pathologies of Section 4.6 common to both, and with the geometry of Section 4.5 available to both through the Fisher information of pure states and the Bures metric of mixed ones.

Table 5.7 The two trainable quantum models of this book.

	Quantum neural network (Ch. 4)	Quantum Boltzmann machine (Ch. 5)
model object	unitary $W(\boldsymbol{\theta})$	Hamiltonian $H(\boldsymbol{\theta})$
state	pure, $WS(\boldsymbol{x})\|0\rangle$	thermal, $e^{-\beta H}/Z$
parameters	rotation angles	couplings of Pauli strings
output	expectation value $f(\boldsymbol{x};\boldsymbol{\theta})$	distribution $p(\boldsymbol{v})$ or state ρ_v
task	regression, classification, PDEs	generative modeling, state learning
loss	square loss, residuals	negative log-likelihood, relative entropy
gradient	parameter-shift rule	$\langle \Lambda_\mu \rangle_{\text{model}} - \langle \Lambda_\mu \rangle_{\text{data}}$ (bound or exact)
what the device does	runs the circuit	prepares and samples the Gibbs state
hard part	barren plateaus, expressivity	the positive phase, Gibbs-state preparation
classical analog	multilayer perceptron	restricted Boltzmann machine

5.7.2 The transverse-field Ising model as a learning problem

The restricted QBM Eq. (5.14) is a transverse-field Ising model on a bipartite graph, and the Hamiltonians learned in Section 5.5 were transverse-field Ising chains. The same model has appeared in every chapter of this book: as the cost and mixer of QAOA, whose annealing limit is the adiabatic passage through the quantum phase transition of the model; as the ground-state problem on which the natural gradient was demonstrated; and here as a generative model and as a target. The three uses are three questions about the same object. QAOA and VQE ask for the ground state of a given H, the thermalizer asks for its Gibbs state, and the QBM asks the inverse question: which H has the data as its thermal statistics? The first two are forward problems with the variational principles of Rayleigh–Ritz and of the free energy; the third is an inverse problem with the maximum-likelihood principle, which for a quantum target is convex and for a classical target is not. Reading the chapters together, a physicist's tool chest of forward solvers becomes, with one change of objective, a set of learning algorithms, and the many-body methods on which they rest (imaginary-time evolution, Monte Carlo sampling, perturbation theory in the form of Duhamel's formula) are the methods of the learning algorithms too.

The limit $\beta \to \infty$ makes the correspondence literal. The Gibbs state becomes the projector onto the ground state, the free energy becomes the energy, and the QBM with a quantum target becomes the problem of finding a Hamiltonian whose ground state is a given state, which for a Hamiltonian in the family is the parent-Hamiltonian problem of many-body physics. The VQT at large β reduces to the VQE, as the $\beta = 3$ rows of Table 5.6 begin to show, and the latent distribution collapses onto one basis state. In the opposite limit $\beta \to 0$ every model is the maximally mixed state, every gradient vanishes, and nothing can be learned: β sets the sharpness of the model, and a device whose effective temperature is not under control cannot represent sharply peaked data however well its couplings are set.

5.7.3 Expressivity, hardware and honest expectations

Is a QBM more expressive than a classical Boltzmann machine? On classical data the exact computations of Section 5.4 answer no, in the sense that matters for learning: the likelihood of computational-basis data is maximized at vanishing transverse fields, and the model falls back on its classical limit. A transverse field can still help as an optimization device, and there are distributions, those whose diagonal is the diagonal of a thermal state with coherent terms, which a QBM with hidden units represents with fewer parameters than an RBM; but the general result [96, 103] is that the diagonal of a Gibbs state of a local Hamiltonian is a Gibbs distribution of a classical energy function with, in general, many-body terms, so that the question is one of economy of parameters rather than of what can be represented at all. On quantum data the answer is yes without qualification, because the classical model cannot represent coherences at all, and Table 5.5 is the elementary demonstration: a target with exchange couplings is not learnable by any family without them.

The hardware question is separate. The negative phase of a Boltzmann machine is a sampling problem, and sampling the Gibbs distribution of a frustrated Ising model is hard for Monte Carlo at low temperature, which is where the models worth having live. A device that reaches thermal equilibrium of the same model physically, as quantum annealers are built to do, would be an engine for the negative phase even if the model it samples is classical, and this quantum-assisted training of classical Boltzmann machines has been the most concrete use of the idea [106, 111]. The temperature of the device is then an unknown to be estimated, and the samples must be corrected for it. On gate-based machines the thermalizer of Sec-

tion 5.6.3 or imaginary-time evolution prepares the state at a controlled β, at the cost of a variational problem inside every gradient step and of circuits deep enough to represent thermal correlations. The honest expectation, as of this writing, is the one of the whole book: the models are sound, the small cases are reproduced exactly as the theory says, the quantum content pays where the data are quantum, and whether a device can run them at a scale where classical sampling fails is a question about devices.

5.8 Summary and perspectives

This chapter has treated the Boltzmann machine as what it is, a statistical-mechanics model turned into a learning algorithm, and followed it into the quantum domain. The classical theory of Section 5.2 rests on three equations: the likelihood as a divergence, Eq. (5.1); the gradient as a difference of data and model averages, Eq. (5.4); and the conditional independence of the restricted model, Eq. (5.7), which makes the hidden sum and the sampler tractable. Contrastive divergence approximates the one quantity that is not tractable, the model average, and Table 5.1 measured its bias. The quantum model of Section 5.3 replaces the energy by a Hamiltonian and the Boltzmann distribution by a Gibbs state, and the one thing that changes in the training is the derivative of the matrix exponential: Duhamel's formula Eq. (5.18) turns the positive phase into an imaginary-time correlation function that no device is known to measure efficiently, the negative phase survives by the cyclicity of the trace, and the practical rule of the literature, Eq. (5.25), is the gradient of a Golden–Thompson bound that cannot see the visible transverse fields. On classical data the exact gradient drove those fields to zero and the bound was good enough; on quantum data the relative entropy of Eq. (5.26) gave a convex problem with a gradient of pure expectation values, Hamiltonians were learned exactly from their Gibbs states, and the operator content of the family set the limit of what could be learned. The free-energy principle Eq. (5.30), the thermofield double Eq. (5.31) and the variational thermalizer Eq. (5.33) showed how a gate-based computer supplies the thermal states that the whole construction presupposes, with the circuits of Chapter 4 and an entropy that is computed classically because it is put into the model by hand.

This chapter ends the sequence of models. Chapter 6 reserves a place for the third kind of learning of Section 1.2, reinforcement learning and its quantum versions, and Chapter 7 looks back over the five models with the reassessment of Section 1.6 in mind and sets out where we think the subject is going.

5.9 Exercises

1. **Conditional independence and the marginal.** Derive Eq. (5.7) from Eq. (5.6) and show that the hidden sum in Eq. (5.5) factorizes into the product of Eq. (5.8). Then derive Eq. (5.9) from Eq. (5.4) by averaging the hidden units over $p(\boldsymbol{h}|\boldsymbol{v})$, and verify numerically with `RBM.gradient_exact` and `numerical_gradient` for $n_v = 5$, $n_h = 3$.
2. **The bias of contrastive divergence.** Train the bars-and-stripes RBM with the exact gradient to a divergence below 10^{-2}. At the trained parameters, average the CD-k gradient Eq. (5.11) over 10^4 independent data sets and compare the average with the exact gradient for $k = 1, 2, 5, 10, 50$. Plot the norm of the bias against k and relate its decay to the mixing of the Gibbs chain Eq. (5.10), which you can measure by the autocorrelation of the visible energy.

3. **Persistent CD.** Repeat the PCD row of Table 5.1 with learning rates $0.2, 0.05, 0.02$ and $50, 500, 5000$ chains. Which combination beats CD-1, and at what cost in Gibbs steps per update? Explain, in terms of how fast the chains must track the moving model, why PCD needs a small rate.
4. **Duhamel's formula.** Prove Eq. (5.18) by writing $G(s) = e^{-(\beta-s)H} e^{-sH'}$ with $H' = H - \varepsilon \Lambda_\mu$ (that is, $\theta_\mu \to \theta_\mu + \varepsilon$), showing that $G(\beta) - G(0) = \int_0^\beta G'(s)\, ds$, and taking $\varepsilon \to 0$. Derive the matrix elements Eq. (5.19) and show that the diagonal rule is their limit as $e_l \to e_k$. Then implement the formula with a Taylor expansion of the divided difference for $|e_k - e_l| < 10^{-6}$, which `exp_derivative` does not do, and show on a Hamiltonian with a degenerate pair of levels that the plain formula loses accuracy while yours does not.
5. **The clamped-state bound.** Prove Eq. (5.22) from the Golden–Thompson inequality and show that the bound is an equality when all transverse fields vanish. Show from Eq. (5.25) that $\partial \widetilde{\mathscr{L}} / \partial \Gamma_i = \beta \langle X_i \rangle_\rho$ for a visible qubit, and that $\langle X_i \rangle_\rho$ has the sign of Γ_i to first order in Γ_i. Then train the XOR model of Section 5.4 with the bound gradient and with the visible transverse fields held fixed at $\Gamma_0 = \Gamma_1 = 1$, as Amin and coworkers do, and compare the divergence and the visible state ρ_v with those of the free model.
6. **Parity on more bits.** Train the restricted QBM with the exact gradient on the even-parity distribution of $n_v = 3$ bits with $n_h = 1, 2, 3$ hidden qubits, and the classical RBM with the same sizes. How many hidden units does each need to reach a divergence below 10^{-2}? Do the transverse fields stay at zero? Keep the models at $n_v + n_h \le 7$ qubits.
7. **Hamiltonian learning with shot noise.** Replace the target expectation values of the listing in Section 5.5 by estimates from N_s measurements of each Pauli string (sample the eigenvalue ± 1 with the probability $(1 \pm \langle \Lambda_\mu \rangle_{\rho_t})/2$), for $N_s = 10^2, 10^3, 10^4$. How does the error in the learned couplings scale with S? Use the convexity of the loss to explain why the noise in the *model* expectation values, which you may also add, is less harmful than the noise in the target ones.
8. **The thermalizer at low temperature.** Run `vqt.py` for $\beta = 10$ and $\beta = 30$ on the same chain, and on the four-qubit chain of `chapter4/qfi.py`, and compare the energy $\mathrm{Tr}[H\rho]$ of the prepared state with the ground-state energy found by the natural-gradient VQE of that program. At what β does the latent distribution collapse onto one basis state, and how does the entropy term in Eq. (5.33) then affect the optimization? Then run $\beta = 0.1$ and show that the latent distribution becomes uniform and the circuit irrelevant.
9. **The thermofield double by circuit.** For one qubit with $H = -hZ$, show that $|\mathrm{TFD}\rangle$ of Eq. (5.31) is produced from $|00\rangle$ by an R_y rotation of the system qubit followed by a CNOT onto the ancilla, and find the angle as a function of βh. Generalize to a commuting Hamiltonian $H = -\sum_q h_q Z_q$ on n qubits. Why does the construction fail for $H = -Z - X/2$, and what replaces it?
10. **Sampling the negative phase.** The programs of this chapter compute $\langle \Lambda_\mu \rangle_\rho$ exactly. Replace it, for the fully visible model of Section 5.5, by an average over M samples drawn from the diagonal of $\rho_{\boldsymbol{\theta}}$ (for the Z-type operators) and from the diagonal of $\rho_{\boldsymbol{\theta}}$ rotated into the X basis (for the X-type ones), which is what a device would provide, and train with $M = 100$ and 1000 samples per step. Compare the convergence with Fig. 5.4 and with the shot-noise discussion of Exercise 7.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter5/` (NumPy and matplotlib only), which import `common/optimize.py` (Adam and the

finite-difference check) and `bookplot.py`, and for the thermalizer the class `QNN` of `chapter4/qnn.py`; they are imported by the notebook `Notebooks/chapter5/chapter5.ipynb`.

`gibbs.py` Pauli strings, Gibbs states, partial traces, entropies, the free energy Eq. (5.29), the relative entropy, the fidelity, the Duhamel derivative Eq. (5.19) and the thermofield double Eq. (5.31); checks of Eq. (5.30) and Fig. 5.5. 2 s.

`rbm.py` The class `RBM` of Eqs. (5.6)–(5.9) with block Gibbs sampling, CD-k Eq. (5.11) and persistent CD; the XOR and bars-and-stripes targets; Table 5.1, Fig. 5.1. 10 s.

`qbm.py` The class `QBM` of Eqs. (5.13)–(5.15): exact gradient Eq. (5.21), clamped states and the bound gradient Eq. (5.25), the relative entropy Eq. (5.26) and its gradient, Adam training; Table 5.2. 1 s.

`classical_data.py` The four models of Table 5.3 on XOR and bars and stripes, the transverse-field scan and the visible state: Fig. 5.3. 100 s.

`quantum_data.py` Hamiltonian learning and the XXZ target: Tables 5.4 and 5.5, Fig. 5.4. 3 s.

`vqt.py` The class `VQT` of Eqs. (5.32)–(5.33) with product and full latent distributions: Table 5.6, Fig. 5.6. 15 s.

Chapter 6
Reinforcement Learning and Quantum Reinforcement Learning

Abstract This chapter is a placeholder. Reinforcement learning, the third kind of learning of Chapter 1, in which an agent acts in an environment and learns a policy from rewards, has quantum versions in which the policy or the value function is a parametrized circuit, the environment is a quantum system to be controlled, or both. A later edition will develop them here, with the same discipline as the preceding chapters: the classical algorithm derived first, the quantum element added, and both run on the same problems. The present text states the scope and the intended structure.

6.1 Scope of this chapter

This chapter is a placeholder for material to be written. It is kept in the book so that the structure announced in Section 1.9 is visible, and so that the labels and the program folder (`BookPrograms/chapter6/`) are reserved.

Reinforcement learning is the third kind of learning named in Section 1.2. An *agent* observes the state s_t of an *environment*, chooses an action a_t according to a *policy* $\pi(a|s;\boldsymbol{\theta})$, receives a reward r_t, and must adjust $\boldsymbol{\theta}$ so as to maximize the expected cumulative reward. There are no labeled samples: the data are generated by the agent's own actions, and the central quantities are the value function of a state, the action-value function, and the policy gradient, which has the same parameter-shift form as the gradients of Chapter 4 when the policy is a circuit.

Quantum reinforcement learning comes in three forms, and the chapter will treat them in this order.

- *Quantum agents for classical environments*: the policy or the action-value function is a parametrized circuit of the kind of Chapter 4, with the state encoded as data, trained by policy-gradient or Q-learning updates; the questions are the ones of Section 1.6, trainability, classical surrogates and total cost, in a setting where the data are generated online.
- *Classical agents for quantum environments*: the environment is a quantum system to be controlled, prepared or measured, the state is a description of it or of the measurement record, and the reward is a fidelity or an estimation accuracy. This is the form closest to the many-body and quantum-control interests of this book and the one in which, following the argument of Section 1.6, we expect the most durable results: reinforcement learning for state preparation, gate design, error correction and adaptive experiment design.
- *Quantum agents for quantum environments*: agent and environment are both quantum and may be entangled, which is the setting of the quantum-enhanced learning-from-experiments results of Sections 1.6 and 7.3.

The intended sections are: Markov decision processes and the Bellman equations; value-based and policy-based classical algorithms (Q-learning, policy gradients, actor–critic) summarized from the reinforcement-learning part of the machine-learning text [9] and implemented in `common/`; the variational circuit as a policy, with its gradient by the parameter-shift rule and its connection to the Fourier picture of Section 4.3; reinforcement learning for quantum control, with the state-preparation and gate-design problems of the companion volume [1] as environments; adaptive measurement and experiment design; and a closing comparison with the classical baselines. Each section will follow the pattern of the preceding chapters, with every number produced by a program and both the classical and the quantum version run on the same problem.

6.2 Exercises

1. **A first environment.** Write a two-state, two-action Markov decision process with known transition probabilities and rewards, solve the Bellman equations for its value function by iteration, and verify the solution by simulation.

Chapter 7
Summary and Perspectives

Abstract The closing chapter looks back over the models of the book with the reassessment of Chapter 1 in mind. It summarizes what the exact computations of the preceding chapters showed about each model, states what quantum machine learning has been shown to gain on classical and on quantum data, and sets out, as a priority list rather than a consensus, the directions we find most promising for the next years: learning quantum states, processes and dynamics; quantum measurements followed by classical learning; structured variational models; quantum reservoirs and fixed feature maps; and fault-tolerant learning primitives. It ends with the perspectives for algorithms, physical applications and hardware that the companion volume and this book share.

7.1 What the book has shown

The four model chapters have built, from the one hybrid model of Eq. (1.1), a kernel method, an optimization algorithm, a neural network with a physics-informed variant, and a generative model, each with its classical counterpart beside it and each tested exactly on problems small enough to allow it. It is worth collecting what those exact computations showed, model by model, because the pattern is the evidence on which the rest of this chapter rests.

The quantum support-vector machine A new kernel, not a new algorithm. The angle-encoding kernel is a product of cosines and is evaluated classically in $\mathcal{O}(p)$ operations; the entangling feature map of Havlíček and coworkers is not known to be, and on the data set built from its own structure it is exact where the Gaussian kernel is not (Chapter 2). On the data sets one actually meets (moons, circles, XOR, Iris), the quantum kernels matched the classical ones and did not exceed them, and the estimated kernel needs a number of shots that grows with the size of the Gram matrix. The advantage, where there is one, is in the data, as Huang and coworkers [21] say, and the model's job is to not stand in its way.

The quantum approximate optimization algorithm A variational eigensolver for a classical spin glass, whose ansatz is Trotterized adiabatic evolution and whose parameters are trained by the same loop as every later model (Chapter 3). At the depths we simulated it reaches the optimum of the smallest instances, an edge and a ring, but not of a six-vertex graph with a triangle; its landscape has the structure that warm starts exploit, and the per-term shift rule makes its gradients exact; what it does not do is beat the classical heuristics that solve these instances instantly, and the portfolio and clustering applications showed the model as a flexible optimizer rather than as a faster one.

The quantum neural network A trigonometric polynomial in the encoded data whose frequencies are fixed by the encoding and whose coefficients are set by the trainable angles (Chapter 4). This single fact explains every experiment: the regression that an 18-parameter circuit fits at the noise floor and a 321-parameter network fits no better, the classifiers that match the kernel machines, the physics-informed circuits that solve a Poisson or a wave equation to machine precision when the solution lies in their function class and not otherwise. The barren plateaus of random circuits were reproduced with their exponential laws, and the natural gradient, imaginary-time evolution and stochastic reconfiguration were shown to be one equation.

The quantum Boltzmann machine Statistical mechanics as a generative model (Chapter 5). On classical data the likelihood drove the transverse fields to zero and the model fell back on its classical limit, so that the training rule of the literature, the gradient of a Golden–Thompson bound, was good enough because the exact gradient wanted the same thing. On quantum data the quantum relative entropy was convex, Hamiltonians were recovered exactly from their thermal states, and the operator content of the model, non-commuting terms that no classical energy function can even represent, decided what could be learned.

The pattern is the one announced in Section 1.6. The quantum models work as their theory says. On the classical data sets a textbook can hold, their gains over classical methods are gains of parameter economy or of structure rather than of accuracy, and in every case we could name the classical surrogate: the product of cosines, the single-layer product form, the classical Boltzmann machine. On quantum data (a Hamiltonian's ground or thermal state, a kernel between quantum states, the operator content of a model Hamiltonian) there is no classical competitor to begin with, and the quantum model is not a replacement for a classical one but the only model there is.

7.2 The reassessment, with the evidence in hand

Section 1.6 stated three connected difficulties of variational quantum machine learning and promised that the chapters would meet each in a concrete form. They did. *Trainability*: the gradient variances of Table 4.2 fall as 2^{-n} and 4^{-n} for the local and global costs, the kernel values of expressive feature maps concentrate exponentially (Section 2.11), and the shot-noise study of Section 2.9.1 shows what finite statistics do to a kernel matrix. *Classical competition*: every model that trained well in this book had a classical surrogate we could write down, and the published results on the link between provable absence of barren plateaus and classical simulability [25] and on classical surrogates of quantum convolutional networks [26] say that this is not an artifact of our small examples. *Total cost*: the kernels of Chapter 2 need $\mathcal{O}(N^2)$ circuit runs of S shots each, the gradients of Chapter 4 need $2P$ circuit evaluations per sample per step, and the Boltzmann machine of Chapter 5 needs a Gibbs state prepared anew for every gradient step; none of these costs appears in the classical competitor.

None of this is a theorem against variational learning. Avoiding barren plateaus does not universally imply classical simulability; the connection has been established case by case, for many commonly proposed architectures, and the surrogate sometimes requires a quantum data-acquisition stage that a classical computer cannot replace. The surrogate of a quantum convolutional network may still need quantum measurements of its input, and what it removes is the repeated quantum training loop. These results are serious challenges to particular designs. What they change is the burden of proof: a proposed quantum learning model must now say which quantum resource it uses and what that resource buys, and "it trains" and "it generalizes" are no longer answers.

7.3 A priority list for the next years

Table 7.1 gives, for the next three to five years, the directions we find promising, in our order of priority. It is a judgment rather than a consensus, and each entry carries the qualification that we think decides it.

Table 7.1 Directions in quantum machine learning for the near term, in our order of priority, with the reason for the ranking and the qualification that will decide each.

Approach	Why we find it promising	Main qualification
Learning quantum states, processes and dynamics	Quantum inputs avoid the classical data-loading bottleneck; coherent access and entangled measurements can give sample advantages	The advantages depend on the task and on the experimental access permitted
Quantum measurements followed by classical learning	Extract informative features once, then train classical models; often more economical than repeatedly optimizing a circuit	Some tasks need quantum memory or collective measurements that an ordinary measurement record cannot preserve
Structured variational models	Symmetry, locality, conservation laws and physical dynamics provide an inductive bias	Trainability and good predictive performance do not by themselves establish a quantum advantage
Quantum reservoir computing and fixed feature maps	Let quantum dynamics generate features and train mainly the classical readout; little optimization overhead	Measurement cost and strong classical reservoir baselines remain decisive
Fault-tolerant learning primitives	Coherent algorithms open possibilities beyond shallow noisy circuits	Practical value depends on logical hardware, data access and full resource estimates

Learning quantum states, processes and dynamics There is concrete evidence behind the first direction. The theory of learning from experiments [28] shows an exponential separation in the number of experiments between an agent that can store and jointly measure copies of a quantum state and one that measures each copy and reasons classically, and the photonic experiment of Liu and coworkers [29] realized, using entanglement, such a separation for the learning of bosonic displacement processes. This is evidence for learning quantum processes, not for a general quantum advantage on ordinary classification data, and the Hamiltonian learning of Section 5.5 is its simplest textbook instance: a convex problem whose data are expectation values of the state to be learned. The sample-complexity results of Anshu and coworkers [104] for Gibbs states, and the provably efficient classical learning of ground-state properties from measurement data [112], set the frame in which such claims should be made.

Quantum measurements followed by classical learning The second direction follows directly from the classical-surrogate results: the essential quantum contribution may be acquiring information, with the useful learning done classically. Classical shadows [113] are the tool, a randomized measurement protocol from which many observables of a state can be predicted from few copies, and the surrogate of the quantum convolutional network [26] is the example: the measurements of the input are quantum, the training is not. The qualification is that some tasks need a quantum memory or collective measurements that no record of single-copy outcomes preserves, which is exactly the resource that the first direction uses.

Structured variational models Variational circuits retain a credible role here. Circuits that respect the symmetries of a learning problem [30, 31] encode its structure and improve generalization in the examples studied, and for many-body physics the natural structure is

particle number, spin, spatial symmetry and the observables of interest, which is the content of the physically motivated ansätze of the companion volume [1]. The qualification is the one of Section 7.2: a structured circuit that trains and predicts well has not thereby been shown to outperform a tensor network or a classical network with the same symmetries built in.

Quantum reservoir computing and fixed feature maps The fourth direction removes the optimization loop. A quantum system with fixed dynamics is driven by the input, its measured observables are the features, and a linear readout is trained classically [32]; the circuit-QED demonstration of Carles and coworkers [33], with a transmon–resonator system generating nonlinear features and no quantum gradient-training loop, is a recent proof of concept. The kernel methods of Chapter 2 are the same idea with the readout a support-vector machine. Whether a quantum reservoir beats a classical one at the same measurement budget is the decisive, and open, question.

Fault-tolerant learning primitives The fifth direction is the one of the original speedup claims of Section 1.4, read with Aaronson's fine print [18] and Tang's dequantizations [19] in mind. Coherent algorithms (phase estimation, amplitude estimation, the linear-system algorithm and quantum principal-component analysis [114]) open possibilities that shallow noisy circuits do not have, and their practical value depends on logical hardware, on how the data are accessed, and on full resource estimates of the kind the companion volume makes for its many-body algorithms.

7.4 For many-body physics and quantum sensing

For the interests of the readers this book was written for, the priority order above has a sharper form. We would prioritize quantum process and Hamiltonian learning and adaptive experiment design, and compare structured variational models against measurement-based classical learning on the same physical problems. The central question should be: *which quantum resource reduces the number of experiments, improves the estimation accuracy, or lowers the total computation at a fixed accuracy?* A proposal that cannot name the resource and the quantity it improves is not yet a proposal. The models of this book that pass the test are the ones whose data are quantum: the kernel between quantum states, the Hamiltonian learned from its Gibbs state, the thermal state prepared variationally for a Hamiltonian of interest. The ones that do not, yet, are the ones that load classical data into a circuit, and they should be taught, as we have tried to teach them, alongside their trainability limits, their classical surrogates and the quantum-data alternatives. They remain useful tools; they are no longer a sufficient research strategy on their own.

7.5 Perspectives

The perspectives of the companion volume and of this book are the same, seen from two sides, and we close with them.

Algorithms On the near-term side: ansätze with the symmetries of the problem built in, error mitigation by zero-noise extrapolation and probabilistic error cancellation, remedies for barren plateaus by locality, layer-wise training and informed initialization, and benchmarks on tasks where the data are generated by quantum systems in the first place. On the fault-tolerant side: Hamiltonian simulation by Trotterization, qubitization and linear combinations of unitaries, randomized and Bayesian phase estimation with fewer qubits, quantum linear

algebra read with its dequantizations, and quantum walks and search as subroutines of Monte Carlo.

Applications in the physical sciences Electronic structure, nuclear structure with the Lipkin, pairing and shell-model Hamiltonians of the companion volume and of the many-body text [34], strongly correlated systems beyond the reach of coupled-cluster theory, finite-temperature properties by imaginary-time evolution and thermal-state preparation, which may be the most natural application of the methods of Section 5.6; lattice gauge theories and the Hubbard model, topological phases, and quantum sensing with Heisenberg-limited parameter estimation, where the learning-from-experiments results of Section 7.3 apply most directly.

Quantum machine learning Generative models beyond the restricted quantum Boltzmann machine, quantum generative adversarial and diffusion models; quantum reinforcement learning for quantum control and for adaptive experiment design, the subject reserved for Chapter 6; tensor networks and classical circuit simulation as both competitor and guide; and classical shadow tomography and cross-entropy benchmarking as the tools for certifying what a device has done.

Hardware Superconducting, trapped-ion, photonic and neutral-atom platforms; surface and color codes and the first logical qubits; and the open question of how many logical qubits are needed for an advantage on a problem anyone cares about. Quantum advantage requires better algorithms *and* better hardware, in tandem, and the scale of fault-tolerant machines will decide which of the algorithms of this book are ever run on data too large to enumerate. The programs of the preceding chapters are written so that readers can run them on such a machine, or on its simulator, with the simulator of `common/` replaced by a backend, and see for themselves whether the picture changes.

References

1. A. Bazavov, Z. Davoudi, M. Hjorth-Jensen, R. LaRose, D. Lee, A. Roggero, *Quantum Computing for Many-body Physics* (Springer, in preparation). Companion volume; the chapters on qubits, gates, measurements, Hamiltonian simulation, phase estimation and variational algorithms are assumed here
2. J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, S. Lloyd, Nature **549**, 195 (2017). DOI 10.1038/nature23474
3. M. Schuld, I. Sinayskiy, F. Petruccione, Contemporary Physics **56**, 172 (2015). DOI 10.1080/00107514.2014.964942
4. M. Schuld, F. Petruccione, *Machine Learning with Quantum Computers*, 2nd edn. (Springer, Cham, 2021). DOI 10.1007/978-3-030-83098-4
5. M. Schuld. Supervised quantum machine learning models are kernel methods (2021). ArXiv:2101.11020
6. T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edn. (Springer, New York, 2009)
7. C.M. Bishop, *Pattern Recognition and Machine Learning* (Springer, New York, 2006)
8. I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning* (MIT Press, Cambridge, MA, 2016)
9. M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning* (Springer, in preparation). Draft available at `https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf`
10. K. Mitarai, M. Negoro, M. Kitagawa, K. Fujii, Physical Review A **98**, 032309 (2018). DOI 10.1103/PhysRevA.98.032309
11. M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, N. Killoran, Physical Review A **99**, 032331 (2019). DOI 10.1103/PhysRevA.99.032331
12. Y. Liu, S. Arunachalam, K. Temme, Nature Physics **17**, 1013 (2021). DOI 10.1038/s41567-021-01287-z
13. M.A. Nielsen, I.L. Chuang, *Quantum Computation and Quantum Information*, 10th edn. (Cambridge University Press, Cambridge, 2010)
14. V. Havlíček, A.D. Córcoles, K. Temme, A.W. Harrow, A. Kandala, J.M. Chow, J.M. Gambetta, Nature **567**, 209 (2019). DOI 10.1038/s41586-019-0980-2
15. P. Rebentrost, M. Mohseni, S. Lloyd, Physical Review Letters **113**, 130503 (2014). DOI 10.1103/PhysRevLett.113.130503
16. A.W. Harrow, A. Hassidim, S. Lloyd, Physical Review Letters **103**, 150502 (2009). DOI 10.1103/PhysRevLett.103.150502
17. V. Giovannetti, S. Lloyd, L. Maccone, Physical Review Letters **100**, 160501 (2008). DOI 10.1103/PhysRevLett.100.160501
18. S. Aaronson, Nature Physics **11**, 291 (2015). DOI 10.1038/nphys3272
19. E. Tang, in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)* (2019), pp. 217–228. DOI 10.1145/3313276.3316310
20. J. Preskill, Quantum **2**, 79 (2018). DOI 10.22331/q-2018-08-06-79
21. H.Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, J.R. McClean, Nature Communications **12**, 2631 (2021). DOI 10.1038/s41467-021-22539-9
22. J.R. McClean, S. Boixo, V.N. Smelyanskiy, R. Babbush, H. Neven, Nature Communications **9**, 4812 (2018). DOI 10.1038/s41467-018-07090-4
23. S. Thanasilp, S. Wang, M. Cerezo, Z. Holmes, Nature Communications **15**, 5200 (2024). DOI 10.1038/s41467-024-49287-w
24. M. Cerezo, G. Verdon, H.Y. Huang, L. Cincio, P.J. Coles, Nature Computational Science **2**, 567 (2022). DOI 10.1038/s43588-022-00311-3
25. M. Cerezo, M. Larocca, D. García-Martín, N.L. Diaz, P. Braccia, E. Fontana, M.S. Rudolph, P. Bermejo, A. Ijaz, S. Thanasilp, E.R. Anschuetz, Z. Holmes, Nature Communications **16**, 7907 (2025). DOI 10.1038/s41467-025-63099-6
26. P. Bermejo, P. Braccia, M.S. Rudolph, Z. Holmes, L. Cincio, M. Cerezo, PRX Quantum (2026). DOI 10.1103/8qt9-72ts. ArXiv:2408.12739
27. T. Hoefler, T. Häner, M. Troyer, Communications of the ACM **66**, 82 (2023). DOI 10.1145/3571725
28. H.Y. Huang, M. Broughton, J. Cotler, S. Chen, J. Li, M. Mohseni, H. Neven, R. Babbush, R. Kueng, J. Preskill, J.R. McClean, Science **376**, 1182 (2022). DOI 10.1126/science.abn7293
29. Z.H. Liu, R. Brunel, E.E.B. Østergaard, O. Cordero, S. Chen, Y. Wong, J.A.H. Nielsen, A.B. Bregnsbo, S. Zhou, H.Y. Huang, C. Oh, L. Jiang, J. Preskill, J.S. Neergaard-Nielsen, U.L. Andersen, Science (2025). DOI 10.1126/science.adv2560. ArXiv:2502.07770
30. M. Larocca, F. Sauvage, F.M. Sbahi, G. Verdon, P.J. Coles, M. Cerezo, PRX Quantum **3**, 030341 (2022). DOI 10.1103/PRXQuantum.3.030341
31. J.J. Meyer, M. Mularski, E. Gil-Fuster, A.A. Mele, F. Arzani, A. Wilms, J. Eisert, PRX Quantum **4**, 010328 (2023). DOI 10.1103/PRXQuantum.4.010328

32. K. Fujii, K. Nakajima, Physical Review Applied **8**, 024030 (2017). DOI 10.1103/PhysRevApplied.8.024030
33. B. Carles, J. Dudas, L. Balembois, others, D. Marković, Physical Review Applied (2026). DOI 10.1103/fmm7-3qm9. ArXiv:2506.22016
34. M. Hjorth-Jensen, *Quantum Mechanics for Many-particle Systems: A Computational Perspective. From Standard Methods to Quantum Computing and Machine Learning* (Springer, in preparation). Draft available at `https://github.com/EducationalMaterialUiO/ManyBodyPhysicsUiO/blob/main/doc/BookManybody/lecturebook.pdf`
35. V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations (2018). ArXiv:1811.04968
36. A. Javadi-Abhari, M. Treinish, K. Krsulich, C.J. Wood, J. Lishman, J. Gacon, S. Martiel, P.D. Nation, L.S. Bishop, A.W. Cross, B.R. Johnson, J.M. Gambetta. Quantum computing with Qiskit (2024). ArXiv:2405.08810
37. B.E. Boser, I.M. Guyon, V.N. Vapnik, in *Proceedings of the Fifth Annual Workshop on Computational Learning Theory (COLT)* (1992), pp. 144–152. DOI 10.1145/130385.130401
38. C. Cortes, V. Vapnik, Machine Learning **20**, 273 (1995). DOI 10.1007/BF00994018
39. S. Boyd, L. Vandenberghe, *Convex Optimization* (Cambridge University Press, Cambridge, 2004)
40. J. Mercer, Philosophical Transactions of the Royal Society A **209**, 415 (1909). DOI 10.1098/rsta.1909.0016
41. J.C. Platt, Sequential minimal optimization: A fast algorithm for training support vector machines. Tech. Rep. MSR-TR-98-14, Microsoft Research (1998)
42. M. Schuld, N. Killoran, Physical Review Letters **122**, 040504 (2019). DOI 10.1103/PhysRevLett.122.040504
43. R. Shaydulin, S.M. Wild, Physical Review A **106**, 042407 (2022). DOI 10.1103/PhysRevA.106.042407
44. H. Buhrman, R. Cleve, J. Watrous, R. de Wolf, Physical Review Letters **87**, 167902 (2001). DOI 10.1103/PhysRevLett.87.167902
45. R.A. Fisher, Annals of Eugenics **7**, 179 (1936). DOI 10.1111/j.1469-1809.1936.tb02137.x
46. J.M. Kübler, S. Buchholz, B. Schölkopf, in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)* (2021), pp. 12,661–12,673
47. N. Cristianini, J. Shawe-Taylor, A. Elisseeff, J. Kandola, in *Advances in Neural Information Processing Systems 14 (NIPS 2001)* (2001), pp. 367–373
48. T. Hubregtsen, D. Wierichs, E. Gil-Fuster, P.J.H.S. Derks, P.K. Faehrmann, J.J. Meyer, Physical Review A **106**, 042431 (2022). DOI 10.1103/PhysRevA.106.042431
49. A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J.I. Latorre, Quantum **4**, 226 (2020). DOI 10.22331/q-2020-02-06-226
50. S. Lloyd, M. Schuld, A. Ijaz, J. Izaac, N. Killoran. Quantum embeddings for machine learning (2020). ArXiv:2001.03622
51. S. Jerbi, L.J. Fiderer, H.P. Nautrup, J.M. Kübler, H.J. Briegel, V. Dunjko, Nature Communications **14**, 517 (2023). DOI 10.1038/s41467-023-36159-y
52. E. Farhi, J. Goldstone, S. Gutmann. A quantum approximate optimization algorithm (2014). ArXiv:1411.4028
53. Röscher, et al. Two-qubit implementation of QAOA for MAX-CUT on an NV-center quantum processor (2026). ArXiv:2604.00949
54. G. Kochenberger, J.K. Hao, F. Glover, M. Lewis, Z. Lü, H. Wang, Y. Wang, Journal of Combinatorial Optimization **28**, 58 (2014). DOI 10.1007/s10878-014-9734-0
55. A. Lucas, Frontiers in Physics **2**, 5 (2014). DOI 10.3389/fphy.2014.00005
56. M.X. Goemans, D.P. Williamson, Journal of the ACM **42**, 1115 (1995). DOI 10.1145/227683.227684
57. E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, D. Preda, Science **292**, 472 (2001). DOI 10.1126/science.1057726
58. T. Albash, D.A. Lidar, Reviews of Modern Physics **90**, 015002 (2018). DOI 10.1103/RevModPhys.90.015002
59. L.T. Brady, C.L. Baldwin, A. Bapat, Y. Kharkov, A.V. Gorshkov, Physical Review Letters **126**, 070505 (2021). DOI 10.1103/PhysRevLett.126.070505
60. L. Zhou, S.T. Wang, S. Choi, H. Pichler, M.D. Lukin, Physical Review X **10**, 021067 (2020). DOI 10.1103/PhysRevX.10.021067
61. E. Farhi, A.W. Harrow. Quantum supremacy through the quantum approximate optimization algorithm (2016). ArXiv:1602.07674
62. S. Bravyi, A. Kliesch, R. Koenig, E. Tang, Physical Review Letters **125**, 260505 (2020). DOI 10.1103/PhysRevLett.125.260505
63. M.P. Harrigan, et al., Nature Physics **17**, 332 (2021). DOI 10.1038/s41567-020-01105-y
64. D.J. Egger, J. Mareček, S. Woerner, Quantum **5**, 479 (2021). DOI 10.22331/q-2021-06-17-479
65. G.E. Crooks. Gradients of parameterized quantum gates using the parameter-shift rule and gate decomposition (2019). ArXiv:1905.13311

66. D. Wierichs, J. Izaac, C. Wang, C.Y.Y. Lin, Quantum **6**, 677 (2022). DOI 10.22331/q-2022-03-30-677
67. D. Loss, D.P. DiVincenzo, Physical Review A **57**, 120 (1998). DOI 10.1103/PhysRevA.57.120
68. J.R. Petta, A.C. Johnson, J.M. Taylor, E.A. Laird, A. Yacoby, M.D. Lukin, C.M. Marcus, M.P. Hanson, A.C. Gossard, Science **309**, 2180 (2005). DOI 10.1126/science.1116955
69. G. Burkard, T.D. Ladd, A. Pan, J.M. Nichol, J.R. Petta, Reviews of Modern Physics **95**, 025003 (2023). DOI 10.1103/RevModPhys.95.025003
70. H. Markowitz, The Journal of Finance **7**, 77 (1952). DOI 10.1111/j.1540-6261.1952.tb01525.x
71. R. Orús, S. Mugel, E. Lizaso, Reviews in Physics **4**, 100028 (2019). DOI 10.1016/j.revip.2019.100028
72. D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, Y. Alexeev, Nature Reviews Physics **5**, 450 (2023). DOI 10.1038/s42254-023-00603-1
73. S. Hadfield, Z. Wang, B. O'Gorman, E.G. Rieffel, D. Venturelli, R. Biswas, Algorithms **12**, 34 (2019). DOI 10.3390/a12020034
74. A.Y. Ng, M.I. Jordan, Y. Weiss, Advances in Neural Information Processing Systems **14**, 849 (2001)
75. M. Schuld, R. Sweke, J.J. Meyer, Physical Review A **103**, 032430 (2021). DOI 10.1103/PhysRevA.103.032430
76. D.P. Kingma, J. Ba. Adam: A method for stochastic optimization (2015). International Conference on Learning Representations; arXiv:1412.6980
77. S.L. Braunstein, C.M. Caves, Physical Review Letters **72**, 3439 (1994). DOI 10.1103/PhysRevLett.72.3439
78. J.J. Meyer, Quantum **5**, 539 (2021). DOI 10.22331/q-2021-09-09-539
79. A. Mari, T.R. Bromley, N. Killoran, Physical Review A **103**, 012405 (2021). DOI 10.1103/PhysRevA.103.012405
80. S. ichi Amari, Neural Computation **10**, 251 (1998). DOI 10.1162/089976698300017746
81. J. Stokes, J. Izaac, N. Killoran, G. Carleo, Quantum **4**, 269 (2020). DOI 10.22331/q-2020-05-25-269
82. X. Yuan, S. Endo, Q. Zhao, Y. Li, S.C. Benjamin, Quantum **3**, 191 (2019). DOI 10.22331/q-2019-10-07-191
83. S. McArdle, T. Jones, S. Endo, Y. Li, S.C. Benjamin, X. Yuan, npj Quantum Information **5**, 75 (2019). DOI 10.1038/s41534-019-0187-2
84. S. Sorella, Physical Review Letters **80**, 4558 (1998). DOI 10.1103/PhysRevLett.80.4558
85. A. Abbas, D. Sutter, C. Zoufal, A. Lucchi, A. Figalli, S. Woerner, Nature Computational Science **1**, 403 (2021). DOI 10.1038/s43588-021-00084-1
86. M. Cerezo, A. Sone, T. Volkoff, L. Cincio, P.J. Coles, Nature Communications **12**, 1791 (2021). DOI 10.1038/s41467-021-21728-w
87. S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, P.J. Coles, Nature Communications **12**, 6961 (2021). DOI 10.1038/s41467-021-27045-6
88. Z. Holmes, K. Sharma, M. Cerezo, P.J. Coles, PRX Quantum **3**, 010313 (2022). DOI 10.1103/PRXQuantum.3.010313
89. M. Raissi, P. Perdikaris, G.E. Karniadakis, Journal of Computational Physics **378**, 686 (2019). DOI 10.1016/j.jcp.2018.10.045
90. S. Wang, X. Yu, P. Perdikaris, Journal of Computational Physics **449**, 110768 (2022). DOI 10.1016/j.jcp.2021.110768
91. O. Kyriienko, A.E. Paine, V.E. Elfving, Physical Review A **103**, 052416 (2021). DOI 10.1103/PhysRevA.103.052416
92. S. Markidis, Frontiers in Applied Mathematics and Statistics **8**, 1036711 (2022). DOI 10.3389/fams.2022.1036711
93. I.E. Lagaris, A. Likas, D.I. Fotiadis, IEEE Transactions on Neural Networks **9**, 987 (1998). DOI 10.1109/72.712178
94. D.H. Ackley, G.E. Hinton, T.J. Sejnowski, Cognitive Science **9**, 147 (1985). DOI 10.1207/s15516709cog0901_7
95. P. Smolensky, in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1*, ed. by D.E. Rumelhart, J.L. McClelland (MIT Press, Cambridge, MA, 1986), pp. 194–281
96. M.H. Amin, E. Andriyash, J. Rolfe, B. Kulchytskyy, R. Melko, Physical Review X **8**, 021050 (2018). DOI 10.1103/PhysRevX.8.021050
97. N.L. Roux, Y. Bengio, Neural Computation **20**, 1631 (2008). DOI 10.1162/neco.2008.04-07-510
98. G.E. Hinton, Neural Computation **14**, 1771 (2002). DOI 10.1162/089976602760128018
99. M.Á. Carreira-Perpiñán, G.E. Hinton, in *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)* (2005), pp. 33–40
100. Y. Bengio, O. Delalleau, Neural Computation **21**, 1601 (2009). DOI 10.1162/neco.2008.11-07-647
101. T. Tieleman, in *Proceedings of the 25th International Conference on Machine Learning (ICML)* (2008), pp. 1064–1071. DOI 10.1145/1390156.1390290
102. M. Kieferová, N. Wiebe, Physical Review A **96**, 062327 (2017). DOI 10.1103/PhysRevA.96.062327
103. H.J. Kappen, Journal of Physics A: Mathematical and Theoretical **53**, 214001 (2020). DOI 10.1088/1751-8121/ab7df6

104. A. Anshu, S. Arunachalam, T. Kuwahara, M. Soleimanifar, Nature Physics **17**, 931 (2021). DOI 10.1038/s41567-021-01232-0
105. N. Wiebe, L. Wossnig. Generative training of quantum Boltzmann machines with hidden units (2019). ArXiv:1905.09902
106. M. Benedetti, J. Realpe-Gómez, R. Biswas, A. Perdomo-Ortiz, Physical Review X **7**, 041052 (2017). DOI 10.1103/PhysRevX.7.041052
107. J. Wu, T.H. Hsieh, Physical Review Letters **123**, 220502 (2019). DOI 10.1103/PhysRevLett.123.220502
108. J. Martyn, B. Swingle, Physical Review A **100**, 032107 (2019). DOI 10.1103/PhysRevA.100.032107
109. G. Verdon, J. Marks, S. Nanda, S. Leichenauer, J. Hidary. Quantum Hamiltonian-based models and the variational quantum thermalizer algorithm (2019). ArXiv:1910.02071
110. C. Zoufal, A. Lucchi, S. Woerner, Quantum Machine Intelligence **3**, 7 (2021). DOI 10.1007/s42484-020-00033-7
111. V. Dixit, R. Selvarajan, M.A. Alam, T.S. Humble, S. Kais, Frontiers in Physics **9**, 589626 (2021). DOI 10.3389/fphy.2021.589626
112. H.Y. Huang, R. Kueng, G. Torlai, V.V. Albert, J. Preskill, Science **377**, eabk3333 (2022). DOI 10.1126/science.abk3333
113. H.Y. Huang, R. Kueng, J. Preskill, Nature Physics **16**, 1050 (2020). DOI 10.1038/s41567-020-0932-7
114. S. Lloyd, M. Mohseni, P. Rebentrost, Nature Physics **10**, 631 (2014). DOI 10.1038/nphys3029

Index

2-design, 66

Adam, 63
adaptive experiment design, 108
adiabatic quantum computation, 40
adjoint test, 23
alternating-operator ansatz, 40
amplitude encoding, 8
angle encoding, 8
  kernel, 21
approximation ratio, 44

bandwidth, 21
barren plateau, 5, 66
  noise-induced, 68
  QAOA, 44
bars and stripes, 90
basis encoding, 8
benchmarks, 5
Bernoulli distribution, 96
BFGS, 63
binomial noise, 23
block Gibbs sampling, 84
Boltzmann distribution, 83
Boltzmann machine, 81
Born rule, 86
Bures metric, 96

CD, *see* contrastive divergence
circuit depth p, 40
clamped expectation, 87
clamped Hamiltonian, 89
classical data, 1
  Boltzmann machines on, 90
classical shadows, 6, 107
classical surrogate, 6
classification
  with a QNN, 69
clustering, 52
CNOT ladder, 58
coherence, 90
combinatorial optimization, 38
companion volume, 7
complementary slackness, 15
concentration of measure, 66
conditional independence, 83
constraint
  penalty, 48
contrastive divergence, 84
conventions, 11
convexity, 92
cost function, 2
cost Hamiltonian, 38
Cramér–Rao bound, 63

data encoding, 8
data re-uploading, 31, 59
decision function, 14
decoherence, 5
density matrix
  as feature vector, 21
dequantization, 6, 32
divided differences, 87
double quantum dot, 47
dual problem, 15
Duhamel's formula, 87

effective dimension, 66
eigenvalue clipping, 29
encoding circuit, 3
energy landscape, 43
energy-based model, 82
entanglement
  in feature maps, 4
entanglement entropy, 95
error mitigation, 108
exchange interaction, 47
exponential concentration, 32
expressivity, 60
  of Boltzmann machines, 99

fault-tolerant learning, 107
feature map, 3, 18
feature space, 18
fidelity, 92
fidelity kernel, 21
finite differences, 45
Fisher information, 63
Fourier series
  of a QNN output, 60

free energy, 94
  of a visible configuration, 83
frequency spectrum, 60
Fubini–Study metric, 63

gate count, 43
Gaussian kernel, 18
generative model, 81
Gibbs sampling, 84
Gibbs state, 86
global cost, 66
Golden–Thompson inequality, 89
gradient descent, 61
Gram matrix, 18

Hamiltonian learning, 92
hard constraints, 72
hardware platforms, 108
hardware-efficient ansatz, 58
harmonic oscillator, 72
Havlíček feature map, 21
heat equation, 72
Heisenberg model, 92
Heisenberg picture, 43, 59
Hessian, 61
HHL algorithm, 4
hidden units, 83
Hilbert space
  dimension, 4
hinge loss, 17
hybrid quantum-classical model, 3
hyperplane, 14

imaginary time
  in the QBM gradient, 87
imaginary-time evolution, 65
input-shift rule, 73
interference, 4
inverse problem, 99
Iris data set, 25
Ising model, 38

Karush–Kuhn–Tucker conditions, 15
kernel, 18
kernel estimation, 23
kernel method, 2, 13
kernel trick, 18
kernel-target alignment, 31
Klein's inequality, 94
Kullback–Leibler divergence, 82

Lagrange multiplier, 15
Lagrangian, 15
latent distribution, 96
learning from experiments, 6, 107
local cost, 66
logistic function, 83
loss function, 61

machine learning, 1
margin, 14
Markov chain, 84
Markov decision process, 103
Markowitz model, 48
matrix exponential
  derivative, 87
MaxCut, 38
maximum likelihood, 82
maximum margin, 14
McLachlan variational principle, 63
Mercer's theorem, 18
mixing Hamiltonian, 40
mixing time, 84
model, 2
multilayer perceptron, 69

natural gradient, 65
negative log-likelihood, 82
negative phase, 83
neural network, 2
neural tangent kernel, 68
NISQ, 5
notation, 11

observable
  choice of, 58
operator content, 92
optimizer, 2
overfitting, 2

parameter-shift rule, 31, 61
  general, 62
  QAOA, 45
  second order, 61
parametrized quantum circuit, 3
parent Hamiltonian, 99
partial trace, 86
partition function, 83
Pauli string, 86
penalty method, 48
PennyLane, 11
per-term shift rule, 45
persistent contrastive divergence, 84
perspectives, 108
physics-informed neural network, 72
physics-informed quantum neural network, 72
plan of the book, 10
Poisson equation, 72
policy, 103
policy gradient, 103
polynomial kernel, 18
portfolio optimization, 48
  with a QNN, 71
positive phase, 83
positive semidefinite, 18
primal problem, 15
priorities, 107
programs
  chapter 1, 12
  chapter 2, 34
  chapter 3, 55
  chapter 4, 79
  chapter 5, 101

PSD projection, 23
purification, 95

Q-PINN, *see* physics-informed quantum neural network
QAOA, *see* quantum approximate optimization algorithm
QBM, *see* quantum Boltzmann machine
Qiskit, 11
QNN, *see* quantum neural network
quadratic programming, 19
quantum advantage, 4, 99
  Boltzmann machines, 99
  in learning, 107
  kernel methods, 32
quantum annealer, 99
quantum approximate optimization algorithm, 37
  as a machine-learning model, 52
quantum Boltzmann machine, 81
  definition, 86
quantum computing, 1
quantum control, 103
quantum data, 6, 92
quantum feature map, 21
quantum Fisher information, 63
quantum Fourier transform, 4
quantum kernel, 21
quantum machine learning, 3
  QAOA, 52
quantum neural network, 57
  model, 58
  versus Boltzmann machine, 98
quantum random-access memory, 4
quantum reinforcement learning, 103
quantum relative entropy, 92
quantum reservoir computing, 6, 107
quantum sensing, 108
quantum support-vector machine, 25
QUBO, 38

radial basis function kernel, 18
Rayleigh–Ritz principle, 94
RBM, *see* restricted Boltzmann machine
regression
  with a QNN, 69
regularization
  constant C, 17
  support-vector machine, 17
reinforcement learning, 2, 103
residual loss, 72
response function, 62
restricted Boltzmann machine, 83
restricted quantum Boltzmann machine, 86
Ritz method, 75

Schrödinger equation
  as a Q-PINN, 77
sequential minimal optimization, 19
shot noise, 5
  kernel, 23, 29
shots, 23
singlet and triplet, 47
slack variable, 17
SMO, *see* sequential minimal optimization
soft margin, 17
softmax, 96
speedup claims, 4
spin glass, 48
spin qubit, 47
square loss, 61
state-vector simulator, 11
stochastic reconfiguration, 65
sufficient statistics, 83
summary of the book, 105
supervised learning, 2
support vector, 15
support-vector machine, 13
SWAP test, 23
symmetry
  in variational circuits, 6

thermal state, 86
  in QAOA, 48
thermofield double, 95
time-dependent variational principle, 65
total cost, 6
trainability, 5
trainable feature map, 31
trainable kernel, 31
training set and test set, 2
transverse field, 86
  trained to zero, 90
transverse-field Ising model, 99
  Gibbs state, 96
  ground state by VQE, 63
trial function, 72
trigonometric polynomial, 60
Trotterization, 40
two circles, 25
two moons, 25

unsupervised learning, 2
upper bound
  on the likelihood, 89

value function, 103
variational principle
  free energy, 94
variational quantum Boltzmann machine, 96
variational quantum circuit, 57
  reassessment, 6, 106
variational quantum classifier, 70
variational quantum eigensolver, 63
  as a limit of the QBM, 99
variational quantum thermalizer, 96
visible units, 83
von Neumann entropy, 94
VQE, *see* variational quantum eigensolver
VQT, *see* variational quantum thermalizer

warm start, 43
wave equation, 76

XOR distribution, 90

XOR problem, 25

XXZ chain, 92

ZZ feature map, 21